

# ROS 2 二维机器人 运动规划实践

从仿真到定位、规划与控制

本书随代码逐章成长。每个例子都有明确的运行入口，  
每个公式都能在对函数和实验中找到用途。

**阅读方法** 先运行、看见现象，再读公式和关键代码，最后完成一个小练习。

**版本范围** 第 01-20 章与选学附录；已实现功能和选做扩展在正文分别标明。

主机 ROS 2 Lyrical / RViz2

独立算法 C++17, ROS 节点 C++20

2026 年 9 月

# 目录

|                           |           |
|---------------------------|-----------|
| <b>第一章 环境搭建与第一个圆盘场景</b>   | <b>1</b>  |
| 1.1 先认识四个名字               | 1         |
| 1.2 确认主机环境                | 1         |
| 1.3 构建并启动                 | 2         |
| 1.4 圆盘为什么用直径?             | 3         |
| 1.5 发布一次, 为什么晚启动也能看到?     | 3         |
| 1.6 动手修改第一个场景             | 4         |
| <b>第二章 二维坐标、时间与接口</b>     | <b>5</b>  |
| 2.1 一个点可以有不同坐标            | 5         |
| 2.2 从公式到一个小函数             | 6         |
| 2.3 逆变换与变换链               | 6         |
| 2.4 运行与理解数值误差             | 7         |
| 2.5 练习: 自己写一次逆变换          | 7         |
| 2.6 仿真时间与真实等待时间           | 8         |
| 2.7 把坐标放到 ROS 的 TF 树中     | 8         |
| 2.8 暂停、单步与重置实验            | 9         |
| <b>第三章 一个可复现的二维世界</b>     | <b>11</b> |
| 3.1 先看世界, 再谈算法            | 11        |
| 3.2 用距离描述圆盘与障碍            | 12        |
| 3.3 从最小化距离推导线段投影          | 12        |
| 3.4 用拐弯方向判断内外与凸包          | 13        |
| 3.5 怎样在圆盘面积上均匀采样          | 14        |
| 3.6 让一个自由网格容纳整个圆盘         | 14        |
| 3.7 从自由网格中找出连通区域          | 15        |
| 3.8 动手修改与练习               | 15        |
| <b>第四章 让圆盘运动: 位置、速度与力</b> | <b>16</b> |
| 4.1 圆心状态与圆盘外形             | 16        |
| 4.2 先让机器人到一个点             | 16        |

|                                    |           |
|------------------------------------|-----------|
| 4.3 扫过的空间也要安全 . . . . .            | 17        |
| 4.4 直接设置位姿的用途 . . . . .            | 17        |
| 4.5 冻结试验与重新开始 . . . . .            | 18        |
| 4.6 给速度：先分清它在哪个坐标系 . . . . .       | 19        |
| 4.7 限速、超时与圆弧安全 . . . . .           | 20        |
| 4.8 从合力得到加速度 . . . . .             | 21        |
| 4.9 从恒加速度到带阻尼的离散模型 . . . . .       | 22        |
| 4.10 惯性、超时和碰撞如何一起工作 . . . . .      | 23        |
| 4.11 实验：质量、步长和制动距离 . . . . .       | 24        |
| 4.12 连续理想参考：给后续传感器一条平滑轨迹 . . . . . | 24        |
| 4.13 运动中的初始条件与导数核对 . . . . .       | 25        |
| 4.14 把运动状态接到后续章节 . . . . .         | 26        |
| 4.15 用轨迹恢复状态与输入：微分平坦 . . . . .     | 27        |
| 4.16 把物理代价的梯度传回轨迹 . . . . .        | 28        |
| 4.17 有车轮方向约束的阿克曼模型 . . . . .       | 29        |
| 4.18 按一个时间步推进小车 . . . . .          | 30        |
| 4.19 从位置曲线恢复小车的运动 . . . . .        | 31        |
| 4.20 停车点如何保存方向 . . . . .           | 32        |
| 4.21 逐个标量计算小车的反向梯度 . . . . .       | 32        |
| 4.22 把模型选择落实到执行器 . . . . .         | 34        |
| <b>第五章 一束光到一圈扫描：CPU 激光雷达</b>       | <b>35</b> |
| 5.1 先让一束光碰到障碍 . . . . .            | 35        |
| 5.2 射线与圆：从圆方程到两个交点 . . . . .       | 35        |
| 5.3 射线与线段：用叉积消去一个未知数 . . . . .     | 37        |
| 5.4 把方向均匀排成一圈 . . . . .            | 38        |
| 5.5 量程、无效读数与采样时间 . . . . .         | 38        |
| 5.6 在 RViz2 同时观察光束和命中点 . . . . .   | 39        |
| 5.7 高斯测距噪声：先有回波，再加误差 . . . . .     | 40        |
| 5.8 从一批读数估计误差分布 . . . . .          | 41        |
| 5.9 从角间隔理解细小障碍的漏检 . . . . .        | 42        |
| 5.10 测量束数增加后的计算代价 . . . . .        | 43        |
| <b>第六章 机器人感觉到什么：IMU 与传感器时间</b>     | <b>45</b> |
| 6.1 静止的加速度计为什么不为零 . . . . .        | 45        |
| 6.2 用三行公式读懂理想 IMU . . . . .        | 46        |
| 6.2.1 为什么安装位置也影响读数 . . . . .       | 46        |
| 6.3 先手算，再运行五个场景 . . . . .          | 47        |

|                           |           |
|---------------------------|-----------|
| 6.4 让每个采样点带上一点白噪声         | 47        |
| 6.4.1 每采样标准差与连续噪声强度       | 48        |
| 6.5 读懂 IMU 消息与协方差         | 48        |
| 6.6 从数据观察噪声分布             | 49        |
| 6.7 零均值误差也会积成漂移           | 50        |
| 6.8 在 RViz 中同时观察激光与 IMU   | 52        |
| 6.9 采样时间不是回调到达时间          | 53        |
| 6.10 先接受运动区间, 再产生观测       | 53        |
| 6.11 动手观察不对齐的采样网格         | 54        |
| 6.12 阿克曼转弯时的 IMU 与采集时间    | 55        |
| <b>第七章 位姿已知时, 如何从观测建图</b> | <b>56</b> |
| 7.1 先给建图一个已知的位姿           | 56        |
| 7.2 同一条里程计消息里的两种坐标系       | 57        |
| 7.3 运行真值适配器并核对 TF         | 57        |
| 7.4 从距离到二维点云              | 58        |
| 7.5 把扫描与同一时刻的位姿配对         | 59        |
| 7.6 格子有面积, 未知也是一种状态       | 60        |
| 7.7 DDA: 沿射线依次走过哪些格子      | 61        |
| 7.7.1 跟着一条射线手算六个格子        | 61        |
| 7.7.2 回波落在地图外时怎么处理        | 62        |
| 7.8 把射线变成概率证据             | 62        |
| 7.8.1 从条件概率到几率相乘          | 62        |
| 7.8.2 取对数, 乘法就变成加法        | 63        |
| 7.8.3 让一帧扫描只贡献一次格子证据      | 63        |
| 7.9 有限回波、无回波和无效读数         | 64        |
| 7.10 在 RViz 中检查“看见了什么”    | 65        |
| 7.11 用几何参考评价观测地图          | 65        |
| 7.12 实验一: 分辨率、噪声与评价指标     | 67        |
| 7.13 实验二: 墙到底变厚了多少米       | 69        |
| 7.14 复现实验与本章练习            | 70        |
| <b>第八章 让两帧扫描对齐: 激光里程计</b> | <b>71</b> |
| 8.1 先对齐两组已知形状的点           | 71        |
| 8.2 把位姿的小变化写成一阶近似         | 72        |
| 8.3 从残差平方和推导位姿增量          | 72        |
| 8.3.1 一个可以手算的三元方程         | 73        |
| 8.3.2 代码中的 LDLT 在做什么      | 73        |

|                           |           |
|---------------------------|-----------|
| 8.4 用 Huber 权重减轻错误关联的影响   | 73        |
| 8.5 点到线：沿墙滑动和穿墙不同         | 74        |
| 8.6 从一小片点云求出墙的法向          | 74        |
| 8.6.1 特征向量从哪里来            | 75        |
| 8.6.2 把推导对应到邻域代码          | 75        |
| 8.7 判断几何约束是否足够            | 76        |
| 8.8 把成功扫描留在一个小窗口中         | 77        |
| 8.9 用估计里程计替换真值入口          | 78        |
| 8.10 实验：束数、噪声与被拒绝的帧       | 79        |
| <b>第九章 让 IMU 预测，让激光纠偏</b> | <b>81</b> |
| 9.1 从一个可以控制噪声的实验开始        | 81        |
| 9.2 八个数描述平面状态             | 81        |
| 9.3 把误差也向前传播一步            | 82        |
| 9.3.1 协方差记录误差的大小与关联       | 82        |
| 9.3.2 逐块求状态转移的偏导          | 82        |
| 9.3.3 从误差乘积得到协方差传播        | 83        |
| 9.4 从加权平均推导卡尔曼增益          | 84        |
| 9.4.1 先估计一个位置数值           | 84        |
| 9.4.2 让同一观测修正多个状态         | 84        |
| 9.4.3 位置观测如何修正速度：一次手算     | 85        |
| 9.4.4 校正后还剩多少不确定性         | 85        |
| 9.4.5 用不确定性衡量一次残差         | 85        |
| 9.5 偏置与一秒缺测实验             | 86        |
| 9.6 校正发生在采样时刻，不是收到消息时     | 88        |
| 9.7 高频预测与扫描位姿各走一个出口       | 89        |
| 9.8 真实扫描：转得更快、少扫半秒        | 90        |
| <b>第十章 回到起点之后：关键帧与位姿图</b> | <b>91</b> |
| 10.1 从一条链变成一张图            | 91        |
| 10.1.1 先手算只有三个位置的图        | 91        |
| 10.2 二维边残差：比较预测与测量        | 92        |
| 10.2.1 把米和弧度换成统一的误差尺度     | 92        |
| 10.3 固定第一帧，就是给地图选坐标原点     | 93        |
| 10.4 对一条边的两端分别求导          | 93        |
| 10.5 把许多小方程装成一个大方程        | 94        |
| 10.5.1 为什么有时只走半步          | 95        |
| 10.5.2 错误回环为何仍需先做几何检查     | 95        |

|                              |            |
|------------------------------|------------|
| 10.6 矩形实验与可独立检验的答案           | 96         |
| 10.7 从关键帧中寻找旧地点              | 97         |
| 10.8 回环更新地图坐标，不改写局部里程计       | 98         |
| 10.9 实际激光与 IMU 的矩形回放         | 99         |
| 10.10 从地图中观察回环的作用            | 100        |
| <b>第十一章 给圆盘找一条路：配置空间与 A*</b> | <b>101</b> |
| 11.1 先运行一张带窄门的地图             | 101        |
| 11.2 圆盘变成一个中心点               | 101        |
| 11.3 A* 的三个数与两种邻接            | 102        |
| 11.4 未知不是免费的捷径               | 103        |
| 11.5 删掉折点之前，检查整条线段           | 103        |
| 11.6 半径、邻接与窄门实验              | 104        |
| 11.7 从观测地图接入 ROS 与 RViz      | 105        |
| 11.8 复现一条来自真实消息的路径           | 106        |
| <b>第十二章 离障碍多远：二维距离场</b>      | <b>107</b> |
| 12.1 先看一个圆的距离场               | 107        |
| 12.2 先确定你计算的是哪一种距离           | 107        |
| 12.3 欧氏距离变换：先平方，后开方          | 107        |
| 12.4 一维最小值为什么可以快速求           | 108        |
| 12.5 连续位置的值与梯度               | 108        |
| 12.6 把距离与安全判定分开              | 109        |
| 12.7 核对距离、梯度与离散误差            | 109        |
| 12.8 分辨率实验与练习                | 110        |
| 12.9 ROS 距离通道与梯度显示           | 111        |
| 12.10 读一次真实距离消息              | 112        |
| <b>第十三章 让整段曲线有地方走：安全走廊</b>   | <b>113</b> |
| 13.1 先比较矩形与凸多边形              | 113        |
| 13.2 一个凸区域的两种表示              | 113        |
| 13.3 从局部矩形开始                 | 114        |
| 13.4 安全顶点不足以证明区域安全           | 114        |
| 13.5 只有通过验证才合并               | 114        |
| 13.6 交叠区域决定连接点               | 115        |
| 13.7 失败时不返回半成品               | 115        |
| 13.8 固定地图上的对照实验              | 116        |
| 13.9 同一观测快照上的 ROS 走廊         | 117        |
| 13.10 参数、观察与独立检查             | 117        |

|                                 |            |
|---------------------------------|------------|
| <b>第十四章 给路径加上时间：五次轨迹</b>        | <b>119</b> |
| 14.1 同样走一米，为什么感觉不同              | 119        |
| 14.2 六个系数对应六个边界条件               | 119        |
| 14.3 从公式到系数与采样                  | 120        |
| 14.4 最容易手算的静止到静止                | 121        |
| 14.5 连接处连续到哪一阶                  | 121        |
| 14.6 实验、保存与验收                   | 122        |
| 14.7 把轨迹作为完整消息发送                | 123        |
| 14.8 把 map 路线固定到 odom           | 123        |
| 14.9 理想执行也要检查整段运动               | 124        |
| 14.10 在 RViz 与实际消息中看执行          | 124        |
| 14.11 观测路线和非整步时间验收              | 125        |
| <b>第十五章 让路点之间更平顺：二维 MINCO</b>   | <b>126</b> |
| 15.1 相同路点、相同时长的比较               | 126        |
| 15.2 能量是系数的二次型                  | 127        |
| 15.3 为什么最优解还要连续 jerk 与 snap     | 127        |
| 15.4 将最优性写成一组线性方程               | 127        |
| 15.5 系数随路点和时长一起变化               | 128        |
| 15.6 为什么不用显式矩阵求逆                | 128        |
| 15.7 带状实现和数值尺度                  | 129        |
| 15.8 独立检查比看图更可靠                 | 129        |
| 15.9 把约束违反量变成软代价                | 130        |
| 15.10 始终保持时长为正                  | 130        |
| 15.11 收敛之后仍要检查                  | 131        |
| 15.12 在观测地图上预览优化结果              | 132        |
| 15.13 补充：以微分平坦映射约束实际力           | 133        |
| 15.14 时长梯度：不能漏掉 snap            | 134        |
| <b>第十六章 换一种消元顺序：二维 Spline2D</b> | <b>135</b> |
| 16.1 先固定问题，再比较实现                | 135        |
| 16.2 未知数改为连接点的速度与加速度            | 135        |
| 16.3 一个块接一个块消元                  | 136        |
| 16.4 能量梯度可以直接看边界                | 136        |
| 16.5 任意代价的伴随如何复用块分解             | 137        |
| 16.6 误差验收与本机计时                  | 137        |
| 16.7 不改曲线，只换成 Bézier 控制点        | 138        |
| 16.8 二分能缩紧界，不能改变事实              | 138        |

|                          |            |
|--------------------------|------------|
| 16.9 加入控制点代价，再独立认证       | 139        |
| 16.10 认证候选与实际执行分开        | 140        |
| 16.11 补充：力约束也要检查整段       | 141        |
| 16.12 阿克曼起停轨迹：分开几何与时间    | 142        |
| 16.13 从小车的物理限制反传到路径      | 143        |
| 16.14 先验证能转弯，再决定是否减速     | 144        |
| <b>第十七章 让有质量的机器人跟上轨迹</b> | <b>145</b> |
| 17.1 位置不是受力模型的控制量        | 145        |
| 17.2 独立 yaw 与执行器限幅       | 145        |
| 17.3 固定参考后，比较原因而非只看最终位置  | 146        |
| 17.4 参考要平滑起步，不能凭空跳出速度    | 147        |
| 17.5 验证、制动与练习            | 147        |
| 17.6 接入 ROS：时间与坐标也属于控制律  | 148        |
| 17.7 补充：阿克曼前馈与局部反馈       | 149        |
| 17.8 阿克曼换道实验与模型匹配        | 150        |
| <b>第十八章 线性 MPC 与约束跟踪</b> | <b>151</b> |
| 18.1 把连续模型积分成预测矩阵        | 151        |
| 18.2 跟踪、前馈和输入变化组成一个二次型   | 151        |
| 18.3 把约束按行装进矩阵           | 152        |
| 18.4 看得懂的 ADMM 求解器       | 152        |
| 18.5 失败状态与短时域的边界         | 153        |
| 18.6 同一八字参考上的实测对照        | 154        |
| 18.7 在 ROS 中观察每次预测与失败    | 155        |
| <b>第十九章 接通在线规划与完整导航</b>  | <b>156</b> |
| 19.1 重规划从未来交接时刻开始        | 156        |
| 19.2 预测未来，不等于提前切换        | 157        |
| 19.3 地图校正与执行参考分开         | 157        |
| 19.4 只在观测自由空间里选择局部目标     | 158        |
| 19.5 优化与备用方案都要通过认证       | 159        |
| 19.6 把轨迹与走廊作为一个事务接收      | 160        |
| 19.7 观测驱动的导航节点           | 161        |
| 19.8 同场景比较三条路线           | 162        |
| 19.9 读懂成绩，也保留边界          | 163        |
| 19.10 从一份配置选择微分平坦模型      | 164        |
| 19.11 从观测就绪到未来起步         | 165        |
| 19.12 选模实验、失败案例与练习       | 166        |

|                             |            |
|-----------------------------|------------|
| <b>第二十章 选学：CUDA 激光雷达加速</b>  | <b>167</b> |
| 20.1 先建立可复现的工具链 . . . . .   | 167        |
| 20.2 同一接口，不改变测量含义 . . . . . | 167        |
| 20.3 一束一个线程 . . . . .       | 168        |
| 20.4 完整扫描收益取决于规模 . . . . .  | 169        |
| 20.5 接入 ROS，再测一次 . . . . .  | 170        |

|                                  |            |
|----------------------------------|------------|
| <b>附录 A 数学回查与数值检查</b>            | <b>171</b> |
| A.1 坐标变换先写清方向 . . . . .          | 171        |
| A.2 最小二乘在解什么 . . . . .           | 171        |
| A.3 协方差与权重 . . . . .             | 172        |
| A.4 用差分检查, 用尺度理解误差 . . . . .     | 172        |
| A.5 凸集、二次型与课程接口 . . . . .        | 172        |
| <b>附录 B 差速扫地机器人: 先约束运动方向</b>     | <b>173</b> |
| B.1 圆盘外形相同, 能做的动作不同 . . . . .    | 173        |
| B.2 手算一次圆弧 . . . . .             | 173        |
| B.3 怎样接到轨迹与控制 . . . . .          | 174        |
| <b>附录 C 进阶感知: 一次只增加一种误差</b>      | <b>175</b> |
| C.1 白噪声不会自己变成偏置 . . . . .        | 175        |
| C.2 统计的是许多终点, 不是一条曲线 . . . . .   | 175        |
| C.3 进一步扩展时, 先写验收边界 . . . . .     | 176        |
| <b>附录 D 记录、回放与外部工具</b>           | <b>177</b> |
| D.1 传感器时间与文件时间是两件事 . . . . .     | 177        |
| D.2 谁来发布时钟与变换 . . . . .          | 177        |
| D.3 做一次真正的录制与回放验收 . . . . .      | 178        |
| D.4 与现有二维 SLAM 对照的接入练习 . . . . . | 178        |
| D.5 从教材跳到 API 文档 . . . . .       | 179        |
| D.6 让实验报告可以被重新运行 . . . . .       | 179        |

# 第一章 环境搭建与第一个圆盘场景

**本章目标** 编译课程工作空间，理解节点怎样发送图形消息，在 RViz2 看见半径为 0.20 m 的圆盘。

**先修** 会打开终端、编辑文本，读懂基本 C++ 变量和函数。

## 1.1 先认识四个名字

节点 (node) 是执行计算并收发消息的程序。话题 (topic) 是带名字和类型的数据通道，例如本章的 `/visualization/robot`。包 (package) 组织源码、依赖和安装规则。工作空间是存放和构建这些包的目录。例如本课程的 `ros2_ws` 中，`src` 存源码，`build` 存编译中间结果，`install` 存供 ROS 使用的程序和配置。

本章只有一个发布节点和一个可视化程序：



RViz2 将收到的几何消息绘制成图形；本章的位置和尺寸由发布节点给定，圆盘保持静止。

## 1.2 确认主机环境

本课程验证平台是 Ubuntu 26.04 和 ROS 2 Lyrical。已有主机不用重装。打开 `bash`，先运行：

```
source /opt/ros/lyrical/setup.bash
printenv ROS_DISTRO
ros2 pkg prefix rviz2
colcon --help
```

前三项应分别加载环境、输出 `lyrical`、找到 `RViz2`。最后一项显示构建工具的帮助信息。若某条命令找不到，先解决这一项，再进入构建步骤。

干净主机应先按 [ROS 官方安装步骤](#) 配置 UTF-8、Universe 与 `ros-apt-source`。已配好软件源的 Ubuntu 26.04 可安装 `ros-lyrical-desktop`、`ros-dev-tools`、`libeigen3-dev`。不要把其他 Ubuntu 版本与 Lyrical 的二进制包混用。

Lyrical 的 ROS 接口按 C++20 编译；本课程独立几何和数学代码采用 C++17。ROS 的 Python 包使用系统 Python，构建时显式指定 `/usr/bin/python3`。

## 1.3 构建并启动

以下路径是当前主机仓库位置；换机器后改成自己的路径。每个新终端都要先加载主机环境，再加载课程工作空间。

```
source /opt/ros/lyrical/setup.bash
cd /home/zdp/ForCodex/ros2_motion_planning_course/ros2_ws
colcon build --symlink-install --cmake-args \
  -DCMAKE_BUILD_TYPE=RelWithDebInfo \
  -DPython3_EXECUTABLE=/usr/bin/python3
source install/setup.bash
ros2 launch motion2d_bringup ch01.launch.py
```

motion2d 保存算法和节点；motion2d\_bringup 保存配置、launch 与 RViz2 视图。后面用到的自定义消息集中在 motion2d\_interfaces 包，本章先使用 ROS 已有的 Marker 类型。

**为什么要两次 source？** 第一条加载主机的 ROS 环境，让终端找到 ros2 和基础包。编译后的 install 目录是本课程新生成的一层环境；第二次 source 将这一层加入当前终端的查找路径。可以把它理解为告诉终端：“除了系统程序，还到这个课程目录找程序。”每个新终端都要完成这两步。

**构建与启动分别做什么？** colcon 根据各包的依赖关系编译 C++，把可运行结果放到 install。launch 再按照启动文件同时创建节点、加载 YAML 参数并打开 RViz2。修改 C++ 后重新 build；修改 YAML 后重启节点。--symlink-install 让安装目录中的配置链接到源码目录，方便编辑时保持两处一致。

RViz2 应显示浅色背景、0.5 m 网格和原点的青色圆盘，Fixed Frame 为 map。在只有终端的环境中，加 rviz:=false 启动节点，再用本章后面的 topic echo 查看同一条消息。

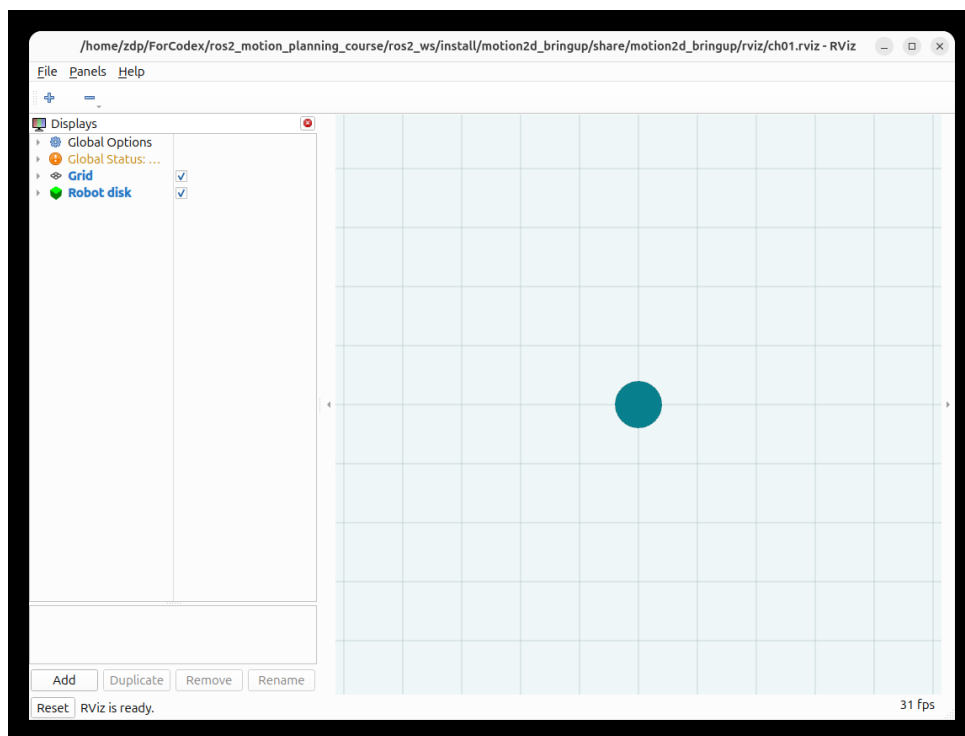


图 1.1: 主机实际运行画面。圆盘直径为 0.40 m，小于一个 0.5 m 网格。

此时 Global Status 可能提示尚无 TF 数据，因为本章还没有发布坐标树。Marker 已直接在 map 帧中，所以仍可正确显示；下一章建立 TF 后该提示消失。

## 1.4 圆盘为什么用直径？

用一个很薄的 CYLINDER 表示圆盘。Marker 的 scale 是物体的完整尺寸，因此

$$d_x = d_y = 2r, \quad r = 0.20 \text{ m} \Rightarrow d_x = d_y = 0.40 \text{ m}. \quad (1.1)$$

scale.z=0.05 只是显示厚度，后续平面碰撞只使用半径。圆柱的 position.z 是中心高度。取 0.025 m、厚度 0.05 m 时，下表面恰好落在 z=0。orientation.w=1 且其他分量为零表示保持原始方向；第二章再说明平面旋转的表示。alpha 控制透明度，0 完全透明，1 完全不透明。

在 hello\_scene\_node.cpp 中，构造消息的顺序是：指定坐标系和标识，设置几何，再设置颜色：

```

33 visualization_msgs::msg::Marker disk;
34 disk.header.frame_id = "map";
35 disk.header.stamp = now();
36 disk.ns = "robot";
37 disk.id = 0;
38 disk.type = visualization_msgs::msg::Marker::CYLINDER;
39 disk.action = visualization_msgs::msg::Marker::ADD;
40 disk.pose.position.x = x;
41 disk.pose.position.y = y;
42 disk.pose.position.z = 0.025;
43 disk.pose.orientation.w = 1.0;
44 disk.scale.x = 2.0 * radius;
45 disk.scale.y = 2.0 * radius;
46 disk.scale.z = 0.05;
47 disk.color.r = 0.03F;
48 disk.color.g = 0.50F;
49 disk.color.b = 0.55F;
50 disk.color.a = 1.0F;
51 //

```

**顺着消息读代码** frame\_id=map 表示 position 中的 x/y 是地图坐标；换坐标系后，同样的两个数字通常表示另一个位置。ns 与 id 共同标识这个图形，后续以同一标识发布会更新它。type 选择圆柱，action=ADD 让 RViz 创建或更新图形。最后 publish(disk) 把整个结构体作为一条消息发出，所以接收者能一次取得同一图形的位置、尺寸和颜色。

程序入口先调用 rclcpp::init 初始化通信，再构造 HelloScene。构造函数读取参数并发布圆盘；spin 让节点继续运行，保存发布者及其消息缓存。在终端按 Ctrl+C 后结束 spin，程序清理通信资源并退出。

## 1.5 发布一次，为什么晚启动也能看到？

QoS 是双方约定的传输方式。reliable 表示可靠传输；transient-local 让运行中的发布者保存最近一条消息。本节点和 RViz2 使用相同的持久性设置。先启动节点，过几秒再打开 RViz2，它就能取得缓存的圆盘。这种方式适合静态场景和配置；后面连续传感器数据会采用不同的队列与传输设置。

第二个终端加载环境，进入工作空间后运行：

```

ros2 topic echo /visualization/robot --once \
--qos-durability transient_local
/usr/bin/python3 ../scripts/check_ch01.py

```

检查程序确实作为晚加入的订阅者接收消息，并核对帧、类型与默认尺寸。此时消息 `stamp` 用系统时间；下一章再引入仿真时钟。

## 1.6 动手修改第一个场景

1. **理解：**节点与话题的区别是什么？发布者先启动、RViz2 后启动时，为什么仍能收到圆盘？
2. **代码：**完成 `exercises/ch01/starter/marker_scale.hpp` 中的 `EXERCISE(ch01-1)`。检查方法、分级提示和独立参考答案位于同目录。
3. **实验：**在 `ch01.yaml` 中把半径改成 0.35 m、x 改为 1.0 m，重启。先预测消息和网格上的尺寸，再观察；完成观察后恢复默认参数，便于后续章节使用相同起点。

应看到直径 0.70 m、圆心 (1,0) 的圆盘。若图形不见了，依次检查 Fixed Frame、话题、alpha 与 QoS。若找不到包，检查是否加载课程的 `install` 环境。

### 独立编译练习

练习的输入是半径，输出是 Marker 的 x/y 尺寸。下面的 `starter` 故意保留一个需要你修改的返回值；它没有被链接到正式演示中。

```
#pragma once

/** @brief Return Marker x/y size (m) for a disk of positive radius (m). */
inline double markerDiameter(double radius)
{
    // EXERCISE(ch01-1): RViz's cylinder scale describes its full width.
    return radius;
}
```

从仓库根目录运行：

```
g++ -std=c++17 -Wall -Wextra -Iexercises/ch01/starter \
    exercises/ch01/check.cpp -o /tmp/ch01_exercise
/tmp/ch01_exercise
```

运行后输出 `PASS` 表示两个半径均得到正确直径。若输出 `FAIL`，先读同目录 `hints` 的提示；将编译命令中的 `starter` 换成 `solutions` 可核对参考解。

## 第二章 二维坐标、时间与接口

**目标** 正确旋转、平移、求逆，并用一个仿真时钟驱动坐标树。

**先修** 第 1 章；二维向量和正弦、余弦。矩阵只需要理解本章的两行乘法。

### 2.1 一个点可以有不同坐标

机器人位于地图坐标  $(1, 2)$ ，朝向正北。雷达观测到正前方 2 m 的点。它在机体系是  $(2, 0)$ ，在地图系是  $(1, 4)$ 。坐标变了，物理上的点没有变。

我们采用右手系：机体  $x$  轴向前、 $y$  轴向左、 $z$  轴向上，逆时针偏航为正，角度使用弧度。一个平面位姿记作  $T_{A \leftarrow B} = (t, \psi)$ ：它把  $B$  系的坐标换成  $A$  系坐标。

#### 从两根坐标轴得到旋转矩阵

先想象只旋转一个向量，暂时把两个坐标系原点放在一起。沿  $B$  系  $x$  轴走 1 m，在  $A$  系中的水平、竖直分量分别为  $\cos \psi, \sin \psi$ 。 $B$  系  $y$  轴比  $x$  轴多转  $\pi/2$ ，其分量是  $-\sin \psi, \cos \psi$ 。任意点  $p_B = (x_B, y_B)$  都可看成“沿第一根轴走  $x_B$ ，再沿第二根轴走  $y_B$ ”，所以

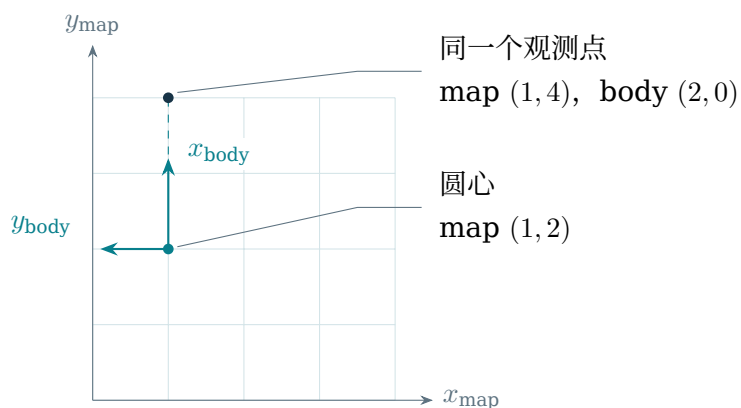
$$x_A - t_x = x_B \cos \psi - y_B \sin \psi, \quad (2.1)$$

$$y_A - t_y = x_B \sin \psi + y_B \cos \psi. \quad (2.2)$$

矩阵只是把这两行计算排在一起：第一列记录旋转后的  $x$  轴，第二列记录旋转后的  $y$  轴。向量默认写成一列，上标  $\top$  表示把行、列互换。

$$R(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi \\ \sin \psi & \cos \psi \end{bmatrix}, \quad p_A = R(\psi)p_B + t. \quad (2.3)$$

先旋转向量，再加上  $B$  原点在  $A$  中的位置  $t$ 。例子中  $\psi = \pi/2$ ， $R(2, 0)^\top = (0, 2)^\top$ ，再加  $(1, 2)^\top$  就得到  $(1, 4)^\top$ 。这个顺序对应两件不同的事：坐标轴改变方向，坐标原点改变位置。



## 2.2 从公式到一个小函数

Pose2D 只保存一个二维向量和一个角度。几何库不包含 ROS 头文件，可以单独用 C++17 编译、调试和测试。

```

13 Eigen::Matrix2d rotation(double yaw)
14 {
15     const double c = std::cos(yaw);
16     const double s = std::sin(yaw);
17     Eigen::Matrix2d result;
18     result << c, -s, s, c;
19     return result;
20 }
21
22 Eigen::Vector2d transformPoint(const Pose2D & pose, const Eigen::Vector2d & point)
23 {
24     return rotation(pose.yaw) * point + pose.position;
25 }
26 //

```

**代码中的每一步** Matrix2d 表示  $2 \times 2$  的数表；逗号初始化依次填写两行。transformPoint 中的矩阵乘法正是上面的两行标量式，pose.position 就是  $t$ 。可以先在纸上算  $\psi = 0$  和  $\psi = \pi/2$ ，再单步调试查看四个矩阵元素。函数所在的 se2 文件名表示平面刚体变换；本章所需运算就是旋转、平移与合成。

## 2.3 逆变换与变换链

从式 (2.3) 两侧减去  $t$ ，再左乘  $R^\top$ ：

$$p_B = R^\top(p_A - t) = R^\top p_A - R^\top t. \quad (2.4)$$

为什么左乘  $R^\top$  能消去  $R$ ？把矩阵直接相乘：

$$R^\top R = \begin{bmatrix} \cos^2 \psi + \sin^2 \psi & -\cos \psi \sin \psi + \sin \psi \cos \psi \\ -\sin \psi \cos \psi + \cos \psi \sin \psi & \sin^2 \psi + \cos^2 \psi \end{bmatrix} = I. \quad (2.5)$$

$I$  是对角线为 1、其余为 0 的单位矩阵, 乘向量后保持原值。因此逆旋转等于转置, 逆位姿为  $(-R^T t, -\psi)$ 。手算例子中, 先做  $(1, 4) - (1, 2) = (0, 2)$ , 再顺时针转  $90^\circ$ , 就回到  $(2, 0)$ 。

若雷达相对机体有固定外参  $T_{B \leftarrow L}$ , 机体相对地图为  $T_{M \leftarrow B}$ , 则

$$T_{M \leftarrow L} = T_{M \leftarrow B} T_{B \leftarrow L}. \quad (2.6)$$

这里的位姿乘号表示“接着做另一个变换”。写出中间点便能展开它:

$$p_B = R_{BL} p_L + t_{BL}, \quad (2.7)$$

$$p_M = R_{MB}(R_{BL} p_L + t_{BL}) + t_{MB} \quad (2.8)$$

$$= (R_{MB} R_{BL}) p_L + (R_{MB} t_{BL} + t_{MB}). \quad (2.9)$$

所以合成的平移为  $R_{MB} t_{BL} + t_{MB}$ , 朝向为  $\psi_{MB} + \psi_{BL}$ 。compose 正是先调用 transformPoint 得到这个平移, 再相加朝向。例如雷达在车体前方 0.5 m, 车体位于  $(1, 2)$  且朝北, 雷达在地图中就位于  $(1, 2.5)$ 。从右向左读下标, 便得到“雷达点  $\rightarrow$  机体点  $\rightarrow$  地图点”的计算顺序。课程默认传感器位于圆心, 外参为单位变换; 保留显式外参能让后续实验改变安装位置。

## 2.4 运行与理解数值误差

按第 1 章的方法重新构建、加载环境, 在工作空间运行:

```
ros2 run motion2d se2_demo
colcon test --packages-select motion2d \
  --event-handlers console_direct+
colcon test-result --verbose
```

示例输出机体点  $(2, 0)$ 、地图点  $(1, 4)$ , 再恢复到  $(2, 0)$ 。浮点运算中的  $\cos(\pi/2)$  可能是接近零的小数, 所以比较使用绝对误差容限  $10^{-12}$ , 不是要求每个计算结果逐位相同。

角度是另一个边界问题。 $179^\circ$  到  $-179^\circ$  的最短旋转只有  $2^\circ$ 。先将差值归一到  $[-\pi, \pi)$ , 再把它作为误差。函数 wrapAngle 使用弧度; 输入 90 表示 90 rad, 不是 90 度。

## 2.5 练习: 自己写一次逆变换

1. **推导:** 为何逆变换中不能只把  $t$  取负? 用  $\psi = 90^\circ, t = (1, 2)$  给一个反例。
2. **代码:** 补完 exercises/ch02/starter/inverse\_point.hpp 的标记函数。它复用正式库的 Pose2D, 检查包括两个旋转方向。
3. **实验:** 把示例朝向换成  $-90^\circ$ , 预测地图点, 再运行核对。还可以将激光外参设为机体前方 0.5 m, 手算其地图位置。

独立编译命令、分级提示和参考答案位于同章习题 README。用正向变换再逆变换的往返结果检查自己: 应回到原来的点, 差别只剩浮点舍入误差。

## 2.6 仿真时间与真实等待时间

假设每个仿真步长是  $\Delta t = 0.005$  s。第  $k$  步的仿真时间为

$$t_k = k\Delta t. \quad (2.10)$$

即使电脑偶尔调度得慢，状态仍然只前进一个固定步长。例如电脑用 0.008 s 才完成一次计算，模拟时刻仍从 0.005 s 增加到 0.010 s；真实等待时间决定程序跑得快慢，仿真时间决定方程积分了多久。

SimClock 用整数 tick 和整数纳秒保存时间，避免反复执行浮点加法。暂停时自动 advance 返回 false，单步只在暂停时允许执行：

```

14 bool SimClock::advance()
15 {
16     if (paused_) {
17         return false;
18     }
19     ++ticks_;
20     return true;
21 }
22
23 bool SimClock::singleStep()
24 {
25     if (!paused_) {
26         return false;
27     }
28     ++ticks_;
29     return true;
30 }
31 //

```

本章由 frame\_demo\_node 唯一发布 /clock；RViz2 和节点都设置 use\_sim\_time=true。暂停后重复发送当前时间，便于新订阅者获得它。判断是否发生新的一步，应比较时间值，而不是数消息。例如收到 0.010、0.010、0.015 s 三条消息，物理状态只从第 2 步推进到第 3 步。

## 2.7 把坐标放到 ROS 的 TF 树中

$$\text{map} \rightarrow \text{odom} \rightarrow \text{base\_link} \rightarrow \{\text{laser}, \text{imu\_link}\}.$$

本章 map 到 odom 为单位变换；传感器位于圆心、与机体系对齐，也是单位变换。这三条静态边通过 /tf\_static 发布。odom 到 base\_link 是带采样时刻的动态边，通过 /tf 发布。

为观察旋转效果，示例固定圆心 (1, 2)，采用解析朝向

$$\psi(t) = \frac{\pi}{2} + 0.2t. \quad (2.11)$$

角速度由求导得到  $\dot{\psi} = 0.2$  rad/s。圆心固定、坐标轴匀速转动，因此图中的蓝色箭头能清楚展示同一个 laser 系向量怎样在地图中转动。

## 消息里为什么出现半角？

ROS 用四个数表示空间朝向。本课程只有绕  $z$  轴的平面旋转，令  $q_x = q_y = 0$ 。接口的平面旋转关系为

$$R_q = \begin{bmatrix} 1 - 2q_z^2 & -2q_zq_w \\ 2q_zq_w & 1 - 2q_z^2 \end{bmatrix}, \quad q_z^2 + q_w^2 = 1. \quad (2.12)$$

用一个角  $\theta$  参数化单位圆： $q_z = \sin \theta, q_w = \cos \theta$ 。双角公式给出  $1 - 2\sin^2 \theta = \cos 2\theta$ 、 $2\sin \theta \cos \theta = \sin 2\theta$ ，因此  $R_q = R(2\theta)$ 。为了表示  $\text{yaw} = \psi$ ，应取  $\theta = \psi/2$ ：

$$(q_x, q_y, q_z, q_w) = (0, 0, \sin(\psi/2), \cos(\psi/2)). \quad (2.13)$$

例如  $90^\circ$  对应  $q_z = q_w = \sqrt{2}/2$ ；零角对应  $(0, 0, 0, 1)$ 。这也解释了第一章为什么把  $w$  设为 1：它满足单位长度并表示零旋转。发布时刻直接取仿真时钟，不能换成回调到达的真实时间：

```
126 geometry_msgs::msg::TransformStamped transform;
127 transform.header.stamp = rclcpp::Time(clock_.nanoseconds(), RCL_ROS_TIME);
128 transform.header.frame_id = "odom";
129 transform.child_frame_id = "base_link";
130 transform.transform.translation.x = x_;
131 transform.transform.translation.y = y_;
132 const double angle = wrapAngle(yaw_ + yaw_rate_ * clock_.seconds());
133 transform.transform.rotation.z = std::sin(angle / 2.0);
134 transform.transform.rotation.w = std::cos(angle / 2.0);
135 broadcaster_.sendTransform(transform);
136 //
```

## 2.8 暂停、单步与重置实验

停止第 1 章程序，重新构建并加载环境后启动：

```
ros2 launch motion2d_bringup ch02.launch.py
```

另一个终端同样加载环境后调用：

```
ros2 service call /sim/pause std_srvs/srv/SetBool "{data: true}"
ros2 service call /sim/step std_srvs/srv/Trigger "{}"
ros2 service call /sim/reset std_srvs/srv/Trigger "{}"
ros2 service call /sim/pause std_srvs/srv/SetBool "{data: false}"
```

暂停时圆盘朝向和箭头冻结；每次单步前进 5 ms；reset 回到时间零、初始朝向，并保持暂停。单步服务在暂停状态使用，方便逐次观察积分结果。

在工作空间运行 `/usr/bin/python3 ../scripts/check_ch02.py` 可检查唯一 TF 父帧、解析朝向、冻结时钟、5 ms 单步、reset 和恢复运行。它会改变演示状态，仅用于本章实验。

重置会使仿真时间向后跳，TF 监听器会清空缓存，这是新试验的正常边界。本节点每隔 1 s 墙钟时间重发静态外参，暂停时同时重发当前动态边，使监听器能在 reset 后重新获得坐标关系。后续估计器也会在这个时刻开始新的积分历史。

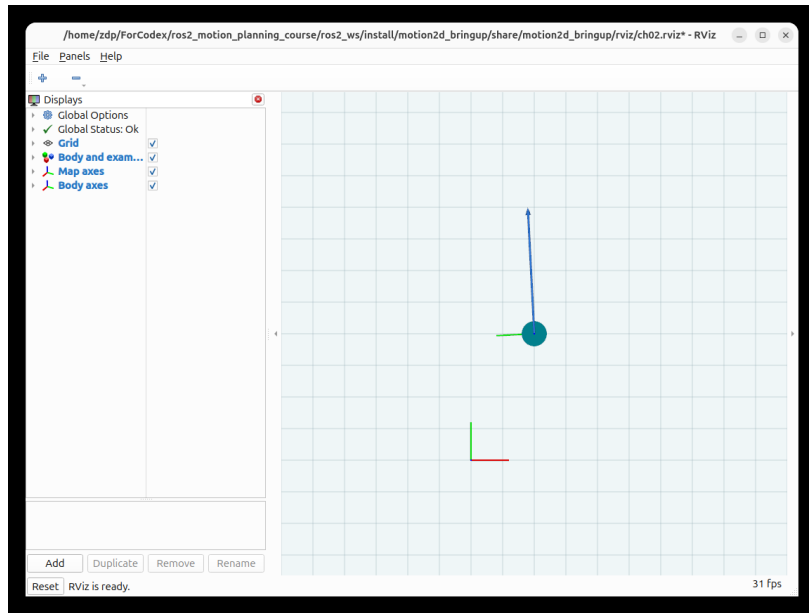


图 2.1: 圆心保持在 (1,2), 蓝色观测向量随机体坐标系转动; 地图坐标轴保持不动。

## 时间练习

先暂停, 再连续调用 `step` 十次, 预测仿真时间增量和朝向增量, 之后查看 `/clock` 和 TF 核对。标准答案是 0.05 s 与 0.01 rad。改变 YAML 中的 `dt` 后要重新推导; 不应继续照搬默认值的检查脚本。

## 第三章 一个可复现的二维世界

**目标** 生成随机圆与凸多边形，计算机器人到障碍的距离，并检查起终点是否连通。

**先修** 前两章、勾股定理和一元函数求导。本章从坐标运算引入点积与二维叉积。

### 3.1 先看世界，再谈算法

停止之前的演示，重新构建并加载工作空间后运行：

```
ros2 launch motion2d_bringup ch03.launch.py
```

RViz2 的 Fixed Frame 设为 map，World 话题选择 /visualization/world。默认世界宽、高都是 20 m，边界以原点为中心。seed=42 时生成 8 个圆和 8 个凸多边形；机器人在  $(-8, -8)$ ，目标在  $(8, 8)$ 。本章先搭好静态场景，第 4 章再让机器人运动。

世界节点保存完整障碍几何，用它计算碰撞和模拟传感器。第 5 章的激光将把这些几何变成有限数量的距离观测；之后的建图程序通过观测逐步认识环境。

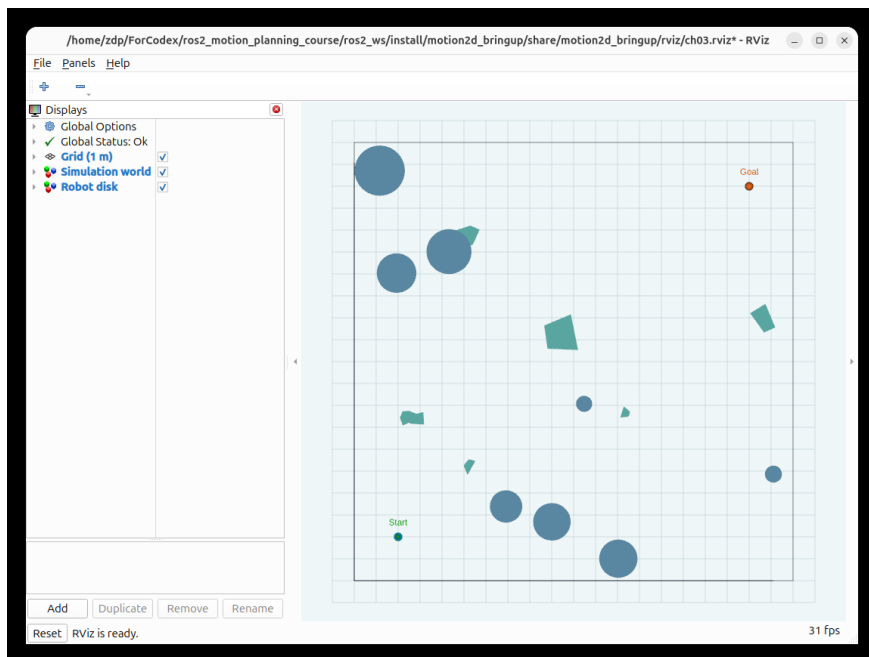


图 3.1: seed=42 的世界。Start 与 Goal 标出起终点；不同障碍可以重叠。

## 3.2 用距离描述圆盘与障碍

我们用位置向量  $p = (p_x, p_y)^\top$  表示圆心，单位为米。两点间距离由勾股定理给出：

$$\|p - c\| = \sqrt{(p_x - c_x)^2 + (p_y - c_y)^2}. \quad (3.1)$$

双竖线表示向量长度。后面常用的点积是两个对应分量相乘再相加： $u^\top v = u_x v_x + u_y v_y$ ，所以  $\|u\|^2 = u^\top u$ 。

设机器人半径为  $r$ ，圆形障碍中心为  $c$ 、半径为  $R$ 。沿圆心连线，两圆之间的空隙为  $\|p - c\| - R - r$ 。空隙大于零时分离，等于零时相切；课程把接触也计为碰撞。因此

$$\|p - c\| > R + r \iff \text{两圆无碰撞}. \quad (3.2)$$

把  $d(p) = \|p - c\| - R$  称为该障碍的**有符号距离**：障碍外为正、边界为零、内部为负。圆盘需要  $d(p) > r$ ；加入规划安全裕量  $\delta$  后，需要  $d(p) > r + \delta$ 。例如中心距 1.4 m、障碍半径 1 m 时，圆心净空为 0.4 m；半径 0.3 m 的机器人能放下，再留 0.2 m 裕量则空间不足。

宽  $W$ 、高  $H$  的矩形世界还要求圆心远离外墙。内部点到四面墙的最小距离为

$$d_{\text{wall}}(p) = \min(W/2 - |p_x|, H/2 - |p_y|). \quad (3.3)$$

这是因为左右墙距离分别为  $W/2 - p_x$ 、 $W/2 + p_x$ ，较小者为  $W/2 - |p_x|$ ；上下墙同理。`clearance` 从外墙、所有圆和多边形中取最小值：

```
11 double clearance(const World2D & world, const Eigen::Vector2d & center)
12 {
13     double distance = std::min(
14         world.width / 2.0 - std::abs(center.x()),
15         world.height / 2.0 - std::abs(center.y()));
16     for (const auto & circle : world.circles) {
17         distance = std::min(distance, signedDistance(circle, center));
18     }
19     for (const auto & polygon : world.polygons) {
20         distance = std::min(distance, signedDistance(polygon, center));
21     }
22     return distance;
23 }
24
25 bool isFree(const World2D & world, const Eigen::Vector2d & center, double radius)
26 {
27     return clearance(world, center) > radius;
28 }
29 //
```

`isFree` 的一个比较就完成了圆盘检查。在自由区域，返回值表示圆心到最近障碍或外墙的距离；负值表示圆心已进入某个障碍或越出世界。第 12 章会将距离存成栅格，供规划反复查询。

## 3.3 从最小化距离推导线段投影

多边形的每条边都是有限线段。设端点为  $a$ 、 $b$ ，记  $e = b - a$ ，线段上的点可以写成  $q(t) = a + te$ ，其中  $0 \leq t \leq 1$ 。先在整条直线上找离  $p$  最近的点。平方根单调递增，最小化距离与最小化距离平方会

得到同一个  $t$ ；后者更方便求导：

$$f(t) = \|p - a - te\|^2 \quad (3.4)$$

$$= \|p - a\|^2 - 2t(p - a)^\top e + t^2\|e\|^2, \quad (3.5)$$

$$f'(t) = -2(p - a)^\top e + 2t\|e\|^2. \quad (3.6)$$

当  $a \neq b$ ,  $f''(t) = 2\|e\|^2 > 0$ , 函数开口向上。令一阶导数为零得到

$$t^* = \frac{(p - a)^\top e}{\|e\|^2}. \quad (3.7)$$

若  $t^* < 0$ ,  $f$  在  $[0, 1]$  内递增, 最小值位于 0；若  $t^* > 1$ , 最小值位于 1；其余情况直接用  $t^*$ 。这三种情况合成一个截断运算：

$$t = \min(1, \max(0, t^*)), \quad q = a + te, \quad d = \|p - q\|. \quad (3.8)$$

例如  $a = (0, 0), b = (2, 0)$ 。对于  $p = (1, 1)$ ,  $t^* = 2/4 = 1/2$ , 最近点为  $(1, 0)$ , 距离 1；对于  $p = (3, 1)$ ,  $t^* = 6/4 = 1.5$ , 截断到 1 后最近点为  $b$ , 距离  $\sqrt{2}$ 。

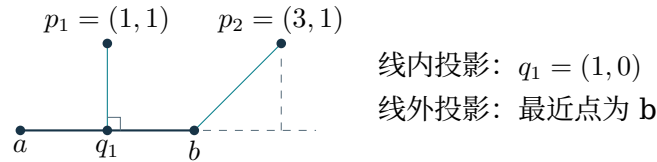


图 3.2: 最近点必须在线段上。第二个点的垂足落在延长线上, 端点  $b$  才是允许的最近点。

```

17 double pointSegmentDistance(
18     const Eigen::Vector2d & point, const Eigen::Vector2d & a, const Eigen::Vector2d & b)
19 {
20     const Eigen::Vector2d edge = b - a;
21     if (edge.squaredNorm() == 0.0) {
22         return (point - a).norm();
23     }
24     const double t = std::clamp((point - a).dot(edge) / edge.squaredNorm(), 0.0, 1.0);
25     return (point - (a + t * edge)).norm();
26 }
27 //

```

代码的 `dot` 对应点积, `squaredNorm` 对应长度平方, `clamp` 实现截断。当边长为零时, 线段只剩端点  $a$ , 直接计算  $\|p - a\|$ 。这样每一种几何情形都对应一个清楚的分支。

### 3.4 用拐弯方向判断内外与凸包

求出到所有边的最小距离后, 还需判断正负。对逆时针排列的凸多边形, 内部点在每条边的左侧。怎样用坐标判断“左侧”? 设两个非零向量的方向角分别为  $\alpha, \beta$ , 则

$$\text{cross}(u, v) = u_x v_y - u_y v_x \quad (3.9)$$

$$= \|u\| \|v\| (\cos \alpha \sin \beta - \sin \alpha \cos \beta) \quad (3.10)$$

$$= \|u\| \|v\| \sin(\beta - \alpha). \quad (3.11)$$

这个数称为二维叉积；它等于有向平行四边形面积。从  $u$  转向  $v$  时，小于半圈的逆时针转动给出正值，顺时针给出负值，共线时为零。对边  $a \rightarrow b$ ，代入  $u = b - a, v = p - a$ ，正值就表示  $p$  在左侧。`signedDistance(Polygon, ...)` 逐边计算距离并检查左侧条件；全部满足时，把最小距离取负。

生成多边形时，先在一个圆盘内抽取若干点，再求它们的凸包。可以把凸包想成包住全部点并拉紧的橡皮筋。`convexHull` 按  $x$  从小到大、同  $x$  时按  $y$  排序，先搭下边界，再反向搭上边界。每次添加新点  $c$ ，检查链末尾的  $a$ 、 $b$ 、 $c$  是否左转；若  $\text{cross}(b - a, c - b) \leq 0$ ，就删掉中间的  $b$ ，再重新检查。

例如依次处理  $(0, 0), (1, 1), (2, 0)$ ，叉积为  $1 \times (-1) - 1 \times 1 = -2$ ，中间点会被移出下边界：它在连接两端的线段上方，应该留给上边界。上下两条链各去掉一个重复端点后合并，就得到逆时针顶点序列。`polygon_samples=8` 指抽样点数；落在凸包内部的点会被舍去，所以最终边数通常更少。

### 3.5 怎样在圆盘面积上均匀采样

令  $u, v$  独立均匀取自  $[0, 1)$ 。“均匀”表示落入长度为  $\ell$  的区间的概率就是  $\ell$ 。角度覆盖一周，因此取  $\theta = 2\pi u$ 。半径的分布需要考虑圆环面积：半径越大，同样厚度的圆环面积越大。面积均匀要求半径不超过  $\rho$  的概率等于面积比例：

$$P(\text{采样半径} \leq \rho) = \frac{\pi \rho^2}{\pi R^2} = \frac{\rho^2}{R^2}. \quad (3.12)$$

把这个比例设为均匀变量  $v$ ，再解出半径，就得到

$$\rho = R\sqrt{v}, \quad p = c + \rho(\cos \theta, \sin \theta). \quad (3.13)$$

例如半径小于  $R/2$  的内圆只占总面积的  $1/4$ ，应接收约  $1/4$  的样本；上式恰好在  $v \leq 1/4$  时进入内圆。

生成器还检查候选形状是否有效、是否越界、是否侵入起终点保护圆。重叠障碍按并集处理。每个形状最多尝试 1000 次；空间太拥挤时会报告生成失败，此时可减小数量或尺寸，再用原种子比较变化。同一参数、随机种子和标准库环境可复现同一个世界，因此实验时应保存配置和软件环境。

### 3.6 让一个自由网格容纳整个圆盘

连接起终点前，先把世界分成小矩形格。若仅检查格中心，机器人移到角落时仍可能碰墙。设格中心为  $c$ 、宽高为  $\Delta x, \Delta y$ ，由勾股定理，格内任意点  $p$  都满足

$$\|p - c\| \leq \epsilon_{\text{cell}} = \frac{1}{2} \sqrt{\Delta x^2 + \Delta y^2}. \quad (3.14)$$

设  $o$  是任意障碍上的点。三角不等式给出  $\|c - o\| \leq \|c - p\| + \|p - o\|$ ，移项可得  $\|p - o\| \geq \|c - o\| - \|p - c\|$ 。再对障碍上的所有  $o$  取最小值，便有

$$d(p) \geq d(c) - \|p - c\| \geq d(c) - \epsilon_{\text{cell}}. \quad (3.15)$$

因此，只要中心净空满足

$$d(c) > r + \delta + \epsilon_{\text{cell}}, \quad (3.16)$$

整个格内的圆心位置都会满足  $d(p) > r + \delta$ 。代码用 `padded_radius` 保存这个额外膨胀后的半径。

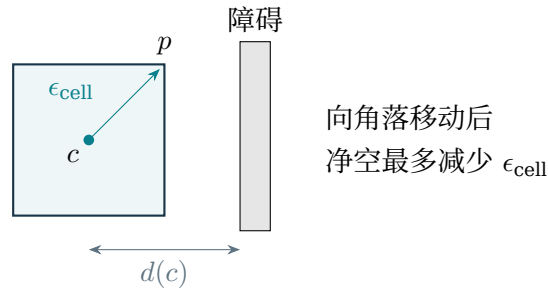


图 3.3: 格中心的距离需要覆盖格内位移。方格边长 0.25 m 时, 额外留出的距离约为 0.177 m。

### 3.7 从自由网格中找出连通区域

`isReachable` 使用四邻接广度优先搜索, 也称 `flood fill`。先把起点格放入队列并标记已访问; 每次取出队首, 把它上、下、左、右尚未访问的自由格依次加入队尾。像水逐格漫开一样, 访问集合逐步覆盖起点所在的连通区域。如果取出了终点格, 便找到连通关系; 若队列先变空, 终点在这个网格中不可达。每个格最多入队一次, 所以过程会结束。

相邻自由格共享一条边, 圆心可以从一个格连续移动到另一个格; 上一节的距离条件保护了整段移动。这种检查为随机世界筛选提供起点, 第 11 章会用 A\* 输出具体路径。默认网格分辨率为 0.25 m。较大的格需要更大的额外净空, 可能把真实窄通道封住; 减小分辨率后, 额外净空随之减小, 代价是格数增加。也可关闭 `require_connected`, 专门观察不连通场景。

### 3.8 动手修改与练习

编辑 `ros2_ws/src/motion2d_bringup/config/ch03.yaml` 后重启。公共 `x/y/radius` 同时供给坐标节点和世界节点, 让显示圆盘与碰撞几何使用同一位置、半径。

1. **手算:** 对  $a = (0, 0)$ ,  $b = (2, 0)$ , 计算  $p = (-1, 1)$  的最近点和距离; 用一阶导数解释为什么选择端点  $a$ 。
2. **代码:** 补全 `exercises/ch03/starter/segment_distance.hpp` 的 EXERCISE(ch03-1)。依次检查垂足在线内、在线外和线段退化三种情况。
3. **推导:** 把格宽和格高都减半, 额外膨胀量变成多少? 同样大小的世界, 格数变成多少?
4. **实验:** 用 `seed=42` 启动两次比较地图, 再改为 43; 随后增加障碍数量, 比较连通性随分辨率的变化。保存参数及失败场景, 检查瓶颈处的通道宽度。

代码练习从仓库根目录独立编译:

```
g++ -std=c++17 -I/usr/include/eigen3 \
-Iexercises/ch03/starter \
exercises/ch03/check.cpp -o /tmp/ch03_exercise
/tmp/ch03_exercise
```

提示和参考解在同章练习目录。将 `starter` 替换为 `solutions` 可运行参考解。使用 `colcon test` 检查正式库; 演示运行时, 在工作空间执行 `/usr/bin/python3 ../scripts/check_ch03.py` 核对默认世界消息。

## 第四章 让圆盘运动：位置、速度与力

**目标** 区分位置、速度和力输入；理解质量、阻尼、碰撞与制动。

**先修** 前三章、一元积分与链式法则。先由位置和速度驱动圆盘，再加入力与质量。

### 4.1 圆心状态与圆盘外形

机器人用圆心描述状态，圆盘描述占据的空间：

$$x = (p, v, \psi, \omega), \quad p = (p_x, p_y), \quad v = (v_x, v_y). \quad (4.1)$$

位置单位 m，速度 m/s，yaw 为 rad，角速度 rad/s。位置和速度在 odom 系表达；本章的 map 与 odom 重合。半径 r 描述碰撞外形，状态向量描述圆心运动。取  $r=0$  就得到几何质点；屏幕仍画一个小标记，方便观察位置。

控制量不同，模型的物理意义就不同。理想到点输入目标位姿；速度模型输入平面速度；惯性模型输入力和偏航力矩。它们分别使用不同话题，消息单位也随控制量而改变。下面从最容易运行的理想到点出发，再运行速度模型与受力惯性模型。

### 4.2 先让机器人到一个点

停止之前的场景，重新构建并加载 install 后运行：

```
ros2 launch motion2d_bringup ch04.launch.py model:=ideal
```

在 RViz2 用 2D Goal Pose 工具点击、拖动，指定目标位置和 yaw。工具向 /command/pose 发布消息。也可在第二个已加载 ROS 环境的终端发送：

```
ros2 topic pub --once /command/pose geometry_msgs/msg/PoseStamped \
  "{header: {frame_id: odom}, pose: {position: {x: -7.0, y: -8.0}, orientation: {w: 1.0}}}"
```

默认从  $(-8, -8)$  移到  $(-7, -8)$ ，图中留下接受过的运动轨迹。目标在下一 tick 才执行；暂停时发目标，圆盘保持不动，单步后才到达目标。

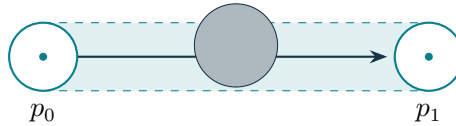
## 4.3 扫过的空间也要安全

起终点安全并不足够。半径  $r$  的圆盘沿线段  $[p_0, p_1]$  移动，扫过的区域是胶囊形。对于圆心  $c$ 、半径  $R$  的障碍，无碰撞要求

$$\text{dist}(c, [p_0, p_1]) > R + r. \quad (4.2)$$

这里直接复用第 03 章的点线段距离。对凸多边形，检查起终点和运动线段到每条边的距离；对矩形边界，内缩后的矩形是凸集，检查两个端点即可保证整条直线留在其中。

端点自由，整段被挡住



```

32 bool sweptDiskIsFree(const World2D & world, const Eigen::Vector2d & from,
33   const Eigen::Vector2d & to, double radius)
34 {
35   // The inset rectangle is convex, so valid endpoints imply a valid segment.
36   if (!isFree(world, from, radius) || !isFree(world, to, radius)) {return false;}
37   for (const auto & circle : world.circles) {
38     if (pointSegmentDistance(circle.center, from, to) <= radius + circle.radius) {
39       return false;
40     }
41   }
42   for (const auto & polygon : world.polygons) {
43     const auto & v = polygon.vertices;
44     for (std::size_t i = 0; i < v.size(); ++i) {
45       if (segmentDistance(from, to, v[i], v[(i + 1) % v.size()]) <= radius) {
46         return false;
47       }
48     }
49   }
50   return true;
51 }
52 //

```

循环中的每次检查都覆盖整个运动线段。圆障碍复用点线段距离；多边形先判断线段是否穿过边界，相交时距离为零，否则取两条线段端点到对方线段距离的最小值。相交判断复用叉积的左右侧符号：两条线段都跨在对方直线两侧且坐标包围盒重叠时相交；包围盒的检查同时排除共线却分离的两段。矩形边界内缩  $r$  后，任意两点的插值  $(1 - \alpha)p_0 + \alpha p_1$  ( $0 \leq \alpha \leq 1$ ) 仍满足四个坐标范围，因此边界检查可以只看两个端点。

## 4.4 直接设置位姿的用途

通过几何检查后，理想模型直接赋值目标位姿：

```

6 State2D idealPose(const Pose2D & target)
7 {
8     State2D result;
9     result.pose = target;
10    result.pose.yaw = wrapAngle(target.yaw);
11    return result; // Zero derivatives mean "unused", not an IMU measurement.
12 }
13 //

```

这个操作适合单独学习几何规划：每次设定一个新的安全位姿。位置在相邻 tick 间直接跳变，跳变时刻的导数无法用有限速度、加速度表示，因此状态中的零导数是占位值。后面的传感器实验会采用连续圆参考或惯性模型，从连续运动方程中取得加速度。

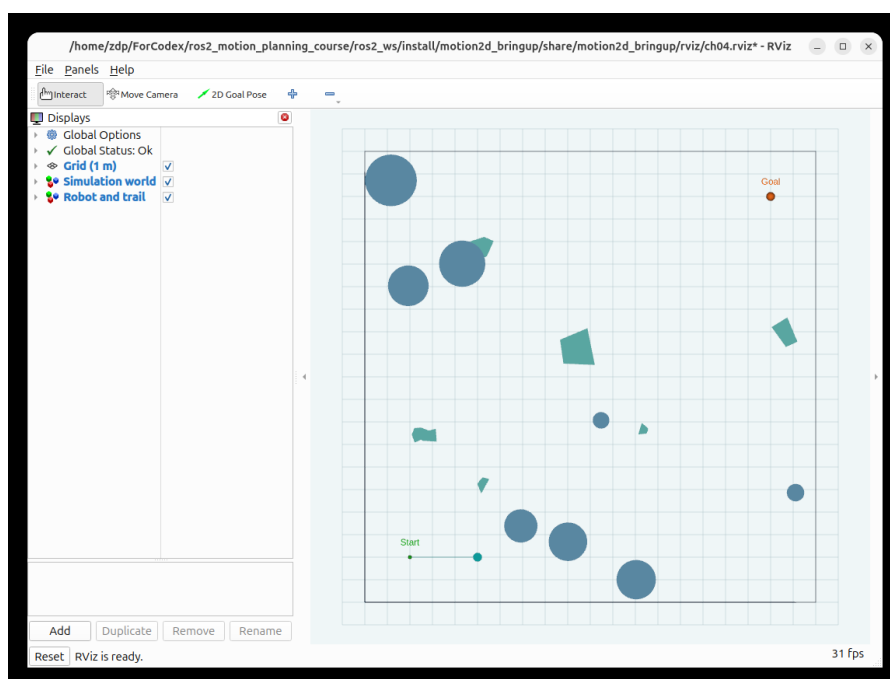


图 4.1: 实际 RViz2 画面：世界、圆盘与已接受的移动轨迹。

## 4.5 冻结试验与重新开始

扫描检查失败时，程序不接受候选状态，位置、速度和时间均保持在上一个安全的 tick。此时圆盘变红，`/sim/status` 显示 `collision_predicted`。冻结保留了导致冲突前的状态，便于检查路径和命令。调用 `reset` 后开始新的运动试验。

```

ros2 service call /sim/reset std_srvs/srv/Trigger "{}"
ros2 service call /sim/step std_srvs/srv/Trigger "{}"
ros2 service call /sim/pause std_srvs/srv/SetBool "{data: false}"

```

`reset` 开始新试验：回初始状态、清空待执行命令与轨迹、仿真时间归零并暂停。单步推进一个 `dt`；恢复运行不改变 `dt`。墙钟定时器决定运行快慢，整数 tick 决定模拟时间。

## 4.6 给速度：先分清它在哪个坐标系

重启场景，选用速度模型：

```
ros2 launch motion2d_bringup ch04.launch.py model:=velocity
```

此时只订阅 `/command/velocity`，RViz2 的位姿目标不参与驱动。在另一个已加载环境的终端连续发送机体速度：

```
ros2 topic pub -r 20 /command/velocity geometry_msgs/msg/TwistStamped \
  "{header: {frame_id: base_link}, twist: {linear: {x: 0.5}, angular: {z: 0.3}}}"
```

odom 系速度方向固定；base\_link 系速度会随机器人转动。同一个  $(0.5, 0)$ ，前者表示一直向 odom 的 x 正方向走，后者表示一直朝圆盘的“前方”走。全向模型允许沿机体 y 轴横移；差速车和本章后面的小车则沿车头方向行驶。

### 积分固定方向的速度

若一个时间步  $h$  内保持 odom 速度  $u$  与角速度  $\omega$ ，分别积分  $\dot{p} = u$ 、 $\dot{\psi} = \omega$ ：

$$p_{k+1} = p_k + \int_0^h u dt = p_k + hu, \quad \psi_{k+1} = \text{wrap}(\psi_k + h\omega). \quad (4.3)$$

wrap 将角度放回  $[-\pi, \pi)$ ，表示的朝向保持相同。

### 积分随车头旋转的速度

若  $u$  在机体系固定，地图中的速度为  $R(\psi_k + \omega t)u_0$ 。先假设  $u = (U, 0)^\top$ ，则  $\dot{p}_x = U \cos(\psi_k + \omega t)$ 、 $\dot{p}_y = U \sin(\psi_k + \omega t)$ 。当  $\omega \neq 0$  时直接积分：

$$\Delta p_x = \frac{U}{\omega} [\sin(\psi_k + \omega h) - \sin \psi_k], \quad (4.4)$$

$$\Delta p_y = \frac{U}{\omega} [\cos \psi_k - \cos(\psi_k + \omega h)]. \quad (4.5)$$

设  $\beta = \omega h/2$ ，用和差化积公式，两个方括号分别变为  $2 \sin \beta \cos(\psi_k + \beta)$  和  $2 \sin \beta \sin(\psi_k + \beta)$ 。因此位移沿步中间的车头方向，长度为  $Uh \sin \beta / \beta$ 。对机体 y 轴速度作同样积分，只需把方向转过  $\pi/2$ ；两轴相加得到

$$p_{k+1} = p_k + h \text{sinc}(\beta) R(\psi_k + \beta) u, \quad \text{sinc}(\beta) = \frac{\sin \beta}{\beta}. \quad (4.6)$$

在零处用极限  $\text{sinc}(0) = 1$ ，运动自然变成直线。由  $\sin \beta = \beta - \beta^3/6 + \beta^5/120 + \dots$ ，小角度时可计算  $1 - \beta^2/6$ ，避开很小的除数。例如  $U = 1 \text{ m/s}$ 、 $\omega = 1 \text{ rad/s}$ 、 $\psi_k = 0$ 、 $h = \pi/2 \text{ s}$ ，积分得到位移  $(1, 1) \text{ m}$ ，正是半径  $1 \text{ m}$  的四分之一圆。

```

16 State2D stepVelocity(const State2D & state, const VelocityCommand & command, double dt)
17 {
18     State2D next = state;
19     const double half_turn = 0.5 * command.yaw_rate * dt;
20     next.pose.yaw = wrapAngle(state.pose.yaw + 2.0 * half_turn);
21     next.yaw_rate = command.yaw_rate;
22     next.yaw_acceleration = 0.0; // Only within the held-command interval.
23     next.acceleration.setZero();
24     if (command.body_frame) {
25         const double sinc = std::abs(half_turn) < 1e-6 ?
26             1.0 - half_turn * half_turn / 6.0 : std::sin(half_turn) / half_turn;
27         next.pose.position += dt * sinc * rotation(state.pose.yaw + half_turn) * command.velocity;
28         next.velocity = rotation(next.pose.yaw) * command.velocity;
29         next.acceleration = command.yaw_rate * Eigen::Vector2d{-next.velocity.y(), next.velocity.x()};
30     } else {
31         next.pose.position += dt * command.velocity;
32         next.velocity = command.velocity;
33     }
34     return next;
35 }
36 //

```

## 4.7 限速、超时与圆弧安全

默认平移速度模长限制为 1 m/s，角速度绝对值限制为 1 rad/s。超出平移限制时，按  $u_{\text{applied}} = u v_{\text{max}} / \|u\|$  等比例缩短向量，保留方向。例如 (1, 1) m/s 的模长为  $\sqrt{2}$  m/s，限制为 1 m/s 后得到  $(1/\sqrt{2}, 1/\sqrt{2})$ 。配置项是 `speed_max` 和 `yaw_rate_max`。

命令默认在模拟时间中有效 0.5 s，到期后在 tick 边界切为零速度，状态为 `command_timeout`。暂停不会消耗有效期。零时间戳从接收时刻起算，非零时间戳从消息时刻起算。改变 dt 时，过期检测最多量化到一个 tick。速度切换时存在瞬时跳变；需要用 IMU 积分连续速度的实验，将采用下面的惯性模型。

### 把圆弧包进可检查的胶囊

圆弧的中间部分会偏离起终点连线。记弦的插值为  $p_{\text{chord}}(t) = p(0) + t[p(h) - p(0)]/h$ ，偏差为  $e(t) = p(t) - p_{\text{chord}}(t)$ 。设步内  $\|a(t)\| \leq a_{\text{max}}$ 。先把加速度积分两次：

$$p(t) = p(0) + tv(0) + \int_0^t (t-s)a(s) ds. \quad (4.7)$$

积分核  $t-s$  的来源是： $s$  时刻产生的速度增量还会作用  $t-s$  时间。把同一个式子分别代入  $p(t)$  和  $p(h)$ ，常数、初速度项相消：

$$e(t) = \int_0^t (t-s)a(s) ds - \frac{t}{h} \int_0^h (h-s)a(s) ds \quad (4.8)$$

$$= -\left(1 - \frac{t}{h}\right) \int_0^t sa(s) ds - \frac{t}{h} \int_t^h (h-s)a(s) ds. \quad (4.9)$$

对两段积分用三角不等式，再用加速度上界替换积分中的模长：

$$\|e(t)\| \leq a_{\max} \left[ \left(1 - \frac{t}{h}\right) \frac{t^2}{2} + \frac{t}{h} \frac{(h-t)^2}{2} \right] \quad (4.10)$$

$$= \frac{a_{\max}}{2} t(h-t) \leq \frac{a_{\max} h^2}{8}. \quad (4.11)$$

最后一步来自  $t(h-t) = h^2/4 - (t-h/2)^2$ 。因此用半径  $r + a_{\max} h^2/8$  检查弦，就能覆盖半径  $r$  的圆盘沿曲线扫过的空间。

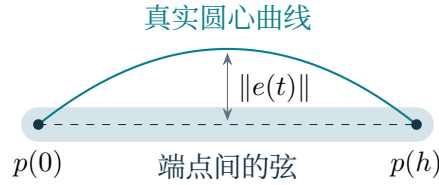


图 4.2: 偏差界随步长平方缩小。检查时，把机器人半径再增加这个偏差上界。

机体恒速度的加速度为  $\dot{R}(\psi)u = \omega R(\psi)Ju$ ，其中  $J = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$  把向量逆时针转  $90^\circ$ 。旋转保持长度，所以  $a_{\max} = |\omega|||u||$ ；odom 恒速度时为零。这个包络可能比真实扫掠区域稍宽；缩短  $h$  可减少狭窄处的保守余量。

**代码练习 ch04-2:** 补完 `exercises/ch04/starter/velocity_step.hpp` 的 odom 系速度积分，并归一化 yaw。用 `check_velocity.cpp` 检查负速度、时间步长和跨越  $\pi$  的角度。编译时沿用本章习题 README 的命令。

**实验:** 在转向后分别发送 odom 与 base\_link 的同一速度，比较轨迹；停止发送，验证超时；暂停两秒再单步，解释为什么暂停不消耗模拟时间。运行下面的实际通信检查：

```
/usr/bin/python3 ../scripts/check_ch04.py --model velocity
```

## 4.8 从合力得到加速度

牛顿第二定律把合力与加速度联系起来。设执行器施加  $F$ ，运动阻力与速度成正比、方向相反，阻力就是  $-c_v v$ 。合力等于  $m\dot{v}$ ；偏航方向则用力矩与转动惯量建立相同关系：

$$\dot{p} = v, \quad m\dot{v} = F - c_v v, \quad (4.12)$$

$$\dot{\psi} = \omega, \quad I_z \dot{\omega} = \tau - c_\omega \omega. \quad (4.13)$$

$F$  为 odom 系平面力，单位 N； $\tau$  为偏航力矩，单位 N·m。 $m$  是质量， $I_z$  是转动惯量。 $c_v, c_\omega$  分别为平移和转动阻尼，从  $c_v v$  必须具有力的单位可得， $c_v$  的单位是 kg/s； $c_\omega$  的单位是 kg·m<sup>2</sup>/s。

全向模型把  $x$ 、 $y$  两轴的受力分开，便于从单轴制动逐步学到平面控制和第 18 章的线性 MPC。本章末尾再加入车轮方向约束，研究阿克曼转向；差速驱动见附录 B。

重启场景后，在第二个终端持续施加力与力矩：

```
ros2 launch motion2d_bringup ch04.launch.py model:=inertial
# second terminal
ros2 topic pub -r 20 /command/wrench geometry_msgs/msg/WrenchStamped \
"{header: {frame_id: odom}, wrench: {force: {x: 1.0}, torque: {z: 0.02}}}"
```

本模式只订阅 `/command/wrench`，只接受 `odom` 系的平面分量。力始终指向 `odom` 的 `x` 正方向，圆盘转向也不改变这个力的方向。停发消息后，默认 0.5 s 超时，撤去的是力，不是速度。

| 配置项                       | 默认值  | 单位与意义                    |
|---------------------------|------|--------------------------|
| <code>mass</code>         | 1.0  | kg, 必须为正                 |
| <code>inertia_z</code>    | -1.0 | 自动取 $mr^2/2$ ; 可显式给正惯量   |
| <code>linear_drag</code>  | 0.0  | kg/s, 非负                 |
| <code>angular_drag</code> | 0.0  | kg·m <sup>2</sup> /s, 非负 |
| <code>force_max</code>    | 2.0  | N, 每个平面轴分别限幅             |
| <code>torque_max</code>   | 0.2  | N·m, 偏航力矩绝对值上限           |

自动惯量来自均匀圆盘。半径  $\rho$ 、厚度  $d\rho$  的圆环面积为  $2\pi\rho d\rho$ ，面密度为  $m/(\pi r^2)$ ，所以圆环质量为  $dm = 2m\rho d\rho/r^2$ 。转动惯量定义为每份质量乘到转轴距离的平方，再求和：

$$I_z = \int_0^r \rho^2 dm = \frac{2m}{r^2} \int_0^r \rho^3 d\rho = \frac{mr^2}{2}. \quad (4.14)$$

参数在启动时读取，改 YAML 后重启。`r=0` 时，另给一个正的偏航惯量作为独立模型参数。惯性模式限制执行器的力和力矩，速度通过运动方程积分得到。

## 4.9 从恒加速度到带阻尼的离散模型

先设阻尼为零。一个步长  $h$  内恒力意味着恒加速度  $a = F/m$ 。积分一次得到  $v(t) = v_k + at$ ，再积分速度：

$$p_{k+1} = p_k + \int_0^h (v_k + at) dt = p_k + hv_k + \frac{1}{2}h^2a, \quad v_{k+1} = v_k + ha. \quad (4.15)$$

这里的  $v_k$  是步初速度。偏航同理，把  $a$  换成  $\tau/I_z$ 。

有线性阻尼时，每个分量都是  $\dot{v} + \lambda v = u$ ，其中  $u = F/m, \lambda = c_v/m$ 。让输入在一个步长内保持常数，称为**零阶保持**。为解这个微分方程，两侧乘  $e^{\lambda t}$ ，左侧恰好成为乘积的导数：

$$\frac{d}{dt}(e^{\lambda t}v) = ue^{\lambda t}. \quad (4.16)$$

从 0 积分到  $t$  后， $e^{\lambda t}v(t) - v_k = u(e^{\lambda t} - 1)/\lambda$ ，所以

$$v(t) = e^{-\lambda t}v_k + \frac{1 - e^{-\lambda t}}{\lambda}u. \quad (4.17)$$

再积分一次求位移。第一项的积分系数为  $B = \int_0^h e^{-\lambda t} dt = (1 - e^{-\lambda h})/\lambda$ ；第二项为  $C = \int_0^h (1 - e^{-\lambda t})/\lambda dt = (h - B)/\lambda$ 。记  $A = e^{-\lambda h}$ ：

$$v_{k+1} = Av_k + Bu, \quad p_{k+1} = p_k + Bv_k + Cu. \quad (4.18)$$

$A$  无量纲， $B$  的单位为秒， $C$  的单位为秒平方，因而上式各项单位一致。对指数作泰勒展开，得到

$$A = 1 - \lambda h + \frac{1}{2}\lambda^2 h^2 + \dots, \quad (4.19)$$

$$B = h - \frac{1}{2}\lambda h^2 + \frac{1}{6}\lambda^2 h^3 + \dots, \quad (4.20)$$

$$C = \frac{1}{2}h^2 - \frac{1}{6}\lambda h^3 + \frac{1}{24}\lambda^2 h^4 + \dots. \quad (4.21)$$

因此  $\lambda \rightarrow 0$  时回到恒加速度式。代码对很小的  $\lambda h$  使用展开，减少两个接近的浮点数相减造成的精度损失；偏航取  $\lambda = c_\omega / I_z, u = \tau / I_z$ ，复用同样的系数。

```

54 State2D stepInertial(const State2D & state, const Wrench2D & command,
55     const InertialParameters & p, double dt)
56 {
57     const auto applied = limitWrench(command, p);
58     const auto xy = gains(p.linear_drag / p.mass, dt);
59     const auto yaw = gains(p.angular_drag / p.inertia_z, dt);
60     const Eigen::Vector2d u = applied.force / p.mass;
61     const double alpha = applied.torque / p.inertia_z;
62     State2D next;
63     next.pose.position = state.pose.position + xy.velocity * state.velocity + xy.position * u;
64     next.velocity = xy.decay * state.velocity + xy.velocity * u;
65     next.acceleration = (applied.force - p.linear_drag * next.velocity) / p.mass;
66     next.pose.yaw = wrapAngle(state.pose.yaw + yaw.velocity * state.yaw_rate + yaw.position * alpha);
67     next.yaw_rate = yaw.decay * state.yaw_rate + yaw.velocity * alpha;
68     next.yaw_acceleration = (applied.torque - p.angular_drag * next.yaw_rate) / p.inertia_z;
69     return next;
70 }
71 //

```

先限幅，再积分。步末加速度由施加的力和步末速度重新计算，后续 IMU 会使用这个运动真值。命令在 tick 切换，位置和速度连续，加速度可以随合力改变。

## 4.10 惯性、超时和碰撞如何一起工作

无阻尼时，零力意味着速度不变；有阻尼、零力时，

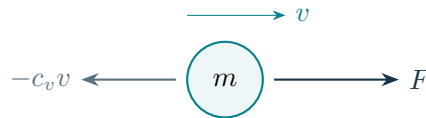
$$v(t) = v(0)e^{-c_v t/m}. \quad (4.22)$$

撤去力以后，机器人继续滑行；阻尼越大，速度衰减越快。超时切换为零输入，同时报告 `command_timeout`。后续控制器要用反向力制动，并考虑剩余空间；如果一直保持反向力，越过零速度后会向反方向加速。

碰撞检查需要本步加速度上界。对  $a = u - \lambda v$  求导，常数输入满足  $\dot{a} = -\lambda \dot{v} = -\lambda a$ 。用刚才的指数解便得到

$$a(t) = a(0)e^{-c_v t/m}, \quad a(0) = \frac{F - c_v v_k}{m}. \quad (4.23)$$

因此本步加速度模长不超过  $\|a(0)\|$ 。继续使用前一节的插值误差界，把  $\|a(0)\| h^2 / 8$  加到检查半径上，可以保守覆盖整个曲线。若包络与障碍相交，仿真停在上一个安全状态，供我们调整命令或路径。



合力  $F - c_v v$  决定加速度；撤去  $F$  后，阻力逐渐减小速度。

默认配置启动后，执行真实通信实验：

```
/usr/bin/python3 ../scripts/check_ch04.py --model inertial
```

脚本检查力与力矩限幅、自动惯量、暂停不推进、超时滑行、反向力制动和重置。质量改为 2 kg 时，可追加 `--mass 2`；其余仍使用默认零阻尼配置。

## 4.11 实验：质量、步长和制动距离

从静止出发，1 N 力持续 1 s，1 kg 的机器人移动 0.50 m、达到 1 m/s；2 kg 的机器人移动 0.25 m、达到 0.5 m/s。再从相同初速  $v_0 = 1$  m/s 出发，用大小  $F_b = 1$  N 的反向力制动，无阻尼时  $v(t) = v_0 - F_b t/m$ 。令速度为零，解出停止时刻；再积分到这个时刻：

$$t_{\text{stop}} = \frac{mv_0}{F_b}, \quad d_{\text{stop}} = \int_0^{t_{\text{stop}}} \left( v_0 - \frac{F_b}{m} t \right) dt = v_0 t_{\text{stop}} - \frac{F_b}{2m} t_{\text{stop}}^2 = \frac{mv_0^2}{2F_b}. \quad (4.24)$$

因此质量翻倍，同速制动距离也翻倍。比较制动时必须明确初速是否相同。

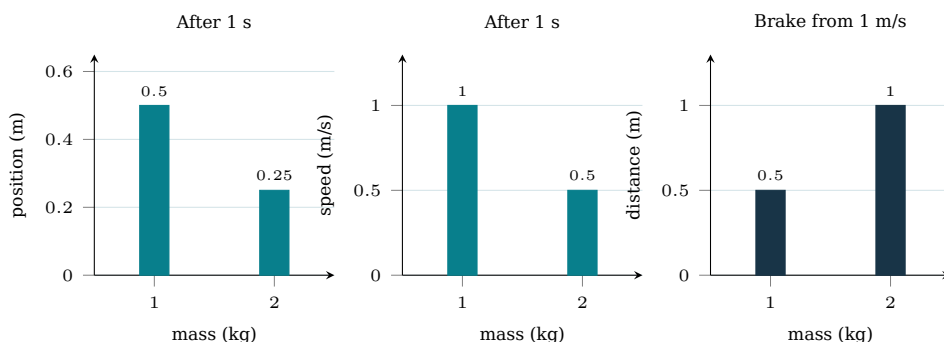


图 4.3: 相同力作用于两种质量。左、中图从静止加速 1 s；右图从相同初速制动。阻尼为零。

核心实验不依赖 ROS 回调时延，直接运行：

```
ros2 run motion2d inertia_demo
```

| m / kg | dt / s | 1 s 位移 / m | 1 s 末速 / (m/s) | 同速制动距离 / m |
|--------|--------|------------|----------------|------------|
| 1      | 0.020  | 0.50       | 1.00           | 0.50       |
| 1      | 0.005  | 0.50       | 1.00           | 0.50       |
| 2      | 0.020  | 0.25       | 0.50           | 1.00       |
| 2      | 0.005  | 0.25       | 0.50           | 1.00       |

两种 dt 得到相同结果，是因为恒定输入的解析积分准确累加，且切换时刻恰好落在步边界。若制动命令在两步之间到达，执行会延后到下一步；步长也决定输入刷新频率和碰撞包络的大小。

**代码练习 ch04-3:** 补完 exercises/ch04/starter/force\_step.hpp。先做无阻尼恒力一步更新，输入已经限幅；平移和偏航都要正确。独立的 check\_force.cpp 覆盖非零初速度、不同质量与转动惯量，运行命令见习题 README。

**拓展实验:** 把 linear\_drag 改为 0.5 kg/s，从有速度的状态撤去力，画出惯性模型速度随时间的衰减，与  $v_0 e^{-c_v t/m}$  比较。

## 4.12 连续理想参考：给后续传感器一条平滑轨迹

还有一种常用实验：先给定一条连续轨迹，让机器人完美执行，单独研究传感器和定位。下面用一个可以直接求导的圆轨迹做这个实验：

```
ros2 launch motion2d_bringup ch04.launch.py model:=reference
```

`reference` 自动按解析式设置位置、速度和加速度，并检查圆盘扫掠空间；后面的轨迹规划章节再研究这条运动需要多大的执行力。轨迹半径 `reference_radius` 默认 1 m，与机器人碰撞半径 `radius` 是两个参数；角频率 `reference_omega` 默认 0.4 rad/s。

令初始位姿为  $(p_0, \psi_0)$ ， $R_0 = R(\psi_0)$ 、 $\theta = \Omega t$ 。先在初始机体系画一个圆心位于  $(0, r_c)$  的圆：半径向量从  $(0, -r_c)$  开始逆时针转动，其分量为  $(r_c \sin \theta, -r_c \cos \theta)$ 。加上圆心，再旋转、平移到地图中，得到位置。随后用  $d\theta/dt = \Omega$  逐项求导：

$$p(t) = p_0 + R_0 \begin{bmatrix} r_c \sin \theta \\ r_c(1 - \cos \theta) \end{bmatrix}, \quad (4.25)$$

$$v(t) = R_0 r_c \Omega \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix}, \quad a(t) = R_0 r_c \Omega^2 \begin{bmatrix} -\sin \theta \\ \cos \theta \end{bmatrix}, \quad (4.26)$$

$$\psi(t) = \text{wrap}(\psi_0 + \theta), \quad \omega(t) = \Omega. \quad (4.27)$$

因此速度模长是  $r_c|\Omega|$ ，加速度模长是  $r_c\Omega^2$ 。每求一次时间导数，就因链式法则多乘一个  $\Omega$ 。方向向量的长度均为 1，因而可直接得到上述模长。代码把这三项和偏航一起返回：

```
7 State2D sampleCircle(const Pose2D & initial, double radius, double omega, double time)
8 {
9     const double theta = omega * time;
10    const auto r0 = rotation(initial.yaw);
11    State2D reference;
12    reference.pose.position = initial.position +
13        r0 * Eigen::Vector2d{radius * std::sin(theta), radius * (1.0 - std::cos(theta))};
14    reference.velocity = r0 * Eigen::Vector2d{
15        radius * omega * std::cos(theta), radius * omega * std::sin(theta)};
16    reference.acceleration = r0 * Eigen::Vector2d{
17        -radius * omega * omega * std::sin(theta), radius * omega * omega * std::cos(theta)};
18    reference.pose.yaw = wrapAngle(initial.yaw + theta);
19    reference.yaw_rate = omega;
20    return reference;
21 }
22 //
```

## 4.13 运动中的初始条件与导数核对

这个圆参考在  $t=0$  就有非零速度。默认起点仍为  $(-8, -8)$ ，初速度是 0.4 m/s，加速度模长为  $0.16 \text{ m/s}^2$ 。`reset` 将时刻、位置、速度和加速度一起恢复到  $t=0$  的值。因此 IMU 积分实验应以 0.4 m/s 作为初速度，才能与这条参考对应。

碰撞余量取  $r_c\Omega^2 h^2/8$ 。默认场景可以走完一圈；换大轨迹半径或换 `seed` 后，若参考穿过障碍，仿真会冻结，便于重新选择圆的大小或位置。

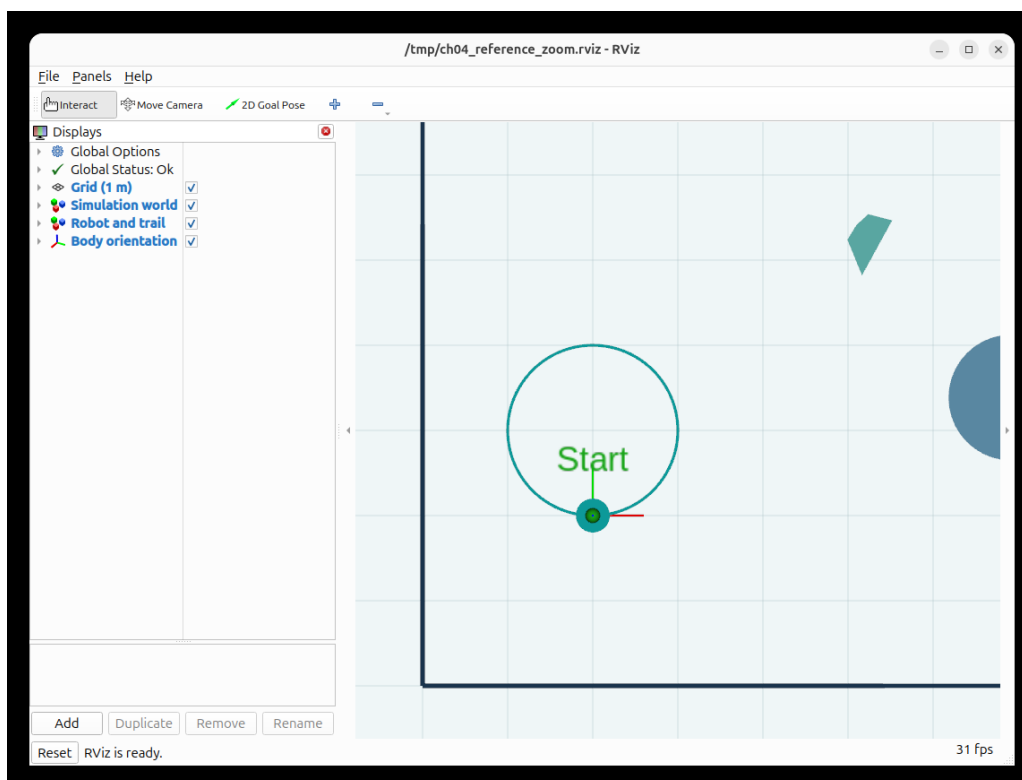


图 4.4: 圆参考的局部放大。轨迹圆心位于起点左侧，初始速度沿车头方向。

## 用中心差分核对解析导数

对任意足够光滑的分量  $f$ ，在  $t$  的两侧展开：

$$f(t + \epsilon) = f(t) + \epsilon f'(t) + \frac{1}{2}\epsilon^2 f''(t) + \frac{1}{6}\epsilon^3 f^{(3)}(t) + O(\epsilon^4), \quad (4.28)$$

$$f(t - \epsilon) = f(t) - \epsilon f'(t) + \frac{1}{2}\epsilon^2 f''(t) - \frac{1}{6}\epsilon^3 f^{(3)}(t) + O(\epsilon^4). \quad (4.29)$$

相减后偶数次项相消，再除以  $2\epsilon$ ，得到  $f'(t) + O(\epsilon^2)$ 。符号  $O(\epsilon^2)$  表示误差的主导大小与  $\epsilon^2$  成比例。因此可以逐分量检查

$$v(t) \approx \frac{p(t + \epsilon) - p(t - \epsilon)}{2\epsilon}, \quad a(t) \approx \frac{v(t + \epsilon) - v(t - \epsilon)}{2\epsilon}. \quad (4.30)$$

先取  $\epsilon = 10^{-3}$  s，再减半，观察误差是否约缩为四分之一。过小时，两个很接近的浮点数相减会放大舍入误差；应寻找误差随步长稳定下降的区间。传感器仍使用解析导数，中心差分用来核对推导和实现。

**代码练习 ch04-4:** 补完 `exercises/ch04/starter/reference_acceleration.hpp`，对速度求时间导数，注意链式法则中的第二个  $\Omega$ 。再把角频率翻倍，验证速度翻倍、加速度变为四倍。用以下命令核对运动消息：

```
/usr/bin/python3 ../scripts/check_ch04.py --model reference
```

## 4.14 把运动状态接到后续章节

本章默认  $dt=0.005$  s。位姿输入接受 `odom/map`；`map` 可用是因为本章二者重合。姿态必须是平面单位四元数，不能把四个分量全填零。时间戳为零表示下一 `tick`；非零时间戳只接受当前模拟时刻附近的命令，过旧或过远的未来命令被拒绝。

运行·理解·修改·验证

/sim/pose、/sim/velocity、/sim/acceleration 都是真值调试输出，平移分量在 odom 系。第 07 章才引入规范的里程计消息与选源，激光与 IMU 将在第 05、06 章根据这些运动量生成；SLAM 接收它们的传感器消息。

1. **理解：** 给一个起终点都自由但中间碰撞的例子。为什么瞬时位置赋值不适合 IMU？
2. **代码 ch04-1：** 完成 exercises/ch04/starter/sweep\_circle.hpp。复用点线段距离，判定圆盘与圆障碍的整段扫描；相切必须判为碰撞。
3. **实验：** 暂停后发目标并单步；改变半径重复同一条贴近障碍的移动。记录 seed、半径、起终点和结果，核对显示圆盘与碰撞半径是否一致。

从仓库根目录编译练习：

```
g++ -std=c++17 -I/usr/include/eigen3 \
-Iros2_ws/src/motion2d/include -Iexercises/ch04/starter \
exercises/ch04/check_sweep.cpp \
ros2_ws/src/motion2d/src/geometry/obstacles.cpp \
-o /tmp/ch04_sweep
/tmp/ch04_sweep
```

提示和参考解分开存放；将 starter 替换为 solutions 可以验证参考解。演示运行时，从工作空间执行：

```
/usr/bin/python3 ../scripts/check_ch04.py
```

脚本会改变场景状态，检查暂停命令、单步、错误帧、碰撞冻结与 reset。算法测试另外覆盖薄墙和零半径质点。

**没有移动时先检查** 是否暂停、是否需要 reset、模型是否与输入话题对应，再检查 frame 和四元数。只运行一个课程 launch，避免多个时钟或 TF 发布者。

## 4.15 用轨迹恢复状态与输入：微分平坦

到目前为止，我们一直由输入积分出运动。规划时可以反过来：先画一条带时间的曲线，再问“要沿它运动，需要怎样的速度和力？”如果选定的一组输出及其有限阶导数，就能通过代数运算恢复全部状态和输入，这组输出叫作**平坦输出**，系统称为**微分平坦系统**。这里的关键是恢复过程只需当前时刻的导数；角度在一个连续支路中表示。

对全向惯性模型，选择  $y = (p_x, p_y, \psi)$ 。位置的一阶导数给出速度，二阶导数给出加速度；把运动方程移项，就恢复了输入：

$$v = \dot{p}, \quad \omega = \dot{\psi}, \quad (4.31)$$

$$F = m\ddot{p} + c_v\dot{p}, \quad \tau = I_z\ddot{\psi} + c_\omega\dot{\psi}. \quad (4.32)$$

所以任意足够光滑的  $p(t), \psi(t)$  都对应一组满足运动方程的状态和输入。执行器上限接下来决定哪些曲线可以实现：例如 1 kg 机器人无阻尼时， $\ddot{p}_x = 3 \text{ m/s}^2$  需要 3 N，而 2 N 执行器允许的同向加速度只有  $2 \text{ m/s}^2$ 。

位置和 yaw 要一起给出。圆盘可以沿直线平移并同时独立转向，单凭  $p(t)$  无法确定 yaw。第 15、16 章的位置优化先保持 yaw 常数，此时偏航力矩为零；需要转向时，可给 yaw 也增加一条曲线，并

用上式约束力矩。对全向速度模型，同样选择位姿作为输出，其一阶导数就给出速度输入。理想到点和圆参考则分别是位置设置方式与预先指定的运动，用于实验中的不同环节。

## 和无人机轨迹规划的联系

四旋翼的推力方向随姿态改变。忽略空气阻力时，设向上的单位向量为  $e_3$ ，机体推力轴单位向量为  $b_3$ ，推力大小为  $f$ ，重力加速度大小为  $g$ 。牛顿定律为  $m\ddot{p} = fb_3 - mge_3$ 。移项后令  $w = m(\ddot{p} + ge_3)$ ，可得

$$fb_3 = w, \quad f = \|w\|, \quad b_3 = w/\|w\| \quad (\|w\| > 0). \quad (4.33)$$

因为  $b_3$  长度为 1，向量  $w$  的长度给出推力大小，方向给出机体倾斜方向。例如悬停时  $\ddot{p} = 0$ ，得到  $f = mg, b_3 = e_3$ ；水平加速时  $w$  向前倾斜，推力也随之倾斜。再指定朝向，就能恢复完整姿态。地面圆盘由地面支撑重力，所以上文的  $F$  表示水平合力。

这种“轨迹导数  $\rightarrow$  物理量”的计算方式也用于 GCOPTER 的轨迹优化。带旋翼阻力的三维模型可继续阅读 Faessler 等人的论文<sup>1</sup>；课程下面用较短的二维方程学习同一计算思路。

## 4.16 把物理代价的梯度传回轨迹

规划器可以惩罚过大的力。它还需要知道：稍微改变速度、加速度时，这个惩罚会怎样变化？这就是多变量链式法则的问题。令  $a = \ddot{p}, j = p^{(3)}$ ， $j$  称为 jerk，单位  $m/s^3$ 。质量、阻尼为常数时，对力再求导得到

$$F = ma + c_v v, \quad \dot{F} = mj + c_v a. \quad (4.34)$$

omniForward 同时返回力、力变化率和偏航力矩：

```
5 OmniFlatOutput omniForward(const OmniFlatInput & x, const InertialParameters & p)
6 {
7     return {p.mass*x.acceleration+p.linear_drag*x.velocity,
8             p.mass*x.jerk+p.linear_drag*x.acceleration,
9             p.inertia_z*x.yaw_acceleration+p.angular_drag*x.yaw_rate};
10 }
11 //
```

设标量代价为  $J$ 。把它对向量  $F$  的各个分量偏导排成列，记为  $\bar{F} = (\partial J / \partial F_x, \partial J / \partial F_y)^\top$ ，称为梯度。当  $F$  改变一个很小的  $dF$  时，一阶变化为  $dJ = \bar{F}^\top dF$ 。如果代价还依赖  $\dot{F}$ 、 $\tau$ ，则把三个贡献相加：

$$dJ = \bar{F}^\top dF + \bar{\dot{F}}^\top d\dot{F} + \bar{\tau} d\tau. \quad (4.35)$$

注意  $d$  表示变量的小变化，点表示对时间求导。代入  $dF = m da + c_v dv$ 、 $d\dot{F} = m dj + c_v da$  及  $d\tau = I_z d\ddot{\psi} + c_\omega d\dot{\psi}$ ，按输入的小变化重新分组：

$$dJ = (c_v \bar{F})^\top dv + (m \bar{F} + c_v \bar{\dot{F}})^\top da + (m \bar{\dot{F}})^\top dj + c_\omega \bar{\tau} d\dot{\psi} + I_z \bar{\tau} d\ddot{\psi}. \quad (4.36)$$

每项前面的系数就是所求梯度：

$$\bar{v} = c_v \bar{F}, \quad \bar{a} = m \bar{F} + c_v \bar{\dot{F}}, \quad \bar{j} = m \bar{\dot{F}}, \quad (4.37)$$

$$\bar{\dot{\psi}} = c_\omega \bar{\tau}, \quad \bar{\ddot{\psi}} = I_z \bar{\tau}. \quad (4.38)$$

<sup>1</sup>Faessler, Franchi, Scaramuzza, [Differential Flatness of Quadrotor Dynamics Subject to Rotor Drag](#), 2018.

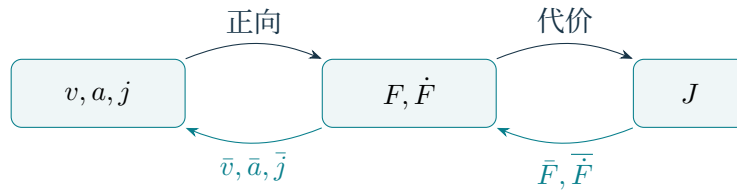


图 4.5: 先计算物理量, 再沿依赖关系反向传递代价变化。每条分支的梯度贡献相加。

例如只取  $J = \frac{1}{2}\|F\|^2 = \frac{1}{2}(F_x^2 + F_y^2)$ , 逐分量求导得到  $\bar{F} = F$ , 于是  $\bar{a} = mF$ 。单轴  $m = 2, c_v = 0.1, v = 0.3, a = 0.4$  时,  $F = 0.83 \text{ N}$ ,  $\partial J / \partial a = 1.66$ ; 加速度增加  $0.001$ , 代价约增加  $0.00166$ 。代码直接算出这些梯度, 不必先构造一个含全部偏导的大矩阵。这种运算常称为向量-雅可比乘积:

```
13 OmniFlatInput omniBackward(const OmniFlatOutput & g, const InertialParameters & p)
14 {
15     return {p.linear_drag*g.force,
16            p.mass*g.force+p.linear_drag*g.force_rate,
17            p.mass*g.force_rate,p.angular_drag*g.torque,p.inertia_z*g.torque};
18 }
19 //
```

正向映射返回曲线实际要求的力, 包括超限部分; 这样惩罚函数才能分辨“超出多少”。仿真执行时再按执行器上限限幅。GCOPTER 的 `forward/backward` 接口也采用先恢复物理量、再传回梯度的分工。

**练习 ch04-5.** 运行 `ros2 run motion2d flatness_demo`, 再完成 `exercises/ch04/starter/flat_force.hpp`。把质量翻倍并保持同一条轨迹, 比较力与加速度梯度; 用前面推导的中心差分检查解析梯度。编译命令:

```
g++ -std=c++17 -I/usr/include/eigen3 -Iexercises/ch04/starter \
    exercises/ch04/check_flatness.cpp -o /tmp/ch04_flatness
/tmp/ch04_flatness
```

第 15、16 章会把这些梯度继续传到曲线系数、路点和时长。

## 4.17 有车轮方向约束的阿克曼模型

小车的前后轮都沿轮子指向滚动。在低速、无侧滑条件下, 把前后两组轮子各等效为一个轮子, 得到自行车模型。取后轴中点为位置  $p$ , 车身朝向为  $\psi$ , 有符号纵向速度为  $s$ , 等效前轮转角为  $\delta$ 。轴距  $L$  是前后轴的距离。

### 由转弯圆心求偏航速度

先在车体系考虑左转。后轮速度沿车身  $x$  轴, 瞬时转弯圆心  $O$  位于后轮正左方, 距离为  $R$ 。前轮的速度方向比车身转过  $\delta$ , 所以从前轮指向  $O$  的线也转过相同角度。图中的直角三角形给出  $\tan \delta = L/R$ , 即  $R = L / \tan \delta$ 。后轮走过弧长  $ds = R d\psi$ , 除以时间增量可得

$$\dot{\psi} = s/R = \frac{s \tan \delta}{L}. \quad (4.39)$$

右转时  $\delta < 0$ , 同一个式子自动给出负的偏航角速度。

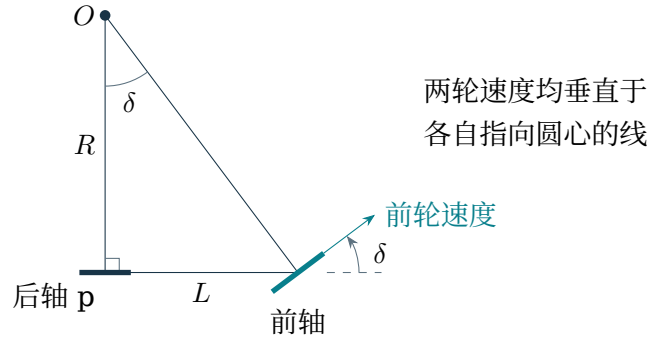


图 4.6: 无侧滑转弯的几何关系。转角决定曲率  $1/R$ ，速度决定沿这个圆行驶的快慢。

给纵向力  $F$  和转向速率  $r_\delta$  作为控制输入，采用与圆盘相同的线性纵向阻尼  $c$ ：

$$x = (p_x, p_y, \psi, s, \delta), \quad u = (F, r_\delta), \quad (4.40)$$

$$\dot{p} = s(\cos \psi, \sin \psi)^\top, \quad \dot{\psi} = s \tan \delta / L, \quad (4.41)$$

$$\dot{s} = (F - cs)/m, \quad \dot{\delta} = r_\delta. \quad (4.42)$$

$F$  的单位为 N， $r_\delta$  为 rad/s。纵向受力改变快慢，转角改变前进方向；当  $s=0$  时，前轮可以转动，车身朝向保持不变。课程把激光、IMU 和圆盘碰撞包络的中心都放在后轴点  $p$ 。若包络需要覆盖实际车身，半径应覆盖从后轴到前悬及车体侧边的最远距离。

## 4.18 按一个时间步推进小车

步内  $F$  和  $r_\delta$  保持常数，纵向速度使用前面推导的带阻尼解析解，转角直接积分为  $\delta(t) = \delta_k + tr_\delta$ 。为使整个步内转角满足  $-\delta_{\max} \leq \delta(t) \leq \delta_{\max}$ ，线性变化只需检查端点，因此还应满足

$$\frac{-\delta_{\max} - \delta_k}{h} \leq r_\delta \leq \frac{\delta_{\max} - \delta_k}{h}. \quad (4.43)$$

`limitAckermannInput` 把这个区间与转向速率本身的上下限取交集，再限制输入。

## 四次斜率估计

对位姿  $q = (p_x, p_y, \psi)$ ，右端函数记为

$$f(t, q) = (s(t) \cos \psi, s(t) \sin \psi, s(t) \tan \delta(t)/L)^\top. \quad (4.44)$$

若只用步初斜率  $f(0, q_k)$  乘  $h$ ，就是一条切线近似；转弯时，步中斜率已经改变。`stepAckermann` 使用经典四阶 Runge-Kutta (RK4)，先用已有斜率预测中间状态，再重新计算斜率：

$$k_1 = f(0, q_k), \quad k_2 = f(h/2, q_k + hk_1/2), \quad (4.45)$$

$$k_3 = f(h/2, q_k + hk_2/2), \quad k_4 = f(h, q_k + hk_3), \quad (4.46)$$

$$q_{k+1} = q_k + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \quad (4.47)$$

$k_1$  在起点取斜率； $k_2$  用起点切线预测半步； $k_3$  用新的半步斜率再估计一次； $k_4$  用  $k_3$  预测终点。最终取加权平均，而不是选择其中某一条切线。

这些权重可以先从只依赖时间的积分理解。设起点、中点、终点的权重为  $A$ 、 $B$ 、 $C$ ，要求公式对  $1, t, t^2$  的积分正确，在单位区间上有

$$A + B + C = 1, \quad B/2 + C = 1/2, \quad B/4 + C = 1/3. \quad (4.48)$$

解得  $A = C = 1/6, B = 2/3$ ，即 Simpson 权重。RK4 在依赖状态的问题中用两个中间状态估计中点斜率，各分配  $1/3$  权重，得到上面的  $1, 2, 2, 1$ 。这里的中间状态本身也是近似值，可用一个  $h$  步与两个  $h/2$  步比较数值误差。当  $s, \delta$  均为常数时， $\dot{\psi}$  为常数，四次阶段中使用的朝向都是正确时刻的朝向；位置更新便退化为对正弦、余弦速度作 Simpson 积分，可以直接和圆弧解析式核对。

## IMU 所需的加速度

记车头方向  $e = (\cos \psi, \sin \psi)^\top$ ，左侧方向  $e_\perp = (-\sin \psi, \cos \psi)^\top$ 。逐分量求导有  $\dot{e} = \dot{\psi} e_\perp$ ，所以对  $v = se$  用乘积法则：

$$a = \dot{s}e + s\dot{\psi}e_\perp. \quad (4.49)$$

第一项改变速率，第二项改变速度方向。匀速转弯仍有第二项，模长为  $s^2/R$ 。同样对  $\dot{\psi} = s \tan \delta / L$  求导，并用  $d(\tan \delta)/dt = (1 + \tan^2 \delta)r_\delta$ ：

$$\ddot{\psi} = \frac{\dot{s} \tan \delta + s(1 + \tan^2 \delta)r_\delta}{L}. \quad (4.50)$$

ackermannKinematics 正是计算这几项，把后轴点的运动导数交给传感器模型。其扫掠余量也沿用  $a_{\max} h^2/8$ ：分别用步内最大速率、最大绝对转角，上界纵向项  $(|F| + c|s|)/m$  和侧向项  $s^2 |\tan \delta|/L$ ，再将两项相加。

## 4.19 从位置曲线恢复小车的运动

选择平坦输出  $y = (p_x, p_y)$ ，先取速度非零、沿车头前进的支路。令  $v = \dot{p}, a = \ddot{p}, j = p^{(3)}$ ，并保存四个便于复用的标量：

$$n = \|v\|, \quad b = v^\top a, \quad h = [v, a], \quad h_p = [v, j]. \quad (4.51)$$

方括号表示第 3 章的二维叉积；这一节中的  $h$  是叉积值。先由速度长度和方向恢复  $s = n, \psi = \text{atan2}(v_y, v_x)$ 。 $\text{atan2}$  同时看两个分量的符号，返回对应象限的方向角。

### 速率、转向和纵向力

由  $n^2 = v^\top v$  求导， $2n\dot{n} = 2v^\top a = 2b$ ，得到  $\dot{n} = b/n$ 。另一方面， $v = ne, a = \dot{n}e + n\dot{\psi}e_\perp$ 。叉积中  $[e, e] = 0, [e, e_\perp] = 1$ ，所以

$$h = [v, a] = n^2 \dot{\psi}, \quad \dot{\psi} = h/n^2. \quad (4.52)$$

曲率  $\kappa$  是单位路程内改变的方向角，因此  $\kappa = d\psi/d\ell = \dot{\psi}/n = h/n^3$ ，单位  $\text{m}^{-1}$ 。与自行车的  $\kappa = \tan \delta / L$  对照后就得到转角；纵向方程则给出力：

$$\delta = \arctan(L\kappa), \quad F = m\dot{n} + cn = mb/n + cn. \quad (4.53)$$

纵向和侧向加速度分别为  $a_\parallel = \dot{n} = b/n, a_\perp = n\dot{\psi} = h/n$ 。

## 转向速率需要三阶导数

为了恢复输入  $r_\delta = \dot{\delta}$ ，再对曲率求导。叉积的乘积法则给出  $\dot{h} = [a, \dot{a}] + [v, j] = h_p$ ，因为  $[a, a] = a_x a_y - a_y a_x = 0$ 。分子、分母都随时间变，因而

$$\dot{\kappa} = \frac{d}{dt}(hn^{-3}) = h_p n^{-3} - 3hn^{-4}\dot{n} = \frac{h_p}{n^3} - \frac{3hb}{n^5}, \quad (4.54)$$

$$r_\delta = \frac{L\dot{\kappa}}{1 + (L\kappa)^2}. \quad (4.55)$$

最后一步用  $d(\arctan z)/dz = 1/(1+z^2)$ 。至此，全部状态和两个输入都由位置的一至三阶导数恢复，得到了这条支路的微分平坦映射。代码按照相同的依赖顺序计算：

```
11 const double b=x.velocity.dot(x.acceleration),h=cross(x.velocity,x.acceleration);
12 const double hp=cross(x.velocity,x.jerk),k=h/std::pow(n,3);
13 const double kdot=hp/std::pow(n,3)-3*h*b/std::pow(n,5);
14 AckermannFlatOutput y;
15 y.speed=n;y.yaw=std::atan2(x.velocity.y(),x.velocity.x());y.yaw_rate=h/(n*n);
16 y.curvature=k;y.steering=std::atan(p.wheelbase*k);
17 y.steering_rate=p.wheelbase*kdot/(1+std::pow(p.wheelbase*k,2));
18 y.longitudinal_acceleration=b/n;y.lateral_acceleration=h/n;
19 y.force=p.mass*y.longitudinal_acceleration+p.linear_drag*n;
20 //
```

**手算匀速圆。**半径  $R$ 、速度  $s$  的圆有  $\kappa = 1/R$ ，所以  $\delta = \arctan(L/R)$ ， $r_\delta = 0$ ， $F = cs$ 。运行 `ros2 run motion2d ackermann_demo`，默认  $L = 0.3 \text{ m}$ 、 $R = 1.2 \text{ m}$ 、 $s = 0.6 \text{ m/s}$ 、 $c = 0.15 \text{ kg/s}$ ，应得到转角约  $0.245 \text{ rad}$ 、力  $0.09 \text{ N}$ 、侧向加速度  $s^2/R = 0.3 \text{ m/s}^2$ 。匀速时力只补偿阻尼；加速段的  $m\dot{n}$  才体现质量。

## 4.20 停车点如何保存方向

上面的恢复式用速度方向表示车头方向。停车时  $v = 0$ ，方向信息消失：同一个静止位置可以对应任意车头方向，因此需要额外保留路径的几何信息。第 16 章将位置写成  $p(t) = q(u(t))$ ，其中几何曲线满足  $q'(u) \neq 0$ 。链式法则给出

$$\dot{p} = q'(u)\dot{u}. \quad (4.56)$$

当  $\dot{u} > 0$ ，车头沿  $q'$ ；停车令  $\dot{u} = 0$ ，但  $q'$  的方向仍保存在路径中。进一步， $\ddot{p} = q''\dot{u}^2 + q'\ddot{u}$ ，于是

$$\frac{[\dot{p}, \ddot{p}]}{\|\dot{p}\|^3} = \frac{[q', q'']\dot{u}^3}{\|q'\|^3\dot{u}^3} = \frac{[q', q'']}{\|q'\|^3} \quad (\dot{u} > 0). \quad (4.57)$$

右侧只依赖几何曲线，可以连续延伸到停车点，用来保存转角。仿真模型可以正常起停；直接平坦映射则要求  $n$  大于最小速度阈值。倒车时，位置的切向与车头方向相反，要另选  $s = -n$  的支路并相应处理朝向。本课程的小车规划先采用前进路径与停稳后的下一段运动。

## 4.21 逐个标量计算小车的反向梯度

阿克曼公式比全向公式长，但反向规则仍是上一节的链式法则。先为每个中间变量建立一个梯度累加器，初值为零；若  $z = f(x, y)$ ，就执行  $\bar{x} += \bar{z} \partial f / \partial x$ 、 $\bar{y} += \bar{z} \partial f / \partial y$ 。一个变量被多个下游量使用时，各条路径的贡献都要相加。

为缩短式子, 记  $z = \kappa$ 、 $D = 1 + (L\kappa)^2$ 。对  $r_\delta = Lz/D$  求导有  $dr_\delta = (L/D)dz - (2L^3\kappa z/D^2)d\kappa$ ；对转角求导有  $d\delta = (L/D)d\kappa$ 。因此先累加

$$\bar{z} += \frac{L}{D}\bar{r}_\delta, \quad \bar{\kappa} += \bar{\kappa}_{\text{out}} + \frac{L}{D}\bar{\delta} - \frac{2L^3\kappa z}{D^2}\bar{r}_\delta. \quad (4.58)$$

$\bar{\kappa}_{\text{out}}$  表示代价直接作用在曲率上的偏导。其余关系按下表倒序传递。每一行都只用乘积、幂函数和商的求导规则；例如  $d(n^{-3}) = -3n^{-4}dn$ 。

| 正向关系                      | 对应的一阶变化（乘以该行输出梯度后累加）                                                    |
|---------------------------|-------------------------------------------------------------------------|
| $F = ma_{\parallel} + cn$ | $dF = m da_{\parallel} + c dn$                                          |
| $a_{\parallel} = b/n$     | $da_{\parallel} = db/n - b dn/n^2$                                      |
| $\omega = h/n^2$          | $d\omega = dh/n^2 - 2h dn/n^3$                                          |
| $a_{\perp} = h/n$         | $da_{\perp} = dh/n - h dn/n^2$                                          |
| $z = h_p/n^3 - 3hb/n^5$   | $dz = dh_p/n^3 - 3b dh/n^5 - 3h db/n^5$<br>$+ (-3h_p/n^4 + 15hb/n^6)dn$ |
| $\kappa = h/n^3$          | $d\kappa = dh/n^3 - 3h dn/n^4$                                          |

例如力贡献先送到  $\bar{a}_{\parallel}$  和  $\bar{n}$ ，再由  $a_{\parallel} = b/n$  送到  $\bar{b}, \bar{n}$ ；转向速率贡献先到  $\bar{z}, \bar{\kappa}$ ，再到  $\bar{h}_p, \bar{h}, \bar{b}, \bar{n}$ 。速度输出  $\mathbf{s}=\mathbf{n}$  的贡献直接加入  $\bar{n}$ 。

最后把四个标量的梯度送回  $\mathbf{v}$ 、 $\mathbf{a}$ 、 $\mathbf{j}$ 。由坐标定义逐项求导：

$$dn = (v/n)^{\top} dv, \quad (4.59)$$

$$db = a^{\top} dv + v^{\top} da, \quad (4.60)$$

$$dh = (a_y, -a_x) dv + (-v_y, v_x) da, \quad (4.61)$$

$$dh_p = (j_y, -j_x) dv + (-v_y, v_x) dj. \quad (4.62)$$

例如  $h = v_x a_y - v_y a_x$ ，对  $v_x, v_y$  求导就是  $a_y, -a_x$ 。朝向的变化可以由  $v = ne$  得到： $dv = e dn + ne_{\perp} d\psi$ ，左乘  $e_{\perp}^{\top}$  后第一项消失，故  $d\psi = (-v_y, v_x)dv/n^2$ 。把各项组合起来：

$$\bar{v} = \bar{n} \frac{v}{n} + \bar{b}a + \bar{h}(a_y, -a_x)^{\top} + \bar{h}_p(j_y, -j_x)^{\top} + \frac{\bar{\psi}}{n^2}(-v_y, v_x)^{\top}, \quad (4.63)$$

$$\bar{a} = \bar{b}v + \bar{h}(-v_y, v_x)^{\top}, \quad (4.64)$$

$$\bar{j} = \bar{h}_p(-v_y, v_x)^{\top}. \quad (4.65)$$

`ackermannBackward` 的最后三个赋值正对应这三行。可以先只给 `force` 输出一个非零梯度，手算力这条分支，再逐一加入转角、转向速率等输出，每次用中心差分检查。这比同时调试所有分支更容易定位错误。

**练习 ch04-6。**补全 `exercises/ch04/starter/curvature_rate.hpp`，推导并实现曲率时间导数，再运行：

```
g++ -std=c++17 -I/usr/include/eigen3 -Iexercises/ch04/starter \
    exercises/ch04/check_ackermann.cpp -o /tmp/ch04_ackermann
/tmp/ch04_ackermann
```

匀速圆的曲率导数恰为零；还应检查速度和曲率同时变化的曲线。固定同一条几何路径、把行驶时间增加一倍，预测转角、侧向加速度和转向速率的变化。利用  $p(t) = q(t/T)$  的链式法则可以逐项核对：方向和曲率保持原值，速度减半，侧向加速度缩为四分之一，转向速率减半。

## 4.22 把模型选择落实到执行器

运行完本章的独立模型例子后，第 19 章提供同一份配置的完整闭环：

```
ros2 launch motion2d_bringup flat_vehicle.launch.py rviz:=false
```

配置文件为 bringup 包中的 config/flat\_vehicle.yaml。编辑顶层 vehicle 并重启：

```
vehicle:
  model: ackermann      # ackermann / inertial
  localization: truth    # truth / slam
  radius: 0.4           # m
  mass: 1.0             # kg
  linear_drag: 0.15     # kg/s
  force_max: 2.0        # N
  wheelbase: 0.3        # m, Ackermann
  steering_max: 0.6     # rad, Ackermann
  steering_rate_max: 1.0 # rad/s, Ackermann
```

launch 把同一处物理值送给模拟器和规划控制节点，避免把仿真设成 3 kg、规划仍按 1 kg 求力。参数在启动时读取；这个文件包含 launch 专用段，不能直接当作节点的 params-file。全向惯量默认  $I = mr^2/2$ ；当半径为零时，必须显式配置正的 inertia\_z。

| 模型        | 实际命令       | 相应规划与控制                |
|-----------|------------|------------------------|
| inertial  | odom 系力/力矩 | MINCO 或 Spline + 前馈 PD |
| ackermann | 纵向力/转向速率   | 正则路径/停车进度 + 局部反馈       |

全向位置优化保持起步 yaw，阿克曼规划使用目标 yaw。planning.backend 只在全向模型中选择 minco 或 spline；小车始终使用专用规划器。两个执行器接口分别对应本章推导的运动方程，规划器使用同一份质量、阻尼与输入上限。

**阿克曼传感器。**激光和 IMU 位于后轴参考点，转弯时 IMU 包含向心加速度。/joint\_states 提供名为 front\_steering 的理想转角编码器，单位 rad；控制器从这个编码器读取当前转角，再计算转向速率。碰撞包络、传感器外参及半径的意义见本章前述约定，非对齐采样见第 06 章。

**学习顺序。**先用真值定位理解模型和规划，再换成激光/IMU SLAM 观察估计误差。第 19 章会讲观测地图、未来起点和停车过程；第 18 章可单独学习全向线性 MPC。掌握停车后接续的前进规划后，可继续用自行车方程学习非线性 MPC、倒车支路和运动中的轨迹接续。

## 第五章 一束光到一圈扫描：CPU 激光雷达

**目标** 用可手算的几何求交构建激光雷达，理解遮挡、角分辨率与测距误差。

**先修** 二维向量、圆方程，以及前四章的坐标、世界与机器人模型。

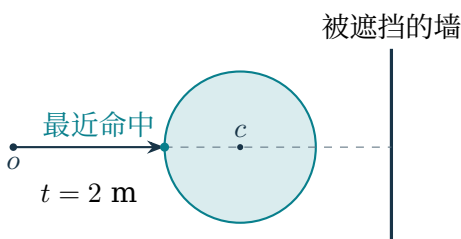
### 5.1 先让一束光碰到障碍

雷达安装在圆盘中心，发出一条没有宽度的射线。机器人位置与  $\text{yaw}$  决定射线的起点和方向。模拟器让射线与环境障碍求交，圆盘外形则交给运动碰撞检查。这样可以把圆心看成雷达的安装点。

射线用起点  $o$ 、单位方向  $d$  和非负距离  $t$  表示：

$$p(t) = o + td, \quad \|d\| = 1, \quad t \geq 0. \quad (5.1)$$

由于  $\|p(t) - o\| = t\|d\| = t$ ，单位方向使  $t$  直接表示米。测距对应最近障碍表面。沿同一方向有多个交点时，较近表面会挡住较远表面。



重新构建工作空间并加载 `install`，先运行不依赖 ROS 回调的几何演示：

```
ros2 run motion2d raycast_demo
```

三个距离应依次为 2、4、2 m。前两个分别是圆和墙的单独求交；最后一个在世界中取最近值。代码位于 `src/examples/raycast_demo.cpp`，这里的墙和圆都可以手算。

### 5.2 射线与圆：从圆方程到两个交点

令射线起点到圆心的向量为  $q = c - o$ 。把射线代入半径  $R$  的圆方程并展开点积：

$$\|o + td - c\|^2 = \|td - q\|^2 = R^2, \quad (5.2)$$

$$t^2(d^\top d) - 2tq^\top d + q^\top q - R^2 = 0. \quad (5.3)$$

单位方向满足  $d^\top d = 1$ 。记沿射线方向的圆心投影为  $h = q^\top d$ ，配方后得到

$$(t - h)^2 = R^2 - (\|q\|^2 - h^2). \quad (5.4)$$

括号中的  $\|q\|^2 - h^2$  是圆心到直线的垂距平方。为了直接计算垂距，取与  $d$  垂直的单位向量  $d_\perp = (-d_y, d_x)^\top$ 。垂直分量为  $d_\perp^\top q = d_x q_y - d_y q_x = d \times q$ 。两个垂直方向的分量满足勾股关系，因此

$$D = R^2 - (d \times q)^2, \quad t_- = h - \sqrt{D}, \quad t_+ = h + \sqrt{D}. \quad (5.5)$$

代码采用叉积形式计算  $D$ ，直接使用垂距，省去了  $\|q\|^2 - h^2$  这两个平方量相减。

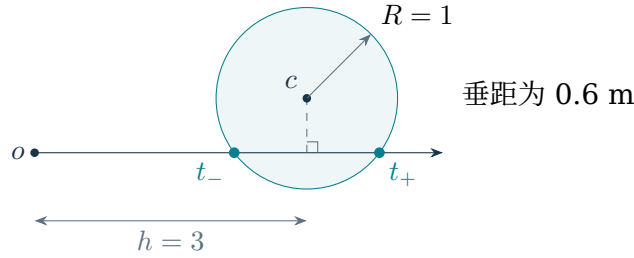


图 5.1: 垂足到两个交点各相距  $\sqrt{1 - 0.6^2} = 0.8$  m，因此根为 2.2 和 3.8 m。

$D < 0$  表示垂距大于半径，直线与圆分离； $D = 0$  时两根相同，对应相切。 $D > 0$  时有两个直线交点，但射线只保留  $t \geq 0$  的部分。先检查较小根  $t_-$ ，若为负再检查  $t_+$ ；两个根都负，圆便位于射线背后。例如起点在圆心时  $h = 0, D = R^2$ ，两根为  $-R, R$ ，正根是向前射出的出口。

```

37 double rayCircle(const Eigen::Vector2d & origin, const Eigen::Vector2d & direction,
38   const Circle & circle)
39 {
40   const Eigen::Vector2d offset = circle.center - origin;
41   const double h = offset.dot(direction);
42   const double perpendicular = cross(direction, offset);
43   const double discriminant = circle.radius * circle.radius - perpendicular * perpendicular;
44   if (discriminant < 0.0) {return kMiss;}
45   const double root = std::sqrt(discriminant);
46   if (h - root >= 0.0) {return h - root;}
47   return h + root >= 0.0 ? h + root : kMiss;
48 }
49 //

```

代码中的 `kMiss` 是正无穷。把它与有限距离取最小值时，有限距离会被保留。几何层由此统一返回“最近非负距离或正无穷”，扫描层再处理量程和测量误差。

**代码练习 ch05-1:** 在 `exercises/ch05/starter/forward_root.hpp` 补完根的选择。从仓库根目录运行：

```

g++ -std=c++17 -Iexercises/ch05/starter \
  exercises/ch05/check_root.cpp -o /tmp/ch05_root
/tmp/ch05_root

```

先手算圆心 (3,0)、半径 1、起点 (0,0)、方向 (1,0) 的两个根；再把起点放到圆心。这两个例子分别检查最近回波和第二个根。

## 5.3 射线与线段：用叉积消去一个未知数

令线段为  $a + ue$ ，其中  $e = b - a, 0 \leq u \leq 1$ 。交点同时位于射线和线段上，因此  $o + td = a + ue$ ，移项得到  $td - ue = a - o$ 。这里有两个未知数  $t, u$ 。利用向量与自身的叉积为零，可以分别消去它们：

$$(td - ue) \times e = t(d \times e) = (a - o) \times e, \quad (5.6)$$

$$(td - ue) \times d = -u(e \times d) = u(d \times e) = (a - o) \times d. \quad (5.7)$$

第二行用了  $e \times d = -(d \times e)$ 。当  $d \times e \neq 0$ ，直接相除：

$$t = \frac{(a - o) \times e}{d \times e}, \quad u = \frac{(a - o) \times d}{d \times e}. \quad (5.8)$$

$t$  的单位为米， $u$  是线段上的无量纲比例。只有  $t \geq 0$  且  $0 \leq u \leq 1$  才同时满足两种限制。例如  $o = (0, 0), d = (1, 0), a = (4, -1), b = (4, 1)$ ，有  $e = (0, 2)$ 、分母 2，分子分别为 8 和 1，故  $t = 4, u = 1/2$ ，交点是墙的中点  $(4, 0)$ 。

当分母为零，两条方向平行。若  $(a - o) \times d \neq 0$ ，说明线段偏离射线直线，二者分离；否则它们共线。共线时，计算两个端点沿  $d$  的投影  $t_a = (a - o)^\top d, t_b = (b - o)^\top d$ ，得到区间  $[\min(t_a, t_b), \max(t_a, t_b)]$ 。它与射线区间  $[0, +\infty)$  的交集有两种结果：

$$\begin{cases} \text{无交点,} & \max(t_a, t_b) < 0, \\ t = \max(0, \min(t_a, t_b)), & \max(t_a, t_b) \geq 0. \end{cases} \quad (5.9)$$

例如投影区间  $[-2, 3]$  包含起点，最近距离为 0； $[2, 4]$  的最近距离为 2； $[-4, -2]$  位于背后。

```

19 double raySegment(const Eigen::Vector2d & origin, const Eigen::Vector2d & direction,
20     const Eigen::Vector2d & a, const Eigen::Vector2d & b)
21 {
22     const Eigen::Vector2d edge = b - a, offset = a - origin;
23     const double denominator = cross(direction, edge);
24     const double tolerance = 1e-12 * std::max(1.0, edge.norm());
25     if (std::abs(denominator) <= tolerance) {
26         if (std::abs(cross(offset, direction)) > tolerance) {return kMiss;}
27         const double t0 = offset.dot(direction), t1 = (b - origin).dot(direction);
28         return std::max(t0, t1) < 0.0 ? kMiss : std::max(0.0, std::min(t0, t1));
29     }
30     const double t = cross(offset, edge) / denominator;
31     const double u = cross(offset, direction) / denominator;
32     return t >= 0.0 && u >= 0.0 && u <= 1.0 ? t : kMiss;
33 }
34 //

```

世界求交 `raycastWorld` 遍历所有圆、所有多边形边，以及矩形边界的四条边，对得到的非负距离取最小值。这样自然处理遮挡，不依赖障碍物存储顺序。算法复杂度随表面数线性增长；先保留这个清楚的 CPU 基准，后续再讨论加速。

**动手核对：**运行 `colcon test --packages-select motion2d`。几何测试覆盖圆相切、背向、线段端点、平行与共线、矩形角点、遮挡和刚体变换不变性。把演示的圆移到墙后，预测最近命中；反转射线方向，解释距离为何改变。

**读代码时留意的数值分支** 方向几乎平行时，除数很小，舍入误差会被放大。实现按边长设置一个小容差，把这个范围内的叉积视作平行，再进入上面推导的分支。可在米制场景中逐渐旋转一条接近平行的边，观察分支切换前后的距离与交点位置。

## 5.4 把方向均匀排成一圈

在雷达坐标系中, 第  $i$  束的角度为  $\alpha_i = \alpha_0 + i\Delta\alpha$ 。初始方向  $\alpha_0$  与  $\text{yaw}$  是不同的量: 前者决定扫描扇区, 后者描述机器人朝向。世界方向与圆心起点是

$$d_i = \begin{bmatrix} \cos(\psi + \alpha_i) \\ \sin(\psi + \alpha_i) \end{bmatrix}, \quad o = p_{\text{robot}}. \quad (5.10)$$

完整一圈像钟面一样闭合。 $N$  个方向之间有  $N$  个间隔, 末方向到首方向也算一段, 所以  $\Delta\alpha = 2\pi/N$ ,  $i$  从 0 到  $N-1$ 。部分视场  $F$  像一把展开的扇子,  $N$  个方向只隔出  $N-1$  段, 两端都要采样, 因此  $\Delta\alpha = F/(N-1)$ , 要求  $N \geq 2$ 。例如从  $-\pi$  开始取 4 束整周扫描, 方向是  $-\pi, -\pi/2, 0, \pi/2$ ; 取 3 束、 $90^\circ$  视场时, 从  $-45^\circ$  到  $45^\circ$  得到  $-45^\circ, 0, 45^\circ$ 。

```

29 double beamIncrement(const LidarConfig & config)
30 {
31     const bool full_turn = std::abs(config.fov - kTwoPi) <= 1e-12;
32     return full_turn ? kTwoPi / config.beams : config.fov / (config.beams - 1);
33 }
34
35 std::vector<float> scanCpu(const World2D & world, const Pose2D & pose,
36     const LidarConfig & config)
37 {
38     std::vector<float> ranges(config.beams);
39     const double increment = beamIncrement(config);
40     for (int i = 0; i < config.beams; ++i) {
41         const double angle = pose.yaw + config.angle_min + i * increment;
42         const Eigen::Vector2d direction{std::cos(angle), std::sin(angle)};
43         const double distance = raycastWorld(world, pose.position, direction);
44         if (distance < config.range_min) {
45             ranges[i] = std::numeric_limits<float>::quiet_NaN();
46         } else if (distance > config.range_max) {
47             ranges[i] = std::numeric_limits<float>::infinity();
48         } else {
49             ranges[i] = static_cast<float>(distance);
50         }
51     }
52     return ranges;
53 }
54 //

```

核心输出只是一组 `float` 距离, 便于与 ROS 分离测试。机器人状态与几何使用 `double`; `Laser-Scan` 用 `float32` 保存距离, 对应代码最后一次类型转换。每束都检查  $C$  个圆和  $E$  条边 (含世界四边),  $N$  束共需约  $N(C + E)$  次基本求交。因此计算量随束数、表面数近似线性增加, 常记作  $O(N(C + E))$ 。

## 5.5 量程、无效读数与采样时间

先求最近真实表面, 再判断它是否可测。近处障碍即使落在盲区内, 仍然挡住后面的墙; 所以整束应标为无效。默认最小量程 0.05 m、最大量程 10 m, 两个端点都包含。

| 情形       | ranges 中的值 | 后续建图含义    |
|----------|------------|-----------|
| 量程内命中    | 有限距离       | 光线清空、终点占据 |
| 量程内没有回波  | $+\infty$  | 只清空量程内光线  |
| 真实命中近于盲区 | NaN        | 整束跳过      |

NaN 表示无有效数值，参与大小比较时结果为假。消费扫描时，先区分有限回波、正无穷和 NaN，再按表中的含义更新地图。例如真正命中 10 m 处的墙，应标记终点占据；正无穷表示量程内无回波，只用于清空沿途。

**接入真实设备时的编码** REP-117 分别用  $-\infty$ 、NaN、 $+\infty$  表示过近、无效和无回波。课程将近距盲区统一编码为 NaN，建图时跳过整束；接入按 REP-117 输出的设备时，应在适配层单独识别  $-\infty$  的过近标记。

**snapshot 是一张快照。** 所有束使用同一时刻的机器人位姿，因此 `time_increment=0`。默认 10 Hz 的帧间周期为 0.1 s，所以 `scan_time=0.1`。header 记录采样时刻，`frame` 为 `laser`；`intensities` 为空。字段的完整说明见 [LaserScan 消息定义](#)。

模拟器默认每 0.005 s 推进一步，雷达每 20 步采样一次。`t=0` 有首帧，暂停期间不重复发布新扫描；碰撞冻结也不继续采样。`reset` 清理采样时序，在 `t=0` 生成新试验首帧。周期与步长对齐时，间隔步数为  $K = 1/(f_s \Delta t)$ ；10 Hz 和 0.005 s 给出  $K = 20$ 。本章的辅助函数 `lidarPeriodTicks` 用来检查这个整数条件。7 Hz 对应约 28.57 步，需要在步内计算采样状态；模拟器使用的这部分调度将在第 6 章展开。

**代码练习 ch05-2：**补全 `exercises/ch05/starter/beam_angle.hpp`，用 `check_angles.cpp` 检查 90/360/720 束与部分视场。理解题：如果盲区被写成  $+\infty$ ，后续建图会把哪里错误标为空闲？

## 5.6 在 RViz2 同时观察光束和命中点

停止上一章场景，构建并加载 `install` 后运行：

```
ros2 launch motion2d_bringup ch05.launch.py
# second terminal, from ros2_ws with the same ROS environment
/usr/bin/python3 ../scripts/check_ch05.py
```

默认沿第 04 章的解析圆参考运动。也可选择 `model:=ideal`，用 2D Goal Pose 手动移动，或选择 `velocity/inertial` 并发送相应控制量。修改 `motion2d_bringup/config/ch05.yaml` 后重启；早期章节默认不启用雷达。

`/scan` 使用 `SensorDataQoS (Best Effort, Volatile)`，RViz 的 `LaserScan` 显示也要匹配。显示辅助光束发布在 `/visualization/lidar`，按采样位姿固定在 `odom` 中，不会在两帧之间跟随当前机器人漂移。SLAM 从 `/scan` 的距离、角度及时间戳处理观测；辅助光束用来检查显示与几何。

**采样实验：**检查脚本会暂停、重置、逐步推进并核对 TF、角度、QoS 与光束数量。默认 19 步内没有第二帧，第 20 步的采样时间为 0.1 s；尝试改为 20 Hz 再手动核对间隔。暂停时晚加入的 `LaserScan` 订阅者不会收到历史帧，恢复运行后等待下一帧即可。

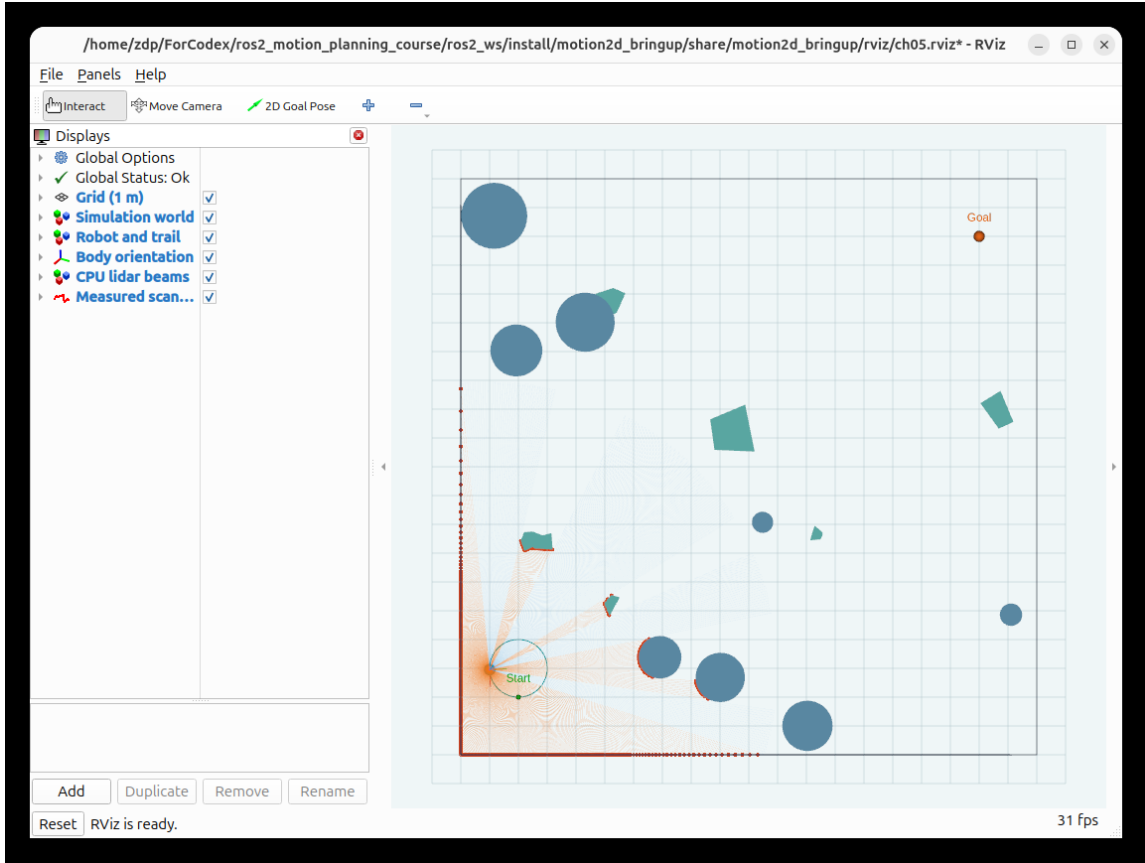


图 5.2: 关闭测距噪声时的扫描。橙色线是有效测距射线，红点是 LaserScan 命中点；淡蓝线只表示量程内无回波，障碍后的空间仍不可见。

## 5.7 高斯测距噪声：先有回波，再加误差

对真实量程内的有效回波  $r_i$ ，加入独立的零均值高斯误差：

$$\tilde{r}_i = r_i + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma_r^2). \quad (5.11)$$

$\sigma_r$  是每束、每次采样的标准差，单位 m，不是方差，也不是噪声密度。默认 `lidar.range_stddev=0.01 m`；设为零即可回到精确几何基准。这个模型用一个参数控制回波散布的宽度，适合先研究随机测距误差对地图的影响。

无回波的  $+\infty$  和盲区 NaN 保持原值。有效距离加噪后若越出量程，记为 NaN。例如 9.99 m 回波加 0.03 m 误差后超过 10 m，整束标为无效；若把它截成 10 m，建图就会在量程边界累积一圈错误终点。

### 零均值与标准差分别描述什么

反复测量同一距离，误差可能为正，也可能为负。记 E 为“重复很多次测量后的平均”这一理想操作， $\mu = E[\epsilon]$  称为期望。零均值模型取  $\mu = 0$ ，表示误差没有固定偏向。方差是偏离均值的平方的期望：

$$\text{Var}(\epsilon) = E[(\epsilon - \mu)^2] = \sigma_r^2. \quad (5.12)$$

先平方使正负误差都贡献非负大小,再平均描述散布程度;最后开平方得到标准差  $\sigma_r$ , 恢复到米的单位。高斯分布用均值和方差确定形状: 它关于均值对称, 越接近均值越常见, 偏离很远的误差越来越少。符号  $\mathcal{N}(0, \sigma_r^2)$  就指定了这个分布。代码用 `normal_distribution(0.0, range_stddev)` 抽样, 其中第二个参数是标准差。相当于先抽均值为零、标准差为 1 的高斯变量  $Z$ , 再计算  $\epsilon = \sigma_r Z$ :  $E[\epsilon] = \sigma_r E[Z] = 0$ ,  $E[\epsilon^2] = \sigma_r^2 E[Z^2] = \sigma_r^2$ 。

```

71 float perturbRange(float range, double error, const LidarConfig & config)
72 {
73     if (!std::isfinite(range)) {return range;}
74     const double measured = static_cast<double>(range) + error;
75     if (measured < config.range_min || measured > config.range_max) {
76         return std::numeric_limits<float>::quiet_NaN();
77     }
78     return static_cast<float>(measured);
79 }
80
81 void addRangeNoise(std::vector<float> & ranges, const LidarConfig & config, std::mt19937 & random)
82 {
83     if (config.range_stddev == 0.0) {return;}
84     std::normal_distribution<double> gaussian(0.0, config.range_stddev);
85     for (auto & range : ranges) {
86         if (std::isfinite(range)) {range = perturbRange(range, gaussian(random), config);}
87     }
88 }
89 //

```

几何函数与加噪函数分开, CPU 和未来 CUDA 可先比较无噪声距离, 再用同一组误差比较输出。`perturbRange` 接受给定误差, 便于这种成对实验。雷达使用独立随机流, `lidar.noise_seed=4242` 不会影响世界种子。`reset` 重置这个随机流; 同一配置、轨迹、采样顺序和标准库下可重放。改变束数会改变雷达的随机数消费顺序, 但不会重新生成障碍物。

**代码练习 ch05-3:** 补完 `exercises/ch05/starter/noisy_range.hpp`, 正确处理给定误差与特殊值。检查入口为 `check_noise.cpp`, 编译命令见本章习题 README。

## 5.8 从一批读数估计误差分布

为了观察散布, 先让所有真实距离固定在 5 m, 远离 0.05 m 和 10 m 的量程边界, 取 50000 个样本:

```
ros2 run motion2d lidar_noise_demo
```

先用已知真实距离减出误差  $e_j = \tilde{r}_j - r_j$ 。用  $M$  个误差的平均估计中心, 再计算它们相对这个中心的平方偏差平均, 得到本实验使用的样本标准差:

$$\bar{e} = \frac{1}{M} \sum_{j=1}^M e_j, \quad \hat{\sigma} = \sqrt{\frac{1}{M} \sum_{j=1}^M (e_j - \bar{e})^2}. \quad (5.13)$$

下面是 `seed=4242` 的演示数据, 比较设置值与从样本算出的值:

| 样本数    | 设定 $\sigma/\text{m}$ | 实测均值/ $\text{m}$ | 实测标准差/ $\text{m}$ |
|--------|----------------------|------------------|-------------------|
| 50,000 | 0.010                | 0.00004150       | 0.00995074        |

## 为什么均值误差随样本数的平方根缩小

每个误差独立、期望为零，因此  $E[e_i e_j] = E[e_i]E[e_j] = 0$  ( $i \neq j$ )。把样本均值的平方展开：

$$E[\bar{e}^2] = \frac{1}{M^2} E\left[\left(\sum_j e_j\right)^2\right] \quad (5.14)$$

$$= \frac{1}{M^2} \left( \sum_j E[e_j^2] + 2 \sum_{i < j} E[e_i e_j] \right) = \frac{M \sigma_r^2}{M^2} = \frac{\sigma_r^2}{M}. \quad (5.15)$$

由于  $E[\bar{e}] = 0$ ，这也是均值的方差。其标准差为  $\sigma_r/\sqrt{M}$ ，常称为**标准误**。样本数增加四倍，平均值的随机起伏约缩小一半；单个样本的标准差仍为  $\sigma_r$ 。本实验以五个标准误作为均值比较范围，方差与设定值的相对差用 3% 作为检查阈值。例如  $\sigma_r = 0.01$  m、 $M = 50000$  时，五个标准误约为 0.000224 m。

上面的方差估计使用  $M$  作分母。展开可得  $\hat{\sigma}^2 = \frac{1}{M} \sum e_j^2 - \bar{e}^2$ ，所以其期望是  $(1 - 1/M)\sigma_r^2$ 。将分母改为  $M-1$  可消除这个有限样本偏差；这里  $M$  很大，两者十分接近。靠近量程边界时，越界样本会被舍弃，例如靠近上限时正误差更容易消失，保留的误差就会向负侧偏移。这解释了实验为何选量程中间的 5 m。

还可以在 ROS 场景中做同样的计算。使用无障碍的 20 m 方形房间，把机器人静止放在中心。一束方向为  $\alpha$  的射线走  $t$  米后，位置为  $(t \cos \alpha, t \sin \alpha)$ 。碰到竖墙需  $t|\cos \alpha| = 10$ ，碰到横墙需  $t|\sin \alpha| = 10$ ，取较先到达的墙：

$$r(\alpha) = \min\left(\frac{10}{|\cos \alpha|}, \frac{10}{|\sin \alpha|}\right) = \frac{10}{\max(|\cos \alpha|, |\sin \alpha|)}. \quad (5.16)$$

停止上一场景，从 `ros2_ws` 运行下列实验：

```
ros2 launch motion2d_bringup ch05.launch.py model:=ideal rviz:=false \
  config:=src/motion2d_bringup/config/ch05_noise_lab.yaml
# second terminal, same ROS environment
/usr/bin/python3 ../scripts/check_ch05_noise.py
```

方向平行于某组墙时，把该分母为零的候选距离视为正无穷，另一组墙给出有限距离。实验使用 360 束、50 Hz、最大量程 20 m；脚本重置后收集  $t=0$  到 3.98 s 的 200 帧，逐束用上述墙距减出残差。房间对角方向最远约 14.14 m，全部墙面都落在量程内。

**理解与实验：**把标准差翻倍，预测方差变成几倍；把采样频率翻倍，解释为什么单个样本的设定标准差不变。修改后同步调整实验脚本中的预期参数，分别比较单样本散布和样本均值的变化。

## 5.9 从角间隔理解细小障碍的漏检

相邻光束在相同距离  $r$  处的端点形成一个等腰三角形。沿角平分线切开后，直角三角形的斜边为  $r$ ，对边为端点间距  $s$  的一半，因此  $s/2 = r \sin(\Delta\alpha/2)$ ，得到

$$s = 2r \sin \frac{\Delta\alpha}{2} \approx r\Delta\alpha. \quad (5.17)$$

近似式来自  $\sin x = x + O(x^3)$ ，适用于小角间隔，角度单位是弧度。距离越远，相同角分辨率对应的间距越大。以 5 m 处为例，90、360、720 束的整周扫描分别对应约 34.90、8.73、4.36 cm 的端点间距。增加束数能让边界更密，但有限束数仍可能漏掉细小物体。

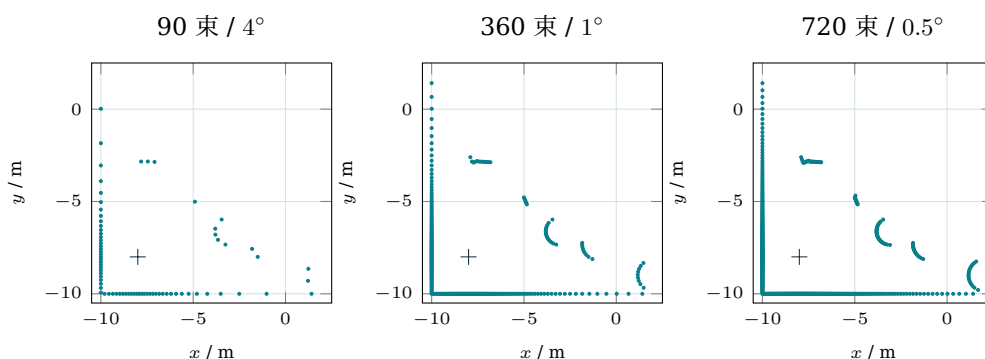


图 5.3: 同一 `seed=42` 世界、同一圆心  $(-8, -8)$ 、`yaw=0`、关闭噪声的有限回波。十字为雷达。点列之间的空白包含光束间隙和被遮挡区域。

再单独构造一个直径  $0.10\text{ m}$  的小圆：圆心距离雷达  $5\text{ m}$ ，方向为  $2^\circ$ 。雷达位于原点，扫描从  $-\pi$  开始。90 束的方向间隔是  $4^\circ$ ，恰好从小圆两侧穿过；360 束有一束命中，720 束有三束命中。从雷达向小圆作切线，直角三角形给出其半角宽度  $\gamma = \arcsin(0.05/5) \approx 0.573^\circ$ 。因而圆占据约  $[1.427^\circ, 2.573^\circ]$  的方向区间：4 度网格中没有方向落入，1 度网格只有 2 度，0.5 度网格则有 1.5、2、2.5 度。

| 束数  | 角间隔/度 | 小圆命中束数 |
|-----|-------|--------|
| 90  | 4     | 0      |
| 360 | 1     | 1      |
| 720 | 0.5   | 3      |

**代码练习 ch05-4:** 补完 `exercises/ch05/starter/beam_gap.hpp`，把角分辨率换算为米制端点间距。然后改变小圆的方向、尺寸或距离，比较是否命中。把小圆再移远，角宽会缩小；它可能重新落在两束之间。用角区间能预测这种漏检。

**实际 ROS 对照:** 复制 `ch05.yaml`，分别将 `lidar.beams` 改为 90、360、720，保持其余参数和世界 `seed` 相同。启动各配置后运行 `check_ch05.py --beams 90`（其余束数类推），核对消息与光束数量。

## 5.10 测量束数增加后的计算代价

从 `ros2_ws` 运行 CPU 基准：

```
ros2 run motion2d lidar_benchmark > ../textbook/data/ch05_benchmark.csv
ros2 run motion2d lidar_benchmark --points > ../textbook/data/ch05_hits.csv
# repository root
make -C textbook REVIEW_CHAPTER=5
```

程序使用 `seed=42` 的默认 8 圆、8 多边形世界和固定起点。每种束数先预热 100 帧，再测 1000 次 `scanCpu` 调用，用单调时钟统计中位数和 P95。例如把 1000 次耗时从小到大排序，中位数描述中间位置，P95 描述约第 950 个位置。前者反映典型耗时，后者帮助观察较慢的调用；两者会随后台负载发生变化。

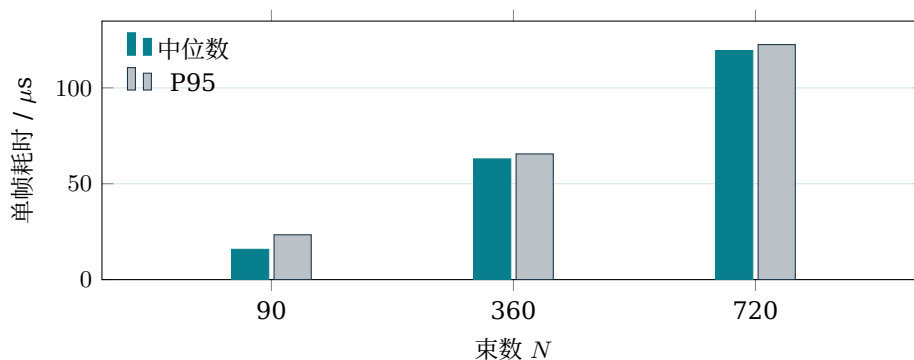


图 5.4: 本机 AMD Ryzen 7 9700X、GCC 15.2、RelWithDebInfo 下的一次测量。仅 CPU 几何扫描，噪声关闭；柱图直接读取实验 CSV。

| 束数  | 中位数/ $\mu s$ | P95/ $\mu s$ | 有限回波数 |
|-----|--------------|--------------|-------|
| 90  | 15.79        | 23.38        | 75    |
| 360 | 62.95        | 65.55        | 301   |
| 720 | 119.48       | 122.57       | 600   |

这里的计时范围是一次 `scanCpu` 调用：方向计算、表面求交、量程编码及结果数组分配。消息发布还会增加噪声生成、数据转换和传输等开销；可以为这些阶段分别加计时点，比较时间花在哪里。复测时记录 CPU、编译选项和系统负载，再观察束数翻倍后几何扫描耗时的变化。

第 20 章将用同一场景、位姿、束数和量程编码比较 CPU 与 CUDA。先对齐无噪声距离，再分别测几何计算、数据传输和完整发布过程，就能判断并行计算在何种规模下有收益。

## 第六章 机器人感觉到什么：IMU 与传感器时间

**目标** 从运动真值产生原始 IMU，区分比力、白噪声和采样时间。

**先修** 第二章的坐标变换、第四章的连续运动，以及第五章的传感器消息。

### 6.1 静止的加速度计为什么不为零

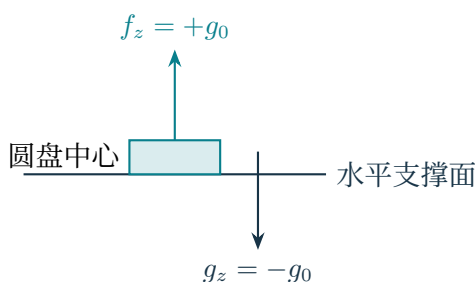
把一个加速度计平放在桌上，令它的  $z$  轴向上，读数约为  $+9.81 \text{ m/s}^2$ 。要理解这个读数，可以从牛顿第二定律出发。把重力以外的力（例如桌面的支持力、驱动力）合写为  $F_{\text{other}}$ ，则质量为  $m$  的物体满足

$$ma = mg + F_{\text{other}}, \quad \frac{F_{\text{other}}}{m} = a - g. \quad (6.1)$$

加速度计测量的正是右边这种**单位质量受到的非重力作用**，称为比力。它在自己的坐标轴上报告读数。设  $R$  将机体系向量转到世界系，转回机体系得到

$$f = R^T(a - g), \quad g = (0, 0, -g_0)^T, \quad g_0 = 9.81 \text{ m/s}^2. \quad (6.2)$$

静止时  $a = 0$ ，桌面支持力抵消向下的重力，因此向上的比力为  $g_0$ 。自由落体时  $a = g$ ，比力为零。这两个例子说明：运动加速度  $a$  与传感器读数  $f$  要通过重力联系起来。反过来求运动加速度，只需左乘  $R$  再加  $g$ ： $a = Rf + g$ 。仿真器用真值姿态生成读数；第九章的估计器用估计姿态将读数转回世界系。



本章 IMU 在圆心，与 `base_link` 同向。机器人只做水平平移和 `yaw` 转动，因此  $a_z = 0$ 、`roll=pitch=0`。陀螺仪理想输出  $(0, 0, \dot{\psi})$ ，单位 `rad/s`，加速度计理想输出的单位是  $\text{m/s}^2$ ，不是  $g$  的倍数。这一约定参考 [REP-145 的 IMU 报告约定（草案）](#)。

## 6.2 用三行公式读懂理想 IMU

二维部分只需要一个旋转转置：

$$\begin{bmatrix} f_x \\ f_y \end{bmatrix} = \begin{bmatrix} \cos \psi & \sin \psi \\ -\sin \psi & \cos \psi \end{bmatrix} \begin{bmatrix} a_x \\ a_y \end{bmatrix}, \quad f_z = g_0, \quad \omega_z = \dot{\psi}. \quad (6.3)$$

例如 yaw 为 90°、世界加速度为 (2, 0) 时，世界的 +x 方向正好是机体的 -y，故  $f = (0, -2, 9.81)$ 。

```

8 ImuSample idealImu(const State2D & state, double gravity)
9 {
10     ImuSample sample;
11     sample.angular_velocity.z() = state.yaw_rate;
12     sample.specific_force.head<2>() =
13         rotation(state.pose.yaw).transpose() * state.acceleration;
14     sample.specific_force.z() = gravity;
15     return sample;
16 }
17 //

```

ImuSample 只保存角速度与比力两个三维向量，算法不依赖 ROS。输入的 acceleration 成员由第四章的运动模型提供。reference 对解析轨迹求导，inertial 从力、质量和阻力求加速度，ackermann 还包含转弯产生的向心项。这三种模型都适合生成 IMU。瞬移位置或瞬间改变速度会使导数中出现突变；需要 IMU 的实验应选择上述连续运动模型。

### 6.2.1 为什么安装位置也影响读数

圆心原地自转时，圆心加速度为零；离圆心越远的点，转动的线速度越大。用  $r$  表示机体系中从圆心指向传感器的固定安装向量，传感器的世界位置为  $p_I = p_C + Rr$ 。定义把平面向量逆时针转 90° 的矩阵

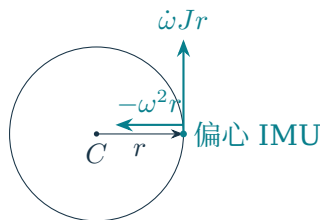
$$J = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}, \quad J^2 = -I.$$

逐项对第二章的旋转矩阵求导可得  $\dot{R} = \omega R J$ ，其中  $\omega = \dot{\psi}$ 。对  $p_I$  求两次导数，利用  $r$  不随时间变化，得到

$$v_I = v_C + \omega R J r, \quad (6.4)$$

$$a_I = a_C + \dot{\omega} R J r + \omega R (\omega J) J r = a_C + R(\dot{\omega} J r - \omega^2 r). \quad (6.5)$$

括号里的第一项沿切向，来自角速度变化；第二项指向转动中心，是向心加速度。例如  $r = (0.2, 0)$  m、 $\omega = 2$  rad/s、 $\dot{\omega} = 0$ ，传感器多测到机体系的  $(-0.8, 0)$  m/s<sup>2</sup>。本章配置  $r = 0$ ，这两个安装位置项都为零；圆心沿曲线运动时， $a_C$  本身仍可非零。



加速度计不能区分静止和匀速直线运动；两者的运动加速度都是零。原始 IMU 也不直接测量位置、速度或绝对 yaw。

## 6.3 先手算，再运行五个场景

重新构建工作空间、加载 install 后运行：

```
ros2 run motion2d imu_demo
colcon test --packages-select motion2d
colcon test-result --verbose
```

演示的 CSV 输出可与下表核对。角速度单位 rad/s，比力单位  $\text{m/s}^2$ 。

| 场景                                  | $\omega_z$ | $f_x$ | $f_y$ | $f_z$ |
|-------------------------------------|------------|-------|-------|-------|
| 静止                                  | 0          | 0     | 0     | 9.81  |
| 匀速 $(2, -1) \text{ m/s}$            | 0          | 0     | 0     | 9.81  |
| $a = (2, 0)$ , $\text{yaw} = \pi/2$ | 0          | 0     | -2    | 9.81  |
| 圆心原地自转                              | -0.3       | 0     | 0     | 9.81  |
| $r = 1$ , $\omega = 0.4$ 的圆参考       | 0.4        | 0     | 0.16  | 9.81  |

圆参考的机体 x 轴沿切向, y 轴朝路径圆心, 故  $f_y = r\omega^2$ 。这里的  $r$  是圆心运动路径的半径, 机器人外形半径仍由 `robot.radius` 配置。由第四章  $p = (r \cos \omega t, r \sin \omega t)$  求两次导数,  $a = -r\omega^2(\cos \omega t, \sin \omega t)$  指向路径圆心；将它投影到沿切向的机体 x 轴为零, 投影到向内的 y 轴为  $r\omega^2$ 。

**代码练习 ch06-1。**在 `exercises/ch06/starter/specific_force.hpp` 补全世界加速度到机体比力的变换, 从仓库根目录编译：

```
g++ -std=c++17 -I/usr/include/eigen3 \
-Iexercises/ch06/starter \
exercises/ch06/check_specific_force.cpp -o /tmp/ch06_force
/tmp/ch06_force
```

理解题：静止、匀速和原地自转有何区别？实验题：将路径  $r$  翻倍, 再单独将  $\omega$  翻倍, 预测和记录陀螺仪、加速度计的变化。验收包括重力符号、正负  $90^\circ$  旋转、 $180^\circ$  旋转；提示与参考解单独保存在练习目录。

**常见错误。**直接把世界加速度写入机体系消息；把  $g_z$  符号写反；将陀螺仪单位写成度每秒；认为“运动很快”就意味着加速度很大。

## 6.4 让每个采样点带上一点白噪声

沿用第五章的高斯噪声模型, 将理想值与随机误差相加：

$$\omega_m = \omega + n_g, \quad f_m = f + n_a. \quad (6.6)$$

每个轴、每个时刻各抽取一次独立误差,  $n_i \sim \mathcal{N}(0, \sigma_i^2)$ 。“白”描述不同时刻的误差互不相关；“高斯”描述单个误差的分布形状。本模型进一步取各轴、各时刻相互独立, 使实现只需六次独立抽样。若整段运动都加同一个常数, 得到的是偏置；附录 C 再研究这种会持续积累的误差。

```

31 ImuSample addImuNoise(ImuSample sample, const ImuNoise & noise, std::mt19937 & random)
32 {
33     std::normal_distribution<double> normal(0.0, 1.0);
34     for (int axis = 0; axis < 3; ++axis) {
35         if (noise.gyro_stddev[axis] > 0) {
36             sample.angular_velocity[axis] += noise.gyro_stddev[axis] * normal(random);
37         }
38         if (noise.accel_stddev[axis] > 0) {
39             sample.specific_force[axis] += noise.accel_stddev[axis] * normal(random);
40         }
41     }
42     return sample;
43 }
44 //

```

默认陀螺仪三轴标准差为 (.002, .003, .004) rad/s, 加速度计为 (.04, .05, .06) m/s<sup>2</sup>。这些是**每采样标准差**, 不同轴可以不同。世界、雷达、IMU 各自拥有随机引擎; 改变雷达束数不会推进 IMU 随机流。`reset` 重新设置 IMU 种子 6060。在同一标准库下, 保持配置、轨迹和采样顺序, 即可逐值重放同一组噪声。跨平台比较时, 更适合比较均值、标准差等统计量。

### 6.4.1 每采样标准差与连续噪声强度

改变采样率时, “保持噪声不变” 有两种含义。本章配置的是每个读数的标准差  $\sigma$ ; 提高频率会在同样一秒内获得更多个标准差相同的独立读数。另一种模型先规定噪声在一段时间内积分的方差。设长度为  $h$  的区间中, 噪声积分记为  $W_h$ , 定义强度  $q$  使

$$\text{Var}(W_h) = qh.$$

这个定义符合独立区间的方差相加: 两段各长  $h$  的积分合并后, 方差为  $2qh$ 。若传感器报告区间平均值  $n = W_h/h$ , 由  $\text{Var}(cX) = c^2 \text{Var}(X)$  可得

$$\sigma^2 = \text{Var}(n) = \frac{qh}{h^2} = \frac{q}{h}, \quad \sigma = \sqrt{q/h}. \quad (6.7)$$

这里  $q$  的单位是“测量单位的平方乘秒”。在这种区间平均模型中, 采样率翻倍使  $h$  减半, 每采样标准差变为原来的  $\sqrt{2}$  倍。使用硬件手册中的噪声密度时, 还需按手册的频谱和滤波约定换算  $q$ 。本章直接配置  $\sigma$ , 因此改频率后  $\sigma$  保持原值。

## 6.5 读懂 IMU 消息与协方差

话题 `/imu/data_raw` 使用 `sensor_msgs/Imu`, 帧为 `imu_link`。`linear_acceleration` 字段装比力, 不能仅凭字段名认为它已经去重力。姿态四元数用单位值占位, 协方差首项 -1 明确表示没有姿态估计。这遵循主机定义及[官方 Imu 消息约定](#)。

协方差描述多个误差怎样一起变化。设  $\mu_i = E[n_i]$ , 定义

$$C_{ij} = E[(n_i - \mu_i)(n_j - \mu_j)]. \quad (6.8)$$

当  $i = j$ , 这就是该轴方差  $\sigma_i^2$ ; 当  $i \neq j$ , 若两个误差常常同时为正或同时为负, 乘积的平均值就偏正。

本章误差独立且零均值，因此  $E[n_i n_j] = E[n_i]E[n_j] = 0$ ，得到

$$C = \begin{bmatrix} \sigma_x^2 & 0 & 0 \\ 0 & \sigma_y^2 & 0 \\ 0 & 0 & \sigma_z^2 \end{bmatrix}.$$

ROS 将矩阵按行摊平成九个数：第  $i$  行第  $j$  列放在  $3i + j$ ，索引从零开始。对角线  $i = j$  因而对应  $4i$ ，即代码中的 0、4、8；单位是测量单位的平方。

六个标准差均设为零可做理想实验。消息协议另有两个约定：协方差全零表示 **协方差未知**，首项为  $-1$  表示 **该测量未提供**。因此原始姿态字段的首项写  $-1$ ；使用零噪声实验数据的滤波器，则通过自身配置指定噪声模型。

```

9  sensor_msgs::msg::Imu toImuMessage(const ImuSample & sample, const ImuNoise & noise,
10  const builtin_interfaces::msg::Time & stamp)
11  {
12      sensor_msgs::msg::Imu message;
13      message.header.stamp = stamp;
14      message.header.frame_id = "imu_link";
15      message.orientation.w = 1.0; // Placeholder, never a heading measurement.
16      message.orientation_covariance[0] = -1.0;
17      message.angular_velocity.x = sample.angular_velocity.x();
18      message.angular_velocity.y = sample.angular_velocity.y();
19      message.angular_velocity.z = sample.angular_velocity.z();
20      message.linear_acceleration.x = sample.specific_force.x();
21      message.linear_acceleration.y = sample.specific_force.y();
22      message.linear_acceleration.z = sample.specific_force.z();
23      for (int axis = 0; axis < 3; ++axis) {
24          message.angular_velocity_covariance[4 * axis] = std::pow(noise.gyro_stddev[axis], 2);
25          message.linear_acceleration_covariance[4 * axis] = std::pow(noise.accel_stddev[axis], 2);
26      }
27      return message;
28  }
29  //

```

**代码练习 ch06-2：**补全 `starter/covariance.hpp`，检验三轴方差和非对角项。例如  $\sigma_z = .004$  rad/s 对应方差  $1.6 \times 10^{-5}$  (rad/s)<sup>2</sup>。

## 6.6 从数据观察噪声分布

先在 `ros2_ws` 运行独立实验：

```

ros2 run motion2d imu_noise_demo
ros2 run motion2d imu_noise_demo \
> ../textbook/data/ch06_noise.csv

```

使用 `seed=6060`、每轴  $N = 60000$  个误差样本。均值和总体标准差分别为

$$\bar{n} = \frac{1}{N} \sum_k n_k, \quad s = \sqrt{\frac{1}{N} \sum_k (n_k - \bar{n})^2}. \quad (6.9)$$

下面直接读取演示生成的 CSV；g 行单位为 rad/s，a 行为 m/s<sup>2</sup>。

| 轴  | 配置标准差 | 实测均值      | 实测标准差    |
|----|-------|-----------|----------|
| gx | 0.002 | -0.000025 | 0.002007 |
| gy | 0.003 | -0.000003 | 0.002990 |
| gz | 0.004 | 0.000002  | 0.003997 |
| ax | 0.04  | -0.000328 | 0.039941 |
| ay | 0.05  | 0.000163  | 0.050240 |
| az | 0.06  | -0.000462 | 0.060063 |

零均值指分布的期望，不要求有限样本均值恰好为零。均值的标准误差为  $\sigma/\sqrt{N}$ ，推导见第五章。以  $\sigma = .004$ 、 $N = 60000$  为例，标准误差约为  $1.63 \times 10^{-5}$  rad/s；可据此判断表中微小的非零均值是否合理。还可以计算  $N^{-1} \sum_k n_{x,k} n_{y,k}$  或相邻样本的乘积平均值，观察它们是否接近零。前者反映轴间相关性，后者反映时间相关性。

**真实 ROS 消息实验。**启动默认第 06 章场景后运行 `check_ch06.py`。它收集 200 Hz、 $t = 0$  到 10 s 的 2001 条圆参考消息。先减掉理想值 (0, 0, .4) 和 (0, .16, 9.81)，再检验残差；不要对包含运动的原始值直接求“噪声均值”。例如对 y 轴读数  $f_{m,y}$ ，先计算  $n_{a,y} = f_{m,y} - .16$ ，再对这些残差求均值和标准差。运动信号与噪声的分离，是读懂传感器实验的第一步。

## 6.7 零均值误差也会积成漂移

先固定已知姿态，只研究一个水平加速度轴和 yaw 角速度的误差，初始误差都为零。这样可以单独看清积分的作用；第九章再把姿态误差和位置误差联系起来。把每个噪声样本看作时长  $h$  内的常值，两次积分得到

$$\delta\psi_{k+1} = \delta\psi_k + n_{g,k}h, \quad (6.10)$$

$$\delta p_{k+1} = \delta p_k + \delta v_k h + \frac{1}{2}n_{a,k}h^2, \quad (6.11)$$

$$\delta v_{k+1} = \delta v_k + n_{a,k}h. \quad (6.12)$$

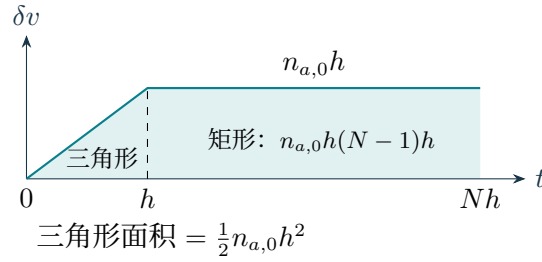
角度只积分一次。把递推式逐步展开，得到  $\delta\psi_N = h \sum_{k=0}^{N-1} n_{g,k}$ 。独立零均值误差的交叉项期望为零，故

$$\text{Var}(\delta\psi_N) = h^2 \sum_{k=0}^{N-1} \text{Var}(n_{g,k}) = \sigma_g^2 N h^2. \quad (6.13)$$

位置积分两次，每个样本的作用时间不同。第  $k$  个加速度误差在自己的区间内产生  $\frac{1}{2}n_{a,k}h^2$  位移误差；区间结束后留下  $n_{a,k}h$  的速度误差，它还会持续  $N - 1 - k$  个区间。两部分相加，得到

$$\delta p_N = h^2 \sum_{k=0}^{N-1} (N - k - \frac{1}{2})n_{a,k}. \quad (6.14)$$

下图只画第零个正误差的贡献：位置误差就是速度误差曲线下的面积。越早出现的加速度误差，留下的速度误差作用越久。



对位置误差求方差，将权重平方后相加，再令  $j = N - 1 - k$ ：

$$\text{Var}(\delta p_N) = \sigma_a^2 h^4 \sum_{j=0}^{N-1} (j + \frac{1}{2})^2. \quad (6.15)$$

这个求和可以直接化简。首尾配对得到  $\sum j = N(N-1)/2$ 。再将  $(j+1)^3 - j^3 = 3j^2 + 3j + 1$  从  $j = 0$  加到  $N-1$ ，左边相邻立方项相消，只剩  $N^3$ ，所以

$$\sum_{j=0}^{N-1} j^2 = \frac{N^3 - 3N(N-1)/2 - N}{3} = \frac{N(N-1)(2N-1)}{6}, \quad (6.16)$$

$$\sum_{j=0}^{N-1} (j + \frac{1}{2})^2 = \sum j^2 + \sum j + \frac{N}{4} = \frac{N(4N^2 - 1)}{12}. \quad (6.17)$$

写成总时长  $T = Nh$ ，结果更容易解释：

$$\text{Var}(\delta p_N) = \sigma_a^2 \left( \frac{hT^3}{3} - \frac{h^3T}{12} \right) \approx \frac{\sigma_a^2 hT^3}{3} \quad (N \text{ 较大}). \quad (6.18)$$

固定每采样标准差时，减小  $h$  会降低累计误差方差。若固定连续强度  $q$ ，则代入  $\sigma_a^2 = q/h$ ，得到  $q(T^3/3 - h^2T/12)$ ，在小步长下接近  $qT^3/3$ 。这说明比较频率前，必须确定保持的是哪一种噪声参数。

```

30     yaw += sample.angular_velocity.z() * dt;
31     position += velocity * dt + .5 * sample.specific_force.x() * dt * dt;
32     velocity += sample.specific_force.x() * dt;
33     //

```

```
ros2 run motion2d imu_noise_demo --drift
```

实验固定  $h = .005$  s、 $T = 10$  s、1000 次试验、seed=6060。两组复用同一随机序列，只把 sigma 倍率从 1 改为 2。表中为位置误差 RMS 与解析标准差，单位 m：

| $\sigma$ 倍率 | 实测位置 RMS | 解析位置标准差 |
|-------------|----------|---------|
| 1           | 0.053555 | 0.05164 |
| 2           | 0.10711  | 0.10328 |

RMS 是  $M$  次独立试验末端误差的均方根： $\text{RMS} = \sqrt{M^{-1} \sum_{i=1}^M (\delta p_N^{(i)})^2}$ 。由于理论均值为零，试验次数足够多时，RMS 接近上述方差的平方根。这里  $M = 1000$  是试验次数， $N = T/h = 2000$  是每次试验的积分步数。递推式对噪声是线性的，所以同一随机序列乘二后，每个末端误差和 RMS 都恰好乘二。

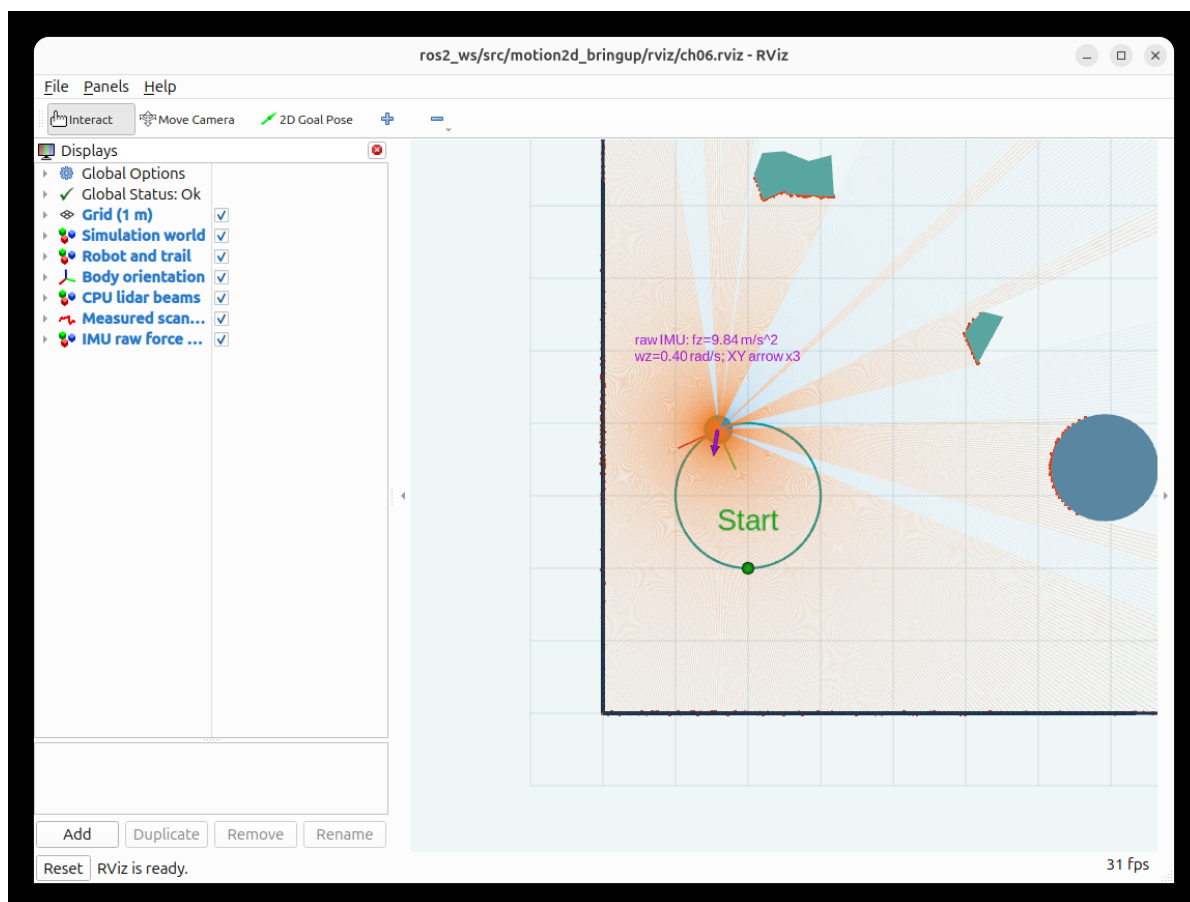
观察单次轨迹时，误差可能增大也可能回落；方差描述的是许多次试验的分散程度。**练习：**用  $N = 2$  手算  $\delta p_2 = h^2(\frac{3}{2}n_{a,0} + \frac{1}{2}n_{a,1})$ ，检验其方差与通式一致，再改变实验时长观察 RMS 的增长。

## 6.8 在 RViz 中同时观察激光与 IMU

从 `ros2_ws` 构建、加载 `install` 后运行，默认选择连续的圆参考：

```
ros2 launch motion2d_bringup ch06.launch.py
# Another terminal, same ROS environment
/usr/bin/python3 ../scripts/check_ch06.py
```

参数在 `motion2d_bringup/config/ch06.yaml`，改后重启。`imu.rate` 默认 200 Hz, `imu.noise_seed` 为 6060; `imu.gyro_stddev`、`imu.accel_stddev` 均为长度 3 的数组。启用 IMU 时选择 `reference`、`inertial` 或 `ackermann` 模型。静止实验可选 `inertial`，保持初速度为零且不施力。



紫色箭头显示实测水平比力，长度按  $1 \text{ m/s}^2 \mapsto 3 \text{ m}$  放大；标签显示竖直比力和  $z$  角速度。噪声会让箭头轻微抖动。Marker 最多 20 Hz，原始 IMU 默认 200 Hz，两者频率不同。显示节点用采样时刻的真值姿态放置这些 Marker，便于观察读数方向。后续 SLAM 的输入是原始 IMU 消息与激光消息。

**常见问题：**只收到激光而无 IMU，先检查本章配置及模型；暂停后新订阅者收不到旧 IMU 属于 Volatile QoS 行为，恢复运行再观察。检查程序会 `reset` 并最终暂停；继续观察时可恢复运行。重置开始新试验，估计器和显示程序也应清理旧试验的时间历史。

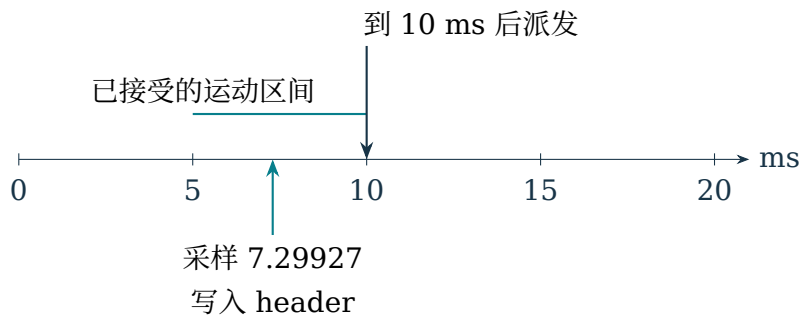
## 6.9 采样时间不是回调到达时间

一个 137 Hz 传感器的周期约为 7.29927 ms，无法写成 5 ms 仿真步长的整数倍。若强行每一个 tick 采样，它会变成 200 Hz；每两个 tick 则变成 100 Hz。本章为每个传感器保留独立的采样序号  $k$ ，共享同一条模拟时间轴：

$$t_k = \text{round}\left(\frac{10^9 k}{f_s}\right) \text{ ns}, \quad k = 0, 1, 2, \dots \quad (6.19)$$

推导只有两步：频率  $f_s$  表示每秒采样次数，所以第  $k$  次采样在  $k/f_s$  秒；乘  $10^9$  换成纳秒，再舍入到最近整数。每个时刻的舍入误差至多半纳秒。若先将 137 Hz 的周期舍入为 7299270 ns，再乘 1000，得到 7299270000 ns；直接计算第 1000 个时刻则为 7299270073 ns。前一种写法将每周期的误差累加了 1000 次。从序号直接计算可以保持长期相位，代价只是相邻纳秒间隔偶尔相差 1。

```
15 std::int64_t SensorScheduler::nextStamp() const
16 {
17     return std::llround(static_cast<long double>(index_) * 1000000000.0L / rate_);
18 }
19 //
```



采样、模拟器派发、订阅端墙钟到达，是三个不同的时间。本实现等一个 tick 通过运动与碰撞检查后派发。若步长为  $h$ ，采样时刻为  $t_s$ ，最早派发时刻为  $t_d = h \lceil t_s/h \rceil$ ，其中向上取整表示等到下一个 tick 边界。由  $t_s/h \leq \lceil t_s/h \rceil < t_s/h + 1$  得  $0 \leq t_d - t_s < h$ 。这描述模拟时间中的等待；墙钟到达还叠加计算、DDS 传输和排队所花的实际时间。因此估计器用 header 中的时间差积分，回调晚到不会改变测量对应的运动时刻。

**代码练习 ch06-3：**补全 starter/sample\_stamp.hpp。检查 200、7、137.5 Hz 以及一小时后的采样点，并与“先舍入周期再累加”比较。

## 6.10 先接受运动区间，再产生观测

设刚接受的运动区间为  $[t_a, t_b]$ 。传感器时刻  $t_s$  落在其中时，只用已经确定的区间输入计算  $x(t_s)$ ，然后生成测量。reference 直接计算绝对时刻的圆轨迹；inertial 和 velocity 从  $x(t_a)$  出发，用本区间保持的命令积分  $t_s - t_a$ ，复用第四章解析解。阿克曼模型同样从区间起点积分到采集时刻，位姿用 RK4，速度、转角与加速度由第四章模型给出。这样一条观测里的位姿和加速度来自同一段运动。

```

181 void publishSensors()
182 {
183     const auto now = clock_.nanoseconds();
184     std::int64_t last_transform = -1;
185     while (true) {
186         const auto never = std::numeric_limits<std::int64_t>::max();
187         const auto laser_time = lidar_schedule_ ? lidar_schedule_->nextStamp() : never;
188         const auto imu_time = imu_schedule_ ? imu_schedule_->nextStamp() : never;
189         const auto stamp = std::min(laser_time, imu_time);
190         if (stamp > now) {break;}
191         const auto sampled = acquisitionState(stamp);
192         publishTruth(sampled, stamp);
193         last_transform = stamp;
194         if (stamp == laser_time) {publishLidar(sampled, stamp); lidar_schedule_->advance();}
195         if (stamp == imu_time) {publishImu(sampled, stamp); imu_schedule_->advance();}
196     }
197     if (last_transform != now) {publishTruth(state_, now);}
198 }
199 //

```

代码先选两个采样网格中较早的时刻；同一时刻可同时生成 IMU 和激光。每个采样位姿先发布对应的真值 TF，最后补齐 tick 末端的 TF。这样按采样 stamp 查询得到该采集时刻的模拟位姿。暂停时仍重复当前 TF 供晚加入者显示，传感器采样序号保持不变。

**边界约定。**恰好在 tick 末端的 IMU 使用刚结束区间的加速度；晚到命令不能重写过去的测量。 $t=0$  是初始状态，之后施力影响之后的区间。瞬移模型只在 tick 末端切换位置，不虚构中间轨迹，也不允许启用 IMU。碰撞拒绝整个候选区间，状态、时间、传感器序号均停在上次接受的位置。

## 6.11 动手观察不对齐的采样网格

停止旧场景，用无噪声空房间配置运行：

```

ros2 launch motion2d_bringup ch06.launch.py rviz:=false \
  config:=src/motion2d_bringup/config/ch06_time_lab.yaml
# Another terminal, same ROS environment
/usr/bin/python3 ../scripts/check_ch06_time.py

```

配置为 200 Hz 物理、137 Hz IMU、7 Hz 雷达，90 束扫描、20 m 正方形房间。启动时暂停，检查脚本走 200 个 tick 到  $T = 1$  s。采样序号满足  $k/f_s \leq T$ ，即  $k \leq f_s T$ ；包含  $k = 0$  时，样本数为  $\lfloor f_s T \rfloor + 1$ 。由此可以先预测结果：

| 传感器          | 1 s 内样本数 | 首个非零 stamp/ns | 派发时刻/ms |
|--------------|----------|---------------|---------|
| IMU (137 Hz) | 138      | 7299270       | 10      |
| 激光 (7 Hz)    | 8        | 142857143     | 145     |

**解析检查。**reference 用圆轨迹核对比力、墙距和 TF。惯性实验的启动参数为 `model:=inertial`，检查脚本需加 `--model inertial`。其质量 2 kg、线性阻力 4 kg/s、转动惯量 0.04 kg m<sup>2</sup>、角阻力 0.02 kg m<sup>2</sup>/s。先静止 1 s，然后施加 (1, -0.3) N 与 0.01 N m。脚本按独立公式求出不断变化的运动，逐条比较比力、90 束墙距及采样位姿 TF。随后另起一次试验向墙运动，检查碰撞冻结。

**reset 开始新试验。**运动状态、命令、采样序号与随机引擎一起重置。时间回退时，消费者清理旧积分与历史；脚本重新建立 TF 监听器，从初态处理新数据。

## 6.12 阿克曼转弯时的 IMU 与采集时间

阿克曼模型提供一个同时检验运动学与采样时刻的例子。取纵向力  $F$ 、转向速率  $r_\delta$  恒定，质量  $m$ 、阻尼  $c > 0$ ，从静止、零转角起步。第四章给出  $\dot{s} = (F - cs)/m$ 、 $\omega = s \tan \delta / L$ ，以及  $a = \dot{s}e + s\omega e_\perp$ ，其中  $e$  沿车头， $e_\perp$  向左。转到机体系后  $R^\top e = (1, 0)^\top$ 、 $R^\top e_\perp = (0, 1)^\top$ ，所以纵向比力就是  $\dot{s}$ ，侧向比力就是  $s\omega$ 。代入积分分解可得

$$s(t) = \frac{F}{c} (1 - e^{-ct/m}), \quad \delta(t) = r_\delta t, \quad (6.20)$$

$$f_x(t) = \frac{F - cs(t)}{m}, \quad f_y(t) = \frac{s(t)^2 \tan \delta(t)}{L}, \quad (6.21)$$

$$\omega_z(t) = \frac{s(t) \tan \delta(t)}{L}, \quad f_z(t) = g_0. \quad (6.22)$$

这些式子在转角尚未达到限幅时成立。 $f_y = s\omega$  的单位为  $(\text{m/s})(1/\text{s}) = \text{m/s}^2$ ，左转时为正、右转时为负。

从仓库根目录、两个已加载 ROS/install 环境的终端运行，仅启动此实验：

```
ros2 run motion2d simulator_node --ros-args --params-file \
  ros2_ws/src/motion2d_bringup/config/ackermann_sensors.yaml
# Another terminal
/usr/bin/python3 scripts/check_ackermann_sensors.py
```

配置  $m = 1 \text{ kg}$ 、 $c = 0.15 \text{ kg/s}$ 、 $L = 0.3 \text{ m}$ ，物理步长  $0.005 \text{ s}$ ，IMU  $137 \text{ Hz}$ 、激光  $7 \text{ Hz}/720 \text{ 束}$ ，噪声关闭。检查器给定  $F = 0.8 \text{ N}$ 、 $r_\delta = 0.12 \text{ rad/s}$ ，单步 40 次到  $0.2 \text{ s}$ ，尚未超过命令有效期和转角界。

清除初始零时刻消息后，应收到  $\lfloor 0.2 \times 137 \rfloor = 27$  条 IMU。逐条以消息 stamp 代入上式，不能以收到消息时的 tick 代入；首个非零激光时刻为  $142857143 \text{ ns}$ ，也不落在  $5 \text{ ms}$  网格上。先用解析式核对速度、转角和 IMU，再将物理步长减半比较位姿变化，就能分别观察传感器公式与 RK4 积分精度。

**练习** 分别把 IMU 改成  $200 \text{ Hz}$ 、将力改为  $0 \text{ N}$ 、把转向速率改成负值，先推导样本数量、纵向和侧向项，再修改实验检查器的预期式。解释为什么零纵向力不总意味着零加速度：已有速度在有阻尼时仍会减速。最后恢复白噪声，减去每个采样时刻的解析信号，比较残差的均值与配置标准差。

本章从运动加速度构造比力，再给读数加入噪声与采样时间。下一章将把有时间戳的激光与位姿配对，逐束更新观测地图。

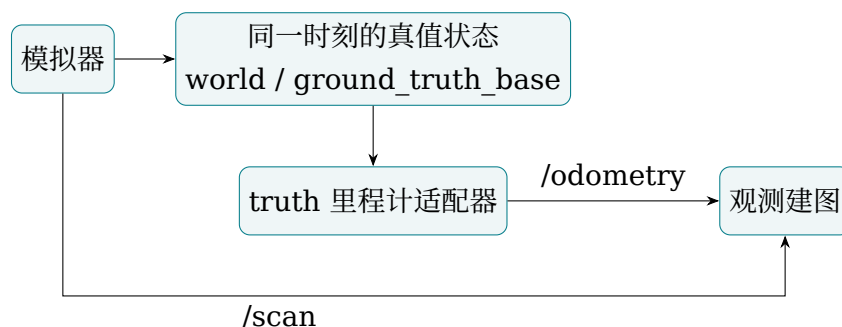
## 第七章 位姿已知时，如何从观测建图

**目标** 将定位与建图分开，用已知位姿解释激光观测，逐步得到规划可用的地图。

**先修** SE(2)、连续运动、激光编码，以及第六章的采样时间。

### 7.1 先给建图一个已知的位姿

机器人知道自己在哪里，怎样把一束距离读数变成地图上的一个格子？本章先用仿真真值提供位姿，把注意力集中在三个步骤：将回波投到统一坐标系，找出射线穿过的格子，再积累自由或占据证据。建图函数的输入始终只有扫描和位姿。后续章节估计出位姿后，可以继续使用这套建图接口。



`/ground_truth/odometry` 保留独立的真值帧标识，不直接建立对应 TF 边。适配器输出统一入口 `/odometry`，使用 `odom / base_link`。本章 `world`、`map`、`odom` 的原点与轴向相同，因此转换只需明确改写帧标识；例如 `world` 中的初始位置  $(-8, -8)$  m，在 `odom` 中仍为  $(-8, -8)$  m。若希望把初始位置设为 `odom` 原点，则应对所有位姿应用第二章的坐标变换，并同步修改 `map` 与 `odom` 的关系。帧名称告诉读者数值属于哪个坐标系，实际坐标转换由  $R, p$  完成。

| TF 边                                      | 唯一发布者                                |
|-------------------------------------------|--------------------------------------|
| <code>map → odom</code>                   | <code>truth_odometry</code> ，单位变换    |
| <code>odom → base_link</code>             | <code>truth_odometry</code> ，随采样状态变化 |
| <code>base_link → laser / imu_link</code> | <code>simulator</code> ，固定圆心外参       |

后续切换到激光里程计或 SLAM 时，对应节点接管这些 TF 边。一条 TF 边由一个节点发布，才能让查询得到的位姿具有一致的来源。

## 7.2 同一条里程计消息里的两种坐标系

依据主机与[官方 Odometry 定义](#)，pose 在 header.frame\_id，twist 在 child\_frame\_id。本章内部保存世界速度  $v_W$ ，写入消息前需要计算

$$v_B = R(\psi)^\top v_W, \quad \omega_B = (0, 0, \dot{\psi}). \quad (7.1)$$

例如  $\psi = \pi/2$ 、 $v_W = (2, 1)$ ，则  $v_B = (1, -2)$  m/s。这个变换只是在机体的两根轴上分解同一个速度向量。设机体前向单位向量  $e_x = (\cos \psi, \sin \psi)$ ，左向单位向量  $e_y = (-\sin \psi, \cos \psi)$ ，则

$$v_{B,x} = e_x^\top v_W = v_{W,x} \cos \psi + v_{W,y} \sin \psi, \quad v_{B,y} = e_y^\top v_W = -v_{W,x} \sin \psi + v_{W,y} \cos \psi.$$

这里表达的是机器人相对世界的速度，只是数值沿机体轴报告。与点坐标变换相比，速度向量无需平移：向量的起点移到哪里，都保持同样的长度和方向。

```

7  nav_msgs::msg::Odometry truthOdometry(
8      const State2D & state, const builtin_interfaces::msg::Time & stamp)
9  {
10     nav_msgs::msg::Odometry message;
11     message.header.stamp = stamp;
12     message.header.frame_id = "world";
13     message.child_frame_id = "ground_truth_base";
14     message.pose.pose.position.x = state.pose.position.x();
15     message.pose.pose.position.y = state.pose.position.y();
16     message.pose.pose.orientation.z = std::sin(state.pose.yaw / 2);
17     message.pose.pose.orientation.w = std::cos(state.pose.yaw / 2);
18     const Eigen::Vector2d body_velocity =
19         rotation(state.pose.yaw).transpose() * state.velocity;
20     message.twist.twist.linear.x = body_velocity.x();
21     message.twist.twist.linear.y = body_velocity.y();
22     message.twist.twist.angular.z = state.yaw_rate;
23     return message;
24 }
25 //
```

位姿、速度、时间都来自同一个 State2D。若分别订阅位置与速度，再按“最新一条”拼接，可能混合不同采样时刻，尤其在转弯时不一致。因此消息转换函数接收完整的 State2D，一次写入该时刻的所有字段。本章真值里程计把协方差设为零；估计里程计将在第九章传播并报告自己的误差协方差。

**练习 ch07-1：**补全 exercises/ch07/starter/body\_velocity.hpp。先手算正 90° 和 180° 旋转，再运行检查。思考圆参考中世界速度为何变化，机体前向速度却始终为  $r\omega$ 。

## 7.3 运行真值适配器并核对 TF

从 ros2\_ws 构建、加载 install 后运行：

```

ros2 launch motion2d_bringup ch07.launch.py rviz:=false
# Another terminal, same ROS environment
/usr/bin/python3 ../scripts/check_ch07_odometry.py
```

默认圆参考、200 Hz 物理。真值还覆盖非整数频率传感器的精确采样时刻。暂停期间里程计和 TF 可重复当前时间，方便晚加入的显示节点；重复时间戳不能算作新观测。检查脚本核对原始/选源消息、机体速度、TF 位姿、唯一时钟与唯一运动 TF 发布者。

```

31     auto selected = truth;
32     selected.header.frame_id = "odom";
33     selected.child_frame_id = "base_link";
34     geometry_msgs::msg::TransformStamped transform;
35     transform.header = selected.header;
36     transform.child_frame_id = selected.child_frame_id;
37     transform.transform.translation.x = selected.pose.pose.position.x;
38     transform.transform.translation.y = selected.pose.pose.position.y;
39     transform.transform.rotation = selected.pose.pose.orientation;
40     dynamic_->sendTransform(transform);
41     output_>publish(selected);
42     //

```

第七章 launch 关闭模拟器的 publish\_truth\_tf, 把运动 TF 交给适配器。运行前停止其他章节的场景, 使上述发布关系保持唯一。去掉 rviz:=false 可观察里程计箭头、机器人与扫描。

```

# Repository root: standalone exercise
g++ -std=c++17 -I/usr/include/eigen3 -Iexercises/ch07/starter \
    exercises/ch07/check_velocity.cpp -o /tmp/ch07_velocity
/tmp/ch07_velocity

```

**常见错误:** 世界速度直接写入 body twist; 把起点重定位为零却没同步地图; 同时打开模拟器与适配器的运动 TF。

## 7.4 从距离到二维点云

单束距离  $r_i$  与方向  $\alpha_i$  给出雷达系点:

$$p_i^L = \begin{bmatrix} r_i \cos \alpha_i \\ r_i \sin \alpha_i \end{bmatrix}, \quad p_i^O = R(\psi(t_s))p_i^L + p(t_s), \quad \alpha_i = \alpha_{\min} + i\Delta\alpha. \quad (7.2)$$

第一个式子来自直角三角形: 斜边长为  $r_i$ , 沿雷达 x、y 轴的投影分别是  $r_i \cos \alpha_i$  与  $r_i \sin \alpha_i$ 。第二个式子使用第二章的刚体变换, 先旋转再平移。 $t_s$  是整帧 snapshot 扫描的采集时间。圆心外参为单位变换, 因此 base\_link 与 laser 共用位姿; 偏置安装时按  $T_{OL} = T_{OB}T_{BL}$  复合外参与机器人位姿。例如  $r = 2$  m、 $\alpha = 0$ 、 $p = (3, 4)$  m、 $\psi = \pi/2$ , 变换结果为 (3, 6) m。

```

20 std::vector<Eigen::Vector2d> projectScan(const Scan2D & scan)
21 {
22     std::vector<Eigen::Vector2d> points;
23     points.reserve(scan.ranges.size());
24     for (std::size_t i = 0; i < scan.ranges.size(); ++i) {
25         const double range = scan.ranges[i];
26         if (!std::isfinite(range) || range < scan.range_min || range > scan.range_max) {continue;}
27         const double angle = scan.angle_min + i * scan.angle_increment;
28         points.emplace_back(range * std::cos(angle), range * std::sin(angle));
29     }
30     return points;
31 }
32 //

```

只有有限且量程内的回波才进入点云。 $+\infty$  表示“在量程内没有回波”, NaN 表示无效; 二者都没有表面端点。将无穷替换为最大量程并画成点, 会凭空造出一圈障碍物。

| 输出                | 坐标系   | 内容           |
|-------------------|-------|--------------|
| /cloud/scan       | laser | 当前帧有效回波      |
| /cloud/registered | odom  | 用采样位姿变换的同一批点 |

两种消息都使用 `PointCloud2` 并保留扫描时间。每点的 `x`、`y`、`z` 各用一个 `float32`，其中 `z=0`。消息只有一行 (`height=1`)，每点占用 12 字节 (`point_step=12`)。`is_dense=true` 表示输出不含无效点。`registered` 表示这些点已经用位姿转换到 `odom`，因而可以在同一张图中比较多帧观测。当前消息只包含一帧；第十章会保留关键帧，将多帧点云拼接成全局地图。

**练习 ch07-2:** 完成 `starter/scan_point.hpp`，用 `check_projection.cpp` 核对手算坐标。进一步说明：“没有点的地方”为什么不能直接标为空闲？点云只记录命中表面，射线才携带沿途自由信息。

## 7.5 把扫描与同一时刻的位姿配对

同一条话题内部按采样顺序发布，不代表不同话题的回调先后相同。`mapping` 使用两个小缓存，按**整数纳秒完全相等**匹配扫描与位姿。模拟器在每个扫描时刻输出一条真值状态，这使精确配对成为可能。例如扫描采于 0.100 s，而当前收到的位姿采于 0.080 s，机器人以 1 m/s 前进时，两者已相差 0.020 m；把旧位姿用于新扫描，会把整帧点云移错这段距离。因此 `poses_` 以时间戳为键保存位姿，`pending_` 保存待处理扫描，`processReady` 用 `poses_.find(scan->first)` 查找相同时刻。找到后，下面的代码才进行投影和坐标变换。

```

118     const auto points = motion2d::projectScan(scan->second.scan);
119     laser_cloud->publish(motion2d::toPointCloud(points, scan->second.header));
120     auto header = scan->second.header;
121     header.frame_id = "odom";
122     registered_cloud->publish(motion2d::toPointCloud(
123         motion2d::registerPoints(points, pose->second), header));
124     //

```

可以按到达顺序手动跟踪两个缓存：

| 新收到的消息     | 缓存中的对应关系      | 动作            |
|------------|---------------|---------------|
| 位姿 0.080 s | 尚无扫描          | 保存位姿          |
| 扫描 0.100 s | 缺少 0.100 s 位姿 | 保存扫描，等待       |
| 位姿 0.100 s | 找到同时间扫描       | 投影、建图、移除待处理扫描 |

两个缓存分别最多保存 1000 个位姿、20 帧扫描。后续已有匹配帧时，代码跳过并丢弃更早的未匹配扫描，让有效数据继续处理。同一正时间的扫描只处理一次；`t=0` 扫描开始新一轮地图。若接入的位姿资源不包含扫描时刻，则需要先插值得到  $T(t_s)$ ，再复用这套投影步骤。

```

# ch07.launch.py rviz:=false is running
/usr/bin/python3 ../scripts/check_ch07_cloud.py
# Separate terminals/domain: mapping node only
ROS_DOMAIN_ID=47 ros2 run motion2d mapping_node
ROS_DOMAIN_ID=47 /usr/bin/python3 ../scripts/check_ch07_join.py

```

后一个脚本按上表发送消息，还会交换扫描与位姿的到达顺序。观察两种顺序都生成相同点云，有助于理解：决定配对的是测量时间，而非回调顺序。默认场景脚本则用实际 `LaserScan` 与里程计逐点重算投影。

`reset` 将时间退回零并清空地图和旧缓存。等新试验的首帧扫描与位姿到齐后再恢复，消费者就可以从同一初始状态重新积累地图。

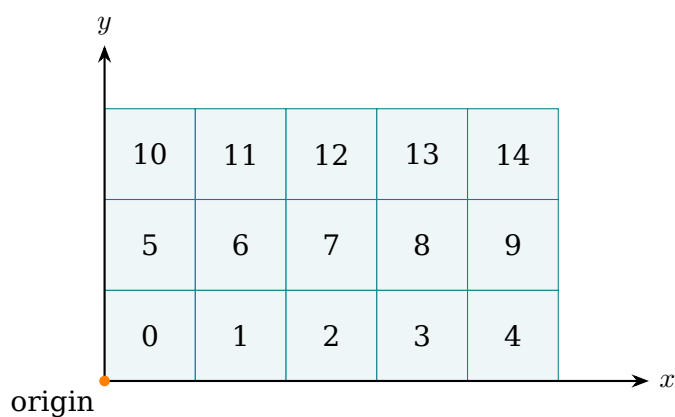
RViz 的 `Registered scan` 显示 `/cloud/registered`，橙色点应落在可见障碍表面附近。世界几何只是显示对照，可在 `Displays` 中关闭；`mapping` 通过 `/scan` 和 `/odometry` 获得全部建图输入。

## 7.6 格子有面积，未知也是一种状态

地图将连续空间切成边长  $\rho$  的正方形， $\rho$  的单位是 m/格。`origin` 是左下角格子的左下角，不是格心。对于地图内的位置  $p = (x, y)$ ,

$$i_x = \left\lfloor \frac{x - o_x}{\rho} \right\rfloor, \quad i_y = \left\lfloor \frac{y - o_y}{\rho} \right\rfloor, \quad j = i_y W + i_x. \quad (7.3)$$

先减原点得到相对距离，再除格长得到走过的格数，向下取整就落到当前格。 $W, H$  是横、纵格数。每完整走过一行跨过  $W$  个元素，因此一维索引为  $i_y W + i_x$ 。格心位于每个方向的半格处，坐标为  $o + \rho(i_x + \frac{1}{2}, i_y + \frac{1}{2})$ 。例如  $o = (-1, -1)$  m、 $\rho = .5$  m、 $W = 6$ ，位置  $(.2, -.1)$  m 对应  $(i_x, i_y) = (2, 1)$ 、 $j = 8$ ，格心为  $(.25, -.25)$  m。边界采用半开区间  $[o_x, o_x + W\rho) \times [o_y, o_y + H\rho)$ ；先检查边界，再取 `floor`，避免负数直接转整数时错误地朝零截断。



$W = 5$ : 存储索引不是米制坐标

默认地图是独立配置的  $220 \times 220$  格、 $\rho = 0.1$  m、`origin`= $(-11, -11)$  m。已知地图存储范围不等于已知障碍物；初始所有格子都未知。参数使用 `mapping.width_cells`、`height_cells`、`resolution` 与 `origin_x/y`。改变分辨率时若要保持物理范围，必须同时调整格数。

**三种含义** `unknown`: 还没有射线信息；`free`: 多次观测支持空闲；`occupied`: 回波支持占据。一片没有点云的区域，可能空旷，也可能被墙遮挡，不能把它全部刷白。

本章使用独立的 `observed` 掩码：尚未观察时输出 -1；已观察但概率恰为 0.5 时输出 50。前者表示没有观测，后者表示已有观测但正反证据相当。地图大小固定；若雷达中心走出存储范围，应扩大配置范围后重新建图。

## 7.7 DDA：沿射线依次走过哪些格子

DDA（数字微分分析）通过比较“下一次穿过哪条格线”遍历射线。将起点  $s$  到终点  $e$  的线段写成  $p(t) = s + t\Delta$ ，其中  $\Delta = e - s$ 、 $0 \leq t \leq 1$ 。这里  $t$  是无量纲的线段比例，与上一节的采样时间无关。设当前格为  $(i_x, i_y)$ ，若  $\Delta_x > 0$ ，下一条竖格线在  $x_b = o_x + (i_x + 1)\rho$ ；若  $\Delta_x < 0$ ，则在  $x_b = o_x + i_x\rho$ 。令  $s_x + t_x\Delta_x = x_b$ ，解得

$$t_x = \frac{x_b - s_x}{\Delta_x}. \quad (7.4)$$

越过一条竖格线后，下条格线沿行进方向再远  $\rho$ ，所以参数增加  $\delta t_x = \rho/|\Delta_x|$ 。同理有

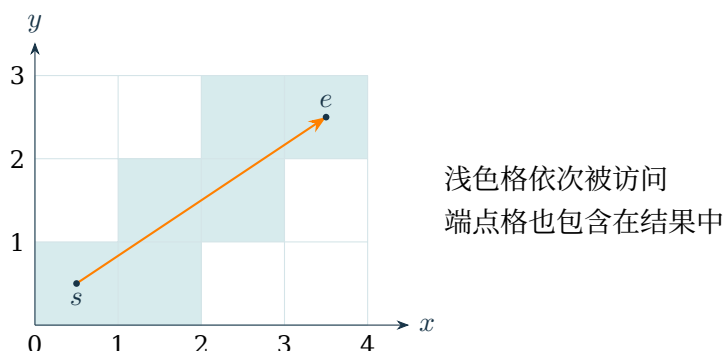
$$t_y = \frac{y_b - s_y}{\Delta_y}, \quad \delta t_x = \frac{\rho}{|\Delta_x|}, \quad \delta t_y = \frac{\rho}{|\Delta_y|}. \quad (7.5)$$

比较  $t_x, t_y$ ：较小者先到，只推进相应轴；然后给该轴参数加上  $\delta t$ 。若某轴  $\Delta$  为零，射线永远不会穿过该轴的下一条格线，将对应参数置为  $+\infty$ 。若两者相等，射线经过角点，同时推进两轴，只擦到角点的旁格不计入观测。

### 7.7.1 跟着一条射线手算六个格子

取单位格长、原点为零， $s = (.5, .5)$ 、 $e = (3.5, 2.5)$ ，则  $\Delta = (3, 2)$ 。第一次穿竖线为  $t_x = .5/3 = 1/6$ ，第一次穿横线为  $t_y = .5/2 = 1/4$ 。后续竖线每隔  $1/3$ ，横线每隔  $1/2$ ：

| 当前格    | 下一竖线参数 | 下一横线参数 | 下一格    |
|--------|--------|--------|--------|
| (0, 0) | 1/6    | 1/4    | (1, 0) |
| (1, 0) | 1/2    | 1/4    | (1, 1) |
| (1, 1) | 1/2    | 3/4    | (2, 1) |
| (2, 1) | 5/6    | 3/4    | (2, 2) |
| (2, 2) | 5/6    | 5/4    | (3, 2) |



```

81 while (x != end_x || y != end_y) {
82     // A negative-direction endpoint can lie exactly on a grid line.
83     // Do not step an axis past the endpoint's half-open cell.
84     if (x == end_x) {next_x = inf;}
85     if (y == end_y) {next_y = inf;}
86     if (next_x < next_y) {
87         x += step_x;
88         next_x += dt_x;
89     } else if (next_y < next_x) {
90         y += step_y;
91         next_y += dt_y;
92     } else { // Exact corner: advance both axes.
93         x += step_x; y += step_y;
94         next_x += dt_x; next_y += dt_y;
95     }
96     cells.push_back(y * config_.width + x);
97 }
98 //

```

代码中的 `step_x`、`step_y` 分别为  $-1, 0, 1$ ，表示格索引的变化方向。`next_x/y` 对应  $t_x, t_y$ ，`dt_x/y` 对应上述增量。循环先判断某轴是否已经到达终点格，抵达后把该轴下一次穿线参数设为无穷。例如负方向终点恰在格线上，半开区间规则可能让当前格已经包含终点；这时应停在该格。

### 7.7.2 回波落在地图外时怎么处理

起点在地图内时，射线会先穿过某个地图边界。求离开参数仍使用  $s_x + t\Delta_x = x_{\text{boundary}}$ ：向右检查右边界，向左检查左边界， $y$  方向同理。取所有离开参数与 1 中的最小值  $t_{\text{end}}$ ，将终点裁为  $s + t_{\text{end}}\Delta$ 。例如地图范围  $[0, 4) \times [0, 3)$ ， $s = (.5, .5)$ 、 $e = (6.5, 2.5)$ ，向右离开参数为  $(4 - .5)/6 = 7/12$ ，向上离开为  $(3 - .5)/2 = 5/4$ ，故裁剪点为  $(4, 5/3)$ 。遍历在右侧最后一列结束。

上边界和右边界不包含在半开地图内，代码用 `nextafter` 将裁剪坐标移到紧邻边界的内部浮点数，再求终点格。原回波在图外，图内格子获得自由证据；只有原回波确实落入图内，终点格才获得占据证据。

**练习 ch07-3：**补完 `starter/grid_traversal.hpp` 的循环。练习已准备好下一格线的参数，只考虑单位格长、原点为零、两个端点在图内；工程核心另外包含边界裁剪。使用 `check_traversal.cpp` 验证水平、竖直、正反对角线、同格和格线终点。先画格子手算访问次序，再运行参考解。

## 7.8 把射线变成概率证据

一条光束经过一个格子，支持“这个格子空闲”；回波落进格子，支持“这里有障碍”。噪声会让相邻帧互相矛盾，因此我们用概率积累证据，而非一次读数就永久决定颜色。

### 7.8.1 从条件概率到几率相乘

用  $O$  表示“该格占据”， $F$  表示“该格空闲”，两者概率和为 1。 $P(O | z)$  读作“已知观测  $z$  后，占据的概率”。条件概率定义为  $P(O | z) = P(O, z)/P(z)$ ，而同一个联合概率也可写成  $P(O, z) = P(z | O)P(O)$ 。

两式合并得到贝叶斯公式；对  $F$  做同样的变换，再相除：

$$\frac{P(O|z)}{P(F|z)} = \frac{P(z|O)P(O)/P(z)}{P(z|F)P(F)/P(z)} = \frac{P(z|O)P(O)}{P(z|F)P(F)}. \quad (7.6)$$

占据概率与空闲概率的比值称为**几率**。例如占据概率.75，对应几率  $.75/.25 = 3$ 。上式表示：新几率等于旧几率乘以本次观测的似然比。在给定格子状态后把各次观测视为独立，就可以逐次使用这一更新。

传感器模型更容易直接设置“看到一条证据后，格子有多大概率占据”。本章以先验  $p_0 = .5$  为基准，设置命中时  $p_{\text{hit}} = .7$ 、穿过时  $p_{\text{miss}} = .4$ 。这叫**逆传感器模型**：从观测推格子状态。将上面的几率公式应用到这一次证据，可解出似然比

$$\frac{P(z|O)}{P(z|F)} = \frac{p_{\text{evidence}}}{1 - p_{\text{evidence}}} \bigg/ \frac{p_0}{1 - p_0}. \quad (7.7)$$

除以先验几率，是为了每次更新只加入新证据的影响。

### 7.8.2 取对数，乘法就变成加法

定义对数几率  $L = \log[p/(1-p)]$ 。由于  $\log(ab) = \log a + \log b$ ，一次更新可写成

$$L_k = L_{k-1} + \log \frac{p_{\text{evidence}}}{1 - p_{\text{evidence}}} - L_0, \quad L_0 = \log \frac{p_0}{1 - p_0} = 0. \quad (7.8)$$

恢复概率也只需代数变形： $e^L = p/(1-p)$ ，于是  $p(1 + e^L) = e^L$ ，即

$$p = \frac{1}{1 + e^{-L}}. \quad (7.9)$$

命中增量为  $\log(.7/.3) \approx .847$ ，穿过增量为  $\log(.4/.6) \approx -.405$ 。两次穿过后  $L \approx -.811$ ，概率为  $1/(1 + e^{.811}) \approx .308$ ；两次命中后概率约为.845。一次命中与一次穿过的增量大小不同，因此它们相加后仍偏向占据。

### 7.8.3 让一帧扫描只贡献一次格子证据

相邻光束常常经过同一格，多束回波也可能来自同一表面。本章将一帧中的重复证据合并，并将每格  $L$  限制在  $[-4, 4]$ ：

$$L_k = \min(4, \max(-4, L_{k-1} + \Delta L)). \quad (7.10)$$

这是代码中的 `clamp`。上限避免长时间观测把置信度推得过高，也让后来的相反证据能较快改变地图。 $L = \pm 4$  对应概率约为.018 与.982。例如先把  $L$  积到 4，之后连续十帧穿过，该值就降到约  $-.055$ ，概率重新接近.5。

```

123 for (std::size_t i = 0; i < evidence.size(); ++i) {
124     if (evidence[i] == 0) {continue;}
125     const double increment = (evidence[i] & 2) ? hit_increment_ : miss_increment_;
126     log_odds_[i] = static_cast<float>(std::clamp(log_odds_[i] + increment,
127         -config_.logodds_limit, config_.logodds_limit));
128     observed_[i] = 1;
129 }
130 //

```

`evidence` 用两个二进制位保存本帧信息：最低位 1 表示穿过，下一位 2 表示命中。按位或 `|=` 合并后，共有四种值：0 为无信息，1 为仅穿过，2 为仅命中，3 为两者都有。`evidence[i] & 2` 检查

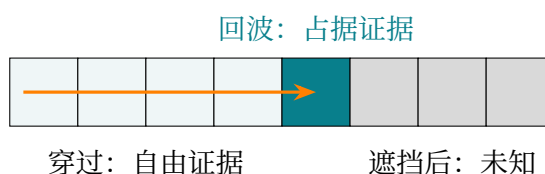
命中位，因此 2 和 3 都使用命中增量。先收集整帧证据再更新，使每帧每格最多加一次增量，且命中优先。这样多束激光共享的起点格不会仅凭一帧就迅速达到置信度下限。

这些概率参数控制证据积累的速度。实际相邻帧仍有相关性，调整参数时应观察地图对噪声和变化障碍的响应；后续更精细的传感器模型可以沿用同一套对数几率存储。

| 消息取值  | 本课程含义                | 本章评估阈值                             |
|-------|----------------------|------------------------------------|
| -1    | 尚未观测                 | unknown                            |
| 0-100 | 占据概率 $\times 100$ 取整 | $\leq 35$ free; $\geq 65$ occupied |
| 36-64 | 已观测但证据不足             | uncertain                          |

这些分类阈值由课程配置决定。第一次穿过得到 40，仍属于证据不足；第二次约为 31，才进入 free。后续规划默认将 unknown 与 uncertain 当作阻塞区域，第十二章再将地图转换成以米为单位的障碍距离。

## 7.9 有限回波、无回波和无效读数



```

107 for (std::size_t i = 0; i < scan.ranges.size(); ++i) {
108     const double range = scan.ranges[i];
109     const bool no_return = std::isinf(range) && range > 0;
110     const bool hit = std::isfinite(range) && range >= scan.range_min && range <= scan.range_max;
111     if (!hit && !no_return) {continue;}
112     const double angle = pose.yaw + scan.angle_min + i * scan.angle_increment;
113     const double length = no_return ? scan.range_max : range;
114     const Eigen::Vector2d end = pose.position +
115         length * Eigen::Vector2d(std::cos(angle), std::sin(angle));
116     const auto hit_cell = hit ? cellIndex(end) : std::nullopt;
117     for (const int cell : rayCells(pose.position, end)) {
118         evidence[cell] |= (hit_cell && cell == *hit_cell) ? 2 : 1;
119     }
120 }
121 //

```

有效有限距离：终点格为占据，之前的格子为空闲证据，不越过回波继续更新。 $+\infty$ ：沿方向清到 range\_max 所在格，不产生占据端点。NaN、负无穷或量程外有限值：整条射线跳过，不清自由区。这保留了第五章近距遮挡与噪声越界的约定。清空从圆心开始，适用于本课程“过近命中记 NaN”的模拟器；真实设备的盲区与异常编码需由适配层重新确认。

一条射线只约束它穿过的格子；噪声也可能让端点前后摆动，所以障碍边界会变厚。遮挡后的格子保持 unknown，等待机器人换个位置再观测。

/map 在每帧成功更新后发布，使用 map 帧、扫描采样时间、可靠且 transient-local 的深度 1 队列。晚打开 RViz 也能读到暂停时的最后地图。reset 的首帧先清空概率与 observed 掩码，再插入 t=0 观测；map\_load\_time 记录本轮首次插入的扫描时间。

## 7.10 在 RViz 中检查“看见了什么”

```
ros2 launch motion2d_bringup ch07.launch.py
# Another terminal, same ROS environment
/usr/bin/python3 ../scripts/check_ch07_map.py
```

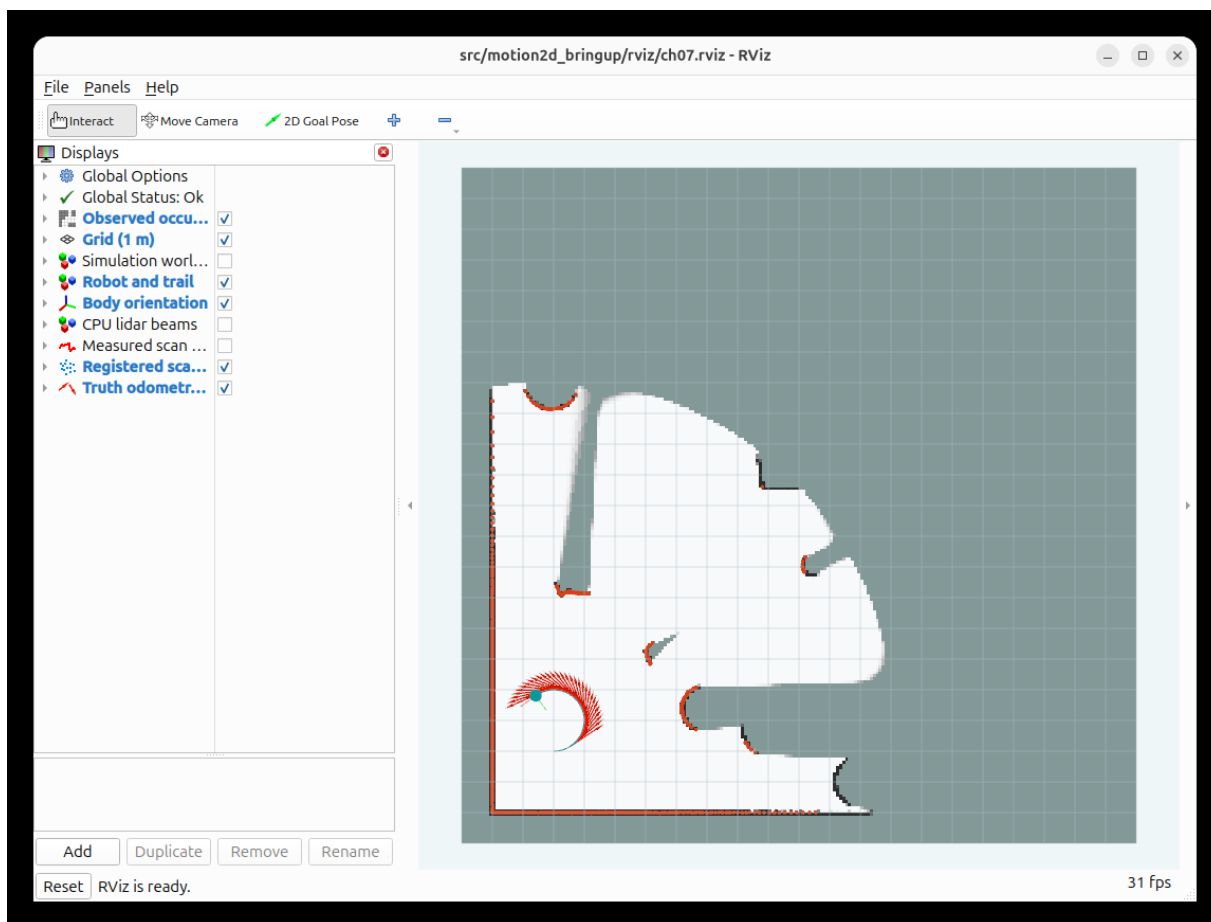


图 7.1: RViz2 中的观测地图。浅色是已观测自由域，深色轮廓是占据证据，灰色是未知。橙色点为当前回波，箭头表示真值里程计位姿。

一米半径的圆参考覆盖起点附近视野。右上方目标周围仍未知，因为激光尚未照到那里。打开 **Simulation world** 可对照形状，但该显示开关不影响 **mapping** 的输入。

## 7.11 用几何参考评价观测地图

为了量化建图结果，**mapping\_demo** 在观测图更新结束后，调用 **rasterizeTruth** 将完整环境几何栅格化，生成一张参考图。参考图可以回答“这个格子是否接触真实障碍”；观测图则回答“当前扫描对这个格子提供了什么证据”。对照两张图，既能检查已观测区域，也能看到仍未覆盖的地方。

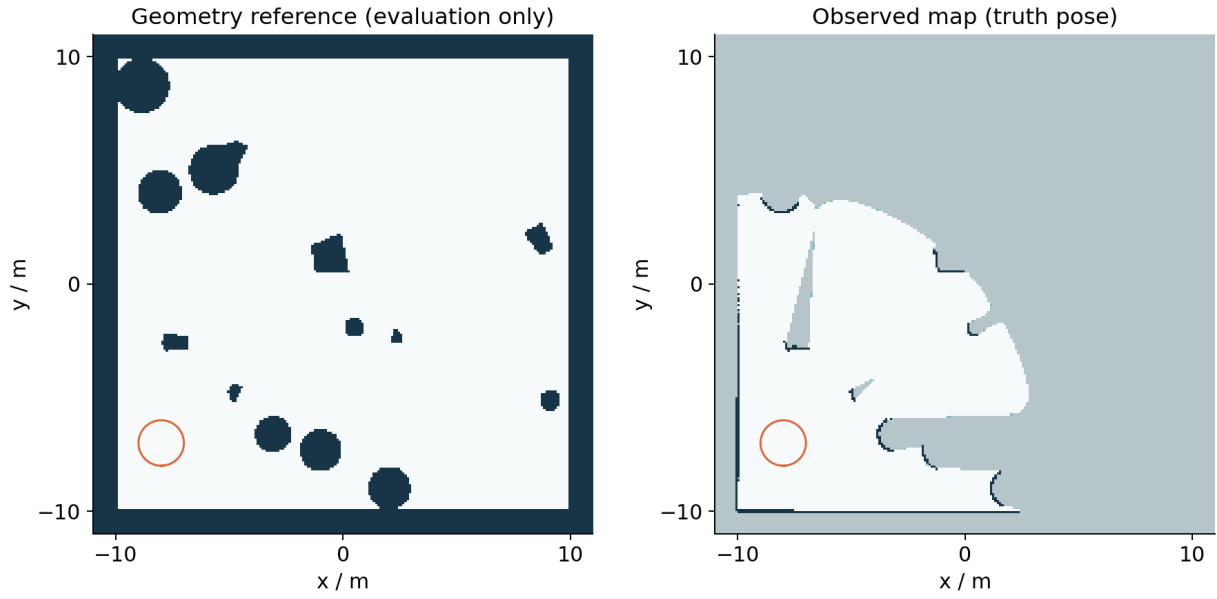


图 7.2: 相同 0.1 m 分辨率, 左侧为独立几何参考, 右侧为 160 帧无噪声观测。灰色未知没有被补全; 橙线是采样轨迹。参考图不作为建图输入。

参考格子只要与实体或房间边界接触就记占据。对于圆, 设格子范围为  $[l_x, u_x] \times [l_y, u_y]$ , 圆心为  $c_o$ 。最小化到格子的平方距离  $(x - c_x)^2 + (y - c_y)^2$  时, 两轴可以分别求最小值: 若  $c_x$  在区间内就取  $x = c_x$ , 否则取最近端点,  $y$  同理。因此

$$x_* = \min(u_x, \max(l_x, c_x)), \quad y_* = \min(u_y, \max(l_y, c_y)), \quad \|c - (x_*, y_*)\|^2 \leq r^2 \quad (7.11)$$

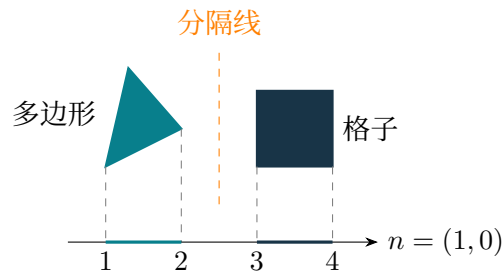
是圆与格子接触的判据。它对应代码的逐轴 `cwiseMax(low).cwiseMin(high)`。

对于凸多边形, 使用**分离轴检验**。沿方向  $n$  的投影是标量  $n^\top p$ ; 对全部顶点取最小和最大投影, 就得到图形在该轴上的区间。若两个区间不相交, 这两个图形显然也不相交。例如多边形投影为  $[1, 2]$ 、格子为  $[3, 4]$ , 位于中间的投影值 2.5 就能画出一条将二者分开的直线。

把多边形顶点记为  $a_i$ , 格子顶点记为  $b_j$ , 上述判断写成

$$\max_i n^\top a_i < \min_j n^\top b_j \quad \text{或} \quad \max_j n^\top b_j < \min_i n^\top a_i.$$

下图沿  $x$  轴投影即可分离。若  $x$ 、 $y$  两轴投影都重叠, 则继续检查斜边的法向。



为什么检查各边的法向就够了? 先想象两个凸多边形分离, 把分隔线平移到刚好碰到一个图形, 再绕接触顶点转动; 在仍能分隔的角度范围端点, 线会贴到某一条边, 或同时碰到两个顶点。后一种情形可换到另一侧的支撑线继续转动, 最终由某条边确定分隔方向。因此只需检查两个图形全部边的法向。<sup>1</sup> 实现收集方格的  $(1, 0)$ ,  $(0, 1)$  与多边形每条边  $e$  的垂直向量  $(-e_y, e_x)$ , 逐轴比较投影区间。任一

<sup>1</sup>可参阅 David Eberly 的**分离轴方法**第 2-3 节中的投影与接触图示。

轴分离就返回无接触；全部区间重叠或刚好相触就记占据。因为  $e^\top(-e_y, e_x) = 0$ ，该方向确实垂直于边。将方向乘正数只会同比例缩放两个投影区间，故比较区间时无需归一化。

评估先记录 **observed** 比例，再排除 **unknown** 与 **uncertain**，仅对有明确判断的格子统计 TP、FP、FN、TN。TP 是正确的占据预测，FP 是把参考自由格判为占据，FN 是把参考占据格判为自由，TN 是正确的自由预测。

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{recall}_{\text{evaluated}} = \frac{\text{TP}}{\text{TP} + \text{FN}}. \quad (7.12)$$

**precision** 的分母是“所有被判为占据的格子”，所以它回答这些判断中有多少正确。**recall** 的分母是“评估区域中真正占据的格子”，所以它回答其中有多少被找到。例如 TP=8、FP=2、FN=4、TN=6，则  $\text{precision}=8/10 = .8$ ， $\text{recall}=8/12 \approx .667$ 。若分母为零，该比例没有定义，记录为 NaN。

这里先排除 **unknown** 与 **uncertain**，所以指标描述的是已有明确判断的区域。还应同时报告观测比例  $N_{\text{observed}}/(WH)$ ，它包含已观测但尚不确定的格子。例如 100 格中只观测 20 格，即使这 20 格全部判断正确，整张地图仍有 80 格等待探索。

一个格子可能一角接触障碍，另一部分仍可被光束穿过。参考图按“整格是否接触”记占据，观测图却可能积累自由证据；因此边缘格会出现 FN，即使位姿和量程都精确。这个差异在分辨率较粗时更明显。

## 7.12 实验一：分辨率、噪声与评价指标

固定世界  $\text{seed}=42$ 、8 圆/8 多边形，沿一米圆参考运动，在  $t = 0, 0.1, \dots, 15.9$  s 采集 160 帧、每帧 720 束、最大量程 10 m。噪声  $\text{seed}=4242$ ， $\sigma = 0/0.05$  m。每组噪声的扫描只生成一次，在三种分辨率上复用；角度和量程采用与 LaserScan 一致的 float32 元数据。

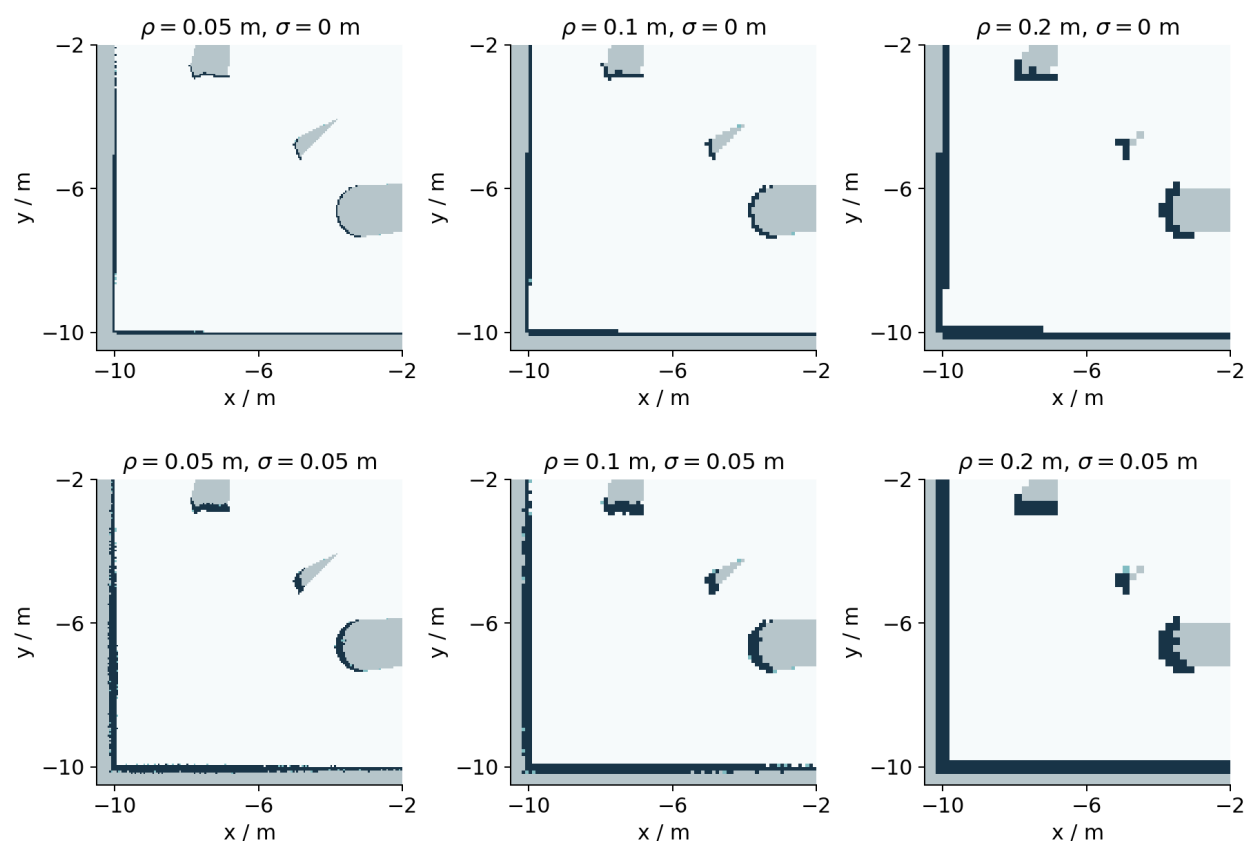


图 7.3: 相同位置的局部放大。白色自由、青色不确定、深色占据、灰色未知。上排无噪声, 下排  $\sigma = 0.05$  m; 列方向分辨率逐渐变粗。

地图始终覆盖  $22 \times 22$  m; 格数依次为  $440^2, 220^2, 110^2$ 。分辨率减半会使二维格数变为四倍, 但不会增加雷达原本没有测到的信息。图中边界随着分辨率和噪声变厚, 细格也会保留角采样造成的间隙。

| $\rho/\text{m}$ | $\sigma/\text{m}$ | 观测比例  | precision | 局部 recall |
|-----------------|-------------------|-------|-----------|-----------|
| 0.05            | 0                 | 0.275 | 1         | 0.63      |
| 0.1             | 0                 | 0.281 | 1         | 0.667     |
| 0.2             | 0                 | 0.29  | 1         | 0.715     |
| 0.05            | 0.05              | 0.281 | 0.977     | 0.807     |
| 0.1             | 0.05              | 0.287 | 0.983     | 0.823     |
| 0.2             | 0.05              | 0.295 | 0.988     | 0.846     |

完整 CSV 同时保存不确定格数与四项计数。噪声可能把端点推到真实表面之后, 增加那里积累的占据证据, 使参考障碍中的更多格子被判占据, 于是 recall 上升。与此同时, 也可能把本来空闲的格子判为占据, 使 precision 下降。因此要把观测比例、两项指标与地图图像放在一起看。

## 7.13 实验二：墙到底变厚了多少米

用空房间隔离形状与遮挡因素：右墙位于  $x = 4.03$  m，雷达静止在原点，161 束覆盖  $[-0.4, 0.4]$  rad、量程 6 m。每组 160 帧、seed=4242，比较  $\sigma = 0, 0.05, 0.10$  m。4.03 避开格线，减少“真实墙恰落在两个格子之间”的量化混淆。

在格心  $y \in [-1, 1]$  m 的每一行，取  $x \in [3, 5]$  m 内被判占据的首末格，计算跨度，再对有占据的行求平均：

$$w_y = (i_{x,\max} - i_{x,\min} + 1)\rho, \quad \bar{w} = \frac{1}{N_{\text{rows}}} \sum_y w_y. \quad (7.13)$$

例如首末格索引为 3 和 5，共跨过第 3、4、5 三格，因此长度为  $3\rho$ ；公式中的 +1 正是将格心间距补成完整格子的跨度。跨度包括中间间隙。没有占据格的行不参加平均，所以表中同时列出有效行数，便于分辨“占据带较窄”与“部分行没有检出墙”。

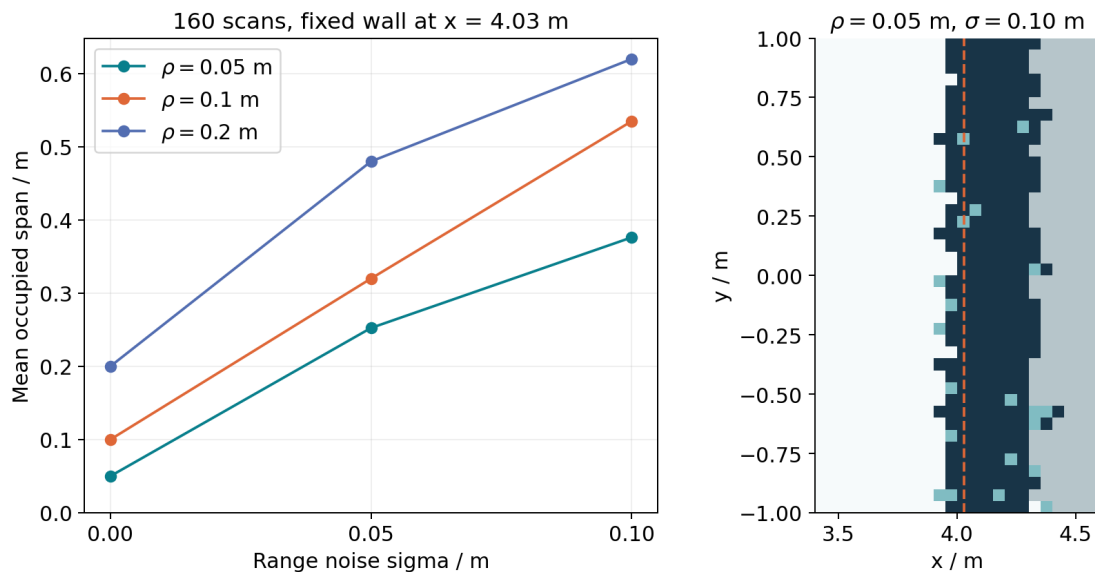


图 7.4: 左：实测平均占据跨度。右：细格、大噪声的局部地图，虚线为真实墙面。简化逆传感器模型会生成偏厚、偏移的占据带。

| $\rho/\text{m}$ | $\sigma/\text{m}$ | 平均跨度/m | 有效行数 |
|-----------------|-------------------|--------|------|
| 0.05            | 0                 | 0.05   | 40   |
| 0.1             | 0                 | 0.1    | 20   |
| 0.2             | 0                 | 0.2    | 10   |
| 0.05            | 0.05              | 0.253  | 40   |
| 0.1             | 0.05              | 0.32   | 20   |
| 0.2             | 0.05              | 0.48   | 10   |
| 0.05            | 0.1               | 0.376  | 40   |
| 0.1             | 0.1               | 0.535  | 20   |
| 0.2             | 0.1               | 0.62   | 10   |

无噪声时，本实验占据带是一格厚；0.1 m 分辨率下，噪声从 0 增至 0.05 m 时，平均跨度由 0.10 增至 0.32 m。实际墙面仍在  $x = 4.03$  m，占据带的宽度来自有限格长、端点噪声与证据积累。细格能表示更多空间细节，但仍保留测距误差；进阶时可用沿射线分布的命中概率代替“只有一个终点格命中”的模型，再比较占据带的变化。

## 7.14 复现实验与本章练习

```
# Repository root, after sourcing the ROS install environment
ros2 run motion2d mapping_demo tmp/ch07_eval
/usr/bin/python3 scripts/plot_ch07.py tmp/ch07_eval textbook/figures
# Exercise ch07-4 (replace starter with solutions for reference)
g++ -std=c++17 -Iexercises/ch07/starter \
    exercises/ch07/check_metrics.cpp -o /tmp/ch07_metrics
/tmp/ch07_metrics
```

绘图使用主机 Matplotlib/NumPy。CSV 保存在实验目录；教材表格读取 textbook/data/ch07\_mapping.csv 与 ch07\_wall.csv。PGM 文件是紧凑实验数据，255 表示未知，其余为概率百分比，绘图按物理 y 轴向上显示。

```
77 for (std::size_t i = 0; i < observed.size(); ++i) {
78     const int value = observed[i];
79     if (value < -1 || value > 100 || truth[i] > 1) {
80         throw std::invalid_argument("invalid occupancy encoding");
81     }
82     if (value == -1) {continue;} // Unknown is not a correct free prediction.
83     ++metrics.observed;
84     if (value > 35 && value < 65) {++metrics.uncertain; continue;}
85     if (value >= 65) {
86         if (truth[i]) {++metrics.true_positive;}
87         else {++metrics.false_positive;}
88     } else {
89         if (truth[i]) {++metrics.false_negative;}
90         else {++metrics.true_negative;}
91     }
92 }
93 //
```

1. **理解：**为什么已知定位仍会得到不完整地图？为什么更大的观测比例或 recall 不一定代表更好？
2. **代码 ch07-4：**完成 starter/observed\_metrics.hpp。必须先排除 -1 与 36-64，再统计四项计数；35/65 是包含端点的阈值。
3. **实验：**保持世界、轨迹、量程、束数与扫描序列相同，仅改变分辨率。报告格数、观测比例、四项计数、不确定数量与米制墙厚。再改变噪声；说明表面变厚如何影响精度与召回率。

本章已经把扫描与位姿连接到观测地图。下一章保留这个接口，研究怎样从相邻扫描的几何关系估计位姿，使机器人在未知自身运动时也能逐步建图。

## 第八章 让两帧扫描对齐：激光里程计

**目标** 从最近邻和最小二乘理解二维配准，再把相对运动积累成里程计。

**先修** 第 01-07 章。先实现只接收两组点的配准内核。

### 8.1 先对齐两组已知形状的点

在 `ros2_ws` 加载环境后运行：

```
ros2 run motion2d scan_matcher_demo
```

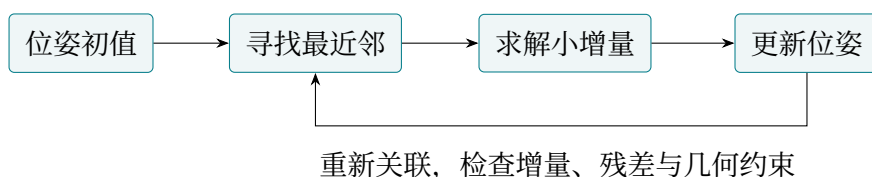
演示先造两面互相垂直的墙，再用已知位姿  $t = (0.12, -0.08)$  m、 $\psi = 0.04$  rad 生成当前帧。配准函数根据两组点和一个位姿初值，尝试找回这个变换。输出包括两种残差的状态、位姿、均方根误差 (RMSE)、关联数与迭代次数。真实雷达点还会受遮挡、噪声和视角变化影响。实验同时使用零初值，以及  $(0.11, -0.075, 0.038)$  的邻近初值。零初值下，点到点结果约为  $(0.1030, -0.0865, 0.0474)$ ：虽然报告收敛，仍有约 1.8 cm 的位置误差。邻近初值能恢复本例答案，点到线则在两个初值下都成功。这个例子说明规则采样也可能形成错误的局部极小值。

设当前帧点为  $p_i$ ，参考帧点为  $q_j$ 。我们求的是  $T_{\text{reference} \leftarrow \text{current}}$ ：

$$e_i = R(\psi)p_i + t - q_{j(i)}, \quad j(i) = \arg \min_j \|R(\psi)p_i + t - q_j\|. \quad (8.1)$$

$j(i)$  表示与第  $i$  个当前点配对的参考点编号； $\arg \min$  表示取距离最小时的编号。残差  $e_i$  是“变换后的当前点减参考点”，单位为米；对齐得越好，残差越接近零。ICP（迭代最近点）交替做两件事：先用当前位姿寻找最近邻，再固定对应关系求位姿的小修正。新的位姿会改变最近邻，因此修正后要重新关联。

例如在一维中，参考点位于 1 m 和 2 m，当前点实际对应 1 m；若初值把它放到 1.6 m，最近邻会选 2 m，后续可能继续向右移动 0.4 m。残差虽然降为零，对应关系却错了。这解释了演示中“增量很小”与“位姿准确”为何可能分开出现。



超过关联距离的点不参与本轮优化。默认门限为 0.5 m，最多 40 次迭代，至少 20 个有效关联。门限不是越大越好：太大容易认错墙，太小则运动稍快就没有足够的点。

## 8.2 把位姿的小变化写成一阶近似

本章直接把  $\theta = (t_x, t_y, \psi)^\top$  作为待求的三个数，更新为  $\theta + \delta$ ，其中  $\delta = (\delta_x, \delta_y, \delta_\psi)^\top$ 。先看一对已经关联的点。展开旋转矩阵，有

$$e_x = p_x \cos \psi - p_y \sin \psi + t_x - q_x, \quad (8.2)$$

$$e_y = p_x \sin \psi + p_y \cos \psi + t_y - q_y. \quad (8.3)$$

分别对三个变量求偏导，记旋转后的点  $u = Rp$ ，得到

$$\frac{\partial e_x}{\partial t_x} = 1, \quad \frac{\partial e_x}{\partial t_y} = 0, \quad \frac{\partial e_x}{\partial \psi} = -p_x \sin \psi - p_y \cos \psi = -u_y, \quad (8.4)$$

$$\frac{\partial e_y}{\partial t_x} = 0, \quad \frac{\partial e_y}{\partial t_y} = 1, \quad \frac{\partial e_y}{\partial \psi} = p_x \cos \psi - p_y \sin \psi = u_x. \quad (8.5)$$

将这些偏导按“残差分量为行、位姿变量为列”排列，就是雅可比矩阵：

$$J_i = \frac{\partial e_i}{\partial \theta} = \begin{bmatrix} 1 & 0 & -u_y \\ 0 & 1 & u_x \end{bmatrix}. \quad (8.6)$$

雅可比是多变量函数的一阶导数。用一阶泰勒展开，两个残差分量的变化为

$$e_i(\theta + \delta) \approx e_i(\theta) + J_i \delta = e_i + \begin{bmatrix} \delta_x - u_y \delta_\psi \\ \delta_y + u_x \delta_\psi \end{bmatrix}. \quad (8.7)$$

例如  $u = (2, 1) \text{ m}$ ，只增加  $0.01 \text{ rad}$  航向，点的位置约变化  $(-0.01, 0.02) \text{ m}$ 。这也符合直觉：绕原点转动时，瞬时位移垂直于半径。这里转动的是  $Rp$ ，平移  $t$  在我们的加法更新中独立变化，因此旋转列使用  $u$ 。

```
101 Eigen::Matrix<double, 2, 3> j;
102 j << 1, 0, -rotated.y(),
103     0, 1, rotated.x();
104 return j;
105 //
```

**代码练习 ch08-1：**补完 `starter/point_jacobian.hpp`。用  $(e(\theta + \epsilon e_j) - e(\theta - \epsilon e_j))/(2\epsilon)$  检查第  $j$  列，其中  $e_j$  是只在第  $j$  项为 1 的单位向量。中心差分来自  $+\epsilon$  与  $-\epsilon$  两个泰勒式相减：常数与二阶项相消，除以  $2\epsilon$  后留下导数及  $O(\epsilon^2)$  误差。取  $\epsilon = 10^{-6}$ ，并在多个朝向核对，可检查旋转方向与符号。

## 8.3 从残差平方和推导位姿增量

一帧有许多点，我们希望整体误差小。固定对应关系后，令

$$F(\delta) = \frac{1}{2} \sum_i w_i \|e_i + J_i \delta\|^2. \quad (8.8)$$

先取所有权重  $w_i = 1$ ；下一节再让可疑的大残差权重变小。平方使正负误差都产生正代价，系数  $1/2$  用来抵消求导时的 2。令  $r_{i,a} = e_{i,a} + \sum_b J_{i,ab} \delta_b$ ，其中  $a$  遍历  $x$ 、 $y$  两个分量。对增量的第  $c$  项求导，链式法则给出

$$\frac{\partial F}{\partial \delta_c} = \sum_i w_i \sum_a J_{i,ac} \left( e_{i,a} + \sum_b J_{i,ab} \delta_b \right) = 0. \quad (8.9)$$

把三个偏导等于零的方程合在一起，得到法方程：

$$H\delta = -g, \quad H = \sum_i w_i J_i^\top J_i, \quad g = \sum_i w_i J_i^\top e_i. \quad (8.10)$$

$H$  为  $3 \times 3$ ,  $g$  为三维向量；即使有上千对点，待解的增量仍只有三个数。 $H$  收集各变量对残差的影响， $g$  收集当前残差指向的改进方向。解出  $\delta$  后更新位姿、重新关联、再求解，这就是本章使用的高斯-牛顿迭代。

### 8.3.1 一个可以手算的三元方程

取  $p_1 = (1, 0)$ 、 $p_2 = (0, 1)$ ，参考点分别为  $q_1 = (1.2, -1)$ 、 $q_2 = (.2, .9)$ 。初值  $t = 0, \psi = 0$ ，固定这两对对应关系，取  $w_i = 1$ 。两点残差都为  $(-.2, .1)$ ，代入各自的雅可比得到

$$H = \begin{bmatrix} 2 & 0 & -1 \\ 0 & 2 & 1 \\ -1 & 1 & 2 \end{bmatrix}, \quad g = \begin{bmatrix} -.4 \\ .2 \\ .3 \end{bmatrix}.$$

前两行给出  $\delta_x = .2 + .5\delta_\psi$ 、 $\delta_y = -.1 - .5\delta_\psi$ ；代入第三行得到  $\delta_\psi = 0$ 。因此  $\delta = (.2, -.1, 0)$ ，正好把两点平移到参考位置。这个例子用于手算内核；实际扫描还有最少关联点数等配置。

### 8.3.2 代码中的 LDLT 在做什么

`h.ldlt().solve(g)` 通过消元解线性方程。对本例，逐项相乘可以验证

$$H = LDL^\top, \quad L = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -.5 & .5 & 1 \end{bmatrix}, \quad D = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

$L$  为下三角矩阵， $D$  为对角矩阵；这种分解把同时求三个未知数，变为逐项代入。令  $b = -g$ ，依次解  $Ly = b$ 、 $Dz = y$ 、 $L^\top \delta = z$ 。第一步从上到下得到  $y = (.4, -.2, 0)$ ，第二步逐项相除得到  $z = (.2, -.1, 0)$ ，第三步从下到上得到同样的  $\delta$ 。Eigen 自动进行分解与代入；调用者直接传入  $H, g$ 。

## 8.4 用 Huber 权重减轻错误关联的影响

平方代价的导数随误差线性增大，因此一个很大的错误对应可能牵动整个位姿。Huber 代价在误差较小时保留平方形状，较大时改为线性：

$$\rho(s) = \begin{cases} \frac{1}{2}s^2, & |s| \leq a, \\ a|s| - \frac{1}{2}a^2, & |s| > a. \end{cases} \quad (8.11)$$

在  $|s| = a$  处，两边值均为  $a^2/2$ ，斜率也相等，所以连接平滑。对标量残差  $s$  求导可得

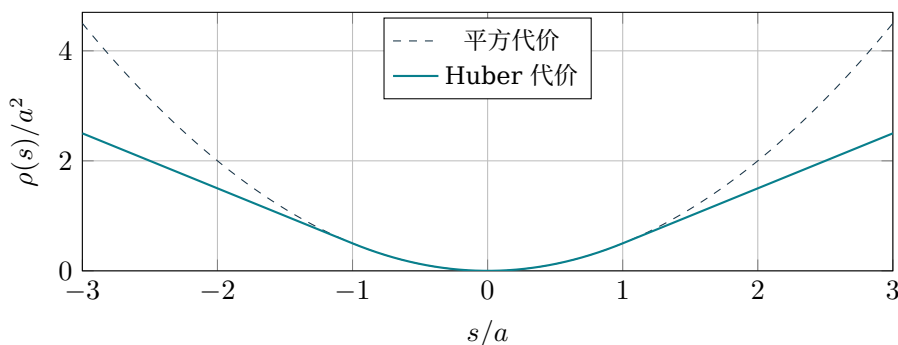
$$\rho'(s) = \begin{cases} s, & |s| \leq a, \\ a \operatorname{sign}(s), & |s| > a. \end{cases}$$

普通加权平方项  $\frac{1}{2}ws^2$  对残差的导数为  $ws$ 。让这一轮线性化的导数与 Huber 相同，就取

$$w = \frac{\rho'(s)}{s} = \min(1, a/|s|), \quad (8.12)$$

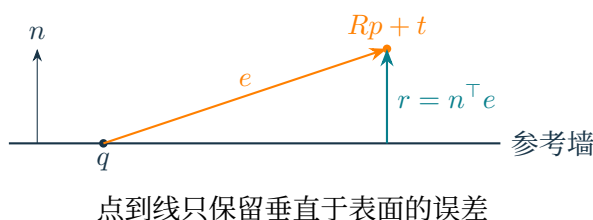
零残差时使用平方分支， $w = 1$ 。每一轮按当前残差重新计算权重，并在该轮求解时固定它们。这就是**迭代重加权最小二乘**。点到点残差为向量，令  $s = \|e\|$ ；链式法则给出  $\partial\rho(\|e\|)/\partial e = \rho'(s)e/s = we$ ，所以同样使用这个权重。

默认  $a = .08 \text{ m}$ 。残差  $.02 \text{ m}$  的权重为 1，残差  $.40 \text{ m}$  的权重为  $.2$ ；其对梯度的贡献幅度从  $.40$  缩到  $.08$ 。最近邻距离门限先去掉相距太远的点，Huber 再减弱保留下来的大残差，两者各有作用。



## 8.5 点到线：沿墙滑动和穿墙不同

考虑参考墙  $y = 0$ ，某次关联的参考点  $q = (0, 0)$ ，变换后的当前点  $u + t = (.3, .1)$ 。点到点残差长为  $\sqrt{.3^2 + .1^2} \approx .316 \text{ m}$ ；但它到墙的垂直距离只有  $.1 \text{ m}$ 。墙面被离散采样后，沿墙  $.3 \text{ m}$  的偏差可能来自采样位置不同。因此可将残差投影到墙的法向，仅使用垂直分量。



参考表面单位法向量为  $n_i$  时，点到线残差和雅可比为

$$r_i = n_i^\top (Rp_i + t - q_i), \quad J_i^{\text{line}} = \frac{\partial r_i}{\partial \theta} = n_i^\top J_i. \quad (8.13)$$

单位长度使点积就是有符号距离。在线性化这一步，参考点和法向量固定，因此求导时只需左乘  $n_i^\top$ 。水平墙取  $n = (0, 1)$ ，则  $r = e_y$ 、 $J^{\text{line}} = (0, 1, u_x)$ ；沿墙平移  $\delta_x$  对残差没有影响。将  $n$  改为  $-n$  会同时改变  $r, J$  的符号，而  $J^\top J$ 、 $J^\top r$  保持不变，故法向量的正反方向不影响本章求解。

## 8.6 从一小片点云求出墙的法向

扫描没有直接提供法向量，需要从参考点附近的形状估计。对一个参考点，收集半径  $.4 \text{ m}$  内的  $M$  个邻点  $q_k$ ，求均值  $\bar{q} = M^{-1} \sum_k q_k$ ，再令  $d_k = q_k - \bar{q}$ 。第六章用协方差描述噪声分散程度，这里用同

样形式描述空间点的分散:

$$C = \frac{1}{M} \sum_k d_k d_k^\top = \begin{bmatrix} A & B \\ B & D \end{bmatrix}, \quad A = \frac{1}{M} \sum_k d_{k,x}^2, \quad B = \frac{1}{M} \sum_k d_{k,x} d_{k,y}, \quad D = \frac{1}{M} \sum_k d_{k,y}^2. \quad (8.14)$$

取任意单位方向  $n$ , 各点沿它的投影为  $n^\top d_k$ , 投影的平均平方为

$$V(n) = \frac{1}{M} \sum_k (n^\top d_k)^2 = n^\top C n. \quad (8.15)$$

墙的切向上点分布较宽, 法向上分布较窄, 所以我们寻找使  $V(n)$  最小的单位方向。这就是主成分分析 (PCA) 在这里的几何意义。

### 8.6.1 特征向量从哪里来

暂用两个分量写  $n = (n_x, n_y)$ , 则  $V = An_x^2 + 2Bn_x n_y + Dn_y^2$ , 其梯度为  $2Cn$ 。因为  $n$  要保持单位长度, 微小变化  $dn$  必须满足  $n^\top dn = 0$  (对  $n^\top n = 1$  求导即可得到)。在最小值处, 沿任意这种切向变化都应有  $dV = 2(Cn)^\top dn = 0$ , 所以  $Cn$  必须与  $n$  平行:

$$Cn = \lambda n. \quad (8.16)$$

这样的方向叫特征向量, 比例  $\lambda$  叫特征值。左乘单位向量  $n^\top$  得  $V(n) = \lambda$ , 因此较小特征值对应较小投影方差, 其方向就是法向。

二维情形可以直接算出两个特征值。把方程展开为  $(A - \lambda)n_x + Bn_y = 0$ 、 $Bn_x + (D - \lambda)n_y = 0$ 。消去其中一个分量, 要存在非零解, 就需

$$(A - \lambda)(D - \lambda) - B^2 = 0.$$

解这个二次方程得到

$$\lambda_{\min, \max} = \frac{A + D \mp \sqrt{(A - D)^2 + 4B^2}}{2}. \quad (8.17)$$

再代回线性方程求方向并归一化。例如五点  $(-.2, 0), (-.1, 0), (0, 0), (.1, 0), (.2, 0)$  的均值为零,  $C = \text{diag}(.02, 0) \text{ m}^2$ 。特征值为 0 和 .02, 较小值的方向是  $(0, 1)$ , 恰好垂直于这条水平线。

### 8.6.2 把推导对应到邻域代码

代码先以当前参考点  $q_i$  为原点, 令  $v_k = q_k - q_i$ , 累积  $\text{sum} = \sum_k v_k$  与  $\text{second} = \sum_k v_k v_k^\top$ 。相对这个原点的均值为  $\mu = M^{-1} \sum_k v_k$ 。展开中心化后的乘积可得

$$\frac{1}{M} \sum_k (v_k - \mu)(v_k - \mu)^\top = \frac{1}{M} \sum_k v_k v_k^\top - \mu \mu^\top - \mu \mu^\top + \mu \mu^\top \quad (8.18)$$

$$= \frac{1}{M} \sum_k v_k v_k^\top - \mu \mu^\top. \quad (8.19)$$

这就是 `second / count - mean * mean.transpose()`。Eigen 的对称矩阵求解器按特征值从小到大排列, `eigenvectors().col(0)` 给出法向。至少 5 点, 且  $\lambda_{\max} > 10^{-8} \text{ m}^2$ 、 $\lambda_{\min} < .15 \lambda_{\max}$  时才保留它。前者要求点具有一定空间跨度, 后者要求点簇明显扁长; 角点或近似圆形点簇常会被这一比例筛掉。

```

44     if (config.metric == MatchMetric::PointToPoint) {
45         const double norm = residual.norm();
46         const double weight = norm <= config.huber_delta ? 1.0 : config.huber_delta / norm;
47         system.h += weight * jacobian.transpose() * jacobian;
48         system.g += weight * jacobian.transpose() * residual;
49         system.squared_error += residual.squaredNorm();
50     } else {
51         const auto & normal = target.normals[nearest];
52         if (normal.squaredNorm() < 0.5) {continue;}
53         const double error = normal.dot(residual);
54         const Eigen::RowVector3d j = normal.transpose() * jacobian;
55         const double weight = std::abs(error) <= config.huber_delta ?
56             1.0 : config.huber_delta / std::abs(error);
57         system.h += weight * j.transpose() * j;
58         system.g += weight * j.transpose() * error;
59         system.squared_error += error * error;
60     }
61     ++system.count;
62     //

```

这段代码把两种残差接到同一个三元法方程。点到点分支使用两行  $J$ ，点到线分支先投影成一行；随后都累积  $wJ^T J$  和  $wJ^T r$ 。squared\_error 单独累积未加权残差平方，便于报告几何误差。

## 8.7 判断几何约束是否足够

把水平墙各点的  $J_i = (0, 1, u_{i,x})$  代入法方程，得到

$$H = \sum_i w_i \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & u_{i,x} \\ 0 & u_{i,x} & u_{i,x}^2 \end{bmatrix}. \quad (8.20)$$

第一行、第一列全为零：任意沿墙平移都不改变残差，扫描没有告诉我们该方向移动了多少。这称为**退化**；从线性方程看，独立约束不足以确定三个增量，也就是矩阵秩不足。

更一般地，对任意增量方向  $d$ ,

$$d^T H d = \sum_i w_i \|J_i d\|^2 \geq 0. \quad (8.21)$$

若某个非零  $d$  使所有  $J_i d = 0$ ，则沿该方向改变位姿几乎不产生代价。与上一节的单位方向最小化一样， $H$  的最小特征值给出最弱方向的局部约束强度。代码将最小值与最大值比较；比值太小时报告退化。本章位姿参数固定使用米与弧度，并在这一参数尺度下设置比值门限。添加一面垂直墙后，沿  $x$  平移也会改变点到线距离，通常就能补上缺失的约束。

几何约束充足后，还要检查关联是否合理。重复的走廊或错误最近邻可以形成一组内部自治的约束，得到很小的残差；合理初值与连续跟踪能减少这类错误。

```

145     const Eigen::SelfAdjointEigenSolver<Eigen::Matrix3d> eigen(system.h);
146     if (eigen.eigenvalues()[0] <= c.min_eigen_ratio * eigen.eigenvalues()[2]) {
147         result.status = MatchStatus::Degenerate;
148         return result;
149     }
150     if (small_step) {
151         result.status = result.rmse <= c.max_rmse ?
152             MatchStatus::Converged : MatchStatus::HighResidual;
153         return result;
154     }
155     if (k == c.max_iterations) {break;}
156     Eigen::Vector3d delta = -system.h.ldlt().solve(system.g);
157     // Keep each local update moderate; this does not rescue a bad initial guess.
158     const double scale = std::max({1.0, delta.head<2>().norm() / .3, std::abs(delta.z()) / .2});
159     delta /= scale;
160     result.pose.position += delta.head<2>();
161     result.pose.yaw = wrapAngle(result.pose.yaw + delta.z());
162     ++result.iterations;
163     small_step = delta.head<2>().norm() < c.translation_tolerance &&
164         std::abs(delta.z()) < c.yaw_tolerance;
165     //

```

`ldlt().solve` 解出增量后，代码还用  $\gamma = \max(1, \|\delta_t\|/.3, |\delta_\psi|/.2)$  缩放它，令  $\delta \leftarrow \delta/\gamma$ 。因为  $\gamma$  至少等于两个比值，单次平移与转角分别不超过 .3 m、.2 rad，有助于让一阶近似保持在当前位姿附近。然后更新位姿，并按新位姿重算关联与残差。

最终报告的几何均方根误差为

$$\text{RMSE}_{\text{point}} = \sqrt{\frac{1}{N} \sum_i \|e_i\|^2}, \quad \text{RMSE}_{\text{line}} = \sqrt{\frac{1}{N} \sum_i r_i^2}.$$

前者包含两个方向的误差，后者只包含法向误差；都以米计量。例如上面的单点例子，两者分别为 .316 m 和 .1 m，比较时应先看清定义。

| 状态              | 下一步检查                   |
|-----------------|-------------------------|
| converged       | 增量、关联数、约束强度及 RMSE 均满足条件 |
| too_few_pairs   | 查看重叠区域、关联门限与法向是否有效      |
| degenerate      | 查看是否只有单墙等约束不足的几何        |
| iteration_limit | 查看初值与迭代过程               |
| high_residual   | 增量已小，检查剩余误差和错误对应        |

结果中的 `information` 保存  $H$ ，可用于观察局部约束强弱。估计误差的协方差还与传感器噪声、关联误差和地图误差有关；第九章将单独建立这一模型。

**动手观察：**改变已知变换与初值，记录增量、RMSE 和恢复的位姿。再删去一面墙，对照上面的零列解释退化；恢复双墙后，观察最弱约束如何变化。

配准内核只处理一次点云对齐；下面由激光里程计管理时间历史、参考点云和失败帧。

## 8.8 把成功扫描留在一个小窗口中

每一帧都与上一帧匹配会连续累积误差。我们把最近几个关键帧转换到 `odom` 系，作为局部参考表面。默认最多保留五帧；累计平移达到 0.2 m 或转角达到 0.15 rad 时才插入。用边长 .06 m 的

方格做降采样（也称体素降采样），每格只保留点的质心。格索引沿用第七章的向下取整，格内质心为  $\bar{p} = M^{-1} \sum_i p_i$ 。它也是使  $\sum_i \|p_i - c\|^2$  最小的代表点：对  $c$  求导得  $2Mc - 2 \sum_i p_i = 0$ ，解出  $c = \bar{p}$ 。这样一面墙上的重复点被压缩成较均匀的参考点；窗口保存的是点坐标，地图的自由证据仍由第七章栅格处理。

若只匹配相邻两帧，得到  $T_{k-1 \leftarrow k}$  后，按第二章的复合规则计算  $T_{O \leftarrow k} = T_{O \leftarrow k-1} T_{k-1 \leftarrow k}$ 。本实现已把窗口点转换到 `odom`，因此配准直接求  $T_{O \leftarrow k}$ 。这解释了同一个内核为什么既能做两帧对齐，也能输出当前里程计位姿。

设前两次成功估计的时刻为  $t_{k-1}, t_k$ ，下次扫描在  $t$  到达。用割线速度提供局部初值：

$$\hat{v}_k = \frac{p_k - p_{k-1}}{t_k - t_{k-1}}, \quad \hat{\omega}_k = \frac{\text{wrap}(\psi_k - \psi_{k-1})}{t_k - t_{k-1}}, \quad p_{\text{init}} = p_k + (t - t_k) \hat{v}_k. \quad (8.22)$$

割线速度就是两次成功位置的变化量除以经过的时间，再把它乘预测时长。航向同样外推，并用 `wrap` 将角差放回  $[-\pi, \pi)$ ，选取较短的转角。例如从  $179^\circ$  到  $-179^\circ$ ，连续运动的角差是  $2^\circ$ ，直接相减则为  $-358^\circ$ 。

若  $0.1 \text{ s}$  和  $0.3 \text{ s}$  的成功位置分别为  $.1 \text{ m}$  与  $.3 \text{ m}$ ，中间一帧失败，割线速度应为  $(.3 - .1) / (.3 - .1) = 1 \text{ m/s}$ ，预测  $0.4 \text{ s}$  的位置为  $.4 \text{ m}$ 。除以固定的  $.1 \text{ s}$  会把速度误算为  $2 \text{ m/s}$ 。外推得到的位姿只作为初值，接下来由新扫描修正。**练习 ch08-2** 的代码入口为 `constant_velocity.hpp`，检查缺帧和跨越  $\pm\pi$  的情况。

```

84 result.match = matchClouds(source, target_, initial, config_.match);
85 result.status = matchStatusName(result.match.status);
86 if (!result.match.accepted()) {return result;}
87 velocity_ = (result.match.pose.position - pose_.position) / dt;
88 yaw_rate_ = wrapAngle(result.match.pose.yaw - pose_.yaw) / dt;
89 pose_ = result.match.pose;
90 last_accepted_ = stamp;
91 if ((pose_.position - keyframe_pose_.position).norm() >= config_.keyframe_distance ||
92     std::abs(wrapAngle(pose_.yaw - keyframe_pose_.yaw)) >= config_.keyframe_angle) {
93     addKeyframe(source, pose_);
94 }
95 //

```

拒绝帧不更新位姿、速度、成功时刻和局部地图。下一帧仍可尝试恢复；连续超过  $1 \text{ s}$  没有成功观测则报告 `tracking_lost`，等待明确 `reset`。窗口控制参考点云的规模，并复用几帧共同观测到的表面。每个关键帧仍带有估计误差，长期运动会逐渐积累漂移；第十章用回环约束修正它。

## 8.9 用估计里程计替换真值入口

重新构建并加载工作区后，运行：

```

1 ros2 launch motion2d_bringup ch08.launch.py
2 # Set rviz:=false for a terminal-only experiment.
3 # Second terminal, from ros2_ws with the same ROS environment:
4 /usr/bin/python3 ../scripts/check_ch08.py

```

激光节点订阅 `/scan`，发布 `/odometry/estimated`；`estimated` 适配器将其接到统一 `/odometry`，并发布 `odom` 到 `base_link` 的 TF。`launch` 将第七章的 `truth` 适配器替换为 `estimated` 适配器。模拟器的独立真值消息用于评估，传感器静态外参继续描述安装位置。

第一帧定义 `odom` 原点与朝向；现在地图中的起点是  $(0, 0)$ 。真实世界起点仍是  $(-8, -8)$ ，两者不能不经变换直接叠加。RViz 默认只显示观测栅格、注册扫描、估计轨迹和机体轴；真值世界与真值机器人显示关闭。`map` 暂与 `odom` 重合。

**跟踪状态** `/estimation/status` 给出每次配准状态。成功时发布该采样时刻的里程计；失败时保留最后一次成功状态，等待下一帧继续匹配。建图节点处理后续能够精确配对的扫描，并丢弃较早的未配对帧。

里程计 `twist` 是成功扫描间的平均速度，在机体系表达。`pose` 协方差暂用固定测量尺度：平移标准差 `.03 m`、航向 `.01 rad`，用来设置后续融合时对激光位姿的信任程度。第九章将说明这些尺度怎样影响滤波增益，并通过误差实验调整它们。`reset` 清空首帧原点、关键帧、速度和轨迹。零时刻重新采样；正时间重复帧被忽略。

**常见错误：**把真值 TF 当初值；把失败帧加入局部地图；拿当前最新位姿变换旧扫描；直接比较不同原点的轨迹。

## 8.10 实验：束数、噪声与被拒绝的帧

在仓库根目录加载工作区环境，运行：

```
1 ros2 run motion2d lidar_odometry_demo tmp/ch08_results
2 /usr/bin/python3 scripts/plot_ch08.py
```

实验固定 `world seed=42`、`noise seed=4242`、圆参考半径 `1 m`、角频率 `0.4 rad/s`， $t = 0, 0.1, \dots, 15.9$  s 共 160 帧。估计器只得到点云；独立评估器用首帧真值定义同一个参考系，再计算位置误差：

$$T_k^{\text{ref}} = (T_0^{\text{truth}})^{-1} T_k^{\text{truth}}, \quad \text{RMSE} = \sqrt{\frac{1}{|\mathcal{A}|} \sum_{k \in \mathcal{A}} \|p_k^{\text{est}} - p_k^{\text{ref}}\|^2}. \quad (8.23)$$

参考轨迹先减去首帧位置，再用首帧旋转的转置转轴，即  $p_k^{\text{ref}} = R_0^\top (p_k^{\text{truth}} - p_0^{\text{truth}})$ ，于是参考与估计都从  $(0, 0)$  出发。 $\mathcal{A}$  是成功帧编号集合， $|\mathcal{A}|$  为其数量。每帧先算欧氏位置误差，再平方、平均、开方，就得到这段轨迹的米制 RMSE。同时报告拒绝数，可看出小误差对应的是大多数扫描还是少量容易配准的帧。下表直接读取 C++ 实验 CSV。无噪声时也可能因离散扫描、可见性和迭代门限而拒绝某些帧。

| 束数  | $\sigma_r/\text{m}$ | 接受  | 拒绝 | 位置 RMSE/m |
|-----|---------------------|-----|----|-----------|
| 90  | 0                   | 155 | 5  | 0.0041    |
| 90  | $3 \cdot 10^{-2}$   | 150 | 10 | 0.0257    |
| 360 | 0                   | 158 | 2  | 0.0018    |
| 360 | $3 \cdot 10^{-2}$   | 154 | 6  | 0.0136    |
| 720 | 0                   | 156 | 4  | 0.0025    |
| 720 | $3 \cdot 10^{-2}$   | 154 | 6  | 0.0101    |

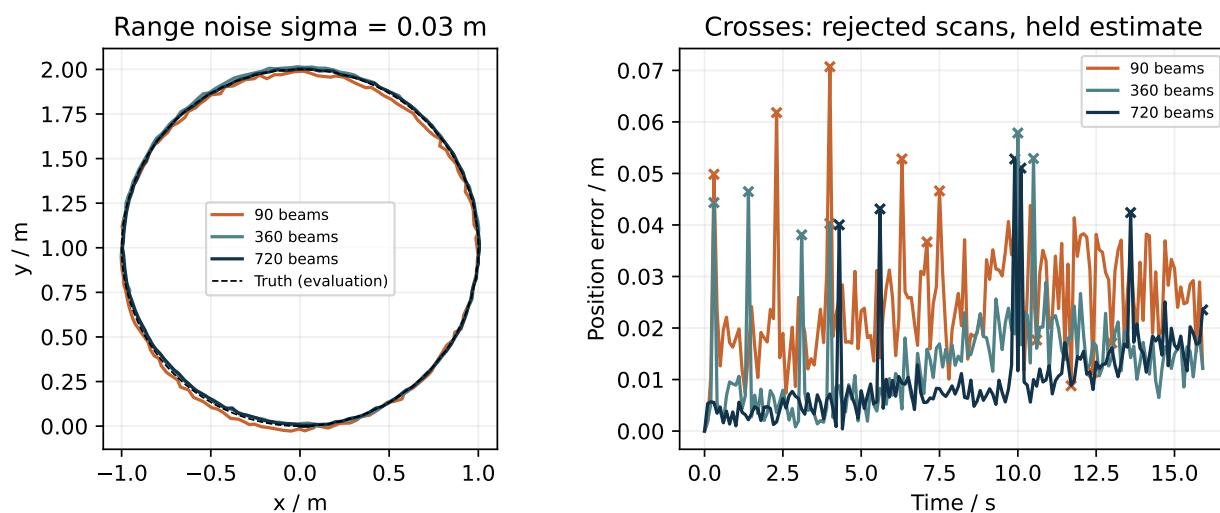


图 8.1: 首帧对齐后的轨迹与位置误差。叉号是拒绝帧, 此时画出保持上次估计的误差; 圆轨迹保持等比例。

比较表格和曲线时, 先观察接受与拒绝发生在哪些时刻, 再看成功帧的位置误差。720 束会提供更密的观测, 同时改变降采样后的参考点、法向和最近邻关系, 所以误差变化还取决于当前场景的几何和噪声。一次只改一个参数, 记录成功率、误差及状态, 才能把这些因素分开理解。

下一章在两次扫描之间用 IMU 预测运动, 再由激光校正, 把高频运动信息与几何观测结合起来。

## 第九章 让 IMU 预测，让激光纠偏

**目标** 在同一时间轴上估计位置、速度、航向与偏置，理解预测和校正的分工。

**先修** 第 06、08 章的 IMU 模型、方差和雅可比。

### 9.1 从一个可以控制噪声的实验开始

重新构建并加载工作区后，从仓库根目录运行：

```
ros2 run motion2d planar_ekf_demo tmp/ch09_filter
/usr/bin/python3 scripts/plot_ch09_filter.py
```

这个实验从解析轨迹生成 IMU 和带噪位姿测量。我们可以指定传感器偏置，也可以暂停位姿测量，观察估计如何变化。先掌握融合的计算，再在本章后半部分将位姿测量换成第八章的扫描匹配输出。

IMU 给出短时间内的运动变化。积分需要初始位置和速度，噪声与偏置还会不断累积。位姿观测的频率较低，却给出了当前位置的约束。融合的关键是两步：每收到 IMU 就**预测**运动；收到位姿观测后，按二者的不确定性**校正**预测。下面从一个数的加权平均推到八个状态的滤波器。

### 9.2 八个数描述平面状态

采用状态列向量

$$x = (p_x, p_y, v_x, v_y, \psi, b_{ax}, b_{ay}, b_g)^\top. \quad (9.1)$$

$p, v$  在 odom 系，单位分别为 m、m/s； $\psi$  是航向，单位 rad。加计偏置  $b_a$  在机体系，单位  $\text{m/s}^2$ ；陀螺偏置  $b_g$  单位 rad/s。偏置描述读数中缓慢变化的偏移，例如静止时陀螺持续多读了 .01 rad/s。滤波器以原始加速度和角速度为输入，orientation 字段留空。因为机器人水平运动且 roll、pitch 为零，重力没有 x/y 分量，由第六章的比力关系得

$$a_k = R(\psi_k)(f_{xy,k} - b_{a,k}), \quad \omega_k = \omega_{m,k} - b_{g,k}. \quad (9.2)$$

先扣除机体系偏置，再旋转到 odom 系，才能与  $v_k$  在同一坐标系中积分。在长度为  $h$  的小区问内，取步首的  $a_k, \omega_k$  为常数，则

$$v_{k+1} = v_k + \int_0^h a_k d\tau = v_k + h a_k, \quad (9.3)$$

$$p_{k+1} = p_k + \int_0^h (v_k + \tau a_k) d\tau = p_k + h v_k + \frac{1}{2} h^2 a_k, \quad (9.4)$$

$$\psi_{k+1} = \text{wrap}(\psi_k + h \omega_k), \quad b_{k+1} = b_k. \quad (9.5)$$

转弯时, 区间内  $R(\psi)$  会变化, 所以固定  $a_k$  是短时间近似。例如保持机体系加速度  $c$ 、角速度  $\omega$ ,  $R(\psi + \omega\tau)c \approx R(\psi)c + \omega\tau R(\psi)Jc$ , 其中  $J = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$  表示转  $90^\circ$ 。被忽略的速度增量约为  $\frac{1}{2}\omega h^2 R J c$ : 步长减半, 这项约减为四分之一。当前内核使用  $0 < h \leq 0.1$  s; 更长的 IMU 缺测由时间管理模块处理。

## 9.3 把误差也向前传播一步

### 9.3.1 协方差记录误差的大小与关联

记估计误差  $\epsilon = x_{\text{true}} - \hat{x}$ , 在当前估计附近把它视为均值零的随机量。协方差的第  $(i, j)$  项是

$$P_{ij} = \mathbb{E}[\epsilon_i \epsilon_j], \quad P = \mathbb{E}[\epsilon \epsilon^\top]. \quad (9.6)$$

对角项  $P_{ii}$  是方差; 非对角项描述两个误差是否经常同向变化。例如速度估大时, 下一刻位置通常也会估大, 位置和速度的误差便产生正相关。 $P_{pv}$  的单位为  $\text{m}^2/\text{s}$ , 与位置方差的  $\text{m}^2$  不同。矩阵把这些关系按状态顺序放在一起, 后面的位置观测就能利用它们修正速度。

### 9.3.2 逐块求状态转移的偏导

设  $c = f_{xy} - b_a$ 。第八章已推导旋转的导数, 因而

$$\frac{\partial a}{\partial \psi} = R(\psi)Jc =: A_\psi, \quad \frac{\partial a}{\partial b_a} = -R(\psi). \quad (9.7)$$

例如  $v^+ = v + ha$ , 对  $v, \psi, b_a$  分别求导, 得到  $I, hA_\psi, -hR$ 。  $p^+ = p + hv + \frac{1}{2}h^2a$  中同样的加速度项只需改乘  $\frac{1}{2}h^2$ ;  $\psi^+ = \psi + h(\omega_m - b_g)$  对偏置求导为  $-h$ 。按  $(p, v, \psi, b_a, b_g)$  分块, 完整雅可比为

$$F = \begin{bmatrix} I & hI & \frac{1}{2}h^2A_\psi & -\frac{1}{2}h^2R & 0 \\ 0 & I & hA_\psi & -hR & 0 \\ 0 & 0 & 1 & 0 & -h \\ 0 & 0 & 0 & I & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}. \quad (9.8)$$

五组的大小依次为 2, 2, 1, 2, 1; 例如第三列第一块是  $2 \times 1$ , 第一行第四块是  $2 \times 2$ , 各个零块也按所在行列确定大小。这张分块表与源码的 `block<2,1>(0,4)` 等调用逐项对应。

再对输入  $u = (f_x, f_y, \omega_m)$  求导, 得到  $8 \times 3$  矩阵

$$G = \begin{bmatrix} \frac{1}{2}h^2R & 0 \\ hR & 0 \\ 0 & h \\ 0 & 0 \\ 0 & 0 \end{bmatrix}. \quad (9.9)$$

这里仍使用五组状态行块, 输入列块大小为 2, 1。输入加速度增加一点, 会同时改变位置和速度; 输入角速度则直接改变航向。

```

20 result.state = state;
21 result.state.head<2>() += dt * state.segment<2>(2) + .5 * dt * dt * a;
22 result.state.segment<2>(2) += dt * a;
23 result.state[4] = wrapAngle(state[4] + dt * (imu.yaw_rate - state[7]));
24 result.f.setIdentity();
25 result.f.block<2, 2>(0, 2) = dt * Eigen::Matrix2d::Identity();
26 result.f.block<2, 1>(0, 4) = .5 * dt * dt * da;
27 result.f.block<2, 1>(2, 4) = dt * da;
28 result.f.block<2, 2>(0, 5) = -.5 * dt * dt * r;
29 result.f.block<2, 2>(2, 5) = -dt * r;
30 result.f(4, 7) = -dt;
31 result.g.setZero();
32 result.g.block<2, 2>(0, 0) = .5 * dt * dt * r;
33 result.g.block<2, 2>(2, 0) = dt * r;
34 result.g(4, 2) = dt;
35 //

```

### 9.3.3 从误差乘积得到协方差传播

用  $\Phi(x, u)$  表示刚才的一步积分，传感器读数满足  $u_m = u_{\text{true}} + n$ 。真实状态与预测之差，按一阶泰勒展开为

$$\epsilon^+ \approx \Phi(\hat{x} + \epsilon, u_m - n) - \Phi(\hat{x}, u_m) + w_b \approx F\epsilon - Gn + w_b, \quad (9.10)$$

$w_b$  表示这一步偏置的随机变化。设已有状态误差、本次采样噪声和偏置变化相互独立、均值为零。将上式乘以自己的转置并取期望，交叉项例如  $\mathbb{E}[\epsilon n^\top] = \mathbb{E}[\epsilon]\mathbb{E}[n^\top] = 0$ ，于是

$$P_{k+1}^- = F_k P_k^+ F_k^\top + G_k \Sigma_u G_k^\top + Q_b. \quad (9.11)$$

上标  $-$  表示校正前， $+$  表示校正后； $\Sigma_u = \mathbb{E}[nn^\top]$ ， $Q_b = \mathbb{E}[w_b w_b^\top]$ 。噪声项的两个负号相乘为正。

看一维加速度噪声  $n_a$  就能核对时间的幂次： $\delta p = -\frac{1}{2}h^2 n_a$ ， $\delta v = -h n_a$ ，所以

$$\Delta P_{pp} = \frac{1}{4}h^4 \sigma_a^2, \quad \Delta P_{pv} = \frac{1}{2}h^3 \sigma_a^2, \quad \Delta P_{vv} = h^2 \sigma_a^2. \quad (9.12)$$

第六章配置的是每采样标准差，这些幂次来自同一个样本在整段时间内保持。例如  $h = .1$  s、 $\sigma_a = .2$  m/s<sup>2</sup>，三项分别为  $10^{-6}$  m<sup>2</sup>、 $2 \times 10^{-5}$  m<sup>2</sup>/s、 $4 \times 10^{-4}$  m<sup>2</sup>/s<sup>2</sup>。

开启 `estimate_bias` 时，偏置的初始标准差默认为 (.05, .05, .01)。为容纳缓慢变化，采用随机游走增量  $w_{b,i} = \sigma_{b,i} \sqrt{h} \xi_i$ ， $\xi_i$  为独立标准正态数。于是  $\text{Var}(w_{b,i}) = h \sigma_{b,i}^2$ ：将时段分成两半后，两段方差仍可相加为同一结果。默认  $\sigma_b = (.0005, .0005, .0001)$ ，单位为相应偏置单位除以  $\sqrt{s}$ 。滤波器据此在  $Q_b$  的偏置对角块加入  $h \text{diag}(\sigma_b^2)$ 。这给出了它愿意容纳的偏置变化速度；实验中的实际偏置另由传感器配置决定。

```

44 covariance_ = step.f * covariance_ * step.f.transpose() +
45     step.g * imu.variance.asDiagonal() * step.g.transpose();
46 if (config_.estimate_bias) {
47     covariance_.diagonal().tail<3>() +=
48         dt * config_.bias_random_walk.array().square().matrix();
49 }
50 //

```

偏置固定零模式把偏置初值、对应协方差和随机游走都置零。这样偏置没有可调整的不确定性，后续增益的偏置行也为零。

## 9.4 从加权平均推导卡尔曼增益

### 9.4.1 先估计一个位置数值

设预测位置为  $\hat{p}^-$ ，预测误差方差为  $P$ ；新观测为  $z = p_{\text{true}} + \nu$ ，测量误差方差为  $R_z$ 。取校正量为两者之差的  $k$  倍：

$$\hat{p}^+ = \hat{p}^- + k(z - \hat{p}^-). \quad (9.13)$$

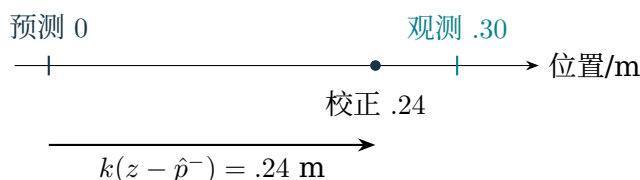
若  $k = 0$  就完全保留预测， $k = 1$  就采用观测。由  $\epsilon^+ = p_{\text{true}} - \hat{p}^+ = (1 - k)\epsilon^- - k\nu$ ，在两项误差独立时，校正后的方差为

$$P^+ = (1 - k)^2 P + k^2 R_z. \quad (9.14)$$

让它最小，对  $k$  求导并令其为零：

$$-2(1 - k)P + 2kR_z = 0 \implies k = \frac{P}{P + R_z}. \quad (9.15)$$

二阶导数  $2(P + R_z) > 0$ ，所以这是最小值。预测越不确定，越愿意向观测移动；观测越不确定，移动越少。例如预测 0 m、标准差 .2 m，观测 .3 m、标准差 .1 m， $P = .04$ ,  $R_z = .01$ ，故  $k = .8$ ，校正位置为 .24 m。代回方差得 .008 m<sup>2</sup>，标准差约 .0894 m。注意输入公式的是标准差的平方。



### 9.4.2 让同一观测修正多个状态

位姿观测  $z = (p_x^{\text{obs}}, p_y^{\text{obs}}, \psi^{\text{obs}})^\top$  从八维状态中选择位置和航向：

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}, \quad z = Hx_{\text{true}} + \nu. \quad (9.16)$$

把“观测减预测”称为**创新**  $r$ 。在航向差很小的局部角度表示中，

$$r = z - H\hat{x}^- = H\epsilon^- + \nu, \quad S = \mathbb{E}[rr^\top] = HP^-H^\top + R_z. \quad (9.17)$$

$r$  的航向分量先做 **wrap**。例如预测 179°、观测 -179°，校正方向为 +2°。**代码练习 ch09-1**：补完 `starter/yaw_residual.hpp`。

设状态第  $i$  项采用行向量  $k_i$  加权， $\hat{x}_i^+ = \hat{x}_i^- + k_i r$ 。它的误差平方期望是

$$\mathbb{E}[(\epsilon_i^- - k_i r)^2] = P_{ii}^- - 2k_i \mathbb{E}[r\epsilon_i^-] + k_i S k_i^\top, \quad (9.18)$$

$$\mathbb{E}[\epsilon^- r^\top] = P^- H^\top. \quad (9.19)$$

展开二次项，利用  $S$  对称，对  $k_i$  各分量求导得到  $2k_i S - 2(P^- H^\top)_{i,:} = 0$ 。将八行放在一起，便是

$$KS = P^- H^\top, \quad K = P^- H^\top S^{-1}, \quad \hat{x}^+ = \hat{x}^- + Kr. \quad (9.20)$$

$K$  有八行三列，表达三个观测残差分别如何修正八个状态。代码通过解  $SK^\top = HP^-$  求出增益，使用第八章介绍的 **LDL<sup>T</sup>** 分解。

### 9.4.3 位置观测如何修正速度：一次手算

只考虑一维位置和速度，步长 .1 s，忽略本步过程噪声，取

$$F = \begin{bmatrix} 1 & .1 \\ 0 & 1 \end{bmatrix}, \quad P_0 = \begin{bmatrix} .01 & 0 \\ 0 & 1 \end{bmatrix}, \quad P^- = F P_0 F^\top = \begin{bmatrix} .02 & .1 \\ .1 & 1 \end{bmatrix}. \quad (9.21)$$

乘法中的 .1 来自“速度误差乘以时间成为位置误差”。只测位置， $H = [1 \ 0]$ ，测量方差为 .01 m<sup>2</sup>，则

$$S = .02 + .01 = .03, \quad K = \frac{1}{.03} \begin{bmatrix} .02 \\ .1 \end{bmatrix} = \begin{bmatrix} 2/3 \\ 10/3 \end{bmatrix}. \quad (9.22)$$

若位置残差为 .03 m，位置增加 .02 m，速度同时增加 .1 m/s。速度增益单位为 s<sup>-1</sup>。类似地，连续预测会建立偏置与位置、航向之间的关联，后续观测便能修正偏置。充分的运动变化和持续的位姿观测有助于区分这些误差来源。

### 9.4.4 校正后还剩多少不确定性

代入  $r = H\epsilon^- + \nu$ ，校正后误差为

$$\epsilon^+ = (I - KH)\epsilon^- - K\nu. \quad (9.23)$$

按前面的方法乘转置、取期望，得到 Joseph 形式

$$P^+ = (I - KH)P^-(I - KH)^\top + KR_zK^\top. \quad (9.24)$$

两个加项各自都是误差变换后的协方差。任意向量  $d$  满足  $d^\top P d = \mathbb{E}[(d^\top \epsilon)^2] \geq 0$ ，称为半正定；保留这种乘积相加的形式有助于减少浮点运算破坏非负方差的问题。算完后再取  $(P + P^\top)/2$  消除微小的不对称舍入误差。

```
70 const Eigen::Matrix<double, 8, 3> gain = solve.solve(h * covariance_).transpose();
71 state_ += gain * residual;
72 state_[4] = wrapAngle(state_[4]);
73 const Matrix8d remainder = Matrix8d::Identity() - gain * h;
74 covariance_ = remainder * covariance_ * remainder.transpose() + gain * noise * gain.transpose();
75 //
```

在刚才的位置速度例子中，代入可得  $P_{pp}^+ = .00667$ 、 $P_{pv}^+ = .03333$ 、 $P_{vv}^+ = .66667$ ，三项都比校正前减小。本章称为扩展卡尔曼滤波（EKF），是因为旋转使状态预测成为非线性函数； $F, G$  用当前估计附近的一阶展开传播误差。

### 9.4.5 用不确定性衡量一次残差

位置用米、航向用弧度，直接相加平方会混淆量纲。若  $S$  为对角阵，归一化残差大小为

$$q = \frac{r_x^2}{S_{xx}} + \frac{r_y^2}{S_{yy}} + \frac{r_\psi^2}{S_{\psi\psi}}. \quad (9.25)$$

每一项都是“残差除以对应标准差”再平方，例如  $r_x = .2$  m、 $\sqrt{S_{xx}} = .1$  m 时贡献为 4。若各分量相关，使用完整二次型  $q = r^\top S^{-1} r$ 。可以将对称正定矩阵写成  $S = U\Lambda U^\top$ ： $U$  的列为相互垂直的单位特征向量， $\Lambda$  放对应正特征值，含义与第八章的主方向相同。令  $\tilde{r} = U^\top r$ ，则  $q = \sum_i \tilde{r}_i^2 / \lambda_i$ ，即转到互不相关的方向后分别按方差归一化。实际只需求解  $Sy = r$ ，再算  $r^\top y$ 。

默认配置在  $q > 16.3$  时跳过这次校正。它筛选的是相对于当前预测过大的残差。若实际存在偏置，而模型把偏置固定为零且  $P$  很小，正常观测也可能被筛掉。因此要同时观察轨迹误差、接受次数和创新大小，再调整模型及噪声参数。在线扫描匹配使用的局部地图含有历史扫描，当前观测与过去估计之间有信息关联。本章用固定  $R_z$  表示位姿测量尺度，可通过重复实验中实际残差的变化来选择参数；同一批历史信息的重复使用会让简单的独立噪声模型低估不确定性。

## 9.5 偏置与一秒缺测实验

合成轨迹为  $p = (\sin(0.6t), 0.5[1 - \cos(0.6t)])$  m、 $\psi = 0.6t$  rad。使用 200 Hz IMU、10 Hz 位姿观测、20 s 时长，4 到 5 s 不提供位姿校正。同一组噪声输入同时送给三个滤波器；seed=9090。加计偏置为  $(.08, -.04)$  m/s<sup>2</sup>，陀螺偏置为 .01 rad/s，滤波器没有收到这些答案。初始速度先验为零、标准差 1 m/s。对轨迹求导有  $v = (.6 \cos(.6t), .3 \sin(.6t))$  m/s，所以实际初速为  $(.6, 0)$  m/s。仅这个初速差就会产生约  $.6t$  m 的位置误差；IMU 积分需要靠后续位姿观测逐渐找回初速。

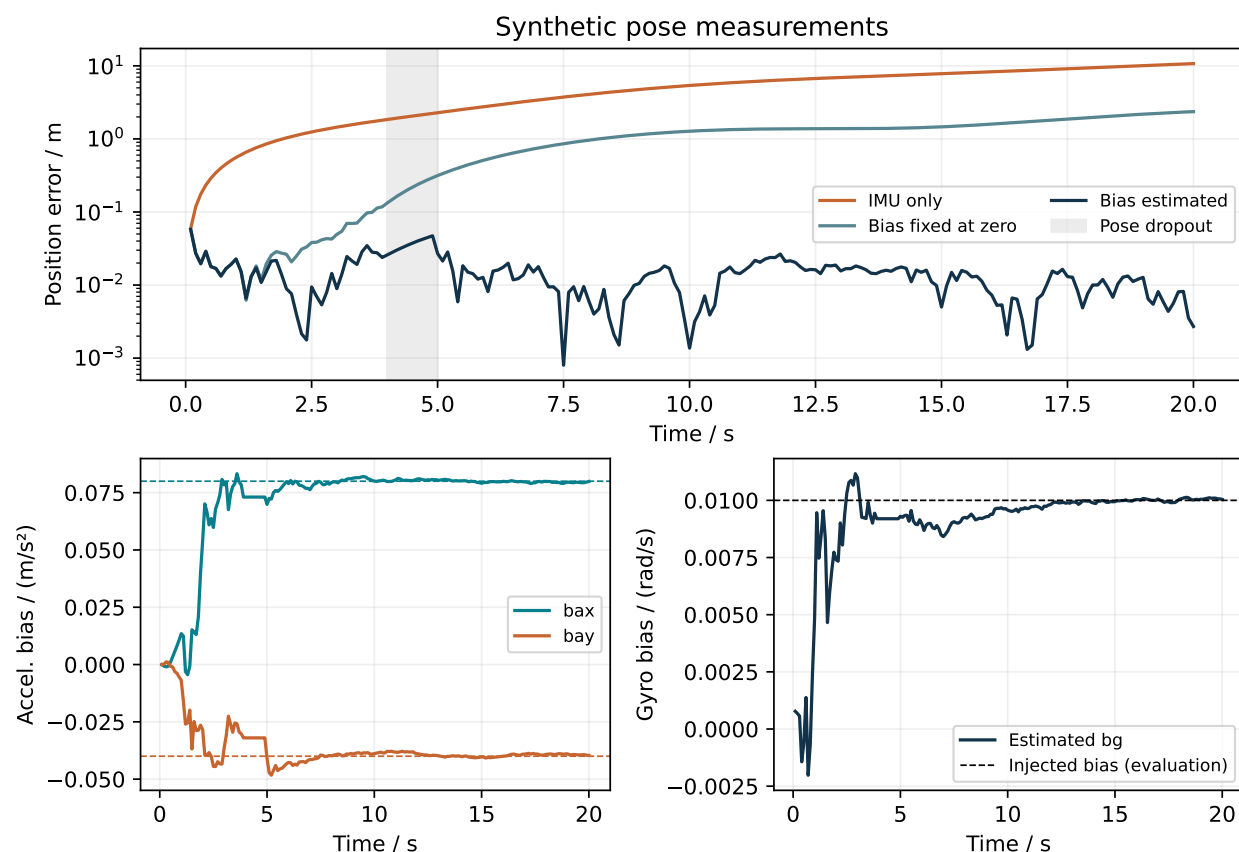


图 9.1: 合成位姿传感器实验。上图纵轴为对数，灰带表示停止位姿校正；下图虚线只用于评估注入的偏置。

| 模式    | 全程 RMSE/m | 缺测段 RMSE/m | 接受  | 拒绝  |
|-------|-----------|------------|-----|-----|
| 仅 IMU | 6.0611    | 2.0622     | 0   | 0   |
| 偏置固定零 | 1.2577    | 0.226      | 36  | 154 |
| 估计偏置  | 0.0184    | 0.0381     | 189 | 1   |

表中误差统计全部 4000 个 IMU 时刻，接受/拒绝计数只针对低频位姿校正。偏置固定零时，模型误差积累并触发大量拒绝；启用偏置估计后，本例全程位置 RMSE 约 0.0184 m。从上图的对数纵轴读取误差时， $10^{-2}$  到  $10^{-1}$  表示增大十倍，与  $10^{-1}$  到  $10^0$  的竖直间距相同。灰带内停止校正后，三个模型的差别更加明显：偏置估得越准，短时间积分中的系统性误差越小。若传感器偏置确实为零，可关闭偏置估计，观察减少待估变量后的表现。

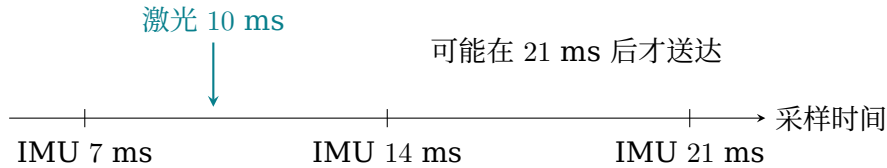
**实验练习：**把注入偏置改为零后复跑，再把观测标准差减半。减半标准差相当于把测量方差降为四分之一；用增益公式预测曲线会怎样变化，再比较全程与缺测段误差、接受与拒绝次数。

## 9.6 校正发生在采样时刻，不是收到消息时

先运行在线版本：

```
1 ros2 launch motion2d_bringup ch09.launch.py rviz:=false
2 # Another terminal with the same ROS environment, from repository root:
3 /usr/bin/python3 scripts/check_ch09.py
```

默认激光 10 Hz、IMU 200 Hz。配准需要时间，激光位姿送达时，滤波器可能已经处理了更新的 IMU。直接拿旧观测校正当前状态，会把消息延迟误当成运动误差。



例如激光时刻为 10 ms，两侧 IMU 为 7 和 14 ms。先用 7 ms 的样本推进到 10 ms，完成位姿校正，再用同一左侧样本推进到 14 ms，随后使用 14 ms 样本继续。所有区间长度先在整数纳秒内相减，最后乘  $10^{-9}$  换算秒。

```
41 void FusionTimeline::advance(FilterSnapshot & state, std::int64_t target) const
42 {
43     for (std::size_t i = 0; i + 1 < samples_.size() && state.stamp < target; ++i) {
44         if (samples_[i + 1].stamp <= state.stamp) {continue;}
45         const auto end = std::min(target, samples_[i + 1].stamp);
46         const double dt = (end - state.stamp) * 1e-9;
47         state.filter.predict(samples_[i].sample, dt);
48         state.stamp = end;
49     }
50 }
51 //
```

FusionTimeline 保存一个校正锚点及有限 IMU 队列。锚点包含状态和协方差：回到旧时刻时，位置、速度、航向、偏置及它们的不确定性一起恢复。激光观测早于或等于最新 IMU 时刻时可以处理；晚于它则由 ROS 节点排队等待。延迟观测校正后，重放到最新 IMU 时刻。默认历史窗口 2 s，超出窗口的旧观测报告 laser\_too\_old，等待下一帧仍在窗口内的观测。校正必须按激光采样顺序处理，重复帧不重复吸收信息。

例如队列已到 21 ms，而 10 ms 激光刚到：从上次校正锚点重放到 10 ms，在这个时刻校正，再依次重放 10 → 14 → 21 ms，恢复到队列末端。7 → 10 和 10 → 14 ms 都使用 7 ms 的 IMU，14 ms 之后才使用 14 ms 的样本。

把同一采样保持区间切成两段，也影响噪声计算。若同一加速度噪声  $n$  保持了  $h_1 + h_2$ ，速度误差为  $n(h_1 + h_2)$ ，方差是

$$\sigma_a^2(h_1 + h_2)^2 = \sigma_a^2(h_1^2 + h_2^2 + 2h_1h_2). \quad (9.26)$$

当前逐步传播将两段当作独立输入，累加前两项；遗漏的交叉项来自“同一个噪声被用了两次”。这是每采样噪声配合拆分积分时需要注意的相关性。可在进阶练习中保存当前样本噪声与状态的互协方差，将交叉项也传播下去。

**代码练习 ch09-2：**补完 starter/held\_duration.hpp。预测区间  $[a, b]$  与样本保持区间  $[c, d]$

的重叠，必须同时晚于两个左端、早于两个右端，故

$$h_{\text{overlap}} = \max(0, \min(b, d) - \max(a, c)). \quad (9.27)$$

先用整数纳秒求这个长度，再换算秒；例如  $[10, 21]$  ms 与  $[7, 14]$  ms 重叠 4 ms。

## 9.7 高频预测与扫描位姿各走一个出口

| 消息                  | 时刻与来源            | 使用者           |
|---------------------|------------------|---------------|
| /odometry/lidar     | 扫描匹配，激光采样时刻      | 融合器           |
| /odometry/estimated | IMU 预测，最新 IMU 时刻 | estimated 适配器 |
| /odometry/scan      | 激光时刻的校正位姿        | 观测建图          |
| /odometry           | 统一选择后的高频估计       | 跟踪控制与显示       |

同一激光时间的预测可能先到，而配准校正稍后才到。若把先到预测立即画进地图，该帧就失去了后来的校正。第九章 launch 将 mapping 的位姿输入重映射到专用的扫描位姿流；配准或 EKF 拒绝的帧不入图，后续成功帧仍可继续。

高频输出在每个 IMU 时刻至多发一次；晚到校正对外体现于下一次 IMU 输出，同一采样时间只发布一次 TF。新校正可能改变随后输出的位置，因此画轨迹时仍可能看到小的修正跳变。当前 map 与 odom 重合；下一章让回环更新 map 到 odom，而局部 odom 保持自己的运动历史。

**根据传感器状态选择后续动作** 超过 0.5 s 没有接受激光，报告 laser\_stale，仍可短时传播 IMU。相邻 IMU 间隔超过 0.1 s，则报告 imu\_gap 并失去初始化，等待新的激光起点。跟踪控制器据此区分短时缺扫和 IMU 中断：前者可短时使用预测，后者需要等待重新初始化。

第一帧激光定义 odom 原点。初速先验仍为零、标准差 1 m/s，后续校正逐渐估计它；匀速与静止都可能给出零水平加速度，因此首帧 IMU 无法区分它们。reset 清空偏置、协方差、锚点和历史；两个输入分别收到新原点时，保留先到的新试验样本。操作时先暂停，等待旧 DDS 队列排空。

**常见错误：**提前使用右侧 IMU；只回滚位置而没回滚速度/协方差；用最新位姿匹配旧扫描；把激光回环校正直接覆盖底层 odom；将过小协方差理解成高精度保证。

移除 rviz:=false 可显示估计轨迹、扫描点云与观测栅格。让机器人经过墙角，观察两种节奏：IMU 让轨迹不断向前延伸，成功配准后新扫描才落入地图。暂停仿真时，采样时钟停止，曲线也应停止增加新点。

**手算巩固：**在 exercises/ch09 中完成协方差传播、标量增益、位置速度关联、随机游走和拆分保持区间的练习。

## 9.8 真实扫描：转得更快、少扫半秒

```
1 ros2 run motion2d lidar_imu_demo tmp/ch09_fusion
2 /usr/bin/python3 scripts/plot_ch09_fusion.py
```

固定 world seed=42, 距离噪声 seed=4242, IMU seed=6060。360 束、距离噪声 0.01 m、激光 10 Hz、IMU 200 Hz, 圆轨迹半径 1 m, 持续 10 s; 3-3.5 s 不提供扫描, 每次激光结果有 30 ms 交付延迟。使用第六章默认白噪声, 无额外固定偏置。两个转速分别为 0.4 和 1.2 rad/s; 真值只用来生成传感器与独立评估。

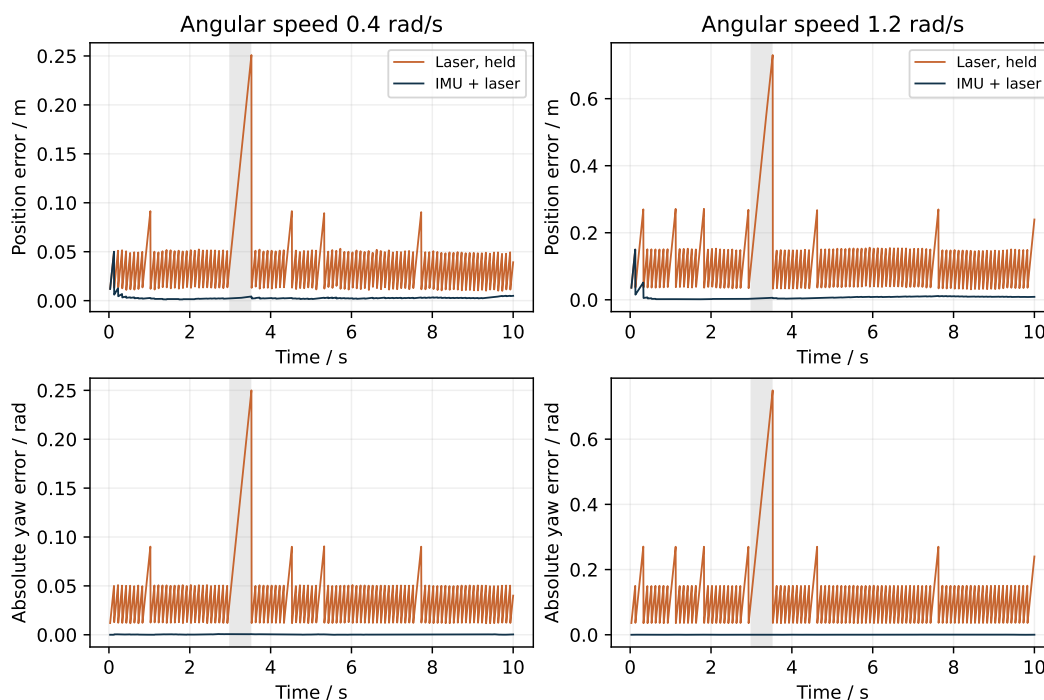


图 9.2: 实际射线与原始 IMU 回放, 灰色区间缺扫。误差在全部可用 IMU 时刻统计, 激光基线保持最新收到的位姿。

| $\omega/\text{rad/s}$ | 模式   | 位置 RMSE/m | 航向 RMSE/rad | 缺扫 RMSE/m |
|-----------------------|------|-----------|-------------|-----------|
| 0.4                   | 保持激光 | 0.0505    | 0.0503      | 0.1518    |
| 0.4                   | 融合   | 0.0044    | 0.0003      | 0.003     |
| 1.2                   | 保持激光 | 0.1522    | 0.1536      | 0.4439    |
| 1.2                   | 融合   | 0.0131    | 0.0003      | 0.0045    |

两个转速下激光分别拒绝 4、7 帧, EKF 均无额外拒绝。融合补偿了采样间运动和交付延迟; 圆周运动的线速度为  $v = r\omega$ , 本例半径为 1 m, 所以两种速度为 .4 和 1.2 m/s。仅 30 ms 延迟就对应约 .012 和 .036 m 的运动距离; 保持上一帧位姿时, 还会叠加激光采样间隔内的位移。融合器用 IMU 传播这段运动, 因而在高频评估中误差更小。可以先把延迟设为零, 再逐渐加大, 用这个数量级解释曲线的变化。

**本章练习:** 完成两道代码练习; 改变转速、延迟或缺扫时长, 比较误差与拒绝次数。

本章一帧扫描的所有光束取自同一时刻; 继续研究逐束采样时, 可用本章的时间轴为每束查询位姿, 补偿扫描过程中的运动。

## 第十章 回到起点之后：关键帧与位姿图

**目标** 用相对运动约束共同调整轨迹，并按新位姿重建观测地图。

**先修** 第 02、08、09 章的坐标变换、最小二乘和传感器融合。

### 10.1 从一条链变成一张图

机器人逐帧估计运动，微小误差会随路程积累。走一圈回到熟悉的墙角时，里程计的终点可能偏离起点。此时旧扫描和新扫描提供了额外信息：这两个时刻其实看到了同一个地点，它们的相对位置可以再测一次。这条跨越很长时间的约束称为**回环边**。

先运行一个可以直接指定相对位姿的实验：

```
1 ros2 run motion2d pose_graph_demo tmp/ch10_graph
2 /usr/bin/python3 scripts/plot_ch10_graph.py
```

81 个节点沿  $4 \times 3$  m 矩形行走，每条相邻节点之间的测量都带有小误差。把这些测量依次复合，得到逐渐漂移的链。最后加入“末帧与首帧重合”的测量，优化器就会让所有节点共同调整。本章先弄清这个调整过程，再从实际扫描中寻找回环。

#### 10.1.1 先手算只有三个位置的图

为了看清“共同调整”，暂时只考虑一维。固定  $x_0 = 0$ ，顺序测量是  $x_1 - x_0 \approx 1$  m、 $x_2 - x_1 \approx 1$  m，额外测量却说  $x_2 - x_0 \approx 1.8$  m。三条边不能同时精确满足；令每条边采用相同的尺度，最小化残差平方和：

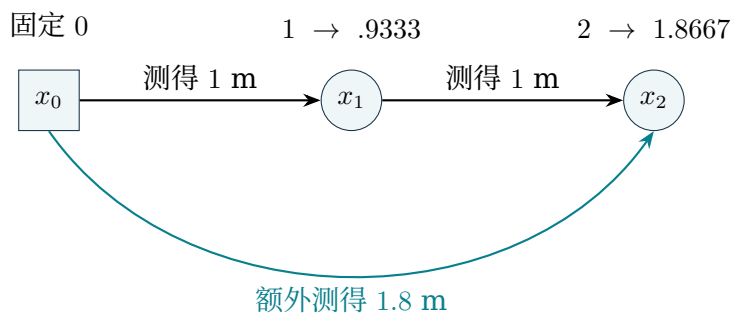
$$C(x_1, x_2) = \frac{1}{2}[(x_1 - 1)^2 + (x_2 - x_1 - 1)^2 + (x_2 - 1.8)^2]. \quad (10.1)$$

分别求偏导，注意第二项中  $x_1$  的系数为  $-1$ ：

$$\frac{\partial C}{\partial x_1} = (x_1 - 1) - (x_2 - x_1 - 1) = 2x_1 - x_2 = 0, \quad (10.2)$$

$$\frac{\partial C}{\partial x_2} = (x_2 - x_1 - 1) + (x_2 - 1.8) = -x_1 + 2x_2 - 2.8 = 0. \quad (10.3)$$

第一式给出  $x_2 = 2x_1$ ，代入第二式得到  $x_1 = .9333$  m、 $x_2 = 1.8667$  m。三个残差分别约为  $-.0667, -.0667, +.0667$  m：新增测量与原来的两个测量各分担了一部分误差。



图中的位置变量可以换成二维位姿，边也可以带不同的可信程度。接下来的计算就是把这个小例子推广到位置与航向一起变化的情况。

## 10.2 二维边残差：比较预测与测量

每个节点保存关键帧在 `map` 中的位姿  $T_i = (R_i, p_i)$ ，它把帧  $i$  中的点变到 `map`。边测量  $Z_{ij} = (R_z, z_{ij})$  把帧  $j$  的点变到帧  $i$ 。由第二章的逆变换与复合规则，当前节点预测的相对变换为

$$T_i^{-1}T_j = (R_i^T R_j, q), \quad q = R_i^T(p_j - p_i). \quad (10.4)$$

$q$  是“站在第  $i$  帧看，第  $j$  帧原点在哪里”。若预测与测量一致， $Z_{ij}^{-1}T_i^{-1}T_j$  就等于单位变换。再做一次复合，得到误差变换的平移与角度：

$$e_{ij} = \begin{bmatrix} R_z^T(q - z_{ij}) \\ \text{wrap}(\psi_j - \psi_i - \psi_{ij}) \end{bmatrix}. \quad (10.5)$$

前两项先在  $i$  帧中相减，再旋转到测量所定义的  $j$  帧方向；最后一项比较相对航向。本章直接用这三个数度量误差，所需运算都是二维旋转、平移和角度归一化。

例如  $p_i = (1, 2) \text{ m}$ 、 $\psi_i = 90^\circ$ ， $p_j = (1, 4) \text{ m}$ 、 $\psi_j = 90^\circ$ ，则  $q = R(-90^\circ)(0, 2) = (2, 0) \text{ m}$ ，相对航向为零。当测量也为  $z_{ij} = (2, 0) \text{ m}$ 、 $\psi_{ij} = 0$  时，残差为零。如果只把  $p_j$  的全局  $x$  增加  $.1 \text{ m}$ ， $q$  变成  $(2, -.1) \text{ m}$ ，残差为  $(0, -.1, 0)$ 。全局向右的小误差，在朝向北的  $i$  帧中表现为向右侧，即局部  $y$  负方向。

### 10.2.1 把米和弧度换成统一的误差尺度

给残差三个方向指定标准差  $\sigma_x, \sigma_y, \sigma_\psi$ ，分别除以标准差：

$$D_{ij} = \text{diag}(1/\sigma_x, 1/\sigma_y, 1/\sigma_\psi), \quad \tilde{e}_{ij} = D_{ij}e_{ij}. \quad (10.6)$$

每个新分量表示偏离了多少个标准差，已经没有单位。这个操作称为白化，这里采用三个方向相互独立的测量尺度；它们对应上式残差所在的坐标方向。例如残差  $(.06, 0, .04)$ 、尺度  $(.03, .03, .02)$ ，得到  $(2, 0, 2)$ ，平方代价为  $\frac{1}{2}(2^2 + 0^2 + 2^2) = 4$ 。尺度减半时，相同误差的平方代价变为四倍，所以该约束会更强地影响轨迹。

顺序边使用  $\frac{1}{2}\|\tilde{e}\|^2$ ；回环边对范数使用第八章的 Huber 损失，默认阈值为 3。它让很大的回环残差以线性速度增加代价，降低单条异常边对整张图的拉动。

### 10.3 固定第一帧，就是给地图选坐标原点

只有相对测量时，把整张图平移或转动都不会改变测量预测。设所有位姿同时左乘固定变换  $A$ ，利用  $(AT_i)^{-1} = T_i^{-1}A^{-1}$ ，有

$$(AT_i)^{-1}(AT_j) = T_i^{-1}A^{-1}AT_j = T_i^{-1}T_j. \quad (10.7)$$

也就是说，同一组相对残差对应无数张整体位置和朝向不同的图。在最小二乘方程里，这表现为有些增量方向不会改变代价，因而解不唯一。一维小例子若连  $x_0$  也一起优化，给三个  $x$  同时加任意常数就会发生这种情况。

固定首帧的  $x, y, \psi$  后，地图的三个整体自由度就确定了。代码把节点 0 留在原处，只把节点  $1, \dots, N-1$  的三个数写进优化变量，总维数为  $3(N-1)$ 。首帧数值来自坐标原点的选择，可以取零，也可以取其他给定位置。若图包含互不连接的两部分，第二部分仍可自行平移旋转，因此程序用图遍历检查所有节点能否从首帧到达，并以 `disconnected` 提示缺少连接约束。

### 10.4 对一条边的两端分别求导

采用加法更新  $p_i \leftarrow p_i + \delta p_i$ 、 $\psi_i \leftarrow \psi_i + \delta \psi_i$ 。记  $J = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$ 。由  $q = R_i^\top(p_j - p_i)$  直接得到

$$\frac{\partial q}{\partial p_i} = -R_i^\top, \quad \frac{\partial q}{\partial p_j} = R_i^\top, \quad \frac{\partial q}{\partial \psi_i} = -Jq, \quad \frac{\partial q}{\partial \psi_j} = 0. \quad (10.8)$$

第三项可从  $R_i^\top = R(-\psi_i)$  看出：对  $\psi_i$  求导为  $-JR_i^\top$ ，再乘位置差。第四项为零，是因为改变末帧的朝向并不改变它的原点位置。测量  $R_z, z_{ij}$  在求导时保持不变，因此  $e_p = R_z^\top(q - z_{ij})$  的每一块偏导只需在左边再乘  $R_z^\top$ 。角度残差对两端航向的导数分别为  $-1, +1$ ，于是

$$A_{ij} = \frac{\partial e}{\partial(p_i, \psi_i)} = \begin{bmatrix} -R_z^\top R_i^\top & -R_z^\top Jq \\ 0 & 0 & -1 \end{bmatrix}, \quad (10.9)$$

$$B_{ij} = \frac{\partial e}{\partial(p_j, \psi_j)} = \begin{bmatrix} R_z^\top R_i^\top & 0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (10.10)$$

例如  $R_i = R_z = I$ 、 $q = (2, 1)^\top$ ，则

$$A = \begin{bmatrix} -1 & 0 & 1 \\ 0 & -1 & -2 \\ 0 & 0 & -1 \end{bmatrix}, \quad B = I. \quad (10.11)$$

只增加起点航向  $.01 \text{ rad}$ ，预测残差约变化  $(.01, -.02, -.01)$ 。可以用这个数值例子核对源码的 `q.y()`，`-q.x()` 符号。航向残差跨越  $\pm\pi$  时先归一化；中心差分的两次扰动应落在同一角度分支内，检验局部导数。

```

10 GraphLinearization linearizeGraphEdge(const Pose2D & from, const Pose2D & to,
11   const Pose2D & measured)
12 {
13   const Eigen::Matrix2d ri = rotation(from.yaw).transpose();
14   const Eigen::Matrix2d rz = rotation(measured.yaw).transpose();
15   const Eigen::Vector2d q = ri * (to.position - from.position);
16   GraphLinearization result;
17   result.residual.head<2>() = rz * (q - measured.position);
18   result.residual.z() = wrapAngle(to.yaw - from.yaw - measured.yaw);
19   result.from.setZero(); result.to.setZero();
20   result.from.topLeftCorner<2, 2>() = -rz * ri;
21   result.from.topRightCorner<2, 1>() = rz * Eigen::Vector2d(q.y(), -q.x());
22   result.from(2, 2) = -1;
23   result.to.topLeftCorner<2, 2>() = rz * ri;
24   result.to(2, 2) = 1;
25   return result;
26 }
27 //

```

## 10.5 把许多小方程装成一个大方程

一条边只依赖两端节点。将白化雅可比记为  $J_i = DA$ 、 $J_j = DB$ ，一阶近似为  $\tilde{e}(\delta) \approx \tilde{e} + J_i \delta_i + J_j \delta_j$ 。第八章已从残差平方和推得  $H\delta = -g$ 。现在对这一条边展开二次代价，与两端有关的贡献恰好是

$$\begin{bmatrix} H_{ii} & H_{ij} \\ H_{ji} & H_{jj} \end{bmatrix} += w \begin{bmatrix} J_i^\top J_i & J_i^\top J_j \\ J_j^\top J_i & J_j^\top J_j \end{bmatrix}, \quad \begin{bmatrix} g_i \\ g_j \end{bmatrix} += w \begin{bmatrix} J_i^\top \tilde{e} \\ J_j^\top \tilde{e} \end{bmatrix}. \quad (10.12)$$

$+=$  表示把本条边的贡献加到已有数值上。普通边  $w = 1$ ；由第八章的 Huber 导数得到回环权重  $w = \min(1, 3/\|\tilde{e}\|)$ ，零残差时取 1。每个  $H_{ij}$  都是  $3 \times 3$  小块，没有边相连的节点对不会直接贡献块。因此尽管图有很多节点，矩阵中仍有大量零，称为**稀疏矩阵**。

节点  $k \geq 1$  对应大向量从  $3(k-1)$  开始的三个位置。下面代码对一条边的两个端点分别遍历，跳过固定节点，把四个小块按行列位置装进去。Triplet 存储“行、列、值”；setFromTriplets 会把落在同一位置的值相加。

```

105   for (int a = 0; a < 2; ++a) {
106     if (indices[a] == 0) {continue;} // Fixed anchor has no optimization variables.
107     const int row = 3 * (indices[a] - 1);
108     gradient.segment<3>(row) += weight * jacobian[a].transpose() * residual;
109     for (int b = 0; b < 2; ++b) {
110       if (indices[b] == 0) {continue;}
111       const int column = 3 * (indices[b] - 1);
112       const Eigen::Matrix3d block = weight * jacobian[a].transpose() * jacobian[b];
113       for (int i = 0; i < 3; ++i) {
114         for (int j = 0; j < 3; ++j) {entries.emplace_back(row + i, column + j, block(i, j));}
115       }
116     }
117   }
118   //

```

在三个位置的一维例子中，收集各边偏导可得  $H = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$ 。在初值  $(x_1, x_2) = (1, 2)$  处， $g = (0, .2)^\top$ 。解  $H\delta = -g$  得  $\delta = (-.0667, -.1333)^\top$ ，正好得到本章开头的答案。二维问题的旋转是非线性的，所以更新后要重新计算残差和雅可比，再求下一步。

### 10.5.1 为什么有时只走半步

一阶近似在增量较小时更可靠。求出  $\delta$  后，先尝试完整增量，若实际代价下降不够，再尝试  $\delta/2, \delta/4, \dots$ ，称为回溯线搜索。设当前总代价为  $C$ ，沿增量方向的一阶变化率是  $g^\top \delta$ 。因为  $H\delta = -g$ ，正定时  $g^\top \delta = -\delta^\top H\delta < 0$ ，这是一个下降方向。代码用条件

$$C(x + \alpha\delta) \leq C(x) + 10^{-4}\alpha g^\top \delta \quad (10.13)$$

要求实际下降达到一阶预测下降量的一小部分，并依次尝试  $\alpha = 1, 1/2, \dots, 1/1024$ 。所有增量分量都很小时报告收敛；若方程奇异、迭代耗尽或所有步长都被拒绝，保留当前图并返回对应状态，便于检查初值和边约束。

### 10.5.2 错误回环为何仍需先做几何检查

考虑一个位置  $x$ ，正常测量为 1 m，错误回环测量为 10 m，两者标准差均为 .1 m。普通平方代价的导数是  $100(x - 1) + 100(x - 10)$ ，令其为零得到  $x = 5.5$  m。若只对回环用阈值 3 的 Huber，在  $x$  接近 1 时，回环白化残差为  $10(x - 10) < -3$ ，Huber 对它的导数为  $-3$ ，再乘链式因子 10，得到对  $x$  的导数  $-30$ 。总导数于是变成  $100(x - 1) - 30$ ，最优位置为  $x = 1.3$  m，且满足所用的线性分支条件。鲁棒损失把错误回环的拉力限制住了，仍然留下 .3 m 偏移。所以回环进入图之前，还需要检查两帧点云是否重叠、配准是否稳定和往返变换是否一致。

## 10.6 矩形实验与可独立检验的答案

固定  $\text{seed}=1010$ , 沿四条边各走 20 步。每步加独立  $0.01\text{ m}$  平移噪声、 $0.002\text{ rad}$  航向噪声, 并注入每步  $0.003\text{ rad}$  航向偏差。顺序边的测量尺度为  $(.03, .03, .02)$ ; 闭环观测为单位变换, 尺度  $(.02, .02, .01)$ 。参考轨迹仅用于评估, 不送给优化器。

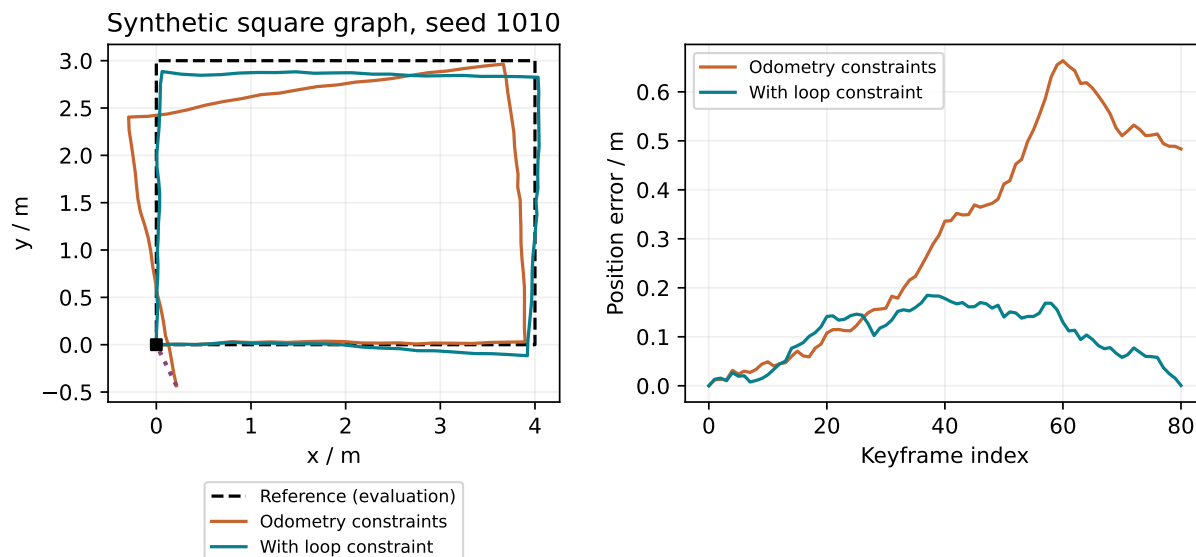


图 10.1: 合成位姿图的优化前后结果。左图等比例, 方块为固定首帧, 紫色虚线指出原始链的闭合缺口。

| 约束  | 位置 RMSE/m | 闭合误差/m | 最终代价  | 迭代 |
|-----|-----------|--------|-------|----|
| 顺序边 | 0.3735    | 0.4835 | 0     | 1  |
| 加回环 | 0.1183    | 0.0006 | 1.059 | 5  |

顺序边代价几乎为零, 因为链本身就是由这些边累乘出来的: 每个相邻位姿都精确满足带误差的测量。加入新约束后边之间有冲突, 最优代价可以大于零, 而参考误差反而下降。因此, 代价衡量图内约束的吻合程度, 位置 RMSE 则衡量相对于参考轨迹的误差; 应结合两者解释结果。

**练习 ch10-1:** 补完 `edge_residual.hpp`, 检查旋转坐标和角度跨界。理解题证明左乘不变性; 实验题改变回环尺度, 比较全程 RMSE、闭合误差、代价与求解状态。常见错误包括变换乘法顺序颠倒、只锚定位置却漏掉航向、把错误闭环交给鲁棒核“自动判断”。

**手算练习:** 在三节点例子中将额外边的标准差减半。这相当于把它的平方项乘以 4; 重新求两个偏导为零的方程, 观察误差如何重新分配。

## 10.7 从关键帧中寻找旧地点

完整管线沿用第九章前端；后端只接收扫描及同一采样时刻的校正位姿。平移达到 0.3 m、航向变化达到 0.25 rad，或距上帧达到 2 s，就保存一个关键帧。其中既有原始距离数组，也有机体体系的素点云和局部法向量。机体系统观测保留了采样时的原始几何，后面改变关键帧的全局位姿时，可用它重新投影地图。

```
1 ros2 launch motion2d_bringup ch10.launch.py rviz:=false
2 # Another terminal, same ROS environment, repository root:
3 /usr/bin/python3 scripts/check_ch10.py
```

排除最近 20 个关键帧后，只在当前估计位置的 0.8 m 范围内寻找候选，航向差小于 0.6 rad。每次最多检查距离最近的三个候选。候选搜索以当前估计为中心，适合累计漂移仍小于搜索半径的场景。漂移超过半径时，旧地点会落在窗口之外；扩大窗口能找回更多候选，也会增加误匹配的机会。成功回环后至少等待五个新关键帧，避免高频重复加入同一个地点的相关证据。

| 几何检查      | 默认条件                                   |
|-----------|----------------------------------------|
| 双向点到线 ICP | 两个方向都收敛，且没有秩退化                         |
| 有效对应比例    | 两个方向均至少 55%                            |
| 残差与初值偏离   | RMSE 不超过 0.05 m；校正不超过 0.5 m / 0.25 rad |
| 往返一致性     | 平移不超过 0.03 m，航向不超过 0.02 rad            |

若正向变换为  $T_{ij}$ ，反向为  $T_{ji}$ ，则一致时  $T_{ij}T_{ji} \approx I$ 。将正向变换写成  $(R_f, t_f)$ 、反向变换写成  $(R_b, t_b)$ ，复合后的平移为  $t_f + R_f t_b$ ，角度为  $\text{wrap}(\psi_f + \psi_b)$ 。例如正向平移 (1, 0) m、转 90°，反向应平移 (0, 1) m、转 -90°；因为  $R(90^\circ)(0, 1) = (-1, 0)$ ，往返平移才会相消。下面代码按这个复合公式检查；通过各项门限后才生成回环边。

```
29 const auto backward = matchClouds(reference.points, current, inverse(result.relative), config.match);
30 if (!backward.accepted() ||
31     static_cast<double>(backward.pairs) / reference.points.size() < config.min_overlap) {
32     result.status = "reverse_rejected"; return result;
33 }
34 const auto cycle = compose(result.relative, backward.pose);
35 if (cycle.position.norm() > config.cycle_translation || std::abs(cycle.yaw) > config.cycle_yaw) {
36     result.status = "inconsistent_cycle"; return result;
37 }
38 result.accepted = true; result.status = "verified";
39 //
```

平行长墙沿墙方向缺少约束，可由第八章的秩检查识别。重复房间则可能在两个方向都得到相似的点云：即使往返变换很小，仍需结合更长时间的运动历史或更有辨识度的场景特征区分地点。可以主动构造这两种场景，分别观察退化拒绝和地点混淆。图优化失败时撤掉新回环边，继续使用已有的局部里程计和顺序边。

当前最多保留 200 个关键帧，达到容量后以 `capacity_reached` 提示停止加入新帧。关键帧越多，逐帧重建地图的计算量也越大；更大场景可将若干关键帧组成子图，按子图组织更新。

## 10.8 回环更新地图坐标，不改写局部里程计

同一机器人有局部位姿  $T_{OB}$  和图优化后的全局位姿  $T_{MB}$ ，满足

$$T_{MB} = T_{MO}T_{OB}, \quad T_{MO} = T_{MB}T_{OB}^{-1}. \quad (10.14)$$

第二式由第一式在右边乘  $T_{OB}^{-1}$  得到。展开为旋转和平移，就是

$$R_{MO} = R_{MB}R_{OB}^{\top}, \quad p_{MO} = p_{MB} - R_{MO}p_{OB}. \quad (10.15)$$

例如同一个关键帧的全局位姿为  $p_{MB} = (10, -2) \text{ m}$ 、 $\psi_{MB} = 180^\circ$ ，局部位姿为  $p_{OB} = (2, 1) \text{ m}$ 、 $\psi_{OB} = 90^\circ$ 。则校正旋转为  $90^\circ$ ，先算  $R_{MO}p_{OB} = (-1, 2)$ ，再相减得到  $p_{MO} = (11, -4) \text{ m}$ 。将其代回  $p_{MB} = p_{MO} + R_{MO}p_{OB}$ ，即可核对  $(10, -2)$ 。程序用最新关键帧的两种位姿计算这个校正，原始 `odom` 位姿继续保存自己的运动历史。**练习 ch10-2** 完成 `map_alignment.hpp`，先手算带  $90^\circ$  旋转的例子。仅把两个位置相减会漏掉旋转，无法得到正确变换。

```
77 for (std::size_t k = 0; k < frames_.size(); ++k) {frames_[k].map_pose = result.poses[k];}
78 alignment_ = compose(frames_.back().map_pose, inverse(frames_.back().odom_pose));
79 ++loops_; last_loop_ = current;
80 //
```

一条 TF 边只有一个发布者：SLAM 发布 `map` 到 `odom`；`estimated` 适配器发布 `odom` 到 `base_link`；模拟器只保留圆心处传感器的静态外参。回环可以让机器人在 `map` 中跳到更一致的位置，但不会把这个全局位移直接写入局部运动链。第十九章会处理全局目标与局部控制参考的衔接。

| 输出                                          | 含义               |
|---------------------------------------------|------------------|
| <code>/map/cloud</code> 与 <code>/map</code> | 按最新图位姿重建的点云与观测栅格 |
| <code>/path/graph_before</code>             | 固定首帧基准中的原始里程计关键帧 |
| <code>/path/graph_after</code>              | 同一基准中的优化关键帧      |
| <code>/slam/edges</code>                    | 顺序边与紫色回环边        |
| <code>/slam/status</code>                   | 更新/拒绝状态及关键帧、回环计数 |

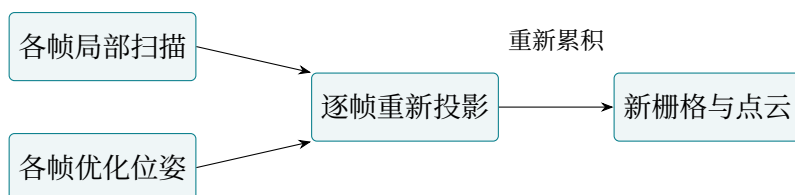
前后路径都用首帧固定的 `map` 基准比较，不再用当前 `map` 到 `odom` 把原始基线变换第二次。高频局部轨迹仍由第九章输出；图更新频率是关键帧频率。

**为什么要重建：**设旧图在  $x = 3 \text{ m}$  有一面墙，优化后这次观测应落在  $x = 4 \text{ m}$ 。直接追加新点会留下两面墙。重建时新建空栅格，再遍历所有保留的原始扫描，按照各自优化后的采样位姿重新做射线清空与端点占据；体素点云也重新生成。本章累积的是关键帧的实际观测，未被光线照到的区域保留未知状态。

只改一条全局 TF，也无法修正不同历史帧之间的相对错位。设旧点云中两个点为  $u, v$ ，施加相同刚体变换后，它们的距离仍为

$$\|(Ru + t) - (Rv + t)\| = \|R(u - v)\| = \|u - v\|. \quad (10.16)$$

最后一步使用  $R^{\top}R = I$ ：旋转保持长度。因此要把两次错开的墙面观测重新对齐，就必须分别使用各自优化后的关键帧位姿。



## 10.9 实际激光与 IMU 的矩形回放

```
1 ros2 run motion2d slam_demo tmp/ch10_slam
2 /usr/bin/python3 scripts/plot_ch10_slam.py
```

使用  $14 \times 12$  m 房间，内部有一个圆与一个三角形。半径 0.2 m 的全向圆盘沿  $6 \times 4$  m 矩形行走，每边 6 s，平滑的时间规律让它在拐角静止（第十四章推导这条五次曲线）；机体 yaw 保持零。四条边均通过整段扫描检查。360 束、距离噪声 0.03 m、激光 10 Hz、IMU 200 Hz，噪声 seed=4242/6060。

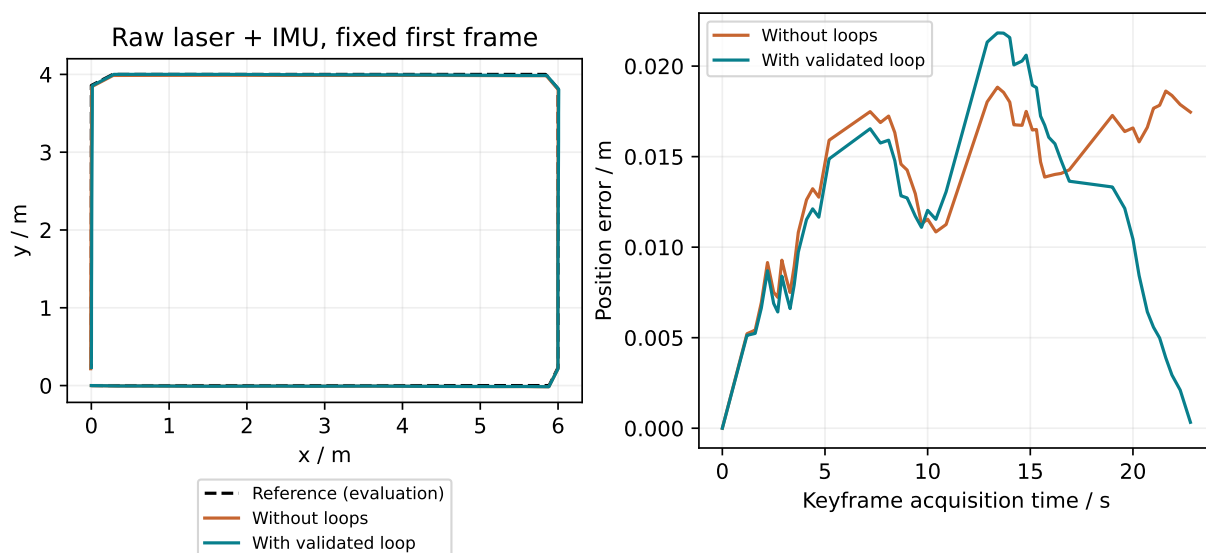


图 10.2: 同一真实传感器前端输出，分别送给回环关闭和开启的图。误差只统计关键帧；参考轨迹仅用于评估。

| 模式   | 关键帧 | 回环 | 位置 RMSE/m | 末关键帧误差/m |
|------|-----|----|-----------|----------|
| 关闭回环 | 53  | 0  | 0.0144    | 0.0175   |
| 开启回环 | 53  | 1  | 0.0132    | 0.0003   |

本例在 22.3 s 检查一个候选并接受回环；最后关键帧时间为 22.8 s。表中的末帧误差取 22.8 s 的关键帧位置与同一时刻参考位置之差；首尾闭合距离则取末帧与首帧之差，两者使用的参照不同。激光拒绝 27 帧，EKF 无额外拒绝；失败扫描没有进入图。

回环显著改善了靠近返回地点的关键帧，但部分中间点误差反而略增。全程位置 RMSE 从约 1.44 cm 降至 1.32 cm；末帧误差则从约 1.75 cm 降至 0.033 cm。末帧接近回环约束的位置，因而改善更明显；其他节点还要兼顾沿途顺序边。所有 CSV、原始地图 PGM 与点云 CSV 均可由上述命令重新生成。

## 10.10 从地图中观察回环的作用

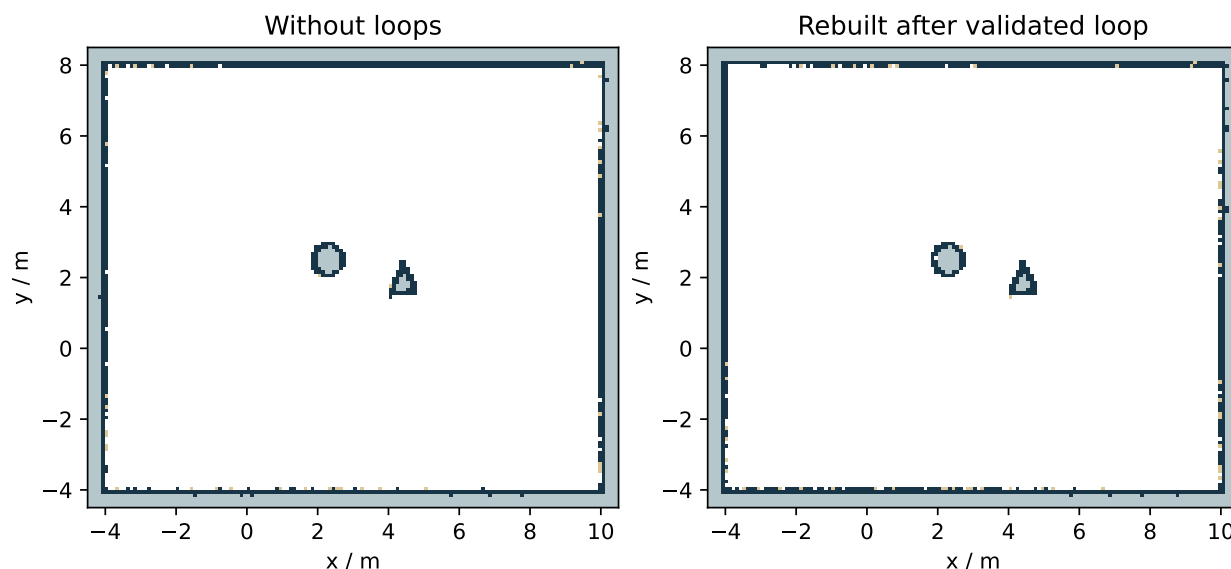


图 10.3: 两图均由关键帧扫描构建, 分辨率 0.1 m。白色自由, 深色占据, 灰色未知, 浅褐色为已观测但不确定; 纵横尺度相同。

本例原始误差只有厘米量级, 小于地图分辨率, 因此两张全景图很接近。可以做一个更容易看清的实验: 只把某个关键帧的墙面观测位姿移动一个格子, 分别尝试追加与重建。追加会留下旧墙, 重建则根据所有关键帧的新位姿重新决定每个格子的证据。圆与三角形内部没有被光线穿透, 仍然保留未知区域。

启动默认 RViz 配置时, 应看到优化关键帧、紫色回环边、重建点云和占据图; 原始世界与真值机器人默认隐藏。可修改配置中的 `slam.enable_loops` 为 `false` 对照。

1. **理解:** 为什么只有修改 TF、不重建历史点云, 会得到坐标正确但几何仍旧扭曲的地图?
2. **代码:** 完成两道练习, 检查边残差的变换顺序和 `map` 到 `odom` 的计算。
3. **实验:** 改变候选距离、关键帧间隔或激光噪声, 记录候选数、接受数、拒绝帧与误差。
4. **失败:** 用单面墙验证退化, 用无重叠云验证拒绝; 达到图容量时应看到明确状态。

本章将激光/IMU 局部定位、几何回环和地图重建连成了一个小场景 SLAM 系统。继续扩大场景时, 可以研究全局地点识别、动态物体处理及子图管理。下一章从真值定位和观测地图开始学习路径搜索, 再逐步将规划与本章的定位建图结果连接起来。

# 第十一章 给圆盘找一条路：配置空间与 A\*

**目标** 将圆盘规划化成中心点搜索，理解栅格、未知区域、路径与轨迹的区别。

**先修** 第 03、07 章。先在给定栅格上检查算法，再接观测地图。

## 11.1 先运行一张带窄门的地图

```
1 ros2 run motion2d astar_demo tmp/ch11_astar
2 /usr/bin/python3 scripts/plot_ch11_astar.py
```

这次输入是手工给定的  $41 \times 31$  栅格，分辨率 0.1 m。中间隔墙有宽 0.7 m 的门；起点 (.75, .75) m，终点 (3.25, 2.25) m。改变半径，观察门从可通行变成不可达。该小实验隔离规划算法，不冒充未知环境建图。

## 11.2 圆盘变成一个中心点

设原始阻塞区域为  $\mathcal{O}$ ，圆盘物理半径为  $r$ ，额外安全裕量为  $m$ 。中心点必须避开膨胀集合

$$\mathcal{O}_{\text{conf}} = \mathcal{O} \oplus B(0, r + m). \quad (11.1)$$

这里  $\oplus$  表示将集合中的每个点向外扩张一个半径  $r + m$  的圆。半径与裕量只加一次；后面使用已经膨胀的地图时不能再重复扣除机器人半径。

原始占据栅格表示方格的面积，不是只有格子中心的一个点。本章还要求输出自由格的整个闭区域都安全，方便后续构造走廊。两个边长  $\rho$  的格子，相对索引差为  $(\Delta i, \Delta j)$ ，其最小距离为

$$d_x = \rho \max(0, |\Delta i| - 1), \quad d_y = \rho \max(0, |\Delta j| - 1), \quad d = \sqrt{d_x^2 + d_y^2}. \quad (11.2)$$

若  $d \leq r + m$ ，候选格子被阻塞。相邻方格共享边或角时距离为零；接触也视为碰撞。

```
47 for (int y = -ny; y <= ny; ++y) {
48     for (int x = -nx; x <= nx; ++x) {
49         const double dx = std::max(0, std::abs(x)-1)*resolution;
50         const double dy = std::max(0, std::abs(y)-1)*resolution;
51         if (std::hypot(dx, dy) <= r + 1e-12) {offsets.emplace_back(x, y);}
52     }
53 }
54 //
```

这种整格安全证书比只检查中心更保守。即使  $r = 0$ ，与障碍接触的相邻格仍会被排除。连续几何可以通过的窄门，粗网格可能判不可达；缩小  $\rho$  能减少这类保守损失。地图外部也视为阻塞，边界附近格子同样要满足距离要求。

## 11.3 A\* 的三个数与两种邻接

$g(n)$  是从起点到节点  $n$  已知的最小代价,  $h(n)$  是到目标的下界, 优先展开

$$f(n) = g(n) + h(n). \quad (11.3)$$

四邻接每步长  $\rho$ , 使用 Manhattan 距离; 八邻接另有长  $\sqrt{2}\rho$  的对角步, 使用 octile 距离。设到目标的索引差绝对值为  $d_x, d_y$ , 则

$$h_4 = \rho(d_x + d_y), \quad (11.4)$$

$$h_8 = \rho[\max(d_x, d_y) + (\sqrt{2} - 1) \min(d_x, d_y)]. \quad (11.5)$$

无障碍时这就是最短距离; 障碍只会让路更长, 因此不会高估。它们还满足一致性, 节点确定后不必反复重新打开。

```

47     for (int dy = -1; dy <= 1; ++dy) {
48         for (int dx = -1; dx <= 1; ++dx) {
49             if ((dx == 0 && dy == 0) || (!diagonal && dx != 0 && dy != 0)) {continue;}
50             const int x = current%width, y = current/width;
51             if (!grid.freeCell(x+dx, y+dy)) {continue;}
52             if (dx != 0 && dy != 0 && (!grid.freeCell(x+dx, y) || !grid.freeCell(x, y+dy))) {continue;}
53             const int next = (y+dy)*width+x+dx;
54             const double candidate = g + grid.geometry.resolution * std::hypot(dx, dy);
55             if (candidate < distance[next]) {
56                 distance[next] = candidate; parent[next] = current;
57                 open.emplace(candidate+heuristic(next), candidate, next);
58             }
59         }
60     }
61     //

```

关键是中间的对角检查: 终点自由不代表连线安全。例如右邻格被堵、上邻格自由, 即使右上格自由, 也不能沿对角线擦过右邻障碍的角。**练习 ch11-1** 补完 `diagonal_step.hpp`, 分别检查两侧被挡的情况。

程序保留每个节点的父节点, 从目标反向回溯, 再反转得到格子序列。输出折线包含精确请求的起终点; 它们可以不在格子中心。因为起终格的整个面积已保证安全, 格内接到中心的短线仍然安全。两个点在同一格时直接相连, 不必绕到格子中心。

## 11.4 未知不是免费的捷径

只有已观测且概率编码不大于 35 的格子作为原始自由格；未知 -1、不确定与占据均默认阻塞。这是本课程的规划策略，不是 ROS 强制规定。显式设置 `unknown_blocked=false` 可以做乐观探索实验，但不能把未观测空间称为已验证安全。

搜索结果必须区分：

| 状态                           | 含义                |
|------------------------------|-------------------|
| found                        | 在当前栅格和规则下找到路径     |
| invalid_start / invalid_goal | 起点或终点越界、被堵或接触阻塞边界 |
| unreachable                  | 两点有效，但自由格图不连通     |

失败时返回空路径和无穷大代价。成功的 `grid_cost` 是中心节点图上的最短距离，不是含精确端点及后续简化的折线总长。两者的定义不能混用。

## 11.5 删掉折点之前，检查整条线段

贪心简化从当前点开始，寻找能直视的最远后续点，再继续。这是减少折点的实用方法，不保证连续空间的全局最短路线。检查线段  $p(t) = a + t(b - a)$ ,  $t \in [0, 1]$  与每个候选阻塞方格的闭区间是否相交。对每个轴施加  $l \leq p(t) \leq u$ ，把允许的  $t$  区间相交；非空就是接触或穿过障碍。

本实现遍历线段包围框中的格子，采用解析区间裁剪。它检查细障碍、擦角和沿边接触，不用固定间距采样“猜测安全”。代价是长斜线可能检查较多格子；之后可优化为 `supercover DDA`，但应保留当前实现作为容易审查的对照。

**建图射线遍历不能直接当作碰撞检查** 第七章 DDA 在恰好经过角点时同时跨两个轴，跳过只被角点触到的旁边格子。这适合那里的观测更新约定，但会漏掉规划中的角点接触。相同的网格不表示相同的遍历语义。

**常见错误：**只看对角终点；把未知默认当作自由；只查简化线段上的少量采样点；扩大半径后仍使用旧的膨胀图；将找到的一串点当成满足速度、加速度和控制约束的轨迹。本章输出只有几何折线，时间分配和连续曲线将在第十四章开始。

## 11.6 半径、邻接与窄门实验

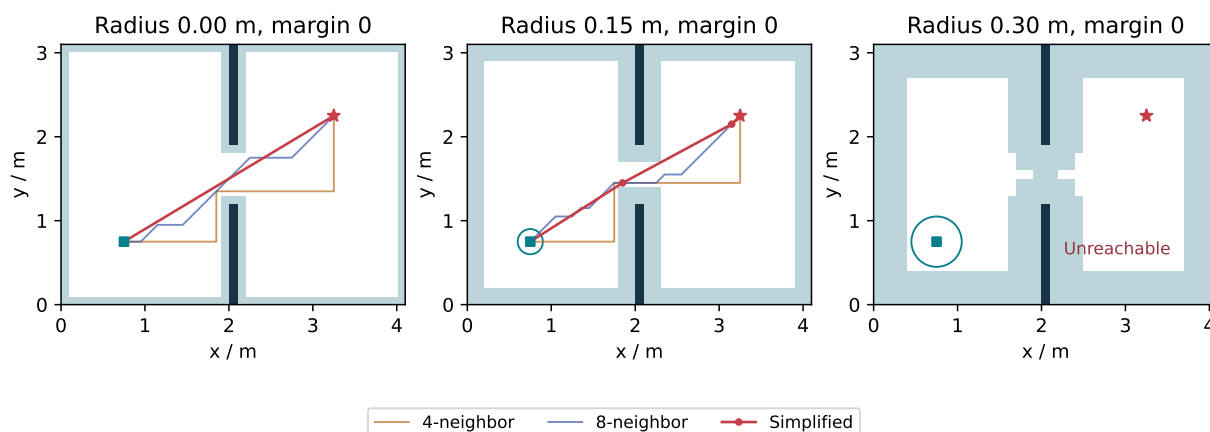


图 11.1: 深色是给定隔墙, 浅蓝是额外阻塞的配置空间格子, 白色可规划。绿方块为起点, 红星为目标; 绿圆保持真实半径与坐标等比例。红线为八邻接结果的简化。

| 半径/m | 邻接 | 状态  | 图代价/m   | 扩展节点 | 简化点数 |
|------|----|-----|---------|------|------|
| 0    | 4  | 找到  | 4       | 317  | 2    |
| 0    | 8  | 找到  | 3.12132 | 142  | 2    |
| 0.15 | 4  | 找到  | 4       | 281  | 4    |
| 0.15 | 8  | 找到  | 3.12132 | 114  | 4    |
| 0.3  | 4  | 不可达 | —       | 283  | 0    |
| 0.3  | 8  | 不可达 | —       | 283  | 0    |

本实验裕量为零, 半径分别为 0、0.15、0.30 m。四邻接成功时图代价为 4 m, 八邻接约为 3.12132 m。两种半径的成功代价相同不表示路径完全相同: 栅格中可以存在多个等长路线。扩展节点数同时受启发式、可用区域和并列排序影响, 不能当作一般性能承诺。

0.7 m 门能在连续几何中容纳直径 0.6 m 的圆盘, 但本例找不到整格都安全的贯通路径。这是明确的离散保守性, 不是把真实机器人偷偷变大; 分辨率、半径与整格安全要求共同决定结果。下一步接入观测地图后, 还要区分几何变窄与尚未看见造成的封闭。

- 理解:** 在小网格上手算  $g, h, f$ , 解释四邻接与八邻接启发式的不同。
- 代码:** 完成邻居检查, 覆盖外边界、两侧被挡和合法对角。
- 实验:** 改变半径、分辨率或未知策略, 记录可达性、代价与简化点数。

**验收:** 30 组 seed=1111 随机栅格与独立 Dijkstra 的最短代价一致; 另检查已知四/八邻接距离、精确端点、同格路径、窄门、未知、外界、穿角、线段擦边/擦角与简化后安全。下面接入 ROS 观测地图和点击目标。

## 11.7 从观测地图接入 ROS 与 RViz

```
1 ros2 launch motion2d_bringup ch11.launch.py
2 ros2 topic pub --once /goal_pose geometry_msgs/msg/PoseStamped \
3   '{header: {frame_id: map}, pose: {position: {x: -7.5, y: -8.0}, orientation: {w: 1.0}}}'
4 ros2 topic echo /plan/status
```

本章默认使用 **ideal** 模型，机器人静止扫描；IMU 关闭，因为本章定位直接来自显式真值适配器。启动后等几帧观测累积，再在 RViz 的 2D Goal Pose 工具点击白色自由区域。工具发布到 `/goal_pose`，规划器只生成几何路径；执行轨迹留到第十四章。目标朝向在本章忽略，未来轨迹和控制器分别处理 yaw。

| 方向 | 接口                                                  | 含义                             |
|----|-----------------------------------------------------|--------------------------------|
| 输入 | <code>/map</code>                                   | 仅凭观测累积的占据栅格                    |
| 输入 | <code>/odometry</code> 、TF                          | 当前位姿与 <code>map/odom</code> 关系 |
| 输入 | <code>/goal_pose</code>                             | <code>map</code> 中的二维目标位置      |
| 输出 | <code>/planning/grid</code>                         | 0 自由、100 阻塞的配置空间掩码             |
| 输出 | <code>/plan/path</code> 、 <code>/plan/status</code> | <code>map</code> 折线与显式规划状态     |

规划器不订阅障碍物列表、仿真位姿或真值里程计。它使用选源后的 `/odometry`；此启动配置的定位来源仍是**真值**，地图来源则是**激光观测**。后续替换定位来源无需修改 A\* 内核。

$${}^{map}T_{base} = {}^{map}T_{odom} {}^{odom}T_{base}. \quad (11.6)$$

TF 按里程计采样时刻查询；未能获取时发布 `waiting_transform` 并等待下一次尝试。不能把 `odom` 坐标直接当作 `map` 坐标。观测地图只接受 `map` 中轴对齐的栅格；原点含旋转时明确拒绝，不能悄悄丢掉旋转。

节点用 5 Hz 定时器合并输入更新：新目标、新地图或起点移动至少 0.05 m 时重新规划。一次搜索始终使用同一张配置空间栅格。地图和状态保留最后一条消息，便于晚加入的 RViz 读取。地图掩码不是占据概率；不要把它再喂给占据建图或重复膨胀。

```
138 const auto result = astar(*grid_, start.position, *goal_, diagonal_);
139 if (!result.success) {publish({}, result.status); return;}
140 const auto path = simplifyPath(*grid_, result.path);
141 publish(path, "found", result.path);
142 //
```

失败状态同时发布空 Path，清除旧结果；错误地图、错误目标坐标系与变换等待也如此。检测到地图或里程计时间回退时清除目标，`reset` 后须重新发送目标。已知完整世界中的远目标可能可达，但如果从当前观测地图看尚被未知区域隔开，本章应报告不可达或无效目标。探索和局部目标选择将在第十九章处理。

## 11.8 复现一条来自真实消息的路径

```
1 # Run ch11.launch.py rviz:=false in a separate terminal first.
2 /usr/bin/python3 scripts/check_ch11.py
3 /usr/bin/python3 scripts/plot_ch11_observed.py
```

检查脚本暂停、reset, 再执行 100 个 5 ms 单步, 固定在 0.5 s。默认 world/noise seed=42/4242, 720 束、10 Hz、测距标准差 0.01 m。地图 220×220 格, 分辨率 0.1 m; 圆盘半径 0.20 m, 额外裕量 0.05 m。脚本保存实际 /map、/planning/grid 与 /plan/path 消息, 再绘制下图。

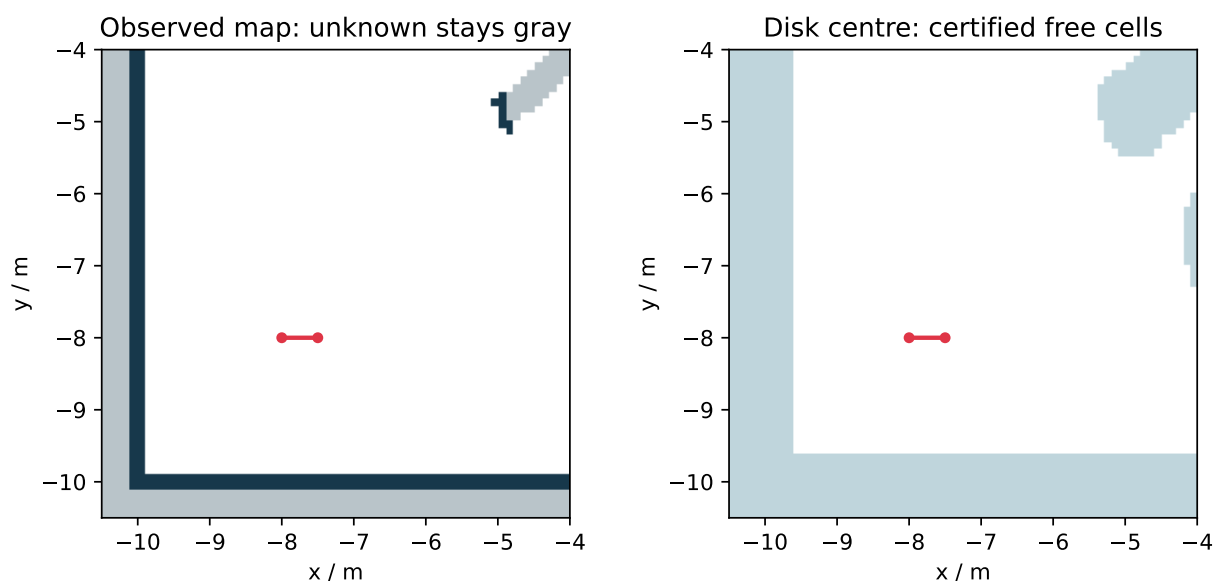


图 11.2: 默认世界左下角的局部视图。左侧灰色为未知, 深色为不满足自由阈值的观测; 右侧浅蓝含额外膨胀。红线从  $(-8, -8)$  到  $(-7.5, -8)$  m, 两侧都保持米制等比例。

本次得到 6872 个整格安全的自由格, 简化路径含两个端点。机器人仍停在起点; 给  $(8, 8)$  m 的目标返回 `invalid_goal`, 错误 frame 返回 `invalid_goal_frame_or_position`, 两种失败都清空路径。还检查输入隔离、时间重置和唯一 TF 来源; 独立测试用旋转  $90^\circ$ 、平移  $(1, 2)$  m 的变换验证坐标组合。RViz 配置已提供, 本次主机验收为无界面 ROS 运行, 未将 GUI 画面列为已验收。

**练习 ch11-2:** 实现 `pathLength`, 累计相邻实际坐标之间的欧氏距离。空路径和单点长度为零; 含重复点时不要额外增加距离。用 3-4-5 三角形与多段折线验证, 比较 A\* 的 `grid_cost`、真实端点折线长度与简化长度。

**排查:** 看不到路径时, 先读 `/plan/status`, 再检查目标 frame、观测是否足够、半径及未知策略。在 RViz 勾选 `Configuration grid` 可查看规划用的掩码。不要为了得到路径而把未知策略悄悄改为乐观。

## 第十二章 离障碍多远：二维距离场

**目标** 把占据标签变成距离，查询连续位置的值与梯度，明确哪些距离可以用于安全判定。

**先修** 第 07、11 章；范数、偏导数和双线性插值。

### 12.1 先看一个圆的距离场

```
1 ros2 run motion2d esdf_demo tmp/ch12_esdf
2 /usr/bin/python3 scripts/plot_ch12_esdf.py
```

实验把半径 1 m 的圆按格子中心采样为占据图，再比较三种分辨率。连续圆的解析有符号距离是  $d_{circle}(p) = \|p\| - 1$ ：圆外正、圆内负、边界零。除圆心外，其梯度为  $p/\|p\|$ ；圆心没有唯一的梯度方向。

### 12.2 先确定你计算的是哪一种距离

本章输入是未膨胀的观测栅格。沿用上一章标签策略：已观测且编码不超过 35 才自由，未知默认阻塞；外围补一圈阻塞格，表达地图外部不安全。在格子中心  $c_i$  定义离散场

$$d_i = \begin{cases} \min_{j \in O_c} \|c_i - c_j\|, & i \text{ 自由}, \\ -\min_{j \in F_c} \|c_i - c_j\|, & i \text{ 阻塞}. \end{cases} \quad (12.1)$$

$O_c, F_c$  是阻塞与自由格中心集合。这是有符号中心距离，不是到完整方格边界的精确距离。例如宽  $\rho$  的直墙格外，相邻自由中心距墙格中心为  $\rho$ ，但距墙边只有  $\rho/2$ 。整张地图阻塞时没有自由中心，内部值为  $-\infty$ ，查询明确返回不可用。

### 12.3 欧氏距离变换：先平方，后开方

平方距离可以分离成 x、y 两项。先逐列，再逐行求最小值：

$$D(x, y) = \min_{i,j} [(x-i)^2 + (y-j)^2 + f(i, j)], \quad f = \begin{cases} 0 & \text{种子格}, \\ +\infty & \text{其他格}. \end{cases} \quad (12.2)$$

$d = \rho\sqrt{D}$  才是米制距离，不能直接把  $D$  当作 ESDF。以阻塞格为种子得到外部距离，以自由格为种子得到内部距离；随后按标签赋予正负号。二维变换复用同一段一维算法；源码入口见本章运行说明。

## 12.4 一维最小值为什么可以快速求

给定数组  $f(q)$ ，每个有限元素贡献一条抛物线  $f(q) + (x - q)^2$ 。我们要的是这些曲线中最低的一条。按中心位置递增加入曲线，保存仍有机会成为最小值的中心索引，以及每条曲线从哪里开始胜出。新曲线与旧曲线  $v$  的交点为

$$s = \frac{f(q) + q^2 - f(v) - v^2}{2(q - v)}. \quad (12.3)$$

若交点早于旧曲线开始胜出的区间，旧曲线永远不再最低，可以删除。每条曲线最多加入和删除一次；得到分界点后，再从左到右读取最小值。这个下包络算法来自 Felzenszwalb 与 Huttenlocher 的 EDT 论文，正文与原始链接见 REFERENCES.md。本课程按公式独立实现，并用暴力最近中心搜索核对结果。

```

22 while (last >= 0) {
23     const int v = sites[last];
24     crossing = ((f[q]+double(q)*q)-(f[v]+double(v)*v))/(2.0*(q-v));
25     if (crossing > begin[last]) {break;}
26     --last;
27 }
28 ++last; sites[last] = q; begin[last] = last == 0 ? -inf : crossing;
29 //

```

## 12.5 连续位置的值与梯度

先换算到以格子中心为采样点的坐标：

$$\xi = \frac{x - o_x}{\rho} - \frac{1}{2}, \quad \eta = \frac{y - o_y}{\rho} - \frac{1}{2}. \quad (12.4)$$

令  $i = \lfloor \xi \rfloor, j = \lfloor \eta \rfloor$ ，局部比例  $u = \xi - i, v = \eta - j$ 。四个中心值按 00, 10, 01, 11 排列：

$$\tilde{d} = (1 - v)[(1 - u)d_{00} + ud_{10}] + v[(1 - u)d_{01} + ud_{11}]. \quad (12.5)$$

对权重求导，并用  $du/dx = dv/dy = 1/\rho$  转为米制梯度。

```

12 const double value = (1-v)*((1-u)*d[0]+u*d[1]) + v*((1-u)*d[2]+u*d[3]);
13 const double dx = ((1-v)*(d[1]-d[0])+v*(d[3]-d[2]))/resolution;
14 const double dy = ((1-u)*(d[2]-d[0])+u*(d[3]-d[1]))/resolution;
15 return {value, {dx, dy}};
16 //

```

查询范围为最外围格子中心组成的闭矩形；越界或角点距离无穷时返回空。梯度在插值块边界可能不连续；跨自由/阻塞相邻中心时，值从  $-\rho$  变到  $+\rho$ ，斜率为 2。所以不要强行把本章梯度解释成处处单位长度的精确 SDF 法向。

## 12.6 把距离与安全判定分开

插值会产生误差，障碍还有方格面积。仅使用  $\tilde{d} > r + m$  可能高估可用净空。设当前查询点的四个插值中心为  $c_k$ ，权重为  $w_k$ ，且四个中心均自由。无符号最近阻塞中心距离  $d_c$  满足

$$|d_c(p) - d_c(c_k)| \leq \|p - c_k\|. \quad (12.6)$$

对四个不等式加权求和，可从插值值减去  $E = \sum_k w_k \|p - c_k\|$  得到  $d_c(p)$  的下界。每个障碍方格离中心最远为半对角线  $h = \rho/\sqrt{2}$ ，再由三角不等式得到

$$\underline{d}(p) = \max \left( 0, \tilde{d}(p) - \sum_{k=1}^4 w_k \|p - c_k\| - \frac{\rho}{\sqrt{2}} \right). \quad (12.7)$$

这就是 `sample` 返回的 `clearance_lower_bound`，针对**方格并集与地图外部**。四个角中存在阻塞格时返回 0；0 只说明未给出正净空证书，不表示一定碰撞。此下界是本课程的保守选择，可能比精确方格距离小很多。

**圆盘半径只处理一次** 本章从未膨胀图计算距离，用  $d > r + m$  验证单个圆心位置。严格大于对应接触也算不安全。若使用第十一章已膨胀的配置空间，则按中心点检查，不再扣一次物理半径。

返回的 `gradient` 是  $\tilde{d}$  的导数，**不是这个下界的导数**。轨迹优化可以借助连续插值的值和梯度获得改进方向；优化完成后必须单独验证安全。离散采样满足净空不保证采样点之间的曲线安全，第十三、十六章会引入凸走廊与整段包含性检查。

## 12.7 核对距离、梯度与离散误差

三个层次分别检查：

1. **算法**：50 张 `seed=1212` 小栅格，逐格枚举所有种子，核对平方距离。无种子、单行及稀疏整列也覆盖。
2. **数值**：直墙的符号/米制距离有解析值；对双线性函数和实际距离场做中心有限差分，梯度误差要求小于  $10^{-9}$ ，查询点避开插值块边界。
3. **几何**：1000 个 `seed=1213` 查询点，逐个障碍方格算点到闭方格的精确距离，再取与外部距离的最小值，确认下界从不超过它，容差  $10^{-12}$  m。

这些检查针对栅格所表达的几何，不能消除传感器噪声、漏检和建图错误。若显式允许未知自由，证书只针对这张乐观地图，不能声称验证过真实未知空间。

## 12.8 分辨率实验与练习

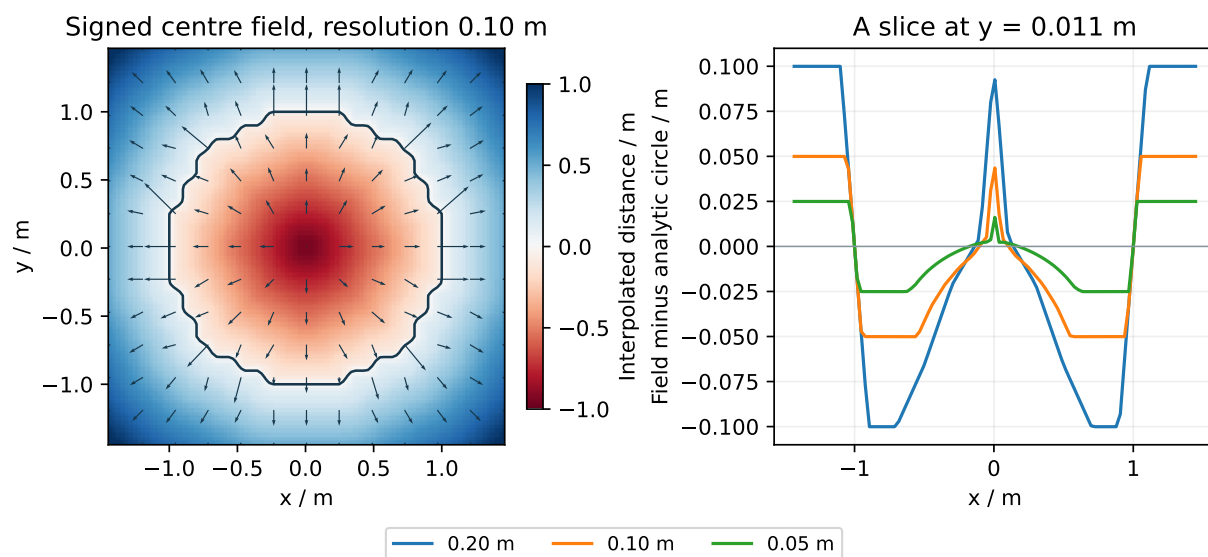


图 12.1: 左: 0.1 m 格的有符号场与插值梯度, 坐标等比例。右: 固定  $y$  的距离误差切片。不同分辨率都存在栅格化和中心距离误差, 靠近界面时插值也影响结果。

| $\rho/\text{m}$ | 格数     | 平均误差/m | 最大误差/m | 中位数/ $\mu\text{s}$ | P95/ $\mu\text{s}$ |
|-----------------|--------|--------|--------|--------------------|--------------------|
| 0.2             | 1,600  | 0.0662 | 0.1865 | 39.9               | 41.1               |
| 0.1             | 6,400  | 0.0315 | 0.0895 | 97.2               | 156.1              |
| 0.05            | 25,600 | 0.0132 | 0.0457 | 431.1              | 442.6              |

给定地图为  $8 \times 8 \text{ m}$ ; 中心位于半径 1 m 圆内的格子标为占据。三组都使用同一查询晶格, 统计  $0.3 \leq \|p\| \leq 1.6 \text{ m}$  环带内与解析圆距离的绝对误差。这是**栅格对圆的近似实验**, 不是同一方格几何的精确距离误差; 原始圆与方格并集不完全相同。每个分辨率预热 5 次, 计时 100 次, 含分类、补边界和两个 EDT, 不含 ROS 序列化。本机更细网格平均误差更小, 但构建更慢; 单个点的误差未必单调, 耗时也不是跨机器承诺。

**练习 ch12-1:** 补全 `bilinearGradient`。给定  $f(x, y) = 2x + 3y + xy$  在  $0, \rho$  四角的值, 验证  $\rho = 0.2 \text{ m}$ 、 $u = 0.3, v = 0.7$  时梯度为  $(2.14, 3.06)$ 。再验证常数场梯度为零。提示、评分和参考解分别位于 `exercises/ch12` 下, 练习失败不影响主演示。

**理解与实验:** 画解析圆场; 解释圆心处不可微、中心距与边界距的差别。改变分辨率, 报告误差、格数和构建时间, 保留相同查询点与计时范围。常见错误是忘记开方、忘记乘分辨率、把索引坐标当米数, 以及使用已经膨胀的地图再扣半径。下面将距离场接入 ROS 可视化。

## 12.9 ROS 距离通道与梯度显示

```

1 ros2 launch motion2d_bringup ch12.launch.py
2 ros2 topic echo /esdf/status
3 # In a separate terminal, with the same ROS_DOMAIN_ID:
4 /usr/bin/python3 scripts/check_ch12.py
5 /usr/bin/python3 scripts/plot_ch12_observed.py

```

`esdf_node` 只订阅 `/map`，每张观测地图独立构建一个距离场。本章启动保持上一章的真值定位和观测建图；ESDF 节点本身不读真值、激光、里程计或 TF。输入与 A\* 共用同一套自由阈值、未知策略和轴对齐地图验证，避免两者解释不同的地图。

| 话题                           | 类型                       | 语义                                    |
|------------------------------|--------------------------|---------------------------------------|
| <code>/esdf/cloud</code>     | <code>PointCloud2</code> | x、y、z、distance 四个 FLOAT32             |
| <code>/esdf/gradients</code> | <code>MarkerArray</code> | map 中的梯度方向箭头                          |
| <code>/esdf/status</code>    | <code>String</code>      | ready / no_finite_field / invalid_map |

点云保持原地图的 `width`、`height`，按  $i = yW + x$  行主序存储；每点 16 字节，行步长为  $16W$ 。x、y 是格子中心的米制坐标，z 固定为零。**distance** 单独存有符号中心距离，单位 **m**，不是 z 高度，也不是占据概率。全阻塞时保留  $-\infty$  并设置 `is_dense=false`，同时发布 `no_finite_field` 和空箭头。地图格式错误时清空点云与标记，不保留上一次的有效显示。

RViz 的 Color Transformer 选择 Intensity, Channel Name 选择 distance。预设用  $[-0.5, 2.0]$  m 固定颜色范围，超出范围只在显示颜色上饱和，消息里的数值不变。Size (m) 默认匹配 0.1 m 分辨率；修改地图分辨率后应相应调整显示方块大小。配置空间掩码、观测图与 ESDF 是不同产物，可在 Displays 中切换比较。

梯度默认每隔 10 格显示一个箭头，箭头方向朝距离增加处。显示长度为  $\min(0.4, 0.2\|\nabla d\|)$  m，箭头头部随短箭头缩放；这只是可视化约定，不能当作速度或控制力。

```

84     const Eigen::Vector2d direction = s->gradient.normalized();
85     const Eigen::Vector2d normal(-direction.y(), direction.x());
86     const double length = std::min(.4, .2*s->gradient.norm());
87     const double head = std::min(.06, .3*length);
88     const Eigen::Vector2d end = p+length*direction;
89     add(p); add(end);
90     add(end); add(end-head*direction+.4*head*normal);
91     add(end); add(end-head*direction-.4*head*normal);
92     //

```

输出继承地图的采样时刻和 `map` 坐标系，采用保留最后一条消息的 QoS。重置不需要维护旧距离场：时间回退的新地图到达后直接重建；晚加入的 RViz 仍能读取当前快照。

## 12.10 读一次真实距离消息

下图来自实际 ROS 的距离点云与箭头消息，固定为 `reset` 后 100 步、0.5 s。图仅截取默认随机世界左下部分；负值同时包含占据、不确定和按策略阻塞的未知区域。颜色不能代替原始占据图中的观测来源信息。

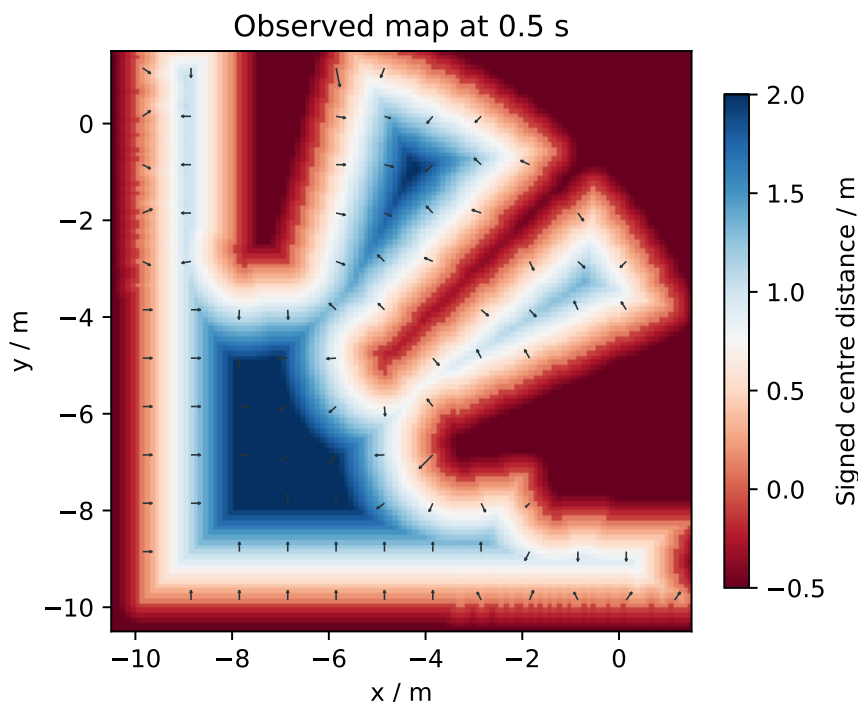


图 12.2: 实际观测距离场与消息中的梯度箭头。坐标为米且等比例，颜色在固定范围内显示。这张图由消息数据绘制，不作为 RViz GUI 截图。

220 × 220 点的独立抽查使用 `seed=1214` 随机选择 100 格，逐个枚举最近阻塞或自由中心，包含外部补边界。消息 `FLOAT32` 与独立计算的最大差为  $4.3 \times 10^{-7}$  m，验收阈值为  $10^{-5}$  m。93 个箭头的长度、方向与字段梯度一致；还验证地图时间戳、输入隔离和 `reset` 后数据重放。

额外独立测试给 `esdf_node` 发送全未知图、全自由图、含旋转的错误图。分别检查  $-\infty$ 、外部距离和清除旧显示，不能用空图的零距离冒充有效场。实际主机已完成无界面 ROS 验收；RViz 配置已提供，GUI 画面未验收。

### 消息练习：

1. 从 `/esdf/cloud` 读取一格的四个字段，手算中心坐标和行字节偏移。
2. 把 `esdf.gradient_stride` 从 10 改为 5，比较箭头密度，说明为何数值距离不变。
3. 对全自由小地图，解释为什么越靠近地图边界，正距离越小。

独立检查入口、命令和评分见本章练习说明。

## 第十三章 让整段曲线有地方走：安全走廊

**目标** 沿 A\* 路径构建整块安全的凸区域，并为后续轨迹分配走廊和连接点。

**先修** 第 11、12 章；凸集、线段与半平面。

### 13.1 先比较矩形与凸多边形

```
1 ros2 run motion2d corridor_demo tmp/ch13_corridor
2 /usr/bin/python3 scripts/plot_ch13_corridor.py
```

本章先使用给定栅格隔离算法，输入是已经膨胀的配置空间和原始 A\* 邻格折线。实验路线从隔墙左下方绕到上端，再回到右下方。先得到一串相互交叠的局部矩形，再尝试合并成较少的凸多边形；每次合并都重新检查整个区域。

### 13.2 一个凸区域的两种表示

顶点表示适合画图；半空间表示适合写约束。对逆时针多边形，边从  $a_i$  指向  $a_{i+1}$ ，内部在边的左侧。令  $e_i = a_{i+1} - a_i$ ，单位外法向为

$$n_i = \frac{(e_{iy}, -e_{ix})^T}{\|e_i\|}, \quad b_i = n_i^T a_i, \quad \mathcal{C} = \{p \mid Ap \leq b\}. \quad (13.1)$$

$A$  的第  $i$  行是  $n_i^T$ ，各行法向长度均为 1；故  $n_i^T p - b_i$  是到该边直线的带符号距离，单位 m。它非正表示点在此半平面中。一个点必须满足所有行，才属于整个区域。

```
87 for (std::size_t i = 0; i < polygon.vertices.size(); ++i) {
88     const auto & a = polygon.vertices[i];
89     const Eigen::Vector2d e = polygon.vertices[(i+1)%polygon.vertices.size()]-a;
90     const Eigen::Vector2d n = Eigen::Vector2d(e.y(),-e.x()).normalized();
91     result.A.row(i) = n.transpose(); result.b[i] = n.dot(a);
92 }
93 //
```

**已经在配置空间内** 走廊容纳的是机器人圆心，输入格子的半径和安全裕量已在第十一章处理。这里不再次缩小一个物理半径。只有半空间约束正确且整个区域安全，才能使用后面的凸性论证。

公开接口为 `makeRegion`、`regionIsFree` 和 `buildCorridor`；结果同时保留顶点与半空间。凸多边形须严格凸、逆时针且非退化。练习 ch13-1 从一条边写出对应的单位外法向和偏移。

### 13.3 从局部矩形开始

对每一段原始邻格路径，取起终点所在格子的包围框。若点恰好在格线或格角上，种子包含两侧格子。框内每格都自由才可开始扩张，然后轮流尝试向左、右、下、上增加一条完整栅格边。任何新增格被阻塞，就不接受这次扩张。每侧还有局部扩展长度限制  $L$ ，对应最多  $\lfloor L/\rho \rfloor$  格；演示设置  $L = 0.6$  m。

```

61 while (changed) {
62     changed = false;
63     if (left > limit_left && freeBox(grid, left-1, left-1, bottom, top)) {--left; changed = true;}
64     if (right < limit_right && freeBox(grid, right+1, right+1, bottom, top)) {++right; changed = true;}
65     if (bottom > limit_bottom && freeBox(grid, left, right, bottom-1, bottom-1)) {--bottom; changed = true;}
66     if (top < limit_top && freeBox(grid, left, right, top+1, top+1)) {++top; changed = true;}
67 }
68 //

```

输出边界向内缩  $10^{-6}\rho$ ，在浮点运算下保留与阻塞格的分离。每条种子线段的两个端点仍须在矩形内，所有边界约定与完整区域检查一致。如果当前最后一个矩形已经包含下一段的两个端点，就无需重复添加。扩张顺序会影响形状；这不是最大面积矩形求解器，合并后的区域也不再受单个种子的扩展窗口限制。

### 13.4 安全顶点不足以证明区域安全

一个四边形的四个顶点、甚至四条边都可能自由，而小障碍被包在正中间。因此 `regionIsFree` 遍历区域包围框内的**阻塞方格**，求它们与走廊多边形的闭集交集。只要交集非空，就拒绝；擦角或沿边接触也算非空。

实现对多边形逐次施加方格的四个半平面约束。跨越某个边界  $n^T p = b$  的边段，设两端残差为  $\alpha = n^T p_0 - b$ 、 $\beta = n^T p_1 - b$ ，交点为

$$p_* = p_0 + \frac{\alpha}{\alpha - \beta}(p_1 - p_0). \quad (13.2)$$

保留半平面内的顶点与必要交点，重复四次。碰撞检查保留退化的点和线，避免漏掉接触。地图外部也被拒绝。检查针对整块区域，不需要靠稠密采样猜测内部是否安全。

### 13.5 只有通过验证才合并

把相邻矩形顶点放在一起求凸包，再调用同一个 `regionIsFree`。通过则替换成凸包，否则保留两个已有区域。转弯内侧的障碍经常导致合并失败，这是应保留的约束。浮点近共线顶点若不满足严格凸性，也放弃该次可选合并。局部矩形刻意有扩展窗口，因此在更大自由空间中，合并后可以出现六边形等非矩形区域。

## 13.6 交叠区域决定连接点

设相邻两个走廊为  $\mathcal{C}_i, \mathcal{C}_{i+1}$ ，先求交集

$$\mathcal{I}_i = \mathcal{C}_i \cap \mathcal{C}_{i+1}. \quad (13.3)$$

它也由凸半平面裁剪得到。这里要求**正面积**，只共用一条边或一个点不够。数值阈值为  $10^{-10}\rho^2$ ；这个阈值只排除退化交叠，实际运动余量仍受地图与半径决定。从交集多边形的顶点  $v_1, \dots, v_K$  取平均作为连接点：

$$q_i = \frac{1}{K} \sum_{k=1}^K v_k. \quad (13.4)$$

这是一个容易计算的内部点，不是多边形的面积质心，也不保证最大净空。起点和终点采用原始请求坐标，最终  $N$  个走廊对应  $N+1$  个路点。第  $i$  条新线段的两个端点都属于  $\mathcal{C}_i$ ，于是由凸性有

$$(1-s)q_i + sq_{i+1} \in \mathcal{C}_i, \quad 0 \leq s \leq 1. \quad (13.5)$$

这证明的是**分配后的直线段整段**包含在安全区域。经过相同路点的五次曲线可能向外鼓出，不能直接复用这条证明；第十六章会检查整段 Bezier 凸包。

## 13.7 失败时不返回半成品

| 状态                  | 含义             |
|---------------------|----------------|
| empty_path          | 没有可用路径         |
| unsafe_path         | 原始折线含越界、穿障或接触  |
| blocked_seed_box    | 线段安全，但整格种子框不可用 |
| no_positive_overlap | 相邻区域没有正面积交叠    |
| invalid_extension   | 扩展长度参数不合法      |

失败结果的区域和路点均为空，调用方应保留明确失败状态。尤其不要把贪心简化后的长对角线直接拿来扩张矩形：线段可以绕过方格，包围框却可能覆盖它。保留原始 A\* 邻格路径更适合本章构造方式；已有长线段可另行重新离散搜索。

验证覆盖单位法向、几何包含、点/线退化交叠、内部障碍、擦边/擦角、非矩形安全凸包、转弯路线与不同栅格尺度。对每个最终区域验证整块无碰撞，并检查分配给它的两个端点。这些是地图所表达的几何证书，仍依赖观测质量与定位质量。

## 13.8 固定地图上的对照实验

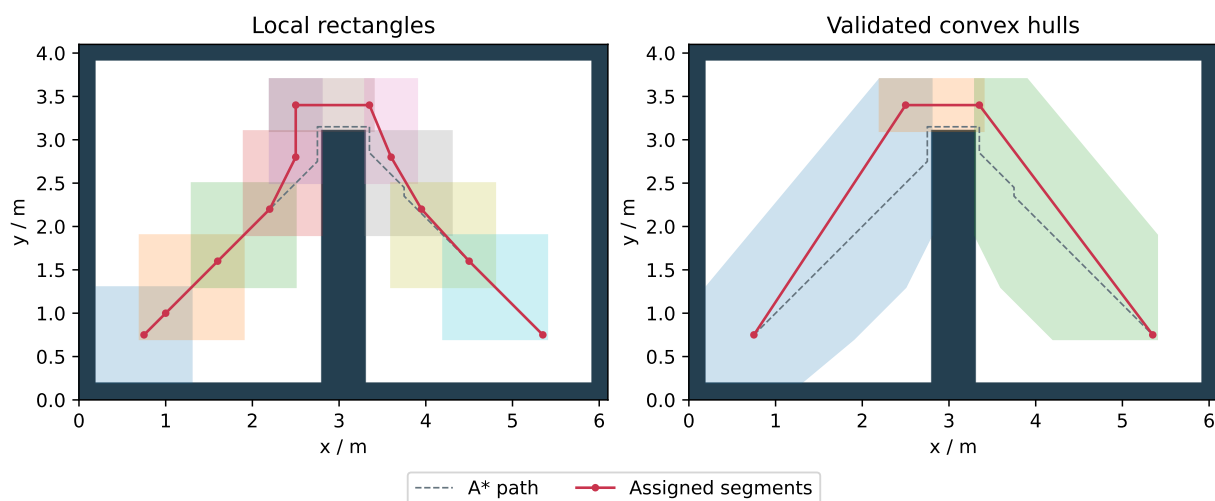


图 13.1: 深色为已膨胀阻塞格, 浅色为走廊, 虚线为原始 A\* 路径。红色线段的两端点均分配到一个凸区域中。两图采用相同坐标比例。

| 凸包合并 | 区域数 | 总顶点数 | 路点数 | 最小交叠/ $m^2$ |
|------|-----|------|-----|-------------|
| 关    | 10  | 40   | 11  | 0.06        |
| 开    | 3   | 19   | 4   | 0.06        |

给定图为  $61 \times 41$  格, 分辨率  $0.1 \text{ m}$ , 半径  $0.15 \text{ m}$ , 额外裕量为零。隔墙上端留出绕行空间, 起终点分别为  $(0.75, 0.75)$ 、 $(5.35, 0.75) \text{ m}$ 。在完全相同的 A\* 路径上, 局部矩形为 10 个, 安全凸包合并后为 3 个; 总顶点数从 40 变成 19。两种构造的最小交叠面积都约为  $0.06 \text{ m}^2$ , 所以连接点没有落在仅共边的退化位置。

区域变少会减少后续轨迹分段和约束的组织成本, 但不代表最短路径、最小 jerk 或最大走廊体积。本章没有给红线分配时间, 也没有直接驱动机器人。

- 理解:** 画一个内部含障碍的凸四边形, 说明只测顶点和边会遗漏什么。
- 代码 ch13-1:** 把 CCW 边转为单位半空间。底边  $(1, 2) \rightarrow (4, 2)$  给出  $-y \leq -2$ ; 斜边  $(4, 2) \rightarrow (7, 6)$  的外法向为  $(0.8, -0.6)$ 。
- 实验:** 改变局部扩展长度与合并开关, 记录区域数、形状与最小交叠面积。保持地图、半径和路径相同, 解释变化原因。

## 13.9 同一观测快照上的 ROS 走廊

```
1 ros2 launch motion2d_bringup ch13.launch.py
2 # 另一个已 source 的终端；暂停、reset、单步和目标检查
3 /usr/bin/python3 scripts/check_ch13.py
4 /usr/bin/python3 scripts/plot_ch13_observed.py
```

本章沿用静止机器人、真值定位与真实扫描建图；默认半径 0.2 m、裕量 0.05 m。RViz 的目标工具发送 map 系目标，规划器先在当前栅格上运行 A\*，再把这次搜索的原始路径直接交给走廊构造。不通过另一个订阅回调拼接不同时刻的路径和地图。下一章才给几何路线增加时间。

```
153 CorridorResult corridor; corridor.status = corridor_enabled_ ? status : "disabled";
154 if (corridor_enabled_ && status == "found") {
155     corridor = buildCorridor(*grid_,raw,merge_convex_,extension_);
156 }
157 corridor_pub_->publish(toCorridorMarkers(corridor.regions,header));
158 corridor_path_pub_->publish(toPath(corridor.waypoints,header));
159 message.data = corridor.status; corridor_status_pub_->publish(message);
160 //
```

| 输出                    | 含义                    |
|-----------------------|-----------------------|
| /plan/path_raw        | 原始 A* 邻格路径            |
| /plan/path            | 第十一章的简化折线             |
| /plan/corridor_path   | 按区域重新分配的连接点           |
| /plan/corridors       | 区域边框 MarkerArray，仅供显示 |
| /plan/corridor_status | 走廊阶段状态，和 A* 状态分开      |

路径与边框共用本次起点里程计的时间戳及 map 坐标系，路径里的姿态单位四元数只是占位，不表示实际 yaw。栅格仍保留地图采样时间；这些时间都不是未来执行时刻。每次 MarkerArray 先发 DELETEALL，再发新边框，区域数变少也不留下旧多边形。错误目标、搜索失败或 reset 都清空路径与边框。

## 13.10 参数、观察与独立检查

- corridor.enabled：本章开启；旧章节默认关闭。
- corridor.merge\_convex：是否尝试安全凸包合并。
- corridor.max\_extension：每侧局部扩展长度，默认 0.6 m。

算法直接使用 ConvexRegion 和半空间，不从显示 Marker 反解约束。RViz 应显示观测配置图、闭合区域轮廓及穿过交叠区的分配路线。本机完成了真实 ROS 消息验收；由于已记录的桌面显示限制，未宣称 GUI 目视验收。

固定 seed=42/4242，reset 后单步至 0.5 s，目标为 (-4, -1) m。得到 73 个原始路径点、4 个走廊；检查脚本使用独立的分离轴方法逐格验证整个多边形，并验证段端点、闭合边框和时间戳。随后换近目标、无效远目标并 reset，核对旧显示确实清除。

**新增实验：**依次发送远近两个已观测目标，记录区域数；解释为什么需要 DELETEALL。将目标改到未知区域，分别观察 A\* 和走廊状态，不能因为上一路径还在截图中就判断当前规划成功。

Observed map: 73 A\* points, 4 safe regions

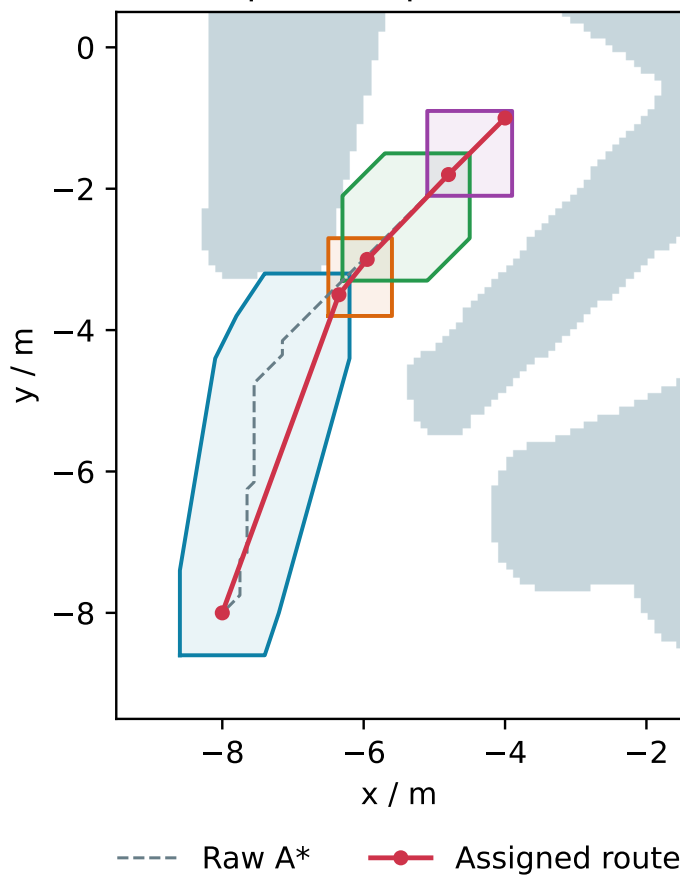


图 13.2: 由实际 ROS 消息绘制的局部观测结果。灰色包含障碍、未知及膨胀阻塞格；彩色区域全部位于白色自由域，浅填充仅用于教材辨认。

## 第十四章 给路径加上时间：五次轨迹

**目标** 从端点位置、速度、加速度求五次曲线，按同一个时间采样，再接入理想执行。

**先修** 第 04、11、13 章；一阶与二阶导数。

### 14.1 同样走一米，为什么感觉不同

```
1 ros2 run motion2d quintic_demo tmp/ch14_quintic
2 /usr/bin/python3 scripts/plot_ch14_quintic.py
```

几何路径只说经过哪里；轨迹还要说何时到达、到达时多快。本章先从静止出发，沿  $x$  轴走 1 m 后静止，分别用 2 s 和 4 s 完成。所有实验使用解析多项式导数，不通过相邻位置差分估计速度或加速度。

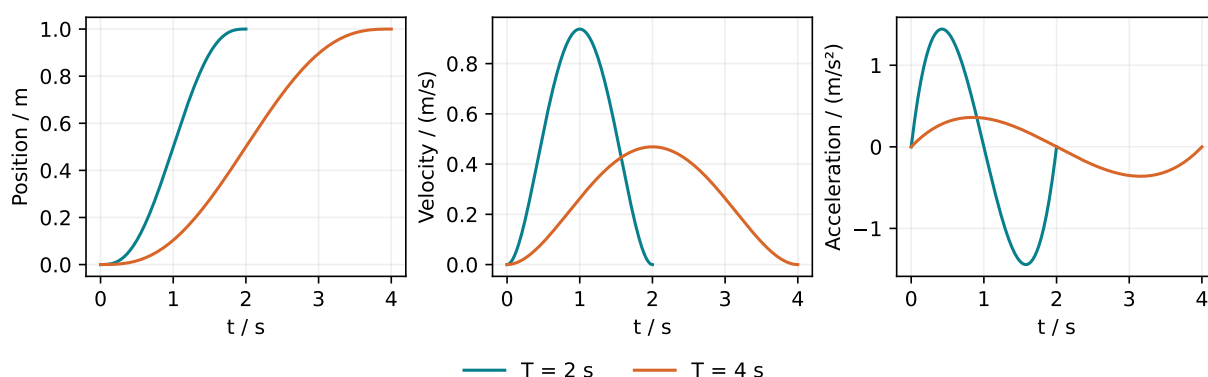


图 14.1: 同一几何线段的时间缩放。时长翻倍，速度缩为一半，加速度缩为四分之一。

### 14.2 六个系数对应六个边界条件

每个坐标分别写成五次式；二维系数  $c_k$  是包含  $x$ 、 $y$  的列向量：

$$p(t) = \sum_{k=0}^5 c_k t^k, \quad \dot{p}(t) = \sum_{k=1}^5 k c_k t^{k-1}, \quad \ddot{p}(t) = \sum_{k=2}^5 k(k-1) c_k t^{k-2}. \quad (14.1)$$

给定  $(p_0, v_0, a_0)$ 、 $(p_1, v_1, a_1)$  与正时长  $T$ ，每个坐标有六个线性约束。前三项直接得到  $c_0 = p_0$ ,  $c_1 = v_0$ ,  $c_2 = a_0/2$ 。为看清时间尺度，先用  $s = t/T$ ，写成  $p(s) = \sum b_k s^k$ ，最后转换  $c_k = b_k/T^k$ 。这里  $b_k$  单位都是 m； $c_k$  的单位是 m/s <sup>$k$</sup> 。

剩余端点残差为

$$\begin{aligned} r_0 &= p_1 - b_0 - b_1 - b_2, \\ r_1 &= T v_1 - b_1 - 2b_2, \quad r_2 = T^2 a_1 - 2b_2. \end{aligned} \quad (14.2)$$

它们分别等于  $b_3 + b_4 + b_5$ 、 $3b_3 + 4b_4 + 5b_5$ 、 $6b_3 + 12b_4 + 20b_5$ 。解这个固定的三阶系统，即得到代码中的三个组合。

### 14.3 从公式到系数与采样

```
16  const double T = duration;
17  Eigen::Matrix<double,2,6> b;
18  b.col(0) = start.position;
19  b.col(1) = T*start.velocity;
20  b.col(2) = .5*T*T*start.acceleration;
21  const Eigen::Vector2d r0 = finish.position-b.col(0)-b.col(1)-b.col(2);
22  const Eigen::Vector2d r1 = T*finish.velocity-b.col(1)-2*b.col(2);
23  const Eigen::Vector2d r2 = T*T*finish.acceleration-2*b.col(2);
24  b.col(3) = 10*r0-4*r1+.5*r2;
25  b.col(4) = -15*r0+7*r1-r2;
26  b.col(5) = 6*r0-3*r1+.5*r2;
27  double power = 1;
28  for (int k = 0; k < 6; ++k) {b.col(k) /= power; power *= T;}
29  //
```

函数 `interpolateQuintic` 接收 `TranslationState`，返回 `QuinticPiece`：一个正 `duration` 和  $2 \times 6$  系数矩阵。列按升幂排列，时间从每段起点开始计秒，不是整条轨迹开始以来的累计秒数。

求导后用 Horner 法从高次项向低次项计算，避免反复调用幂函数：

```
14  Eigen::Vector2d value = Eigen::Vector2d::Zero();
15  for (int k = 5; k >= order; --k) {
16      double factor = 1;
17      for (int j = 0; j < order; ++j) {factor *= k-j;}
18      value = value*time + factor*piece.coefficients.col(k);
19  }
20  return value;
21  //
```

公开 `derivative` 支持 0 到 5 阶，拒绝外推。第 0 阶为位置，第 3 阶为 `jerk`，单位  $\text{m/s}^3$ 。所有维度都共用相同  $T$ ；分别给 `x`、`y` 分配不同时长会改变二维运动的含义。

## 14.4 最容易手算的静止到静止

令  $\Delta p = p_1 - p_0$ 、 $v_0 = v_1 = a_0 = a_1 = 0$ ，则

$$p(t) = p_0 + \Delta p h(s), \quad h(s) = 10s^3 - 15s^4 + 6s^5, \quad s = t/T. \quad (14.3)$$

因为  $h'(s) = 30s^2(1-s)^2 \geq 0$  且  $h(0) = 0, h(1) = 1$ ，这一段始终在线段  $[p_0, p_1]$  上。最大速度为  $1.875\|\Delta p\|/T$ ，不是平均速度  $\|\Delta p\|/T$ 。若整条线段已经分配到安全凸走廊中，本特殊构造保持其几何包含性；任意非零边界导数没有这个结论。

## 14.5 连接处连续到哪一阶

将第  $i$  段终点和第  $i+1$  段起点设置为同一组  $(p, v, a)$ ，便有  $C^2$  连续。jerk 可以跳变；五次本身不意味着所有阶导数连续，也不意味着满足速度、力或碰撞约束。

PolynomialTrajectory 构造时检查正时长、有限系数、总时长与连接条件。连接容差为位置、速度和加速度分别  $10^{-7}$  对应 SI 单位的绝对误差。采样范围为闭区间  $[0, \sum_i T_i]$ ；内部连接时刻选择右段，最终时刻选择末段终点。这让  $p/v/a$  一致，而 jerk 的单侧定义明确。区间外抛出错误，由执行器单独决定等待或终点保持。

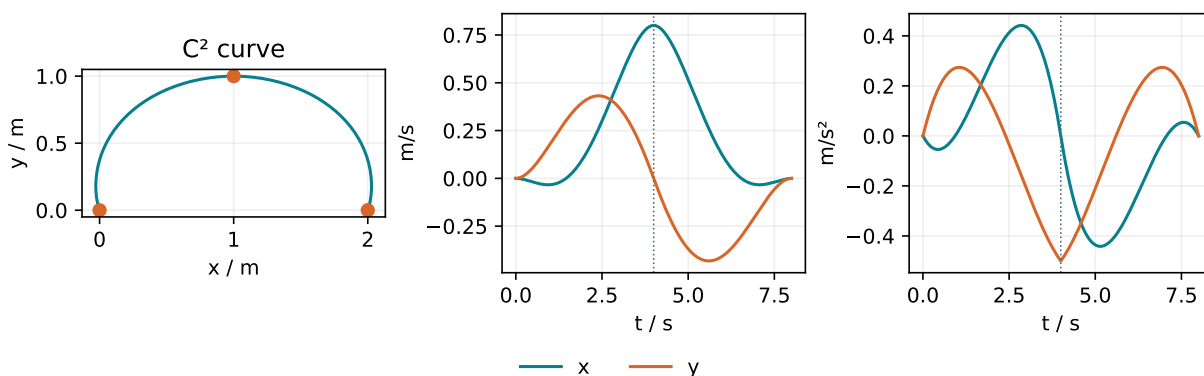


图 14.2: 两个 4 s 分段，连接点 (1,1) m，连接速度 (0.8,0) m/s、加速度 (0, -0.5) m/s<sup>2</sup>。虚线是连接时刻；图形坐标等比例，速度与加速度连续。

最简单的 stopAtWaypoints 在每个路点设  $v = a = 0$ ，分配

$$T_i = \max \left( T_{\min}, \frac{\|q_{i+1} - q_i\|}{v_{\text{nom}}} \right). \quad (14.4)$$

$v_{\text{nom}}$  是平均速度参数，不是峰值上限。重复点产生正时长的原地停留段。每个中间点都停下易于验证，但速度不流畅；第十五、十六章将求解内部导数，减少 jerk 能量。

## 14.6 实验、保存与验收

演示输出 `samples.csv`、`summary.csv` 和 `coefficients.csv`。系数文件逐行保存案例、段号、时长、坐标轴与六个升幂系数，可用相同采样器重建；不包含 `yaw`。本章平移参考与 `yaw` 分离，不能把切线方向默认为机器人的朝向。

| 案例 | 总时长/s | 速度峰值   | 加速度峰值  | jerk 峰值 |
|----|-------|--------|--------|---------|
| 0  | 2     | 0.9375 | 1.4434 | 7.5     |
| 1  | 4     | 0.4688 | 0.3608 | 0.9375  |
| 2  | 8     | 0.8    | 0.5    | 0.8826  |

各段采样到同一总时长网格的 2001 个点；这些峰值是采样统计，不冒充任意多项式的解析全局界。案例 0/1 的速度与 jerk 峰值也能解析验证；案例 2 使用上述非零连接导数。端点测试采用 `seed=1414` 的 100 组边界，并用有限差分检查导数；还覆盖重复点、连接时刻和非法时长。

1. **推导：**从六个静止端点约束推导  $h(s)$ ，说明  $T$  翻倍后的各阶导数缩放。
2. **代码 ch14-1：**完成升幂系数的一阶、二阶导数；避免重复时间缩放。
3. **实验：**修改中间点速度，比较曲线形状与连续性；解释为何路点安全不足以证明曲线安全。

## 14.7 把轨迹作为完整消息发送

```

1 ros2 launch motion2d_bringup ch14.launch.py
2 ros2 topic pub --once /goal_pose geometry_msgs/msg/PoseStamped \
3   '{header: {frame_id: map}, pose: {position: {x: -7.5, y: -8.0}, orientation: {w: 1.0}}}'
4 ros2 service call /trajectory/execute std_srvs/srv/Trigger '{}'
5 ros2 topic echo /sim/trajectory_status

```

先等待扫描建图与目标规划成功，再调用服务。点击目标负责产生几何路线；调用服务将当前路线转换为一次完整执行。`trajectory_node` 仅订阅区域连接路径、选源里程计与 TF，不读仿真真值。

新增小包 `motion2d_interfaces` 定义两个消息，避免在 Path 的姿态或时间字段里暗藏系数。

| Trajectory2D 字段 | 明确语义                         |
|-----------------|------------------------------|
| header          | 发布时间；执行坐标系固定为 odom           |
| start_time      | 仿真时钟上的绝对执行起始时刻               |
| pieces          | 非空五次分段；每段 duration、x[6]、y[6] |
| yaw             | 独立常值朝向，单位 rad                |

每段系数以局部秒升幂存储；时长必须有限且为正，连接需满足  $C^2$ 。消息转换不重新采样，往返测试逐项保留系数与时长。零起始时刻就是仿真原点，不是“收到后立即执行”的特殊值。

## 14.8 把 map 路线固定到 odom

执行节点在当前里程计时刻查询  ${}^{odom}T_{map}$ ，转换所有路点，然后求逐点停靠轨迹。发布时冻结该变换，后续采样不跟随 TF 改写参考。在线重规划与回环衔接在第十九章处理。

```

70 std::vector<Eigen::Vector2d> points;
71 for(const auto & pose : path_>poses) {
72     const auto & p=pose.pose.position;
73     if(pose.header.frame_id!="map" || !std::isfinite(p.x) || !std::isfinite(p.y) || p.z!=0)
74         {throw std::invalid_argument("Expected finite planar path points in map");}
75     points.push_back(transformPoint(odom_from_map,{p.x,p.y}));
76 }
77 if((points.front()-start.position).norm())>1e-7)
78     {throw std::invalid_argument("Route start differs from current pose; wait for replanning");}
79 const auto stamp=now();
80 TimedTrajectory command{stopAtWaypoints(points,speed_,.5),
81     stamp.nanoseconds()+std::llround(lead_*1e9),start.yaw};
82 pub_>publish(toTrajectoryMessage(command,stamp));
83 //

```

默认平均速度 0.4 m/s、最短分段 0.5 s，执行起点比发布时间晚 0.25 s。若路线起点与当前位置不一致，拒绝发布并等待新路线；运动中再次发送轨迹由本章执行器明确拒绝。服务成功表示已发布，最终接收结果看 `/sim/trajectory_status`。

## 14.9 理想执行也要检查整段运动

仿真器新增 `model=trajectory`, 只订阅 `/plan/trajectory`。接收时检查起点与当前位姿相同, 两端  $v = a = 0$ , `yaw` 不跳变、起始时刻不在过去。执行前静止等待, 结束后静止保持; 暂停冻结仿真时间, `reset` 清空曲线。这仍是指定参考的理想模型, 没有质量或力上限的跟踪误差; 惯性模型与控制器将在后续章节接入。

位置、速度和加速度都由同一解析函数产生, 传感器在自身采样时间调用它。为避免弯曲段越过障碍仍被接受, 沿用第四章的线性插值误差上界:

$$\delta = \frac{a_{\max} \Delta t^2}{8}, \quad \|\ddot{p}(t)\| \leq \sum_{k=2}^5 k(k-1) \|c_k\| t_{\max}^{k-2}. \quad (14.5)$$

右式来自三角不等式, 忽略系数间抵消, 可能保守。跨多个分段时对每个相交片段求界再取最大值; 将圆盘半径加上  $\delta$  后检查端点连线的整段扫掠。该证明依赖连接处加速度连续且起终点平滑接入保持段。

```

24 double offset=0, bound=0;
25 for(const auto & piece : curve.pieces()) {
26     if(end>=offset && begin<=offset+piece.duration) {
27         const double hi=std::clamp(end-offset,0.,piece.duration);
28         double sum=0, power=1;
29         for(int k=2;k<6;++k) {
30             sum+=k*(k-1)*piece.coefficients.col(k).norm()*power;
31             power*=hi;
32         }
33         bound=std::max(bound,sum);
34     }
35     offset+=piece.duration;
36 }
37 return bound;
38 //

```

撞击风险时保持上一个有效状态并冻结时钟, 报告 `collision_predicted`; 不会采集未来区间的传感器样本。此检查使用仿真世界作为物理碰撞裁判, 不向规划器提供世界几何。

## 14.10 在 RViz 与实际消息中看执行

`/plan/trajectory_path` 是稀疏采样预览: 总头记录发布时间, 每个 `PoseStamped` 记录未来执行时间。真正执行始终使用完整系数; 预览折线的密度不影响仿真精度。RViz 应显示洋红色参考、圆盘与实际轨迹; 本机完成无界面 ROS 验收, GUI 限制仍见验收记录。

## 14.11 观测路线和非整步时间验收

```

1 # 默认ch14在运行时
2 /usr/bin/python3 scripts/check_ch14.py
3 /usr/bin/python3 scripts/plot_ch14_execution.py
4 # 停止上一个launch，再运行独立空房间时间实验
5 ros2 launch motion2d_bringup ch14.launch.py rviz:=false \
6   config:=$PWD/ros2_ws/src/motion2d_bringup/config/ch14_time_lab.yaml
7 /usr/bin/python3 scripts/check_ch14_time.py

```

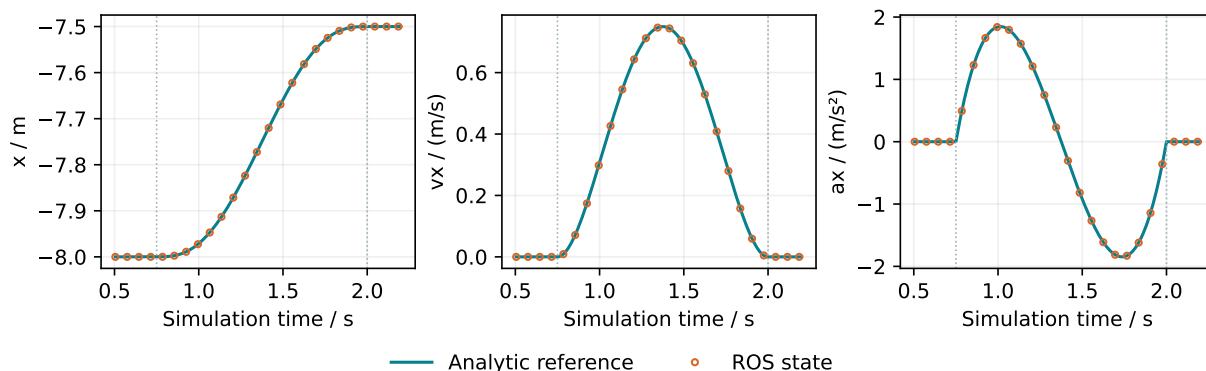


图 14.3: 实际观测地图上执行半米路线。0.75 s 开始、2 s 结束；实测  $p/v/a$  与解析参考重合，用空心点区别采样结果与参考曲线。

第一组从真值定位的观测地图规划  $(-8, -8) \rightarrow (-7.5, -8)$  m，reset 后至 0.5 s 发布目标。逐步执行 340 tick，位置、速度、加速度最大误差分别约  $1.8 \times 10^{-15}$  m、 $3.9 \times 10^{-15}$  m/s、 $1.4 \times 10^{-14}$  m/s<sup>2</sup>。这是真值理想执行的数值一致性，不是带噪 SLAM 或闭环控制性能。

第二组关闭噪声，20 m 空房间、yaw=0.4 rad、90 束、137 Hz IMU 与 7 Hz 雷达。独立  $6 \times 6$  线性方程生成两段曲线，开始、连接、结束分别为 0.203、0.661、1.298 s，均不对齐 5 ms tick。1.5 s 内 206 条 IMU、11 帧扫描的采样时刻和解析值通过检查；最大状态/IMU 误差约  $7.3 \times 10^{-14}$ ，激光误差约  $4.8 \times 10^{-7}$  m，后者受 FLOAT32 存储影响。

还检查了等待、终点保持、拒绝运动中替换、消息非法输入、输入模式隔离、reset 和穿墙曲线冻结。**新增练习：**如果仅把 IMU 时间戳改成正确值，却使用 tick 末端的加速度，为什么仍会出错？运行上述非整步实验并给出一个具体采样时刻的误差。

## 第十五章 让路点之间更平顺：二维 MINCO

**目标** 给定路点和时长求最小 jerk 曲线，把系数代价传回路点与时长。

**先修** 第 14 章；矩阵方程、链式法则，本章补充必要的最优化条件。

### 15.1 相同路点、相同时长的比较

```
1 ros2 run motion2d minco_demo tmp/ch15_minco
2 /usr/bin/python3 scripts/plot_ch15_minco.py
```

上一章在每个路点停下，易于构造，但未选择合适的中间速度和加速度。本章固定路点  $q_i$  与分段时长  $T_i > 0$ ，保留两端的  $p/v/a$ ，求解整个区间的

$$\min J = \sum_{i=1}^N \int_0^{T_i} \|p_i'''(t)\|^2 dt. \quad (15.1)$$

jerk 是加速度的变化率。 $J$  常被称为控制能量，这里单位是  $\text{m}^2/\text{s}^5$ ，不是物理焦耳。MINCO 是一种利用路点和时长确定最小控制代价轨迹的表示；课程根据 [作者论文](#) 中的最优性思想，独立实现固定二维、五次、最小 jerk 子问题。

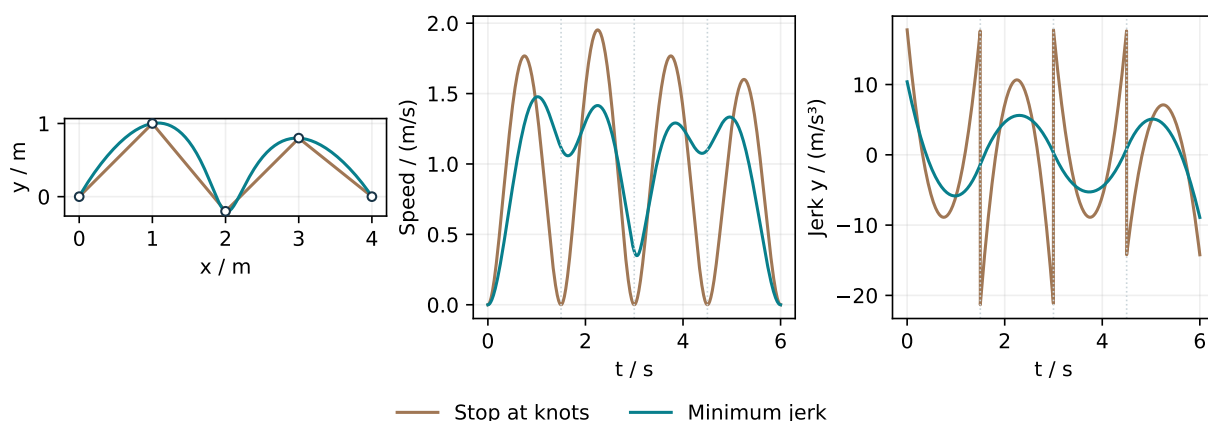


图 15.1: 四段均为 1.5 s。圆点是固定路点，虚竖线是连接时刻。最小 jerk 解允许内部点有非零速度，连接处 jerk 连续；此图没有施加避障约束。

## 15.2 能量是系数的二次型

将单段系数按升幂排成  $C \in \mathbb{R}^{6 \times 2}$ , 则  $J_i = \text{tr}(C^T Q(T_i) C)$ 。  $Q$  只有第 3 到 5 次项对应的部分非零; 对  $k, l \geq 3$ ,

$$Q_{kl}(T) = \frac{k(k-1)(k-2)l(l-1)(l-2)}{k+l-5} T^{k+l-5}. \quad (15.2)$$

这只是先求导、相乘再积分; 代码 `jerkGram` 没有数值积分误差。练习 `ch15-1` 用单段  $J = 720\|p_1 - p_0\|^2/T^5$  检查因子与幂次。

## 15.3 为什么最优解还要连续 jerk 与 snap

只要求  $p/v/a$  连续时, 内部速度和加速度仍可调整。让曲线发生一个小变化  $\delta p$ , 对  $\delta J = 2 \int p''' \cdot \delta p''' dt$  做三次分部积分, 得到

$$\frac{\delta J}{2} = [p''' \cdot \delta p'' - p'''' \cdot \delta p' + p''''' \cdot \delta p]_0^T - \int_0^T p^{(6)} \cdot \delta p dt. \quad (15.3)$$

每段内部变化任意, 故最优解满足  $p^{(6)} = 0$ , 即至多五次。在固定位置的内部路点处  $\delta p = 0$ , 而两侧共享的  $\delta v, \delta a$  可以自由变化。相邻段端点项的符号相反, 因此它们的系数必须抵消: `jerk` 与 `snap` 分别连续。合上原有  $p/v/a$  连续, 便得到  $C^4$ ; 第五阶导数仍可跳变。

## 15.4 将最优性写成一组线性方程

每段六个系数, 每个坐标共  $6N$  个未知数。方程数量恰好也是  $6N$ :

- 首尾各三个  $p/v/a$  条件, 共 6 行。
- 每个内部连接: `jerk`、`snap` 连续各一行, 左段终点固定位置一行,  $p/v/a$  连续三行, 共 6 行。

把二维系数堆为  $C \in \mathbb{R}^{6N \times 2}$ , 一次求解两个右端项:

$$A(T)C = B(q). \quad (15.4)$$

每一行只涉及相邻两段, 所以按代码顺序排列后, 上下带宽均为 6。

```

35 for(int i=0;i<n-1;++i) {
36     const double T=durations_[i]; const int r=6*i+3;
37     insert(r,i,polynomialBasis(T,3)); insert(r,i+1,polynomialBasis(0,3),-1); // jerk continuity
38     insert(r+1,i,polynomialBasis(T,4)); insert(r+1,i+1,polynomialBasis(0,4),-1); // snap continuity
39     insert(r+2,i,polynomialBasis(T,0)); rhs.row(r+2)=interior[i].transpose();
40     for(int d=0;d<3;++d) {
41         insert(r+3+d,i,polynomialBasis(T,d)); insert(r+3+d,i+1,polynomialBasis(0,d),-1);
42     }
43 }
44 //

```

`insert` 将对应阶导数的基函数系数放入矩阵; 例如零时刻 `jerk` 对  $c_3$  的系数是 6。矩阵约束已经包含最优性, 求解阶段无需再迭代下降能量。`trajectory()` 返回上一章的统一采样对象。

## 15.5 系数随路点和时长一起变化

以后添加避障或速度代价，先计算它们对系数、时长的直接偏导。记  $G = \partial K / \partial C$ 、 $g_T = \partial K / \partial T$ ；固定系数时，jerk 代价有

$$G_i = 2Q(T_i)C_i, \quad (g_T)_i = \|p_i'''(T_i)\|^2. \quad (15.5)$$

但是改变  $T_i$  后，保持边界条件需要重新求系数；只保留这个非负上限项，会丢掉关键变化。例如单段静止到静止的完整导数为  $dJ/dT = -3600\|\Delta p\|^2/T^6$ ，反而是负值。

对  $AC = B$  求微分，得到  $AdC = dB - (dA)C$ 。为了避免对每个路点单独求一次  $dC$ ，先求一个伴随方程：

$$A^T \Lambda = G, \quad dK = \langle \Lambda, dB - (dA)C \rangle + g_T^T dT. \quad (15.6)$$

$\langle X, Y \rangle = \sum_{ij} X_{ij}Y_{ij}$  表示逐项乘积求和。一个路点只改变  $B$  对应的一行；故直接取该行  $\Lambda$ 。某一时长只改变其所在段的右端导数值；再对时间求导，阶数增加一阶。

```

79  const Eigen::MatrixX2d lambda=factor_.solve(gradient,true);
80  TrajectoryGradient result{Eigen::MatrixX2d::Zero(n-1,2),direct};
81  for(int i=0;i<n;++i) {
82      const auto C=coefficients_.middleRows<6>(6*i); const double T=durations_[i];
83      const int row=i<n-1 ? 6*i+3 : 6*n-3;
84      const std::vector<int> orders=i<n-1 ? std::vector<int>{4,5,1,1,2,3} : std::vector<int>{1,2,3};
85      for(std::size_t j=0;j<orders.size();++j) {
86          result.times(i)-=lambda.row(row+j).dot(polynomialBasis(T,orders[j])*C);
87      }
88      if(i<n-1) {result.waypoints.row(i)=lambda.row(6*i+5);}
89  }
90  return result;
91  //
```

中间连接对应的导数阶数为 [4, 5, 1, 1, 2, 3]，与上一页六行方程逐一对应；最后一段仅有末端  $p/v/a$ ，使用 [1, 2, 3]。energyPartials 给出直接偏导；propagate 输出总路点/时长梯度，也能处理未来其他可微代价。

## 15.6 为什么不用显式矩阵求逆

代码先将  $A$  分解为  $LU$ ，正向求解先解  $Ly = B$ 、再解  $UC = y$ 。伴随复用同一组因子，先解  $U^T y = G$ 、再解  $L^T \Lambda = y$ 。固定带宽下，每行只读写常数个邻居，储存、分解和每次求解都随段数线性增长。这指算法复杂度；实际速度对照在第十六章测量。

## 15.7 带状实现和数值尺度

```

24   for(int k=0;k<size_;++k) {
25       const double pivot=get(k,k);
26       if(!std::isfinite(pivot) || std::abs(pivot)<1e-14)
27       {throw std::runtime_error("Unsafe MINCO pivot; check time/coordinate scales");}
28       const int last=std::min(size_-1,k+bandwidth_);
29       for(int i=k+1;i<=last;++i) {
30           at(i,k)/=pivot;
31           const double multiplier=get(i,k);
32           for(int j=k+1;j<=last;++j) {at(i,j)-=multiplier*get(k,j);}
33       }
34   }
35   //

```

BandedLu 只保存每行附近 13 个元素。教学实现使用本章特定顺序的无主元 LU，不是任意矩阵的通用稳定求解器。若主元非有限或绝对值小于  $10^{-14}$ ，明确报告数值失败。正时长保证数学问题可解，不保证任意悬殊尺度下的浮点计算稳定；优先使用秒、米和适中的分段时长。

## 15.8 独立检查比看图更可靠

测试另建一个密集等式约束二次型：只约束位置和 p/v/a 连接，不加入 jerk/snap 连续条件。jerk 平方积分用三点 Gauss-Legendre 求积计算，它对该四次被积式精确。将能量的驻点条件与等式约束联立为密集线性系统 (KKT)，求解后与带状结果比较，避免用同一组最优性方程自证。1-6 段、seed=1515 的小问题，系数相对误差门限为  $2 \times 10^{-9}$ ，连接到四阶检查门限为  $10^{-8}$ 。

对路点和时长采用步长  $10^{-5}$  的中心差分，检查完整伴随梯度；测试还增加系数平方与显式时长平方项，避免只验证 jerk 这个特殊目标。允许误差为  $2 \times 10^{-6} \max(1, |g_{FD}|)$ ，不是用差分替代实现中的解析梯度。

| 模式    | jerk 积分 | 速度峰值  | 加速度峰值 |
|-------|---------|-------|-------|
| 逐点停靠  | 766.104 | 1.953 | 4.008 |
| MINCO | 122.003 | 1.478 | 2.773 |

演示两端为 (0,0), (4,0) m，中间为 (1,1), (2,-0.2), (3,0.8) m，首尾静止。四段各 1.5 s；能量是解析积分，速度/加速度峰值来自 3001 个采样点。本次能量降低，曲线也偏离折线；**没有施加障碍、走廊或动力学约束。**

1. **推导：**为什么只固定内部位置，最优解仍有连续 jerk 与 snap？
2. **代码 ch15-1：**完成 jerkGram 的一个条目，检查低次项为零、积分指数和因子。
3. **实验：**固定相同边界、路点和时长，对比能量；改变一段时长检查解析梯度的符号与数值。

## 15.9 把约束违反量变成软代价

令  $[h]_+ = \max(0, h)$ , 用  $w[h]_+^3$  惩罚违反; 它的一阶导数是  $3w[h]_+^2$ 。速度和加速度使用平方范数残差, 走廊使用单位法向量残差, ESDF 使用原始中心距离:

$$h_v = \|v\|^2 - v_*^2, \quad h_a = \|a\|^2 - a_*^2, \quad h_c = n^T p - b + \epsilon, \quad h_d = d_* - d(p). \quad (15.7)$$

走廊已经在圆盘配置空间检查过;  $\epsilon$  是额外优化余量, 不再加一遍半径。  $d(p)$  用于可微软惩罚; 独立检查仍使用第十二章的净空下界。各项单位不同, 权重负责尺度换算。总目标为  $w_J J + w_T \sum T_i + \int \sum w[h]_+^3 dt$ 。

每段在  $s_j = j/K$  处求梯形积分, 局部时间  $t_j = s_j T$ 。改变  $T$  时, 积分权重  $T/K$  和采样位置都变化。下列代码里的 `gp/gv/ga/gj` 是惩罚对 `p/v/a/jerk` 的偏导, `weight` 在两端取  $1/2$ ; 本章补充的力变化率代价会用到 `gj`:

```
76     result.cost+=weight*dt*penalty;
77     result.coefficients.middleRows<6>(6*i)+=weight*dt*(
78         b0.transpose()*gp.transpose()+b1.transpose()*gv.transpose()+b2.transpose()*ga.transpose()+
79         polynomialBasis(t,3).transpose()*gj.transpose());
80     // Both integration weight T/K and sample location t=s*T depend on T.
81     result.times(i)+=weight*(penalty/c.quadrature_steps+
82         dt*s*(gp.dot(v)+gv.dot(a)+ga.dot(jerk)+gj.dot(derivative(piece,t,4))));
83     //
```

先求直接偏导, 再经 MINCO 的 `propagate` 传回路点和时长, 不能只改变采样间隔。测试固定系数, 分别扰动十二个系数和时长, 检查全部软项; 差分步长  $10^{-6}$ 。

## 15.10 始终保持时长为正

令  $T_i = T_{\min} + e^{\tau_i}$ , 优化无约束变量  $\tau_i$ 。链式法则给出  $dK/d\tau_i = (dK/dT_i)(T_i - T_{\min})$ :

```
43     for(int k=0;k<nt;++k)
44         value.gradient(nq+k)=gradient.times(k)*(T[k]-config.min_duration);
45     //
```

不把越界时间硬裁剪后假装梯度不变。指数溢出、线性求解失败、ESDF 越出有效域都使该试探步无效。BFGS 用 Armijo 回溯寻找有限且下降的步; 逆 Hessian 更新使用实际变量差和梯度差。内层 MINCO 随段数线性增长, 外层教学版密集 BFGS 使用二次内存, 两者不要混淆。状态分别报告 `converged`、`max_iterations`、`line_search_failed` 和 `time_limit`; 时间预算在迭代和试探步之间检查, 不是硬实时保证。

## 15.11 收敛之后仍要检查

```
1 ros2 run motion2d corridor_demo tmp/ch13_corridor
2 ros2 run motion2d trajectory_optimization_demo tmp/ch15_optimization
3 /usr/bin/python3 scripts/plot_ch15_optimization.py
```

这是提供栅格的算法实验，非未知环境导航。沿用第十三章墙体、0.1 m 栅格、0.15 m 圆盘和三段走廊，初始时长为对应折线长度除以 0.6 m/s；两端静止。速度/加速度软目标均为 0.8，独立上限均为 1 (SI)；ESDF 目标 0.4 m，净空检查要求严格大于 0.15 m。每段积分 24 格，独立检查 200 格。

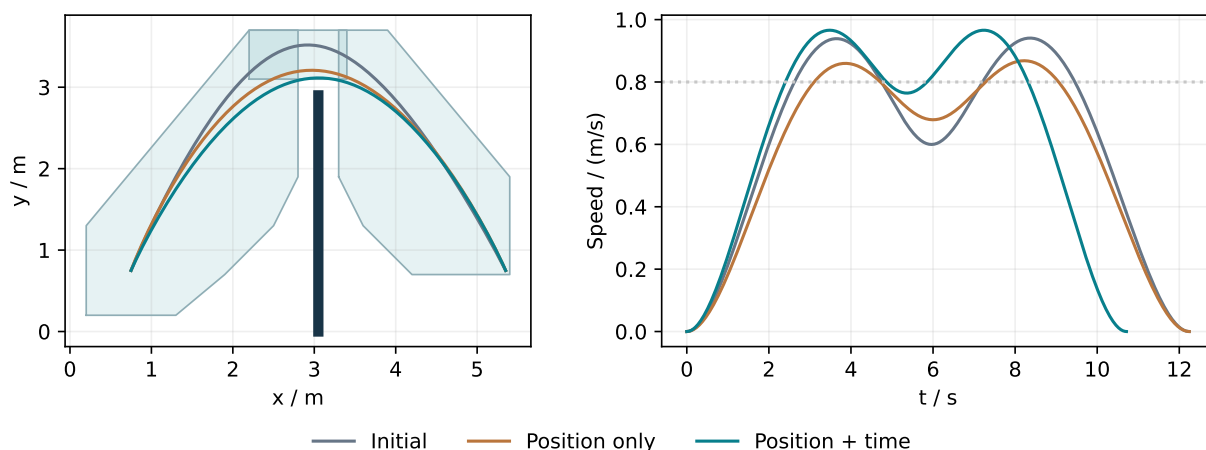


图 15.2: 相同初值下分别锁定全部变量、只优化内部位置、联合优化位置与时间。浅色多边形为走廊。联合优化缩短时间并降低目标，但局部穿出走廊，不能直接作为安全命令。

| 模式    | 总代价     | 时长/s    | 走廊最大残差/m | 净空下界/m |
|-------|---------|---------|----------|--------|
| 初始    | 12.5815 | 12.2428 | -0.0500  | 0.4607 |
| 仅位置   | 12.3159 | 12.2428 | 0.0166   | 0.2160 |
| 位置与时间 | 11.4043 | 10.7129 | 0.0233   | 0.1322 |

三个问题都返回 `converged`，只有初始曲线通过本次采样检查。优化变量为零时，`converged` 仅表示没有可调变量。小代价、驻点和可行性是不同问题。即使 `samples_feasible` 为真，也仅是有限采样；下一章用 Bézier 凸包给出整段的充分条件。本功能只输出诊断，未向机器人发送未经认证的优化轨迹。

- 推导：**只有单段 `jerk` 与时间代价时，证明最优时长为  $T^* = (3600w_J\|\Delta p\|^2/w_T)^{1/6}$ ，并用实验核对。
- 代码 ch15-2：**补全正时长变量的链式梯度。改变  $T_{\min}$  后仍应通过差分。
- 实验：**逐次加大走廊权重，记录残差、迭代数、终止状态；解释为什么权重很大仍不构成连续证明。

## 15.12 在观测地图上预览优化结果

```
1 ros2 launch motion2d_bringup ch15.launch.py rviz:=true
2 # A second terminal, same ROS_DOMAIN_ID:
3 /usr/bin/python3 scripts/check_ch15.py
```

从 RViz 的 Goal Pose 工具设置目标后，planner 在同一份观测地图上依次生成 A\*、走廊、ESDF 与优化曲线。未知空间继续阻塞；没有读取模拟器障碍物。输入仍为 /map、/odometry、/goal\_pose 和坐标变换。起终点取静止 p/v/a，初始分段时间为连接点距离除以 0.5 m/s，至少 0.5 s。这是静止机器人上的单次预览；移动边界和在线衔接留到第十九章。

| 接口                            | 含义                                |
|-------------------------------|-----------------------------------|
| /plan/optimized_preview       | map 系几何 Path，约 0.05 s 采样；没有绝对执行时间 |
| /plan/optimization_status     | 求解状态、独立采样状态和 preview_only 标签      |
| optimization.enabled          | 第十五章默认 true，旧章节默认 false           |
| optimization.max_iterations   | 默认 200；终止于迭代上限仍保留诊断               |
| optimization.max_wall_seconds | 默认 0.2 s；步间检查预算，不是实时截止保证          |

走廊和距离场由同一回调中的栅格构造，避免把旧走廊与新地图混用。非法目标、规划失败或时间回退清空预览；数值失败显示原因。预览不发布 /plan/trajectory，也不向理想到点接口发命令。

RViz 预期：观察地图上出现紫色曲线，青色折线与走廊边框继续可见。实际主机已检查 ROS 消息、端点、清理与输入隔离；当前桌面限制下未验收 GUI 画面。若没有曲线，先读状态：未知目标没有可行路径，ESDF 查询越域和求解失败也不能显示为成功。若出现曲线但报告 sampled\_violation，可调整权重和初值，再核对实际残差。

- 理解：**为什么两个订阅回调中分别缓存走廊和距离场，可能使一次检查失去意义？
- 代码阅读：**在 planner\_node.cpp 找到 publish 中同一地图快照的使用顺序，列出所有输出清空路径。
- 验收：**默认起点 (-8, -8) m、目标 (-7.5, -8) m，检查曲线端点、preview\_only 标签；步进仿真确认机器人未移动，随后发送非法坐标系目标，确认旧曲线消失。

## 15.13 补充：以微分平坦映射约束实际力

第4.15节已把轨迹导数变成真实执行器量。现在将质量和阻尼带进优化器，而不只给加速度一个与质量无关的上限。正向得到  $F = ma + c_v v$ 、 $\dot{F} = mj + c_a a$ ，再用无量纲残差

$$h_F = \frac{\|F\|^2}{F_*^2} - 1, \quad h_{\dot{F}} = \frac{\|\dot{F}\|^2}{R_*^2} - 1, \quad P = w_F [h_F]_+^3 + w_R [h_{\dot{F}}]_+^3. \quad (15.8)$$

$F_*$  单位 N,  $R_*$  单位 N/s。平方范数在零力处也可微，不用除以力的模。先求物理量偏导，再调用第04章的反向映射：

```

53     if(c.dynamics) {
54         const auto & d=c.dynamics;
55         const auto physical=omniForward({v,a,jerk,0,0},d.parameters);
56         OmniFlatOutput g;
57         g.force=hinge(physical.force.squaredNorm()/(d.force*d.force)-1,c.force_weight)
58             *2*physical.force/(d.force*d.force);
59         g.force_rate=hinge(physical.force_rate.squaredNorm()/(d.force_rate*d.force_rate)-1,c.force_rate_weight)
60             *2*physical.force_rate/(d.force_rate*d.force_rate);
61         const auto flat=omniBackward(g,d.parameters);
62         gv+=flat.velocity;ga+=flat.acceleration;gj+=flat.jerk;
63     }
64     //
```

没有启用 **dynamics** 时，旧章节仍采用原来的几何代价。启用后，MINCO 与 Spline2D 共享这一个代价函数和解析偏导；内层如何消去系数，并不改变平坦映射。

**完整反传路线：**物理量代价  $\rightarrow (\bar{F}, \bar{\dot{F}}) \rightarrow (\bar{v}, \bar{a}, \bar{j}) \rightarrow (\bar{C}, \bar{T}) \rightarrow (\bar{q}, \bar{T}) \rightarrow (\bar{q}, \bar{\tau})$ 。前两步是平坦映射反传，中间是多项式与积分链式法则，随后才是 MINCO/Spline 伴随。这正是参考 GCOPTER 的分工，具体二维公式与代码由本课程独立实现。

规划约束检查未饱和的期望力。仿真执行仍受力限幅；两者不能互相替代。当前只优化二维位置，yaw 保持常数，所以没有声称优化偏航力矩或无人机倾角。

## 15.14 时长梯度：不能漏掉 snap

一段内  $p = C^T \beta_0(t)$ ,  $v = C^T \beta_1(t)$ , 依次到 jerk  $j$  和 snap  $s_4$ 。对归一化积分节点  $\alpha \in [0, 1]$ , 局部采样时刻  $t = \alpha T$ :

$$\bar{C} += w \frac{T}{K} (\beta_0 \bar{p}^T + \beta_1 \bar{v}^T + \beta_2 \bar{a}^T + \beta_3 \bar{j}^T), \quad (15.9)$$

$$\bar{T} += w \left[ \frac{P}{K} + \frac{T}{K} \alpha (\bar{p}^T v + \bar{v}^T a + \bar{a}^T j + \bar{j}^T s_4) \right]. \quad (15.10)$$

这里是**固定系数的直接时长偏导**；系数本身随段时长改变的部分，随后由伴随求出。漏掉最后一项时，力变化率惩罚仍有正常的数值，优化方向却是错的。测试分别扰动全部系数、时长、内部路点，再用 MINCO 和 Spline2D 重做，不能只测最终代价下降。

**可复现实验。**在加载工作空间后运行：

```
ros2 run motion2d force_planning_demo
# Exercise from repository root:
g++ -std=c++17 -Iexercises/ch15/starter \
    exercises/ch15/check_flat_time.cpp -o /tmp/ch15_flat_time
/tmp/ch15_flat_time
```

两端静止，从 (0,0) 到 (2,0.3) m，阻尼 0.3 kg/s，质量分别 1、3 kg。初始时长均 2 s；只调时长，软目标是独立上限的 85%，独立力上限 1 N、力变化率上限 2 N/s。程序输出两种后端的时长、连续力界和求解状态；任一项未通过便非零退出。软目标与验证限值分开，使优化器为软约束残差留有空间。

**配置入口。**planner 与 navigation 都读取以下可选项；规划质量、阻尼必须与模拟器相同：

```
planning.dynamics.enabled: true
planning.dynamics.mass: 1.0
planning.dynamics.linear_drag: 0.15
planning.dynamics.force_max: 1.5
planning.dynamics.force_rate_max: 4.0
```

这里 force\_max 是**向量范数**上限。模拟器的原始 force\_max 是每轴上限；将范数上限设为不大于每轴上限是保守且一致的选择。反馈控制还需要保留执行器余量。

**练习 ch15-3：**补全移动采样时刻的直接偏导；解释质量翻倍但时长固定时，旧加速度约束为什么不能保证同样的力界。观察更大质量对应的时长变化，先看 certified 与 status，再比较代价；不要拿不可行解宣称规划更快。

## 第十六章 换一种消元顺序：二维 Spline2D

**目标** 理解块三对角系统，验证同一最小 jerk 问题的系数与梯度，实测性能。

**先修** 第 14、15 章；本章固定二维五次，不扩展到任意阶通用框架。

### 16.1 先固定问题，再比较实现

```
1 ros2 run motion2d spline_demo tmp/ch16_spline
2 bash scripts/benchmark_ch16.sh
3 /usr/bin/python3 scripts/plot_ch16_spline.py
```

第一条无需下载。第二条是可选上游对照，使用 [SplineTrajectory](#) 的 QuinticSplineND<2>，固定版本为 126525e49a43b0548bc6960e4979dd6b6258f289。上游为 MIT 许可证；对照脚本只在临时目录读取它，课程核心独立实现。本章的 Spline2D 和上一章 MINCO 解决同一个 **clamped 最小 jerk 问题**：相同首尾 p/v/a、内部位置和正时长，结果应相同。它不是另一条曲线，也不是把路点当 B-spline 控制点。

### 16.2 未知数改为连接点的速度与加速度

单段边界状态写为  $W_i = [p_i, v_i, a_i, p_{i+1}, v_{i+1}, a_{i+1}]^T$ ，每行有 x/y 两列。第十四章已能从边界状态求六个系数，因此存在  $C_i = E_i(T_i)W_i$ 。用单位边界依次调用 `interpolateQuintic`，可以直接构造  $E_i$  的六列；无需初学者先抄一大组公式。代入 jerk 二次型：

$$J_i = \text{tr}(W_i^T R_i W_i), \quad R_i = E_i^T Q_i E_i. \quad (16.1)$$

所有位置固定，首尾 v/a 固定，只有内部  $z_k = [v_k, a_k]^T$  待求。每段只含相邻两个节点，所以驻点条件形成对称块三对角系统：

$$\begin{bmatrix} H_0 & B_1^T & 0 \\ B_1 & H_1 & B_2^T \\ 0 & B_2 & H_2 \end{bmatrix} \begin{bmatrix} z_0 \\ z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} r_0 \\ r_1 \\ r_2 \end{bmatrix}. \quad (16.2)$$

每个块为  $2 \times 2$ ，右端项有 x/y 两列。这是结构示例，实际块数为  $N - 1$ 。从各段  $R_i$  取出对应 v/a 行列相加，已知位置和边界贡献移至右侧。

## 16.3 一个块接一个块消元

令  $D_0 = H_0$ ，依次计算

$$L_i = B_i D_{i-1}^{-1}, \quad D_i = H_i - L_i B_i^T. \quad (16.3)$$

先解下三角，再解对角块，最后回代。每步固定规模，内存与时间随段数线性增长。代码只对  $2 \times 2$  正定块用 Cholesky 求解两个单位右端，缓存其逆作用；未显式求整张大矩阵的逆。

```

49  for(int i=0;i<m;++i) {
50      if(i) {
51          const Eigen::Matrix2d off=lower_[i];
52          lower_[i]=off*inverse_diagonal_[i-1];
53          diag[i]=lower_[i]*off.transpose();
54      }
55      Eigen::LLT<Eigen::Matrix2d> factor(diag[i]);
56      if(factor.info()!=Eigen::Success) throw std::runtime_error("Spline block is not positive definite");
57      inverse_diagonal_[i]=factor.solve(Eigen::Matrix2d::Identity());
58  }
59  const Eigen::MatrixX2d free=solveBlocks(rhs);
60  for(int i=0;i<m;++i) states_.middleRows<2>(3*i+4)=free.middleRows<2>(2*i);
61  //

```

得到所有节点  $v/a$  后，再经  $E_i W_i$  还原为通用五次轨迹。非正时长、维度不匹配、非有限数值或非正定分解均明确失败。任意悬殊时长仍可能损害浮点精度。

## 16.4 能量梯度可以直接看边界

第十五章分部积分给出的边界项也能用于梯度。内部位置的导数为两段第五阶导数之差：

$$\frac{dJ}{dq_i} = 2 \left( p_{i-1}^{(5)}(T_{i-1}) - p_i^{(5)}(0) \right). \quad (16.4)$$

每段时长导数可在该段起点计算：

$$\frac{dJ}{dT_i} = -\|p_i'''\|^2 + 2p_i'''' \cdot p_i'' - 2p_i^{(5)} \cdot p_i'. \quad (16.5)$$

这是固定端点、路点的完整导数；最优内部导数的变化因驻点条件消去。把起点导数代成  $v = c_1, a = 2c_2, j = 6c_3, s = 24c_4, p^{(5)} = 120c_5$ ：

```

89  for(int i=0;i<n-1;++i)
90      g.waypoints.row(i)=240*(coefficients_.row(6*i+5)-coefficients_.row(6*i+11));
91  for(int i=0;i<n;++i) {
92      const auto C=coefficients_.middleRows<6>(6*i);
93      g.times(i)=-36*C.row(3).squaredNorm()+96*C.row(4).dot(C.row(2))-240*C.row(5).dot(C.row(1));
94  }
95  //

```

该专用路径无需额外伴随。若加入 ESDF 等其他代价，仍需下一节的一般传播。

## 16.5 任意代价的伴随如何复用块分解

记未知内部导数为  $z$ ，其能量驻点条件为  $F(z, q, T) = 0$ ，且  $F_z = H$ 。给定系数偏导  $G_i$ ，先计算边界偏导  $E_i^T G_i$ ，累加得到  $K_z$ 。再解  $H\lambda = K_z$ ，因此

$$dK = K_q dq + K_T dT - \lambda^T (F_q dq + F_T dT). \quad (16.6)$$

这里  $K_q, K_T$  是固定  $z$  的直接偏导。propagate 复用构造时的块分解求伴随。为了计算  $E_T$ ，令  $M(T)C = W$  为六个端点条件；由  $E = M^{-1}$  得  $E_T = -EM_T E$ 。  $M_T$  只有末端三行非零，分别是速度、加速度、jerk 基函数。然后对  $R = E^T Q E$  用乘积法则；这几步均为每段固定大小矩阵，仍是线性复杂度。

## 16.6 误差验收与本机计时

测试固定 seed=1616，1-12 段、非零边界 v/a、不同正时长。系数与 MINCO 的相对误差要求低于  $10^{-9}$ ；MINCO 已另与独立密集 KKT 对照。解析能量梯度与通用伴随交叉检查；含速度/加速度软惩罚的一般传播用  $10^{-5}$  中心差分检查。

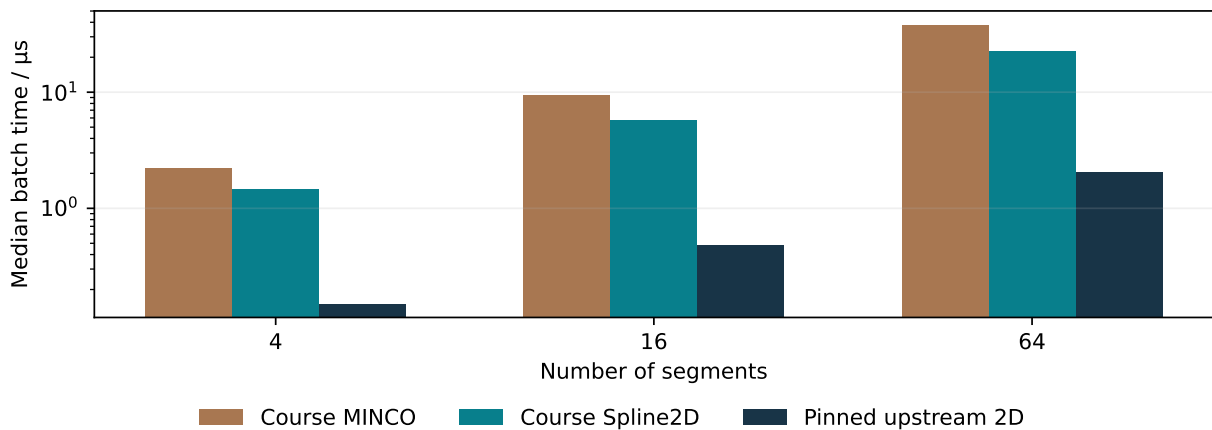


图 16.1: 同一进程、相同输入下的构造、能量及完整能量梯度总耗时；纵轴为对数。不包含地图、碰撞检查、I/O、ROS 或外层非线性优化。

| 段数 | 教学 MINCO/ $\mu\text{s}$ | 教学 Spline2D/ $\mu\text{s}$ | 固定上游/ $\mu\text{s}$ |
|----|-------------------------|----------------------------|---------------------|
| 4  | 2.247                   | 1.476                      | 0.152               |
| 16 | 9.453                   | 5.793                      | 0.484               |
| 64 | 38.000                  | 22.735                     | 2.049               |

GCC 15.2、Eigen 3.4、C++17，统一 -O3、-fno-fast-math，无 LTO；预热 10 次，21 组各 20 次，取组均值中位数。同问题系数相对误差低于  $4.2 \times 10^{-14}$ ，能量梯度低于  $6.5 \times 10^{-13}$ 。教学实现保留较多矩阵运算便于阅读；上游采用专门化公式。本机小实验不能替代完整规划器基准，也不承诺固定加速比。

1. **理解：**两种实现消去了不同变量，为何仍得到同一条曲线？
2. **代码 ch16-1：**补全 Schur 块，和独立  $4 \times 4$  密集方程的解比较。
3. **实验：**改变段数与时长尺度，同时报告数值误差、失败状态及耗时，禁止只展示更快的一次。

## 16.7 不改曲线，只换成 Bézier 控制点

将秒制多项式改用归一化时间  $s = t/T$ 。五次 Bernstein 基函数  $B_j^5(s) = \binom{5}{j} s^j (1-s)^{5-j}$  非负且和为一，因此

$$p(s) = \sum_{j=0}^5 P_j B_j^5(s), \quad P_j = \sum_{k=0}^j \frac{\binom{j}{k}}{\binom{5}{k}} c_k T^k. \quad (16.7)$$

$P_j$  是另一种表示的控制点；它们通常不在曲线上。若全部控制点位于同一个凸多边形，它们的任何凸组合都在其中，故整段曲线都在其中。这里的多边形必须先经过第十三章的**完整区域配置空间检查**，不能只检查多边形顶点。不能把“控制点处 ESDF 足够大”替换成该证明：一般自由空间不是凸集。

速度与加速度也有控制点，其凸包分别包含整段导数：

$$V_j = \frac{5}{T}(P_{j+1} - P_j), \quad A_j = \frac{20}{T^2}(P_{j+2} - 2P_{j+1} + P_j). \quad (16.8)$$

由范数凸性， $\max_j \|V_j\|$  和  $\max_j \|A_j\|$  是整个区间的上界。它们可能偏保守，但不是某几个轨迹采样点的最大值。

## 16.8 二分能缩紧界，不能改变事实

de Casteljau 递推把相邻控制点取平均，得到精确表示左右半段的新控制点。重复二分后，对每个小段作同样检查。曲线与时间未改变；位置凸包逐步缩紧，速度/加速度用对应小段时长计算。默认深度 5，每段检查 32 个子段；接口允许 0-10。

```

36 for(int j=0;j<6;++j) {
37     const Eigen::VectorXd h=region.A*P.row(j).transpose()-region.b;
38     increase(result.corridor_residual,
39         h.allFinite() ? h.maxCoeff() : std::numeric_limits<double>::infinity());
40 }
41 for(int j=0;j<5;++j) {
42     const auto velocity=5/T*(P.row(j+1)-P.row(j));
43     increase(result.speed_bound,velocity.norm());
44 }
45 for(int j=0;j<4;++j) {
46     const auto acceleration=20/(T*T)*(P.row(j+2)-2*P.row(j+1)+P.row(j));
47     increase(result.acceleration_bound,acceleration.norm());
48 }
49 //

```

`certifyBezier` 必须收到每段对应的已验证区域。所有半空间残差不大于  $10^{-9}$ ，且导数上界不超过物理上限加同样的数值容差，才报告 `certified`。 $10^{-9}$  是浮点比较容差，不是机器人避障裕量。未通过只说明该充分条件无法认证，也可能确实越界；不能据此断言曲线必然碰撞。

## 16.9 加入控制点代价，再独立认证

沿用上一章的正时长变量与外层 BFGS，新增 `optimizeSpline` 选择实际第二种内核。两种实现共用同一份优化器和代价；没有复制一套配置工厂。对控制点的走廊残差、导数范数违反量加入三次正部惩罚。`bezierCost` 将梯度传回系数和时长，显式包含  $T^k$  的导数；之后由样条伴随传回路点和时间。它是控制点代价之和，不是对时间的数值积分，权重也不能照搬上一章后就假设同尺度。

```
1 ros2 run motion2d bezier_demo tmp/ch16_bezier
2 /usr/bin/python3 scripts/plot_ch16_bezier.py
```

同一墙体栅格、初始路点/时长和端点；第三组额外使用控制点代价，走廊权重 50000、额外余量 0.04 m。三组的目标定义并不完全相同，不把总代价大小用作横向优劣结论，改看时长与约束检查。

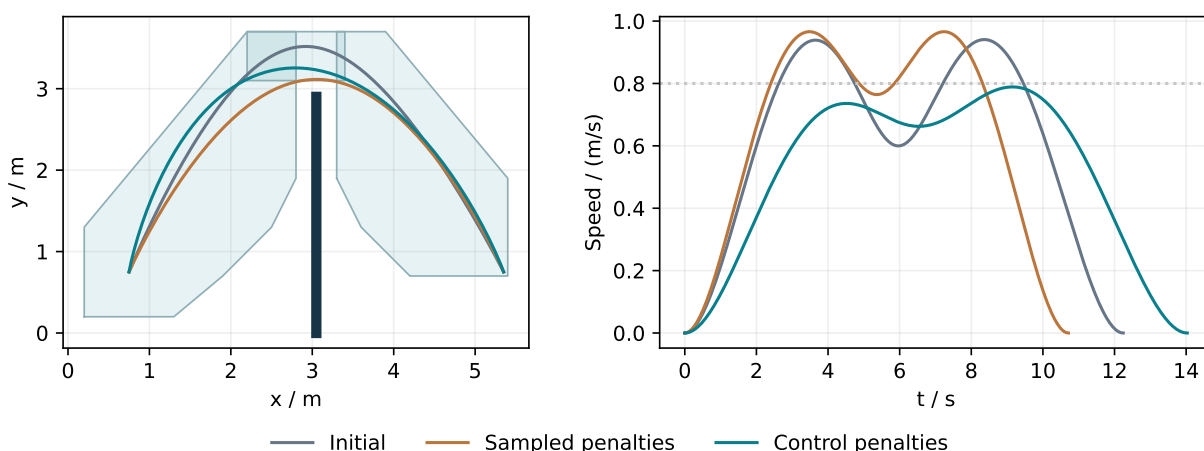


图 16.2: 只使用轨迹采样软惩罚仍可能离开凸走廊；控制点惩罚把本次解推回可认证范围。青色曲线与其他曲线均从同一初值开始，后验认证独立于优化器状态。

控制点版本本次返回 `converged`，整段走廊最大残差为  $-0.03427$  m，并通过速度/加速度连续上界检查；原采样惩罚版本残差为正，认证失败。即使在地图上认证成功，也只证明参考圆心曲线相对于这份固定栅格满足所设条件。定位误差、追踪误差及地图更新需要额外裕量和后续在线处理。

验证包括：Bernstein/de Casteljau 值与原多项式导数逐点一致；控制点代价的系数/时长差分；端点安全但中间越界的二次曲线反例；静止到静止 1 m、2 s 曲线的速度和加速度解析峰值。后者未细分时界较保守，细分后能认证速度  $\leq 1$  m/s、加速度  $\leq 1.5$  m/s<sup>2</sup>。

1. **推导：**为什么控制点在凸走廊内是充分条件，却不是必要条件？
2. **代码 ch16-2：**补全归一化基底映射中的二项式比例，保留  $T^k$ 。
3. **实验：**从深度 0 增到 5，记录上界收紧和耗时变化；确认轨迹采样结果完全不变。

## 16.10 认证候选与实际执行分开

```
1 ros2 launch motion2d_bringup ch16.launch.py rviz:=true
2 # Place a goal in observed free space, then explicitly execute:
3 ros2 service call /trajectory/execute std_srvs/srv/Trigger '{}'
4 # Automated paused / single-step acceptance:
5 /usr/bin/python3 scripts/check_ch16.py
```

本章配置采用 Spline2D、控制点软惩罚和独立连续认证。只有优化器收敛且 `certifyBezier` 通过, `planner` 才发布 `/plan/certified_candidate`。它携带完整五次系数、规划时 `odom` 时间戳和固定 `yaw`; `start_time=0` 表示候选曲线的局部原点, 不触发执行。空 `pieces` 撤销候选; 未收敛或认证失败时仍可看到诊断预览, 但无法作为本章执行源。

将 `map` 中曲线固定到 `odom` 时, 令  $p_o = Rp_m + t$ , 则

$$c_{o,0} = Rc_{m,0} + t, \quad c_{o,k} = Rc_{m,k} \ (k > 0). \quad (16.9)$$

只对常数项加平移; 速度、加速度和更高阶系数只旋转。刚性变换保持范数与包含关系。候选不会因之后的 `map/odom` 变换更新而反复移动; 第十九章再处理新地图下的重规划。

`trajectory.source=certified` 让服务读取此候选。发送实际命令前检查:

- 候选与当前 `odom` 时间差在  $[0, 0.5]$  s, 起点与当前位置一致。
- 机器人静止、首尾 `v/a` 为零、`yaw` 与当前一致, 才能与等待/停止段连续衔接。
- 通过后才设置绝对开始时间 `now+0.25` s, 并发布 `/plan/trajectory`。

求解与执行分开, 能避免点击目标时不知不觉让机器人运动; 仿真仍保留原来的整段扫掠检查。静态地图上的曲线认证不能替代实际执行时的检测。

**ROS 验收与常见错误** 默认真值定位、观测建图, 起点  $(-8, -8)$  m、近目标  $(-7.5, -8)$  m。脚本确认候选曲线经过认证但机器人保持静止; 显式服务调用后, 用接收的系数独立计算 `p/v/a`, 与每个仿真 `tick` 对照。发送非法目标后候选清空、执行服务拒绝; `reset` 后也不能复用旧候选。另有带旋转和平移的单元测试, 检查导数只旋转、常数项才平移。

`RViz` 预期同时显示几何优化预览和已发送命令的带时轨迹, 圆盘沿后者运动。本机 ROS 消息和无界面执行已验证; `GUI` 仍未在当前桌面限制下验收。若服务拒绝, 先看“候选过期/机器人未静止/起点不一致”, 而不是绕开条件发送旧系数。

1. **理解:** 为什么对每个系数都加 `TF` 平移, 会使速度、加速度一起出错?
2. **实验:** 先预览再步进, 确认没有自动执行; 执行后验证 `p/v/a`, 而不只检查最终位置。
3. **扩展准备:** 若当前机器人正在运动, 写出新轨迹起点必须匹配的 `p/v/a` 条件; 暂不删除本章静止检查。

## 16.11 补充：力约束也要检查整段

第 15 章的力软惩罚仍是有限采样。对本课程线性受力映射， $F(t)$  与  $\dot{F}(t)$  都是多项式，可以继续使用 Bézier 凸包。若  $p(t) = \sum_{k=0}^5 c_k t^k$ ，越界系数视为零，则

$$F_k = m(k+2)(k+1)c_{k+2} + c_v(k+1)c_{k+1}, \quad (\dot{F})_k = (k+1)F_{k+1}. \quad (16.10)$$

将力多项式转换成 Bézier 控制点  $B_k$ 。范数是凸函数，所以  $\|F(t)\| \leq \max_k \|B_k\|$ ；对力变化率同理。细分收紧的是整段充分上界，不是增加几个采样点后声称连续安全。

```

93 Eigen::Matrix<double,2,6> F=Eigen::Matrix<double,2,6>::Zero(),rate=F;
94 for(int k=0;k<5;++k) F.col(k)+=d.parameters.linear_drag*(k+1)*piece.coefficients.col(k+1);
95 for(int k=0;k<4;++k) F.col(k)+=d.parameters.mass*(k+2)*(k+1)*piece.coefficients.col(k+2);
96 for(int k=0;k<5;++k) rate.col(k)=(k+1)*F.col(k+1);
97 const auto M=bezierMap(piece.duration);
98 out.force_bound=std::max(out.force_bound,vectorBound(M*F.transpose(),depth));
99 out.force_rate_bound=std::max(out.force_rate_bound,vectorBound(M*rate.transpose(),depth));
100 //

```

certifyForce 可单独检查力；启用 dynamics 后，certifyBezier 会同时要求走廊、速度、加速度、力和力变化率通过。因而第 19 章的优化候选及回退轨迹也都受同一检查，不会在优化失败后悄悄换成未检查力的回退曲线。

这项认证针对固定质量/阻尼、固定地图中的名义轨迹；跟踪误差与模型偏差仍需额外余量。分段连接若只满足  $C^2$ ，力连续但力导数可以跳变；本检查给出每段两侧的有限变化率界，没有限制更高阶导数，也不约束力导数的跃变大小。

1. **推导：**写出只有一维匀加速且含阻尼时的力控制点，验证上界包含真实峰值。
2. **代码阅读：**在 certifyBezier 最后找到 certifyForce；解释为何检查失败必须阻止发送候选。
3. **实验：**将 force\_planning\_demo 的质量从 1 改到 3，再把细分深度从 0 改到 5，记录上界与密集采样峰值。上界应不小于采样峰值，不能用密集采样替代上界。

## 16.12 阿克曼起停轨迹：分开几何与时间

不能直接将全向 MINCO 的任意曲线交给阿克曼车。本节使用第 04 章模型，固定路点位置与朝向，优化正的切向长度和段时长。几何路径仍是五次多项式，复用第 14 章的端点插值：

$$\begin{aligned} q_i(0) &= p_i, & q_i(1) &= p_{i+1}, \\ q'_i(0) &= \ell_i e_i, & q'_i(1) &= \ell_{i+1} e_{i+1}, \\ q''_i(0) &= 0, & q''_i(1) &= 0. \end{aligned} \quad (16.11)$$

其中  $e_i = (\cos \psi_i, \sin \psi_i)^T$ ,  $\ell_i > 0$ 。连接处共享朝向；几何端点二阶导数为零，所以端点转角为零。实际时间进度取

$$u = h(r) = 10r^3 - 15r^4 + 6r^5, \quad r = t/T, \quad p(t) = q(h(t/T)). \quad (16.12)$$

$h'(0) = h'(1) = h''(0) = h''(1) = 0$ ，每段都从静止平滑起步、静止结束。这是一种容易理解的起停规划，不是连续高速过弯的完整最优控制器。 $q'(u)$  在整个区间内仍须非零；切向长度为正只保证端点，不能替代整段检查。

用第 04 章映射对  $q', q'', q'''$  求几何量  $n_g, a_g, \kappa, \delta, \delta'_g$ 。令  $d = \dot{u}, e = \ddot{u}$ ，物理量为

$$s = n_g d, \quad \dot{s} = a_g d^2 + n_g e, \quad (16.13)$$

$$F = m(a_g d^2 + n_g e) + c n_g d, \quad r_\delta = \delta'_g d, \quad (16.14)$$

$$\dot{\psi} = \psi'_g d, \quad a_\perp = a_{\perp,g} d^2. \quad (16.15)$$

几何朝向、曲率和转角保持不变。即使  $d = 0$ ，这些公式仍能使用已保存的几何路径：

```

14 AckermannFlatOutput warpAckermann(const AckermannFlatOutput & q, const ProgressSample & h, const
    AckermannParameters & p) {
15     auto y=q; y.speed=q.speed*h.rate; y.yaw_rate=q.yaw_rate*h.rate;
16     y.longitudinal_acceleration=q.longitudinal_acceleration*h.rate*h.rate+q.speed*h.acceleration;
17     y.force=p.mass*y.longitudinal_acceleration+p.linear_drag*y.speed;
18     y.steering_rate=q.steering_rate*h.rate;
19     y.lateral_acceleration=q.lateral_acceleration*h.rate*h.rate; return y;
20 }
21 //
```

**边界：**这条规划路线只做前进、逐段停车；没有倒车支路、动态障碍预测，也不接受运动中的任意新参考。A\* 只提供几何初值，不能证明小车一定能通过每个弯。

## 16.13 从小车的物理限制反传到路径

对速度、纵向力、转角、转向速率、侧向加速度使用与第 15 章相同的归一化立方铰链代价。先通过时间变换反传，再通过几何平坦映射反传：

$$\bar{z}_{\text{physical}} \longrightarrow (\bar{z}_g, \bar{d}, \bar{e}) \longrightarrow (\bar{q}', \bar{q}'', \bar{q}''') \longrightarrow (\bar{C}, \bar{\ell}, \bar{T}). \quad (16.16)$$

例如  $\bar{F}$  给出  $\bar{a}_g \leftarrow md^2\bar{F}$ ,  $\bar{n}_g \leftarrow (me + cd)\bar{F}$ 。固定归一化节点  $r$  时,  $u = h(r)$  不随  $T$  变, 而  $\partial d/\partial T = -d/T$ ,  $\partial e/\partial T = -2e/T$ 。因此本节的直接时长链与第 15 章按秒表示多项式的链不同：

```

89  const auto warped=warpAckermannBackward(geometry,h,g,l.vehicle);
90  const auto flat=ackermannBackward(x,warped.geometry,l.vehicle);
91  out.cost+=w*dt*cost;
92  out.coefficients+=w*dt*(polynomialBasis(h.u,0).transpose()*gp.transpose()+
93    polynomialBasis(h.u,1).transpose()*flat.velocity.transpose()+
94    polynomialBasis(h.u,2).transpose()*flat.acceleration.transpose()+
95    polynomialBasis(h.u,3).transpose()*flat.jerk.transpose());
96  // At fixed alpha, u=h(alpha) is fixed; only rate and acceleration depend on T.
97  out.time+=w*(cost/steps-dt/piece.duration*(warped.rate*h.rate+2*warped.acceleration*h.acceleration));
98  //

```

不是把两种表示的时间偏导相加, 否则会重复计数。几何系数对端点切向长度的导数来自五次 Hermite 插值, 例如对左端速度的系数向量为  $(0, 1, 0, -6, 8, -3)^T$ 。长度取  $\ell = e^\eta$ , 时长取  $T = 0.1 + e^\tau$ , 最后再应用指数链式法则。

本节用小型 BFGS 优化这些变量。它不是 MINCO 的另一种名字: 固定朝向与非完整约束改变了可调参数。正向模型、反向偏导和端点插值分别保留清晰的函数。后续若引入新的曲线表示, 仍可复用同一个物理映射及梯度。

**练习 ch16-3:** 固定几何路径和  $r$ , 推导  $\partial F/\partial T$ , 完成 starter 目录中的 warp\_force.hpp。分别指出惯性项与阻尼项的时长缩放次方。

```

g++ -std=c++17 -Iexercises/ch16/starter \
    exercises/ch16/check_warp.cpp -o /tmp/ch16_warp
/tmp/ch16_warp

```

## 16.14 先验证能转弯，再决定是否减速

`certifyAckermann` 将几何 Bézier 控制点递归二分，在每个小区间求导数凸包。首先检查所有位置控制点在对应的已验证圆盘配置空间走廊内。再沿该小段弦方向  $e_c$  取  $n_{\min} = \min_i e_c^T V_i > 0$ ，证明整段切向非零； $n_{\max} = \max_i \|V_i\|$ 。

叉积与点积是双线性的，故枚举速度/加速度控制点对，可得到整段  $||[q', q'']|| \leq H$ 、 $|q'^T q''| \leq B$ ；速度/jerk 控制点对给出  $||[q', q''']|| \leq H'$ 。于是

$$|\kappa| \leq H/n_{\min}^3, \quad (16.17)$$

$$|\kappa'_g| \leq H'/n_{\min}^3 + 3HB/n_{\min}^5. \quad (16.18)$$

进度导数有全局界  $d \leq 1.875/T$ 、 $|e| \leq 10/(\sqrt{3}T^2)$ 。组合这些界得到速度、力、转角、转向速率和侧向加速度的**连续充分上界**。它可能比实际峰值保守，尤其是转向速率的分母用下界 1；不能把认证失败直接解释为数学上绝对无解。算法会明确报告“不获认证”。

1. 优化器先收敛，再查位置、切向和转角；任何一项不获认证就拒绝曲线。
2. 几何已通过而动力学界过大时，每次将时长乘 1.2，最多尝试 20 次，并重新认证。
3. 只有全部通过才返回可执行曲线。输出 `retiming_steps`，明确最终时间含保守减速，不将它冒充优化器的原始最优解。

转角取决于路径曲率；延长  $T$  不会降低它。因此没有“转不过弯就一直减速”的回退。没有实现的倒车或绕行搜索也不会被全向位置跳变代替。

**运行：**

```
ros2 run motion2d ackermann_planning_demo tmp/ackermann_planning
```

默认从 (0,0,0) 到 (2,0.3,0)，起终点静止；控制跟踪见下一章。边界与走廊是这个离线实验显式提供的输入，不声称它来自未知环境探索。

**实验：**把最大转角设得很小，确认证认证拒绝且不返回执行曲线。再保持几何不变增加时长，比较转角不变、动态界降低。阅读 `continuous bounds` 测试，核对密集采样只用于交叉检查上界，没有被称为连续证明。

## 第十七章 让有质量的机器人跟上轨迹

**目标** 从参考 p/v/a 计算真实力与力矩，区分反馈、前馈、模型失配和饱和。

**先修** 第 04、14 章；本章参考已给定，先隔离跟踪问题。

### 17.1 位置不是受力模型的控制量

```
1 ros2 run motion2d pd_tracking_demo tmp/ch17_pd
2 /usr/bin/python3 scripts/plot_ch17_pd.py
```

理想执行直接取参考位置。有质量的机器人满足  $m\dot{v} = F - c_v v$ ，即使立刻改变力，速度和位置也不会瞬间跳到目标。控制器从同一 odom 坐标系读取当前位置/速度及参考 p/v/a，先求期望加速度，再换算成力：

$$e_p = p_r - p, \quad e_v = v_r - v, \quad F = \hat{m}(a_r + K_p e_p + K_d e_v) + \hat{c}_v v. \quad (17.1)$$

帽子表示控制器使用的参数估计；它不应悄悄读取模拟器真值参数。 $a_r$  是前馈：提前提供沿曲线运动所需的加速度。位置项把偏离拉回，速度项减小来回振荡； $K_p$  单位为  $s^{-2}$ ， $K_d$  为  $s^{-1}$ 。

### 17.2 独立 yaw 与执行器限幅

yaw 误差使用最短包角差，角速度误差按 rad/s 计算；再用转动惯量和角阻力换算力矩。本章圆盘可全向受力，yaw 不决定平移方向；差速轮式车是另一种动力学模型。

```
15 const Eigen::Vector2d acceleration=feedforward+
16     gains.kp.cwiseProduct(reference.pose.position-state.pose.position)+
17     gains.kd.cwiseProduct(reference.velocity-state.velocity);
18 result.requested.force=model.mass*acceleration+model.linear_drag*state.velocity;
19 const double alpha=yaw_feedforward+gains.yaw_kp*wrapAngle(reference.pose.yaw-state.pose.yaw)+
20     gains.yaw_kd*(reference.yaw_rate-state.yaw_rate);
21 result.requested.torque=model.inertia_z*alpha+model.angular_drag*state.yaw_rate;
22 result.applied=limitWrench(result.requested,model);
23 //
```

trackPd 同时返回 requested 和 applied，方便看出要求了多少、实际执行了多少。每个力轴分别限制在  $[-F_{\max}, F_{\max}]$ ，力矩同理；没有把状态速度硬裁掉。若力饱和，机器人可能跟不上轨迹，轨迹应减速或重新分配时间。

### 17.3 固定参考后，比较原因而非只看最终位置

实验使用空 8 m 方形边界、半径 0.2 m 圆盘；净空由边界到圆盘外缘的距离计算，仅用于评估，不输入控制器。实际质量 1 kg、线阻力 0.15 kg/s、转动惯量  $0.02 \text{ kg m}^2$ 、角阻力 0.01；两轴  $K_p = 4, K_d = 3$ , yaw 同值。每次运行 24 s，动力学 200 Hz、控制 50 Hz，命令在四个动力学步内保持。这是无通信延迟的核心实验，ROS 链路的表现单独验收。

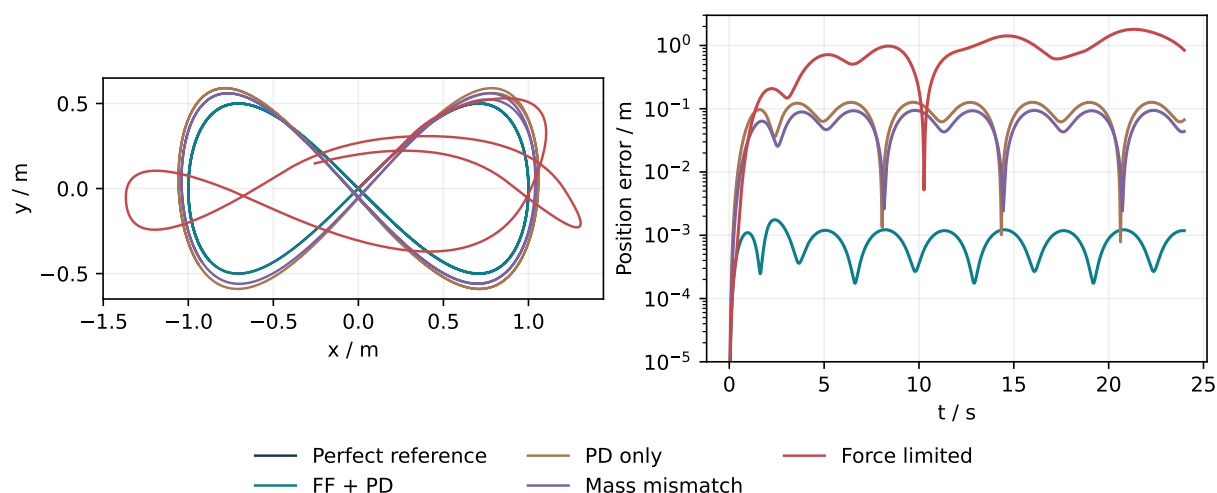


图 17.1: 八字参考上的几何路线与位置误差。误差纵轴为对数，范围下方的早期值不显示；理想参考误差为零，不画在对数轴上。每轴 0.2 N 限力导致明显跟踪失败，即使场景仍有足够净空。

| 控制条件          | 圆形 RMS/m | 八字 RMS/m |
|---------------|----------|----------|
| 前馈 + PD, 模型匹配 | 0.000393 | 0.000898 |
| 关闭加速度前馈       | 0.06112  | 0.09151  |
| 估计质量 0.6 kg   | 0.04262  | 0.06691  |
| 每轴力限 0.2 N    | 0.97127  | 0.95082  |

关闭前馈仍保留阻力补偿；质量失配组同时将估计转动惯量改为  $0.012 \text{ kg m}^2$ 。其他动态组的每轴力限为 2 N、力矩限为 0.2 N m。完整 CSV 还记录最大位置误差、yaw RMS、输入峰值、最小净空和饱和控制次数。模型匹配的八字实际每轴力峰值约 0.508 N；限力组最大位置误差约 1.802 m。理想模式直接采样参考，位置误差为零；其力峰值记 NaN，避免误解成“零牛顿也能完成运动”。

## 17.4 参考要平滑起步，不能凭空跳出速度

直接令相位  $\theta = \omega t$ ，圆轨迹起点已带速度；本章将相位时钟平滑启动。给定渐入时长  $R = 2 \text{ s}$ ，在  $s = t/R \in [0, 1]$  时使用

$$\tau = R(s^3 - \frac{1}{2}s^4), \quad \dot{\tau} = 3s^2 - 2s^3, \quad \ddot{\tau} = 6s(1 - s)/R; \quad \theta = \omega\tau. \quad (17.2)$$

$t \geq R$  后  $\tau = t - R/2$ 、 $\dot{\tau} = 1$ 、 $\ddot{\tau} = 0$ ，因此起点静止，连接处  $\mathbf{p}/\mathbf{v}/\mathbf{a}$  连续。局部坐标下两种路线分别为

$$p_{\text{circle}}(\theta) = r \begin{bmatrix} \sin \theta \\ 1 - \cos \theta \end{bmatrix}, \quad p_{\text{eight}}(\theta) = r \begin{bmatrix} \sin \theta \\ \frac{1}{2} \sin 2\theta \end{bmatrix}. \quad (17.3)$$

用链式法则计算  $v = p_\theta \dot{\theta}$ 、 $a = p_{\theta\theta} \dot{\theta}^2 + p_\theta \ddot{\theta}$ ，再按参考初始 `yaw` 旋转到 `odom`。独立朝向为  $\psi = \psi_0 + 0.4 \sin(\theta/2)$ ，角速度/角加速度也解析求导。默认  $r = 1 \text{ m}$ 、 $\omega = 0.5 \text{ rad/s}$ ；参考尺度与机器人碰撞半径是两件事。

## 17.5 验证、制动与练习

模型精确、无饱和、理想连续反馈时，可将控制律代入得到

$$\ddot{e}_p + K_d \dot{e}_p + K_p e_p = 0. \quad (17.4)$$

此式解释增益作用，却不能直接证明有采样延迟和输入饱和时仍按同样规律收敛。

测试零误差时前馈对应的实际加速度，检查质量和阻力的单位；验证跨  $\pm\pi$  的 `yaw` 误差，检查限幅及阻尼制动力始终反向做功。圆形/八字的  $\mathbf{p}/\mathbf{v}/\mathbf{a}$  与角导数用中心差分检查，包含渐入连接点。这些检查验证控制实现的意义，而不只是图看起来贴合。

`dampingBrake` 按当前速度给出  $F = -\hat{m}k_b v$ 、 $\tau = -\hat{I}_z k_b \omega$  并限幅。它产生一个新的制动命令，后续 MPC 失败时不会继续发送旧优化结果。制动需要时间和距离，不代表能在任何障碍前停下，也不自动满足输入变化率限制。

1. **推导：**去掉加速度前馈时，误差方程右侧多出什么？模型质量估计偏小时又如何变化？
2. **代码 ch17-1：**补全逐轴力限幅，解释它和向量范数缩放为什么不同。
3. **实验：**同样参考依次改变质量估计、阻力和力上限；报告 RMS、最大误差、力峰值与净空，不能只说“增益更大所以更稳定”。

## 17.6 接入 ROS：时间与坐标也属于控制律

```

1 ros2 launch motion2d_bringup ch17.launch.py rviz:=false
2 # 另一个已 source 的终端；初始暂停，检查脚本负责推进
3 /usr/bin/python3 scripts/check_ch17.py
4 # 手动运行时解除暂停
5 ros2 service call /sim/pause std_srvs/srv/SetBool '{data: false}'

```

默认受力模型从静止开始，控制器每 0.02 s 使用一个新的里程计时刻，输出 odom 系 Wrench-Stamped。Odometry 的 pose 在 odom，twist 却在 child\_frame\_id=base\_link：

$$v_{\text{odom}} = R(\psi)v_{\text{body}}. \quad (17.5)$$

不能把机体系速度直接带入世界系反馈；参考速度的发布也必须按参考 yaw 反向旋转。控制器只订阅选择后的 /odometry；真值仅由明确的 truth 适配器提供，控制器不读取仿真内部状态。

| 接口                              | 含义                             |
|---------------------------------|--------------------------------|
| /control/reference              | 同观测时刻的参考位姿与机体系速度               |
| /control/reference_acceleration | odom 系平移加速度和 yaw 角加速度          |
| /control/wrench_requested       | 限幅前的力与力矩，实际值在 /command/wrench  |
| /control/status                 | tracking、saturated 或明确的制动/拒绝原因 |
| /tracker/enable                 | SetBool；关闭后立即产生新的限幅阻尼制动        |

同一时间戳只处理一次；暂停不会重复控制，仿真时间回退时清空参考并重建起点。观测超过 0.12 s 未更新，或收到积压的旧观测，则以最后可用速度产生新的限幅制动；这是失去观测时的退化措施，不能保证停车距离或当前速度准确。暂停时 /clock 不动，不按墙钟误触发超时。

**外部轨迹与验证** reference.source=trajectory 时等待第 14 章的系数消息，必须有当前静止状态、匹配的静止首尾边界和未来起点。本章执行中拒绝替换；错误新消息保留原有效参考。第 19 章再处理运动中的 p/v/a 接续。主机 12 s ROS 实验匹配 600 个观测时刻，位置 RMS 为 0.000940 m、最大 0.001758 m，yaw RMS 为  $6.52 \times 10^{-5}$  rad，最小净空 2.79979 m。check\_tracker\_boundary.py 用独立 /clock 与合成里程计检查旋转、重复时间、旧数据、断流制动及回退清理。RViz 预期为八字参考、圆盘轨迹与观测地图；本机本轮仅验收无界面 ROS 链路。

**练习 ch17-2** 补全机体速度到 odom 的旋转；用  $\text{yaw}=\pi/2$ 、机体系速度 (0.2, 0.3) m/s 验算。实验暂停后等待墙钟 1 s，确认参考没有新增；再仅推进 /clock，观察超时制动。

## 17.7 补充：阿克曼前馈与局部反馈

小车需要纵向力和转向角速度，不能使用全向的两个平移力。从第 16 章曲线取参考  $s_r, \dot{s}_r, \delta_r, \dot{\delta}_r$ ，测量后轴位姿、纵向速度和转角。转角来自显式转向编码器接口，而不是从零速里程计倒算。令  $e$  为当前前向单位向量， $e_\perp$  为左向， $\tilde{p} = p_r - p$ ：

$$F = m\{\dot{s}_r + k_p e^T \tilde{p} + k_v(e^T v_r - s)\} + cs, \quad (17.6)$$

$$\delta_d = \delta_r + k_\psi \text{wrap}(\psi_r - \psi) + \text{atan2}(k_y e_\perp^T \tilde{p}, |s| + s_0), \quad (17.7)$$

$$r_\delta = \dot{\delta}_r + k_\delta(\delta_d - \delta). \quad (17.8)$$

$s_0 > 0$  平滑低速反馈；它不用于篡改平坦映射的分母。控制器不在测量速度上重新求逆，因此可以从静止起步。实际命令仍经过纵向力、转向速率和下一 tick 转角限制。

```

10 AckermannInput trackAckermann(const AckermannState & x, const AckermannReference & ref,
11     const AckermannParameters & p, const AckermannGains & g, double dt) {
12     const Eigen::Vector2d e(std::cos(x.pose.yaw), std::sin(x.pose.yaw)), left(-e.y(), e.x());
13     const Eigen::Vector2d error=ref.state.pose.position-x.pose.position;
14     const double steering=ref.dynamics.steering+g.yaw*wrapAngle(ref.state.pose.yaw-x.pose.yaw)+
15         std::atan2(g.lateral*left.dot(error), std::abs(x.speed)+g.soft_speed);
16     AckermannInput u;
17     u.force=p.mass*(ref.dynamics.longitudinal_acceleration+g.position*e.dot(error)+
18         g.speed*(e.dot(ref.state.velocity)-x.speed))+p.linear_drag*x.speed;
19     u.steering_rate=ref.dynamics.steering_rate+g.steering*(steering-x.steering);
20     return limitAckermannInput(x,u,p,dt);
21 }
22 //

```

这是参考附近的局部跟踪律，不是任意朝向下的全局停车控制器。尤其在终点，单纯纵向修正不能消除任意横向误差。规划需要保留足够的碰撞及执行器余量；发生明显偏离时应停止试验并重新规划。第 18 章原有线性全向 MPC 也不能换一个模型名就控制小车；阿克曼的 NMPC 属于后续扩展。

## 17.8 阿克曼换道实验与模型匹配

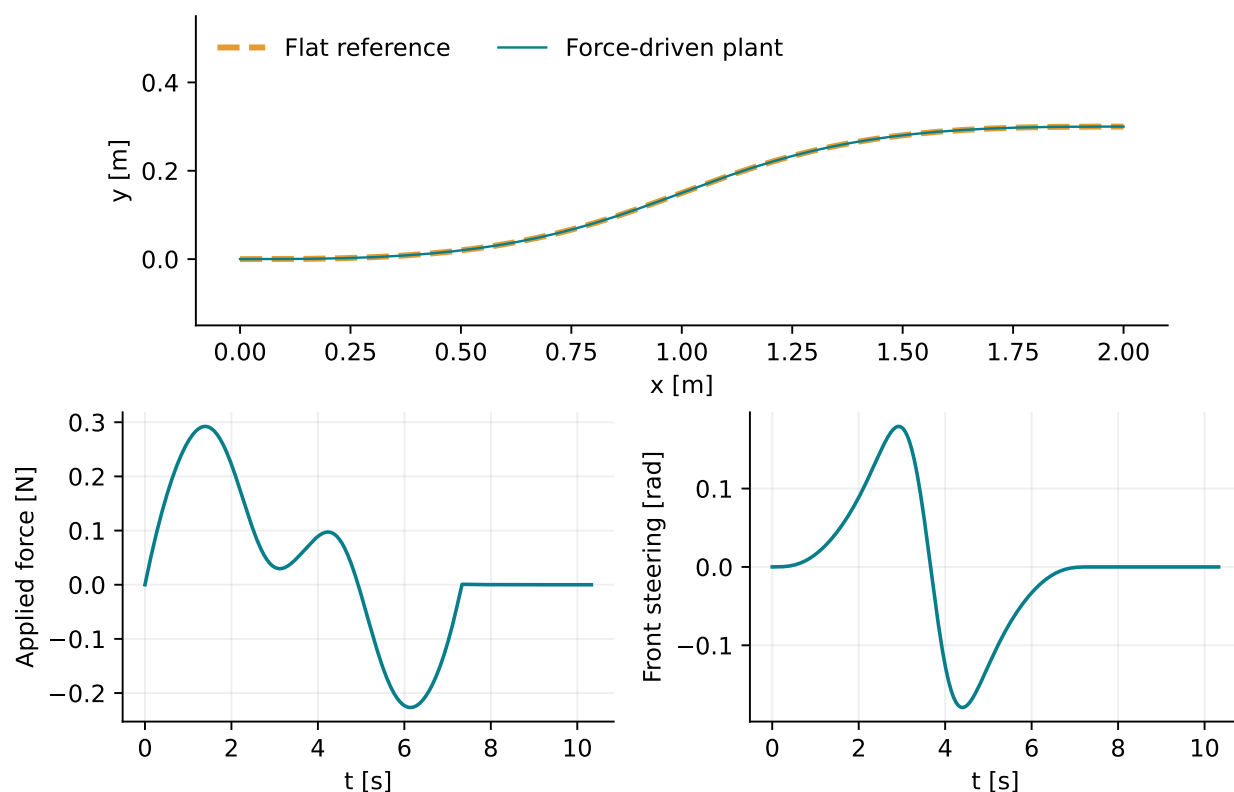


图 17.2: 真实算法输出的起停换道: 虚线参考与受力积分轨迹几乎重合。横向几何采用相同单位长度, 下面分别显示纵向力和前轮转角。

默认  $m = 1 \text{ kg}$ 、 $c = 0.15 \text{ kg/s}$ 、 $L = 0.3 \text{ m}$ , 规划上限依次为速度  $0.7 \text{ m/s}$ 、力  $1 \text{ N}$ 、转角  $0.5 \text{ rad}$ 、转向速率  $0.7 \text{ rad/s}$ 、侧向加速度  $0.8 \text{ m/s}^2$ ; 反馈执行器保留更大的上限。本次优化收敛后做了 2 次保守减速, 最终时长约  $7.327 \text{ s}$ 。力连续上界约  $0.535 \text{ N}$ 、转角界  $0.190 \text{ rad}$ 、转向速率界  $0.443 \text{ rad/s}$ 。

离线反馈与积分都用  $0.005 \text{ s}$ , 模型参数一致, 无观测噪声或延迟; 峰值位置误差约  $0.000171 \text{ m}$ 。这是核对前馈、导数和闭环接口的基线, 不代表实际传感器下也有同样精度。从仓库根目录生成数据和图:

```
ros2 run motion2d ackermann_planning_demo tmp/ackermann_planning
/usr/bin/python3 scripts/plot_ackermann.py
```

**练习:** 仅把控制器中的质量估计调成真实值的 1.3 倍, 保留被控模型和同一参考, 比较误差、峰值力及饱和次数。再关闭前馈做对照; 每次只改一个因素。对转向轴也做类似实验, 观察为什么只有位置终点看起来正确并不能验证全程姿态。

# 第十八章 线性 MPC 与约束跟踪

**目标** 将受力模型、参考轨迹和线性约束写成 QP；只执行本次有效解的第一步。

**先修** 第 04、13、14、17 章；yaw 仍使用独立前馈 + PD。

## 18.1 把连续模型积分成预测矩阵

```
1 ros2 run motion2d mpc_demo tmp/ch18_mpc
2 /usr/bin/python3 scripts/plot_ch18_mpc.py
```

取  $x = [p_x, p_y, v_x, v_y]^\top$ 、 $u = [F_x, F_y]^\top$ ，控制间隔为  $h$ 。在一步内保持力不变，令  $\lambda = c_v/m$ ， $e = e^{-\lambda h}$ ， $g = (1 - e)/\lambda$ ， $f = (h - g)/\lambda$ ，则

$$x_{k+1} = Ax_k + Bu_k, \quad A = \begin{bmatrix} I & gI \\ 0 & eI \end{bmatrix}, \quad B = \begin{bmatrix} (f/m)I \\ (g/m)I \end{bmatrix}. \quad (18.1)$$

$\lambda \rightarrow 0$  时  $g = h, f = h^2/2$ 。小阻力应使用第 04 章的稳定极限，避免两个相近数相减。`linearInertialModel` 复用精确积分器的基向量响应来提取 A/B；输入取力限内的幅值后归一化。控制与仿真共用动力学，测试另核对零阻力解析值。

重复代入  $N$  步，将未来状态和力依次堆叠为  $X = [x_1; \dots; x_N]$ 、 $U = [u_0; \dots; u_{N-1}]$ ：

$$X = S_x x_0 + S_u U, \quad (S_x)_k = A^k, \quad (S_u)_{k,j} = \begin{cases} A^{k-1-j} B, & 0 \leq j < k, \\ 0, & j \geq k. \end{cases} \quad (18.2)$$

预测步数与预测时间不同： $N = 20, h = 0.02 \text{ s}$  只看未来 0.4 s。位置与速度均在 `odom`，力单位 N。

## 18.2 跟踪、前馈和输入变化组成一个二次型

未来参考在  $t + h, \dots, t + Nh$  采样，前馈力  $U_r$  由  $\hat{m}a_r + \hat{c}_v v_r$  堆叠。令  $D$  为主对角 I、下一条块对角  $-I$  的差分矩阵， $d = [u_{\text{previous}}; 0; \dots]$ ，代价为

$$J = \|S_x x_0 + S_u U - X_r\|_{\bar{Q}}^2 + r \|U - U_r\|^2 + s \|DU - d\|^2. \quad (18.3)$$

$\bar{Q}$  每步对位置与速度分别加权； $r > 0$  使力的二次型正定。写成  $\frac{1}{2}U^T P U + q^T U + \text{const}$  后，

$$P = 2(S_u^T \bar{Q} S_u + rI + sD^T D), \quad (18.4)$$

$$q = 2\{S_u^T \bar{Q}(S_x x_0 - X_r) - rU_r - sD^T d\}. \quad (18.5)$$

上一拍必须用**实际施加的力**，包括制动输入；不能用旧参考或未限幅请求替代。

## 18.3 把约束按行装进矩阵

求解器的统一形式为  $l \leq CU \leq u$ ，这里  $C$  是约束矩阵，避免与动力学  $A$  混淆。分别堆叠以下线性行：

- 力：  $-F_{\max} \leq U_i \leq F_{\max}$ ；两个方向独立。
- 变化率：  $-\dot{F}_{\max}h \leq DU - d \leq \dot{F}_{\max}h$ ，单位 N。
- 速度：从  $S_x x_0 + S_u U$  取出各步速度，限制两个分量。
- 走廊：每步选择已验证的配置空间凸区，代入  $H_k p_k \leq b_k$ 。

输入  $N$  个凸区与  $N$  个预测状态一一对应；区域已经考虑圆盘半径，不再膨胀一次。这些约束只限制预测节点，不证明节点之间、定位误差下或跟踪偏离后的连续安全。ESDF 是非线性距离场，若直接加入非线性约束，就需要另讲 SQP/NMPC，不能继续称作此处的线性 QP。

```

60 out.q=2*(prediction_.Su.transpose()*Q_*(free-target)
61   -config_.force_weight*feedforward-config_.increment_weight*D_.transpose()*d);
62 out.A=Eigen::MatrixXd::Zero(6*n+extra,2*n);
63 out.lower=Eigen::VectorXd::Constant(6*n+extra,-std::numeric_limits<double>::infinity());
64 out.upper=Eigen::VectorXd::Zero(6*n+extra);
65 out.A.topRows(2*n).setIdentity();
66 out.lower.head(2*n).setConstant(-model_.force_max);out.upper.head(2*n).setConstant(model_.force_max);
67 out.A.middleRows(4*n,2*n)=D_;
68 out.lower.segment(4*n,2*n)=d.array()-config_.force_rate_max*config_.dt;
69 out.upper.segment(4*n,2*n)=d.array()+config_.force_rate_max*config_.dt;
70 //

```

## 18.4 看得懂的 ADMM 求解器

本课程独立实现小型稠密正定 QP 求解器，不调用 OSQP 库。它采用分裂变量  $z$  将线性系统求解与区间投影分开，主要参考 [OSQP 官方算法说明](#)，省去稀疏排序、自动缩放和抛光等工程功能。

```

57 const Eigen::VectorXd old_y=y;
58 result.x=factor.solve(s.sigma*result.x-p.q+p.A.transpose()*(rho*z-y));
59 const Eigen::VectorXd ax=p.A*result.x;
60 z=(ax+y/rho).cwiseMax(p.lower).cwiseMin(p.upper);
61 y+=rho*(ax-z);
62 result.primal_residual=norm(ax-z);
63 result.dual_residual=norm(p.P*result.x+p.q+p.A.transpose()*y);
64 //

```

这里 `solve` 使用缓存的 LDLT 分解；每 25 次迭代按归一化残差调整  $\rho$ ，变化足够大时重新分解。只有原始残差  $CU - z$  与对偶残差  $PU + q + C^T y$  均满足绝对/相对容差才返回 `solved`。默认容差均为  $10^{-5}$ 、最多 1500 次；无约束测试对照解析解  $-P^{-1}q_0$ 。

## 18.5 失败状态与短时域的边界

成功后仅施加  $u_0$ ；下一次重读状态、重建 QP。上次成功 U 左移一块、重复尾块，作为本次初值；失败立即清空热启动并生成新的限幅阻尼力。稠密构建、分解和迭代适合本章小问题，不承诺硬实时。

| 状态                | 可得结论                    |
|-------------------|-------------------------|
| solved            | 残差在设定容差内；不是连续碰撞或闭环稳定性证明 |
| primal_infeasible | 上下界矛盾，或找到近似归一化不可行证书     |
| max_iterations    | 给定迭代数内没收敛；不能因此断言问题不可行   |
| time_limit        | 迭代间检查到墙钟预算用完；当前迭代可超出预算  |
| numerical_failure | 非有限迭代；不能继续使用该输入序列       |

不可行证书满足  $C^T v \approx 0$  且  $u^T v_+ + l^T v_- < 0$ ；非有限单侧边界不能参与错误的有限乘积。无效外部维度或非正定价直接报错；示例对超时和迭代不足分别验收。制动仍只保证力限幅，**不保证输入变化率、停止路径或失去观测后的速度准确性**。

**一个可复现的不可行例子** 测试用 1 kg、阻力 0.15、力限 0.8 N、变化率 0.8 N/s,  $h = 0.1$  s、 $N = 12$ ，走廊右边界  $x = 0.3$  m，却让代价追逐  $x_r = 1$  m。前几次 QP 都满足各自节点约束，第八次则不可行。独立检查每一步所能用的最小  $x$  力  $\max(-F_{\max}, u_{\text{previous}} - j\dot{F}_{\max}h)$ ，即使这样尽早减力，第 12 步仍到约 0.30618 m，超过边界。有限时域末端还带着速度，下一次多看到一步时便发现来不及停。终端安全集合、可停车参考和约束收紧是进一步方法；本章不宣称已实现递归可行性保证。

### 练习与独立检查

1. 推导一步 A/B 的零阻力极限，并核对 kg、N、m/s 的单位。
2. **ch18-1**: 补全输入差分矩阵；解释为什么首块要减上一拍实际输入。
3. 显式滚动积分并计算各项代价，用中心差分核对凝聚后梯度  $PU + q$ 。
4. 固定参考比较 PD/MPC、预测长度和控制频率；记录失败次数、实际变化率与求解耗时，不能只画误差。

## 18.6 同一八字参考上的实测对照

使用第 17 章相同八字、24 s、200 Hz 物理积分、1 kg 与 0.15 kg/s 阻力，初始静止，yaw 使用相同 PD；本表没有通信延迟或估计噪声。MPC 默认位置/速度权重 40/2、力权重 0.02、增量权重 0.005；不加终端集合。2 N 组速度分量限 1 m/s、力变化率限 5 N/s。

| 方法      | N  | 控制/Hz | 位置 RMS/m              | 耗时 P95/ms |
|---------|----|-------|-----------------------|-----------|
| 前馈 + PD | -  | 50    | $8.98 \times 10^{-4}$ | -         |
| MPC     | 10 | 50    | $4.63 \times 10^{-5}$ | 0.0300    |
| MPC     | 20 | 50    | $9.37 \times 10^{-6}$ | 0.0826    |
| MPC     | 40 | 50    | $5.56 \times 10^{-6}$ | 0.4114    |
| MPC     | 20 | 20    | $1.44 \times 10^{-5}$ | 0.0790    |

所有常规组均无求解失败。耗时包含 QP 组装、分解、迭代和热启动更新，不含 ROS 或绘图；主机 Ryzen 7 9700X、GCC 15.2、RelWithDebInfo，未锁频或绑核。较长时域在此例更精确也更慢；20 Hz 组  $Nh = 1$  s，与 50 Hz 的 0.4 s 不是同一个前瞻时间。极小误差来自准确模型与未来参考，不能照搬为真实传感器系统的性能。

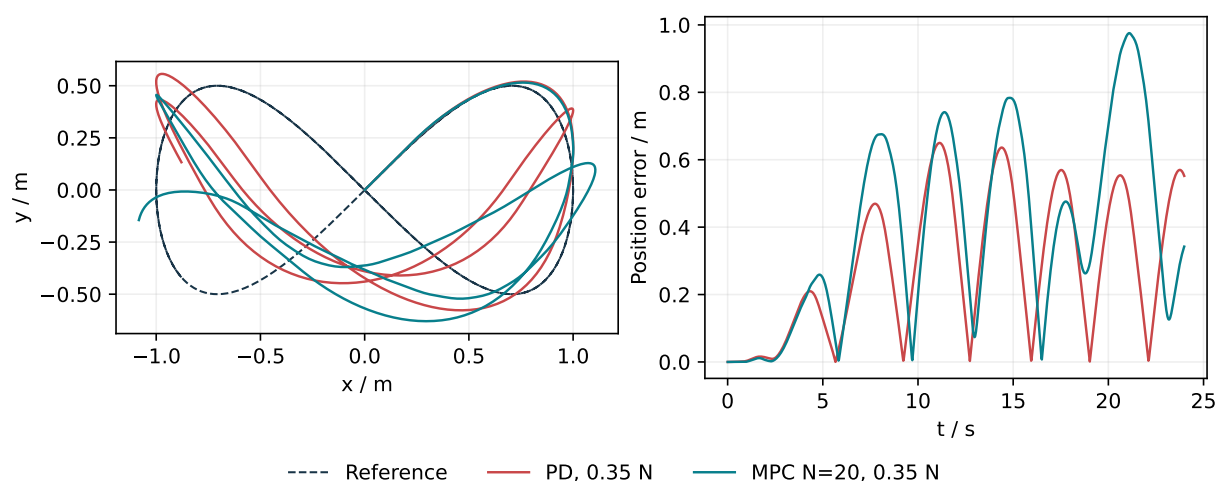


图 18.1: 将力限收紧到每轴 0.35 N。两种方法都无法紧贴原参考；MPC 还施加每轴 0.5 m/s 和 2 N/s 约束。它的失败会切换到制动，因此不能把整条实际输入曲线都称为满足 QP 约束。

限力 PD 的 RMS 为 0.3533 m，MPC 为 0.4771 m；MPC 并非所有参数下都更好。PD 最大速度分量 0.5992 m/s；MPC 约 0.500001 m/s，后者的微小超量在数值容差量级。MPC 有 21 次 `max_iterations`，不能据此断言不可行；制动切换导致实际最大力变化率达 35 N/s。其最小净空仍有 2.6933 m，但空房间的这次结果不证明任意障碍场景安全。完整 `summary.csv` 与 `failures.csv` 分开记录各次误差、耗时、状态及新的制动力。

## 18.7 在 ROS 中观察每次预测与失败

```
1 ros2 launch motion2d_bringup ch18.launch.py rviz:=false
2 # 另一个终端；默认暂停，检查脚本会推进 12 s
3 /usr/bin/python3 scripts/check_ch18.py
4 # 相同配置切回第 17 章控制器：重新启动时加 controller:=pd
```

`tracker_node` 的 `control.controller` 选择 `pd/mpc`；两者复用相同的观测时间、坐标变换、参考系数和物理限幅接口。MPC 预测步长取 `control.rate_hz` 的倒数；默认 50 Hz、20 步。这些是启动参数，改 YAML 后重启；不要把参数服务接受了新值误解为已重建控制器。正常链路假设最近发布的限幅力已被模拟器接收，并将它用于下一次增量约束。

| 接口                                | 用法                                                                                          |
|-----------------------------------|---------------------------------------------------------------------------------------------|
| <code>/control/mpc_status</code>  | <code>MpcStatus</code> , <code>header</code> 是观测时刻, <code>status/iterations</code> 为求解状态和次数 |
| <code>compute_seconds</code>      | 组装 QP、求解和热启动更新的墙钟耗时, 单位 s                                                                   |
| <code>primal/dual_residual</code> | 终止条件所用残差; 未迭代时为正无穷                                                                          |
| <code>max_violation</code>        | 仅 <code>solved</code> 有效; 失败为 NaN, 不假报零违反量                                                  |
| <code>/control/prediction</code>  | <code>odom</code> 系预测位置, 逐点时间是未来执行采样时刻                                                      |

预测路径供 RViz 观察, 实际控制仍使用力。失败发布空预测, 清除热启动, 输出新的限幅阻尼力; `yaw` 同时切到阻尼力矩。观测中断、过旧、暂停去重和时钟回退沿用第 17 章策略。默认求解墙钟预算 0.01 s; 这是迭代间检查的预算, 不保证整条 ROS 链路或单次分解在预算内完成。

**主机实测** 12 s、600 个对齐观测时刻, 600 次全部 `solved`; 位置 RMS 为  $1.148 \times 10^{-5}$  m、最大误差  $3.519 \times 10^{-5}$  m, `yaw` RMS 为  $6.52 \times 10^{-5}$  rad; 最小净空 2.8000 m, 组装与求解耗时 P95 为 0.234 ms。这是空房间、真值定位、精确模型的受力跟踪实验, 不是带 SLAM 误差的导航成绩。检查脚本同时验证预测点数、未来时间戳、残差与暂停/reset, GUI 本轮未验收。

**故意制造超时, 检查真正发出的消息** 在独立域使用下列启动参数:

```
1 use_sim_time=true
2 control.controller=mpc
3 mpc.max_wall_seconds=1.0e-12
```

执行脚本 `check_mpc_failure.py`, 由它提供唯一 `/clock` 和合成里程计。机体系前向速度 0.2 m/s、`yaw`= $\pi/2$  时, 世界系制动力应为 (0, -0.4) N。验收 `time_limit`、零迭代、空预测、NaN 违反量与这个新制动力, 不只检查日志里有没有“超时”。

**练习** 记录 50 Hz、N=10/20/40 的 `MpcStatus` 耗时分位数, 与核心实验区分; 改变预算触发失败, 并检查制动力时间戳和方向。再暂停仿真, 确认墙钟等待不会产生新的预测点。

## 第十九章 接通在线规划与完整导航

**目标** 接续运动中的参考，再将地图、SLAM、规划和控制串成可测量的闭环。

**先修** 第 08-18 章；先单独验证交接，随后接入观测地图。

### 19.1 重规划从未来交接时刻开始

```
1 ros2 run motion2d reference_handover_demo tmp/ch19_handover
2 /usr/bin/python3 scripts/plot_ch19_handover.py
```

假设当前仿真时间为  $t_0$ ，预留一段计算和传输时间后，在  $t_s > t_0$  切换到新曲线。新曲线不能从当前测量位置、零速度重新起步；应固定它的首边界为旧参考在  $t_s$  的值：

$$p_{\text{new}}(0) = p_{\text{old}}(t_s), \quad v_{\text{new}}(0) = v_{\text{old}}(t_s), \quad a_{\text{new}}(0) = a_{\text{old}}(t_s). \quad (19.1)$$

这样交接前仍执行旧前缀，交接后才沿新路线走。实际机器人可能偏离参考，偏差由反馈处理；若偏离已超出可接受范围，应制动和重新建立参考，而不是伪称仍有无缝接续。

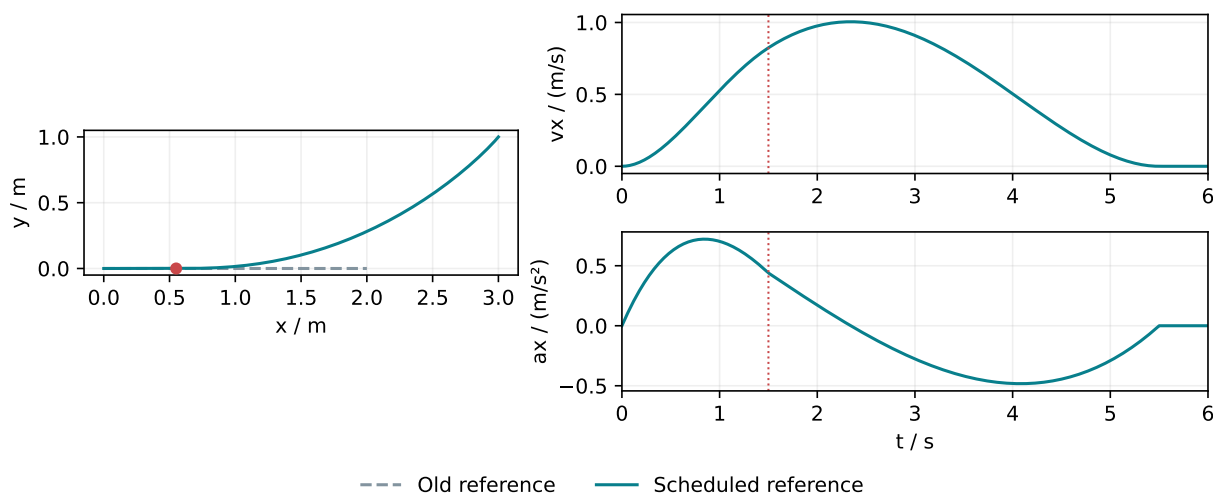


图 19.1: 在 1.5 s 交接。旧曲线原本沿  $x$  轴到达 (2,0)，新曲线转向 (3,1)；红点和竖虚线标记交接时刻。速度与加速度保持连续，jerk 不要求连续。

此时  $p=(0.550415,0)$  m、 $v=(0.823975,0)$  m/s、 $a=(0.439453,0)$  m/s<sup>2</sup>。将新首速度简单置零，会立即产生与旧参考不匹配的速度跳变。ReferenceSchedule 保存一条活动参考和至多一条未来候选；末端必须静止，才能连续接到终点保持。这是参考调度实验，尚不以曲线平滑来声称已验证障碍或物理输入约束。

## 19.2 预测未来，不等于提前切换

MPC 会采样未来多个时刻。如果 `sample` 一查到未来就移动“当前曲线”指针，下一次采样当前时刻时可能已经取错参考。因此 `sample` 为只读函数，按查询时刻选择活动曲线或未来候选；`advance` 只由真正推进的观测时刻调用。接口不保留过去所有曲线的历史，供当前与未来查询使用。

```

45  const auto old=sample(next.motion.start_ns);
46  const auto first=next.motion.curve.sample(0);
47  const auto last=next.motion.curve.sample(next.motion.curve.duration());
48  if((first.position-old.pose.position).norm()>1e-7 ||
49     (first.velocity-old.velocity).norm()>1e-7 ||
50     (first.acceleration-old.acceleration).norm()>1e-7 ||
51     std::abs(wrapAngle(next.motion.yaw-old.pose.yaw))>1e-7)
52     return {false,"discontinuous_handover"};
53  if(last.velocity.norm()>1e-7 || last.acceleration.norm()>1e-7)
54     return {false,"nonstationary_end"};
55  pending_=std::move(next);
56  return {true,"accepted"};
57  //

```

过去的起点、尚未交接时积压的第二个候选，以及不连续边界都会被拒绝。拒绝不能覆盖已有效的参考；`reset` 则显式清空活动与候选并设置新的静止锚点。紧急重置会中断参考导数连续性，不能将它包装为普通  $C^2$  重规划。

## 19.3 地图校正与执行参考分开

SLAM 回环可能改变 `map` 到 `odom` 的对齐关系。规划时冻结同一个变换，令  $p_m = R p_o + t$ ；把地图中的曲线交给控制器前，使用

$$p_o = R^T(p_m - t), \quad v_o = R^T v_m, \quad a_o = R^T a_m. \quad (19.2)$$

对五次系数而言，每列都旋转，只有常数项平移；区域的所有顶点也使用同一变换，再由顶点构建半空间。`yaw` 参考随坐标旋转一次，不从路径切向推断。已发出的 `odom` 系数随后保持不变；新的地图校正影响下一次规划，而不直接跳动当前控制参考。

**验证与练习** 测试包含交接前后 1 ns 的 `p/v/a`、提前预读未来、待交接状态、非法候选保留旧曲线、终点保持与显式 `reset`。另将曲线和区域经平移 + 旋转送到 `map` 再变回 `odom`，验证所有导数和区域一致。

1. **ch19-1**: 补齐 `p/v/a` 边界比较；同位置、不同速度或加速度必须拒绝。
2. 把新参考错误地连接到  $t_0$  的状态，观察为何到  $t_s$  时不能再匹配。
3. 分别说明定位误差、规划不满足约束、执行跟踪误差；平滑接续只解决其中的参考衔接。

## 19.4 只在观测自由空间里选择局部目标

```
1 ros2 run motion2d replanner_demo tmp/ch19_replanner
2 /usr/bin/python3 scripts/plot_ch19_replanner.py
```

规划输入是一份不可变快照：原始占据栅格、由它膨胀的配置空间栅格、同源未膨胀 ESDF。目标在地图之外或明确占据时先报告错误；未知目标并不允许搜索穿越未知区域。从未来交接点所在自由格做八邻接可达搜索，斜向仍禁止切角。若全局目标已可达，就用 A\* 找到它；否则，在可达自由格中选择靠近未知区、且更接近目标的一个格。

这些候选与原始未知格之间的搜索窗口由膨胀半径和分辨率确定。它们是**局部探索候选**，不是承诺一定能发现通路的完整探索器。失败状态分别说明当前局部搜索的结果：

- `no_progress_frontier`：候选的目标距离改善不足 0.1 m。
- `no_reachable_frontier`：没有可达的未知边缘候选。

两者都不代表“整个未知世界一定无路”。选好可达目标后，再沿 A\* 路线弧长截取一段，默认最多 2 m；截断点仍位于已验证自由线段上。

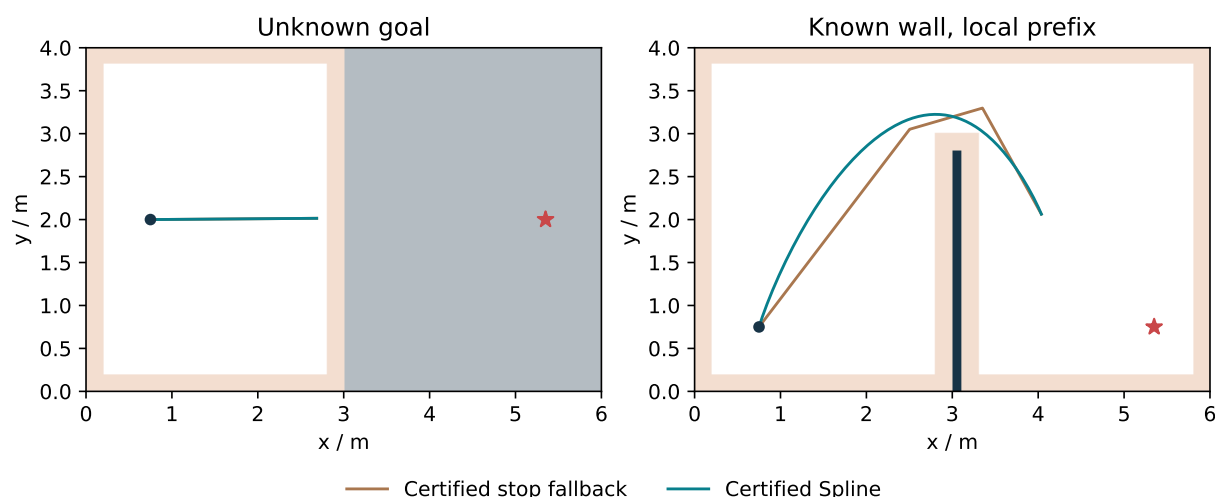


图 19.2：两个手工占据网格测试夹具，不是在线 SLAM 成绩。灰色未知、深蓝占据、浅橙为配置空间阻塞；星号是全局目标，曲线只到达当前局部终点。左图不穿越未知区，右图采用最多 5 m 的路线前缀绕墙。

例子使用  $60 \times 40$  格、0.1 m 分辨率、半径 0.1 m 与 0.05 m 余量。左图只有  $x < 3$  m 区域已观测，仍能向未知边缘逐段接近；右图全图已知，但本次只执行一段路线前缀。若新扫描揭示更多自由空间，下次快照会产生新的候选；若停在看不到更多区域的位置，必须明确报告停滞。

## 19.5 优化与备用方案都要通过认证

对同一路线构建凸走廊，从第 19.1 节的未来  $p/v/a$  起步，终点设为静止。Spline 优化同时使用 ESDF、走廊和速度/加速度软项；只有求解收敛且独立 Bézier 证书通过才接受。本章核心默认开启控制点惩罚，走廊权重 50000、额外内缩 0.04 m；软速度/加速度目标 0.45/0.5，证书上限 0.7 m/s 与  $0.8 \text{ m/s}^2$ 。

若优化失败，不复用一个不合格的低代价值结果，而是尝试明确的备用方案：各路点停下的五次段，首段仍保留原来的  $p/v/a$ 。尝试基准时长与 1.5/2/3 倍时长，每次都重新检查同一组连续证书。增加时长不一定能修复问题：固定非零首速度时，较长时间也可能把曲线推到走廊外。

```

103 for(double scale:{1.,1.5,2.,3.}) {
104     std::vector<QuinticPiece> pieces; TranslationState left=start;
105     for(std::size_t i=0;i<times.size();++i) {
106         TranslationState right; right.position=corridor.waypoints[i+1];
107         pieces.push_back(interpolateQuintic(left,right,times[i]*scale)); left=right;
108     }
109     PolynomialTrajectory candidate(std::move(pieces));
110     if(certifyBezier(candidate,corridor.regions,optimization.limits).certified) {
111         result.curve=std::move(candidate); result.success=true;
112         result.status="fallback_stop_segments:"+route.status; return finish();
113     }
114 }
115 //

```

| 夹具          | 备用段总时长/s | Spline 总时长/s |
|-------------|----------|--------------|
| 未知目标，局部前缀   | 7.258    | 11.424       |
| 绕墙，5 m 路线前缀 | 19.466   | 12.994       |

四组均通过各自连续证书；示例为便于复现关闭优化墙钟预算，实际节点应限制计算时间。Spline 优化的是带能量与软项的总代价，不保证每次都比备用方案更短；单段控制点惩罚也可能偏保守。返回结果单列 `optimized` 或 `fallback_stop_segments`，并保留优化器原始状态。所有方案都不合格时返回 `no_certified_local_trajectory`，上层进入明确的制动处理。

**验证与练习** 单元检查未知目标、已知断开的地图、占据/越界目标、路线长度、配置区域整体验证和非零首速度。另把首速度设为 2 m/s，超过证书上限，必须拒绝全部备用方案。比较有无控制点惩罚时的时长与认证结果；画出原始占据和配置栅格，解释半径为何只能处理一次。

## 19.6 把轨迹与走廊作为一个事务接收

```

1 # Terminal 1: standalone receiver, no simulator
2 ros2 run motion2d tracker_node --ros-args \
3   -p use_sim_time:=true -p reference.source:=navigation \
4   -p control.odometry_timeout:=10.0
5 # Terminal 2: synthetic /clock and observations
6 python3 scripts/check_navigation_boundary.py

```

`NavigationReference` 把带绝对起点的五次系数、逐段凸区域、地图快照时刻放进同一条消息。否则分别到达的“新曲线 + 旧走廊”可能组成一个从未被规划器验证过的组合。区域顶点用 `float64`，避免 `float32` 舍入使贴边曲线的接收检查发生变化。接收器检查消息坐标、区域数量、有限值及曲线在区域中的连续证书；区域是否属于观测自由空间仍由规划器负责，控制器并不读取地图或世界。

| 话题                                           | 语义                             |
|----------------------------------------------|--------------------------------|
| <code>/navigation/reference</code>           | 原子候选，坐标 <code>odom</code>      |
| <code>/control/accepted_reference</code>     | <code>Int64</code> ，已接收候选的起点纳秒 |
| <code>/navigation/stop</code>                | <code>Empty</code> ，显式清空参考并制动  |
| <code>/control/reference</code>              | 当前观测时刻的参考位姿与机体速度               |
| <code>/control/reference_acceleration</code> | 同时刻的 <code>odom</code> 加速度     |

发布成功不等于控制器已经接受。规划端应等到对应起点的确认，再更新自己的参考调度；每个待确认请求使用不同的未来起点，不能同时积压多个候选。运动中的新曲线严格匹配旧参考的 `p/v/a/yaw`，拒绝候选时保留原曲线。MPC 向前查询时，同时选择对应时刻的区域；第一条曲线尚未开始时，静止保持使用它的第一个区域。

**启动与停车** 第一次启动，或显式制动后的重新启动，要求观测速度与角速度不超过 `0.02`，首点在当前位置 `0.03 m` 内，`yaw` 差不超过 `0.02 rad`，并且首 `v/a` 为零。这个小容差用于静止状态的测量波动，不能用于掩盖运动中的不连续交接。制动会清空未来曲线、预测和显示路径，使用最新状态计算限幅阻尼力；它不承诺制动过程仍满足旧曲线的连续证书或力变化率限制。

理想模式另设 `control.controller=ideal`，直接发送下一控制周期的参考位置，仿真必须配套 `ideal` 模型并关闭 IMU；带质量模式仍发送力/力矩。两者的物理含义不同，完整导航实验会分开报告。

**验收与练习** 合成时钟测试先从静止沿 `x` 轴运动，再于 `0.7 s` 接续转向曲线。检查确认只出现两次、交接前仍是旧曲线、交接处 `p/v/a` 相等、错误替换不覆盖有效参考，以及显式停车发布新的阻尼力、移动中拒绝重新锚定。练习：故意去掉第二条消息的一块区域，再重复发送；说明为什么既不能确认，也不能丢掉原参考。

## 19.7 观测驱动的导航节点

```

1  ros2 launch motion2d_bringup ch19.launch.py route:=B
2  ros2 service call /sim/pause std_srvs/srv/SetBool "{data: false}"
3  # RViz: use the 2D Goal Pose tool, or publish a map goal
4  ros2 topic pub --once /goal_pose geometry_msgs/msg/PoseStamped \
5  "{header: {frame_id: map}, pose: {position: {x: -4.0, y: -4.0}, orientation: {w: 1.0}}}"

```

NavigationNode 将几个小模块串起来：地图回调生成同源占据/膨胀/ESDF 快照，选定里程计推进调度，激光时间戳检测传感器是否还在更新。规划默认 5 Hz，MPC 控制 50 Hz；理想路线直接以 200 Hz 发送下一仿真步的位置。暂停时仿真时间不变，墙钟计时器醒来也不重复规划；时间回退清空目标和旧快照，需要重新给目标。

每次预留 0.15 s 交接时间，优化墙钟预算为 0.04 s。A\*、走廊构建和传输也需要时间，因而还必须检查接收确认是否赶在起点之前。节点保存至多一个待确认候选，控制器确认之前不把它当作已经执行的参考。下面的核心路径把同一个对齐变换用于首边界、整条曲线和所有区域：

```

143  const auto old=schedule_.sample(start_ns);const auto R=rotation(map_from_odom.yaw);
144  TranslationState boundary{map_from_odom.position+R*old.pose.position,R*old.velocity,R*old.acceleration};
145  const auto result=replanObserved(snapshot_>grid,snapshot_>raw,snapshot_>field,boundary,*goal_.config_);
146  planner_status_=result.status;optimization_status_=result.optimization_status;planning_seconds_=result.
    seconds;
147  if(!result.success) {stop("planning_failed_braking");return;}
148  NavigationReference mapped{*result.curve,start_ns,wrapAngle(old.pose.yaw+map_from_odom.yaw)},result.regions,
    snapshot_>stamp};
149  awaiting_=transformReference(mapped,inverse(map_from_odom));
150  reference_pub_>publish(toNavigationMessage(*awaiting_,header().stamp));
151  //

```

TF 使用最近可用的 map 到 odom 对齐；高频 IMU 时刻通常比 SLAM 最近一次 TF 更晚，不对未来外推。该对齐在本次求解期间保持不变。已有 odom 参考不随回环修改；新快照会把剩余区域变换回 map，检查整个区域仍属于自由配置空间。失效则制动，重新静止起步。区域检查偏保守，但不会把旧地图证书冒充为永久有效。

**明确的失败处理** 扫描超过 0.35 s、里程计超过 0.12 s 或地图超过 3 s 未更新，都会清空参考并制动。规划无认证结果、确认超时、MPC 求解失败也采用相同的显式停车路径。等待静止后可重试仍保留的目标；这不是完备探索，也不保证总能恢复或到达。用结构化状态区分规划失败、等待静止、等待确认、执行和到达，另记录优化器状态、耗时、停车次数。

**练习** 补完 ch19-2 的仿真时间触发条件，验证首次、暂停、恰好到周期、候选待交接和时间回退。运行 check\_navigation\_failure.py 的独立自由网格夹具，观察里程计仍更新而激光中断时的制动，再观察缺少确认时的制动。夹具地图只用于接口测试，不计入未知环境导航成绩。

## 19.8 同场景比较三条路线

```
1 # Starts/stops its own isolated launches; source workspace first.
2 python3 scripts/run_ch19.py --seeds 42 43 44
3 python3 scripts/plot_ch19_navigation.py
```

A 使用真值适配器、观测建图和理想点控制；B 将执行改为带质量模型与 MPC；C 将定位改为真实模拟激光/IMU 前端、EKF 和关键帧 SLAM。C 不启动真值适配器或真值投影建图节点，唯一 map 到 odom 的变换由 SLAM 发布。三组都只有观测地图；真值与完整障碍物只用于最后的评价。

世界为  $20 \times 20$  m、8 个圆与 8 个凸多边形，圆盘半径 0.2 m，相同种子对应相同几何。传感器为 720 束、10 Hz、6 m 量程、每束标准差 0.01 m；B/C 使用 200 Hz IMU，噪声沿用第 10 章，动力学阻尼与第 18 章相同。地图分辨率 0.1 m，规划额外跟踪余量 0.15 m。节点采用采样软惩罚以减少单段控制点代价过保守的时长，但仍由独立连续证书决定是否执行。

物理起点为  $(-8, -8)$ 、目标为  $(-4, -4)$ ，初始 yaw=0。A/B 中 map 与 world 相同；C 的初始 map 原点在第一帧，故同一目标约为  $(4, 4)$ 。当前 C 配置要求初始 yaw=0，使估计 odom 轴与物理力输入轴一致；扩展初始朝向时须同时适配力向量坐标。到达条件为选定定位距目标小于 0.12 m 且速度小于 0.08 m/s；评价还要求真实目标误差小于 0.25 m。

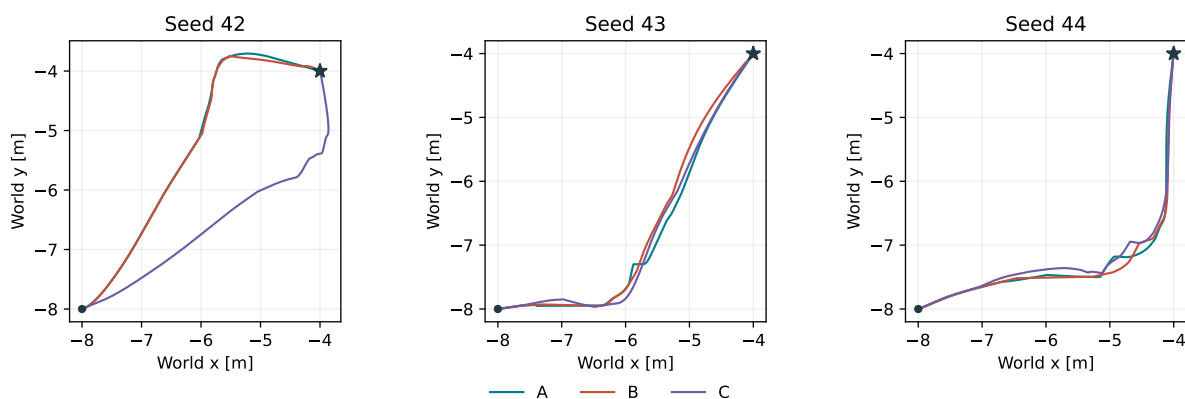


图 19.3: 主机 ROS 实测的圆心轨迹，统一画在 world 坐标中，圆点为起点、星号为目标。此图只比较实际路径，不是地图或 RViz 截图；障碍净空由独立几何评价器计算。

**评价约定** ATE 只在第一帧做一次 SE(2) 对齐，之后比较 SLAM map 位姿与真实位姿；不对整条路径重新拟合来减小漂移。RPE 使用相隔 1 s 的相对位姿平移误差，跟踪误差则比较同一时间戳的选定里程计与 odom 参考。净空是完整世界几何到真实圆心的距离减去物理半径，不能用有噪声的估计 ESDF 代替。

## 19.9 读懂成绩，也保留边界

| 路线/种子 | 到达/s  | ATE/mm | 跟踪/mm | 净空/m  | 停车 |
|-------|-------|--------|-------|-------|----|
| A/42  | 20.80 | 0.00   | 0.00  | 0.444 | 10 |
| B/42  | 24.05 | 0.00   | 0.41  | 0.406 | 6  |
| C/42  | 26.44 | 5.40   | 2.02  | 0.370 | 6  |
| A/43  | 26.18 | 0.00   | 0.00  | 0.481 | 13 |
| B/43  | 24.02 | 0.00   | 0.46  | 0.474 | 6  |
| C/43  | 24.91 | 4.95   | 2.04  | 0.417 | 6  |
| A/44  | 24.52 | 0.00   | 0.00  | 0.530 | 7  |
| B/44  | 27.61 | 0.00   | 0.38  | 0.574 | 7  |
| C/44  | 28.23 | 5.76   | 1.76  | 0.435 | 7  |

表中 ATE 和跟踪均为 RMS，净空为全程最小真实圆盘净空。这九次均到达，碰撞尝试为 0；A 的直接设点跟踪误差为 0，A/B 的定位误差为 0 是真值模式的定义，不能作为估计算法的成绩。B/C 均未出现 MPC 未收敛；停车主要来自观测变化使旧区域失效，以及重规划没有认证结果。停止后重新建立静止参考是显式中断，不能把这些位置的加速度称为连续。

C 的三次 1 s 平移 RPE 分别为 2.57、2.21、2.55 mm。全部试验的规划 P95 为 6.96–10.13 ms，B/C 的 MPC P95 为 0.38–0.46 ms；规划时长包含局部路线、走廊和轨迹求解，不含地图回调的膨胀/ESDF 构建、DDS 传输和显示。这些是顺序运行、未锁频的主机测量，不是实时上界。

初始未知格约占 88.9%–92.9%，到达时仍有约 74.7%–76.4% 未知；导航只探索本次任务需要的区域，并没有读入完整世界地图。详细数值保存于 `textbook/data/ch19_navigation.csv`，脚本另输出逐时刻定位、跟踪、状态和 QP 数据。异步回调到达次序与有限墙钟预算会影响具体重规划时刻；相同种子保证几何和噪声流，不保证操作系统调度或最终路径逐位相同。

**怎样解释成功率** 9/9 是三个固定种子、同一起终点、短距离静态场景的本次结果，样本太少，不能推断任意未知环境都有这样的成功率。长窄通道、不可见凹陷、低激光重叠和动态障碍可能导致停滞；当前前沿选择只追求更靠近目标，不具备完整的全局探索策略。这些单程任务也没有主动制造回环；实际回环实验见第 10 章，本章另用曲线/区域坐标往返和新旧参考测试验证地图校正不会直接重写已发出的 odom 曲线。

**验收与继续练习** 检查节点订阅表：导航、控制、激光里程计、EKF 和 SLAM 都不能订阅 `ground_truth` 或 `sim` 位姿。分别增大噪声、减小激光量程、降低重规划频率，记录**失败状态**和**停车次数**，不要只保留成功曲线。当前主机完成无界面 ROS 实验，RViz 的预期内容已配置；桌面图形问题尚未解除，未声称截图验收通过。

## 19.10 从一份配置选择微分平坦模型

本节将第 04、15-17 章的新增模块接到观测地图。它采用停止到停止的小型闭环：先看清模型与约束如何影响真实输入，再学习更复杂的运动中接续。

```
# Repository root; source ROS and ros2_ws/install first.
mkdir -p tmp
cp ros2_ws/src/motion2d_bringup/config/flat_vehicle.yaml \
  tmp/vehicle.yaml
ros2 launch motion2d_bringup flat_vehicle.launch.py \
  rviz:=false config:=/absolute/path/to/tmp/vehicle.yaml
```

先将最后一行替换为自己的绝对路径。物理参数和模型选择见第 04 章新增配置节。

定位选项在 **vehicle** 段中，字段名为 **localization**，可取 **truth** 或 **slam**。所有物理量同时传给仿真和规划控制，修改配置后重启，不运行中切换动力学。

| 选择    | 全向 inertial        | 阿克曼 ackermann       |
|-------|--------------------|---------------------|
| 平坦输出  | $(p_x, p_y, \psi)$ | $(p_x, p_y)$ , 前进支路 |
| 规划变量  | 路点、分段时长            | 正的切向长度、分段时长         |
| 物理软约束 | 力、力变化率             | 力、转角、转向速率、侧向加速度     |
| 控制输出  | $F_x, F_y, \tau$   | $F, r_\delta$       |
| 朝向    | 保持起步 yaw           | 采用目标 yaw            |

全向采用 MINCO/Spline 与前馈 PD；阿克曼采用正则几何五次段、五次停车进度和局部反馈。**planning.backend** 仅影响全向选择，不能把全向五次时间曲线当作阿克曼几何系数。原第 18/19 章线性 MPC 和未来运动接续仍使用原入口，本节没有实现阿克曼 NMPC。

**参考限制和执行器限制分开。**默认物理力上限 2 N，规划参考上限 1 N，物理转角/转向速率 0.6 rad/1 rad/s，规划为 0.5 rad/0.7 rad/s。全向另限制力变化率 2 N/s；阿克曼另限制侧向加速度 0.8 m/s<sup>2</sup>，两者速度上限均为 0.7 m/s。优化软目标取规划上限的 85%，执行前连续界按完整规划上限验收。反馈保留执行器余量，最终指令受物理限幅；参考证书不等于实际跟踪误差或制动过程的保证。

**数据来源。**truth 也只凭扫描建图；slam 使用激光里程计、IMU 融合和关键帧地图，不启动真值适配器。阿克曼控制额外读取显式理想转角编码器；SLAM 仍只使用扫描和 IMU。默认半径 0.4 m，配置栅格额外留 0.15 m 跟踪余量，未知格阻塞。

## 19.11 从观测就绪到未来起步

启动时暂停；在另一个已加载环境的终端恢复，然后给出当前可达的近目标：

```
ros2 service call /sim/pause std_srvs/srv/SetBool '{data: false}'
# truth mode: start (-8,-8), goal (-6,-7.7), yaw=0
ros2 topic pub --once /goal_pose geometry_msgs/msg/PoseStamped \
  '{header: {frame_id: map}, pose: {
    position: {x: -6.0, y: -7.7}, orientation: {w: 1.0}}}'
ros2 topic echo /flat/status
```

SLAM 第一帧作为初始 **map** 原点，同一相对目标改为 (2,0.3)。当前配置要求初始 **yaw=0**，保持估计 **odom** 和全向施力轴一致。目标必须携带有效平面四元数；阿克曼采用其中的终点朝向，全向仍保持起步朝向。

SLAM 的第一帧自由概率 0.4 尚未低于规划的 0.35 阈值，**waiting\_observed\_free\_start** 会等下一份观测地图。后续关键帧增加证据才继续，不为尽快开车降低自由判据。A\* 和凸走廊都来自同一份膨胀地图；固定路径曲率不适合小车时，专用优化器可明确失败。

**求解完成不等于现在开始**。仅在观测速度、角速度以及阿克曼转角接近零时求解，预算默认 1 s。成功后还要等求解结束之后的一条新里程计，以其 **stamp** 加 0.3 s 设执行起点；避免使用计算期间排队的旧观测。路径和区域由同一 **map** 到 **odom** 变换冻结，后续定位校正不重写已发出的系数。新地图若使区域失效，则清空参考并制动。

```
ros2 topic pub --once /flat/stop std_msgs/msg/Empty '{}'
```

```
ros2 service call /sim/reset std_srvs/srv/Trigger '{}'
```

暂停保持同一个模拟时间，不重复推进控制；**reset**、非法目标和显式停车清空旧参考。有新鲜里程计但激光/地图/转角过期时，以新状态给限幅阻尼输入；里程计也超过 120 ms 时，改为零输入，避免反复使用旧速度制动而使车辆反向加速。这是明确的故障处理，不是原轨迹连续证书的一部分。

**接口与显示**。阿克曼曲线通过 **/flat/ackermann\_trajectory** 输出，几何参数  $u$  无量纲，每段另存物理秒和绝对起点；空曲线表示撤销。**/flat/reference** 是当前观测时刻的 **Odometry** 参考。RViz 配置显示观测地图、注册点云、选定里程计、**/plan/path** 和凸区域；完整世界与仿真圆盘属于可选真值对照。此主机完成无界面验证，未声称新增 GUI 画面已验收。

## 19.12 选模实验、失败案例与练习

启动匹配的配置后，从仓库根目录运行；脚本会主动 `reset`、推进和暂停：

```
/usr/bin/python3 scripts/check_flat_vehicle.py \
--model ackermann --localization truth --output tmp/flat_check
```

分别修改配置和脚本参数，完成两种模型与两种定位的四组试验。默认 `seed42`，世界  $20 \times 20$  m、8 圆/8 多边形；质量 1kg、半径 0.4m，720 束/10Hz 激光、每束标准差 0.01m，200Hz IMU 沿用第 06 章白噪声。相对目标为 (2, 0.3) m，初终朝向均为零。

| 模型/定位     | 规划后端   | 参考时长/s | 峰值跟踪/mm |
|-----------|--------|--------|---------|
| 阿克曼/truth | 专用     | 7.327  | 0.680   |
| 全向/truth  | Spline | 11.118 | 0.145   |
| 阿克曼/SLAM  | 专用     | 7.325  | 4.046   |
| 全向/SLAM   | MINCO  | 11.115 | 4.931   |

跟踪误差比较相同 `stamp` 的选定里程计与参考，不是 **SLAM 的 ATE**。四组均完成，暂停保持时刻/状态，非法目标和 `reset` 后无旧轨迹重放；真实节点订阅图确认规划、估计节点不读真值，只有一个时钟发布者。数据保存在 `textbook/data/flat_vehicle_validation.csv`。样本是固定种子的短距离静态场景，不能作为任意环境成功率或两类优化器速度优劣的结论。

**保留失败。**将规划转角上限改成 0.005 rad，再给同一换道目标，验收脚本加 `--expect-failure`：主机报告规划失败，未发布非空执行曲线。因为  $\delta = \arctan(L\kappa)$  只取决于几何，单纯延长时长不能减小转角。拒绝的是这个固定几何路点/朝向问题，不是证明整个未知世界没有其他可行路线。

**ch19-3 配置练习** 先逐项改变质量、规划力上限、转角上限，每次只改变一个量，记录状态、认证界、时长与跟踪误差。解释物理上限、参考软目标和后验认证之间的关系，失败样本也写入报告。再试验运动中显式停车，说明为什么不能声称制动仍满足原曲线证书。代码练习 `ch19-1/2` 仍用于原运动接续入口；新增平坦梯度练习在 `ch04-5/6`、`ch15-3`、`ch16-3`。

**扩展边界。**此小车规划前进、逐段停车，固定几何路点和朝向；没有倒车、Hybrid A\*、前沿探索或阿克曼 NMPC。窄通道、急弯、远处未知目标可以失败。下一步扩展时先加入相应几何搜索和控制模型，再沿用这里的单位、时间、解析反向和独立验收约定。

## 第二十章 选学：CUDA 激光雷达加速

**目标** 保留 CPU 基线，逐束验证 GPU 结果，并分清计算、传输和完整消息成本。

**先修** 第 05 章、C++ 数组；不要求为前面章节安装 CUDA。

### 20.1 先建立可复现的工具链

默认 CPU 构建不查找 CUDA。本机使用 RTX 5060、驱动 595.91.07、GCC 15.2 和 CUDA 13.2.1。主机原先缺少 `nvcc`；脚本将官方组件按锁定 SHA-256 下载到 `tmp`，保留许可证，不修改驱动。13.0.2 与本机 `glibc` 2.43 的头文件冲突，故选用已实测的 `nvcc` 13.2.78。

```
1 # Repository root; optional Linux x86_64 toolchain
2 python3 scripts/setup_cuda_local.py
3 source /opt/ros/lyrical/setup.bash
4 cd ros2_ws
5 colcon build --symlink-install --cmake-args \
6   -DCMAKE_BUILD_TYPE=RelWithDebInfo \
7   -DPython3_EXECUTABLE=/usr/bin/python3
8 colcon build --packages-select motion2d --symlink-install \
9   --cmake-args -DMOTION2D_ENABLE_CUDA=ON \
10  -DCMAKE_CUDA_COMPILER="$PWD/../tmp/cuda-13.2.1/toolkit/bin/nvcc" \
11  -DCMAKE_CUDA_HOST_COMPILER=/usr/bin/g++ \
12  -DCMAKE_CUDA_ARCHITECTURES=120
13 source install/setup.bash
14 ros2 run motion2d lidar_cuda_benchmark > ../tmp/ch20_cuda.csv
```

120 是本机 GPU 的计算能力，不适用于所有显卡。现代 CMake 可使用 `native` 探测本机；跨机器运行时需按目标选择架构。禁用时将上述 CUDA 开关显式设为 `OFF`；CMake 会记住旧设置，只删掉命令中的 `ON` 不能将它关闭。

### 20.2 同一接口，不改变测量含义

`CudaLidar` 构造时接收静态世界及最大束数，`scan` 接收位姿和相同 `LidarConfig`，返回相同 `float` 数组。扫描仍是圆心处的瞬时均匀扫描；完整一周用  $2\pi/N$ ，部分视场包含两端。近于最小量程为 `NaN`，超出最大量程为  $+\infty$ ，不能把二者都解释为自由空间。

设备几何使用 `double`，输出才转为 `float`；本实验关闭 FMA 融合，没有 `fast-math`。这些选择便于与 CPU 对照，但不是最高性能方案。相同误差  $e_i$  在两个无噪声输出上分别调用 `perturbRange`；不能仅凭“两个独立生成器种子相同”就认定样本相同，因为不同后端的生成顺序或特殊值跳过方式可能不同。

## 20.3 一束一个线程

第  $i$  束仍计算

$$\theta_i = \psi + \theta_{\min} + i\Delta\theta, \quad d_i = \min\{d_{i,\text{circles}}, d_{i,\text{edges}}, d_{i,\text{walls}}\}. \quad (20.1)$$

一束对应一个线程；线程内部遍历所有圆和边，顺序求最小命中。本章没有再做障碍物并行归约，也没有 BVH；先保持与 CPU 公式逐行可比。B 束、C 个圆、E 条边的总工作量仍为  $O(B(C + E))$ ，并行只改变执行安排。

```

35 __global__ void scanKernel(const double * geometry,int circles,int edges,
36   double ox,double oy,double yaw,double angle_min,double increment,int beams,
37   double range_min,double range_max,float * ranges) {
38   const int i=blockIdx.x*blockDim.x+threadIdx.x;
39   if(i>=beams) return;
40   const double angle=yaw+angle_min+i*increment,dx=cos(angle),dy=sin(angle);
41   const double * cx=geometry,*cy=cx+circles,*r=cy+circles;
42   const double * ax=r+circles,*ay=ax+edges,*bx=ay+edges,*by=bx+edges;
43   double nearest=CUDART_INF;
44   for(int j=0;j<circles;++j) nearest=fmin(nearest,circleHit(ox,oy,dx,dy,cx[j],cy[j],r[j]));
45   for(int j=0;j<edges;++j) nearest=fmin(nearest,segmentHit(ox,oy,dx,dy,ax[j],ay[j],bx[j],by[j]));
46   ranges[i]=nearest<range_min ? CUDART_NAN_F : nearest>range_max ? CUDART_INF_F : float(nearest);
47 }
48 //

```

每块 128 个线程，总块数为  $\lceil B/128 \rceil$ ；末块多出的线程通过边界判断返回。几何打包成一个连续的结构分量数组：圆的  $cx$ 、 $cy$ 、 $r$ ，然后是边的  $ax$ 、 $ay$ 、 $bx$ 、 $by$ 。外框也作为四条边加入；所有线程复用这些静态数据。相邻线程读取相同障碍物参数，有利于缓存复用，但不能仅凭采用 SoA 就声称已经更快。

**内存与生命周期** 构造时分配设备几何、输出数组和计时事件，上传一次；后续扫描重复使用。位姿和角度等少量标量作为内核参数传入，输出仍须回传 CPU，供噪声和 ROS 消息使用。此版本使用默认流、同步返回，暂不引入多流、双缓冲和跨帧异步调度。CudaLidar 是一个小类，其内部 CUDA 资源只存在于可选实现中，CPU 头文件不依赖 CUDA Runtime。

**先检查正确性** GPU 测试覆盖 40 个自由随机位姿、圆相切、共线边、部分视场、NaN 与无回波，逐束分别比较特殊值类别和有限距离，容差为  $2 \times 10^{-5}$  m。再施加同一批噪声样本并重复检查。启用 CUDA 却没有可用 GPU 时测试应失败，不能静默切到 CPU 后仍报告“GPU 验收通过”。

## 20.4 完整扫描收益取决于规模

```
1 # Copy benchmark CSV to textbook/data only when recording a new run.
2 python3 scripts/plot_ch20_cuda.py
```

每组预热 20 次,再运行 60 次,分别记录 P50/P95;CPU 与 GPU 使用相同位姿和几何。0/16/128/512 个障碍物、90/720/4096/16384 束共 16 组,圆与四边形各占一半,外框另计四条边。计时场景为独立合成夹具,不把随机生成器的连通性筛选时间计入扫描。

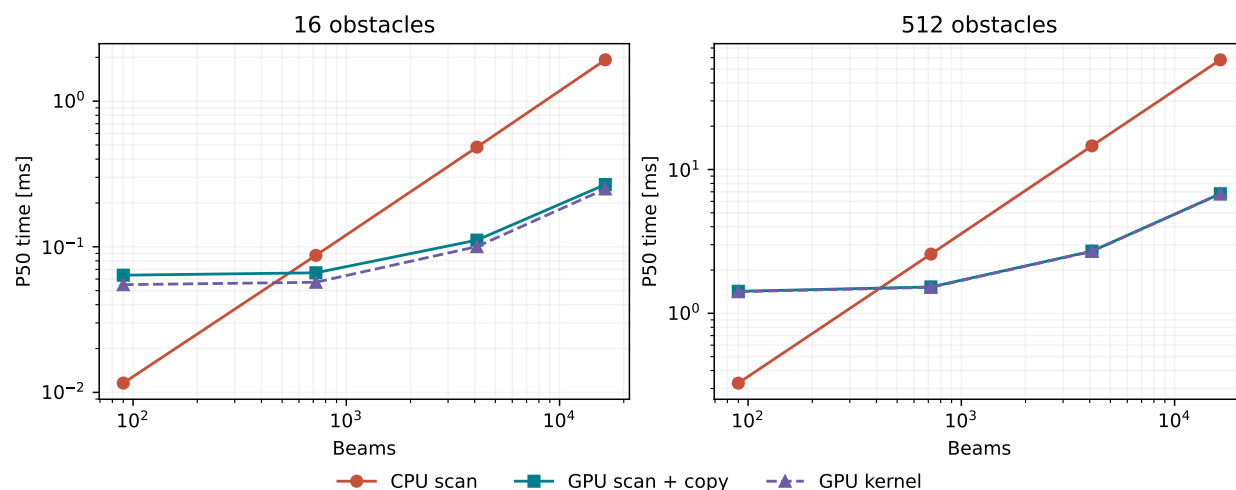


图 20.1: RTX 5060 主机实测,两个坐标轴都用对数。驻留 GPU 扫描包含内核启动、同步和结果回传;虚线只表示内核,不能替代整条曲线。

| 障碍/束      | CPU/ms | 内核/ms  | 驻留扫描/ms | CPU/扫描 |
|-----------|--------|--------|---------|--------|
| 16/90     | 0.0116 | 0.0549 | 0.0638  | 0.18   |
| 16/720    | 0.0873 | 0.0571 | 0.0663  | 1.32   |
| 16/16384  | 1.923  | 0.250  | 0.267   | 7.19   |
| 512/90    | 0.327  | 1.413  | 1.422   | 0.23   |
| 512/16384 | 57.913 | 6.766  | 6.784   | 8.54   |

表中均为预热后的 P50。少量束时 GPU 明显更慢:并行度不足、启动固定成本和 double 算术代价可能超过并行收益。720 束、16 障碍只有约 1.32 倍驻留扫描收益,不能写成数量级加速。

一次 setup 包括主机打包、分配、上传和事件创建;首组约 61 ms,还可能包含上下文初始化。之后的 setup 约 0.029-0.110 ms;单独上传约 0.002-0.012 ms,不应每帧重复支付。回传约 0.005-0.013 ms,已包含在驻留扫描里;不能把三个独立分位数相加当作总时长分位数。CUDA 事件测内核,墙钟测同步后的回传和整次扫描;只量异步 launch 的返回时间会严重低估成本。

本次 16 组输出的 float 距离最大差为 0,随机位姿与噪声检查也通过;这不是所有场景位级一致的承诺,仍保留浮点容差及特殊值检查。练习 ch20-1 补齐 ray-circle 根选择,并解释为什么增加障碍、但不增加束数,未必能让 GPU 获益。完整 ROS 噪声、消息构建和发布成本仍需在节点中另测,不能由此表推断。

## 20.5 接入 ROS，再测一次

```

1 ros2 launch motion2d_bringup ch20.launch.py backend:=cuda beams:=720
2 ros2 service call /sim/pause std_srvs/srv/SetBool "{data: false}"
3 # Separate experiment: starts and closes its own CPU/CUDA launches.
4 python3 scripts/run_ch20.py

```

仿真器通过 `lidar.backend` 在 `cpu` 与 `cuda` 之间选择；参数只在启动时读取。CUDA 构建关闭时若请求 `cuda`，会明确报错，不静默切回 CPU。几何和输出缓存跨扫描复用；`reset` 只重置仿真时间与同一条噪声随机流，不重复上传不变的世界。不论后端如何，**采集时间戳、激光坐标、量程编码和消息 QoS 都相同**。RViz 可直接查看 `/scan` 命中点；`/clock` 仍只有仿真器一个发布者。

默认关闭性能诊断，选学 `launch` 开启 `lidar.profile`，输出结构化 `/sim/lidar_timing`。其中扫描、噪声、消息构造、`publish` 调用耗时相加为本次总耗时；GPU 另列事件内核与回传，CPU 对这两个不适用字段填 NaN。总耗时截至 `publish` 返回，不包含订阅者接收、RViz 渲染、调试光束或诊断消息自身。这说明发布线程花了多久，不是 DDS 传输延迟的直接测量。

| 后端/束       | 扫描/ms  | 噪声/ms  | 发布/ms  | 总 P50/ms | 总 P95/ms |
|------------|--------|--------|--------|----------|----------|
| CPU/90     | 0.0559 | 0.0041 | 0.0091 | 0.0706   | 0.0752   |
| CUDA/90    | 0.1203 | 0.0039 | 0.0086 | 0.1344   | 0.1774   |
| CPU/720    | 0.2993 | 0.0248 | 0.0092 | 0.3358   | 0.5049   |
| CUDA/720   | 0.1262 | 0.0246 | 0.0095 | 0.1623   | 0.2101   |
| CPU/16384  | 3.5454 | 0.2585 | 0.0105 | 3.8232   | 4.4095   |
| CUDA/16384 | 0.3700 | 0.4665 | 0.0111 | 0.8418   | 0.9521   |

各阶段列为 P50；消息构造约 0.0007–0.0022 ms，完整 CSV 中另列。分位数不满足可加性，不能用表中各列 P50 的和校验总 P50。90 束时 GPU 仍更慢；720 束与 16384 束的发布线程总 P50 分别约改善 2.07 和 4.54 倍。噪声仍在 CPU 执行，在大束数下已经是不可忽视的一部分；两组运行中的 CPU 时钟、缓存和调度也会影响阶段耗时。

这里采用第 05 章 `seed=42` 的随机世界、解析圆运动、10 Hz 扫描与 0.01 m 高斯测距噪声，不同于上一节的合成计时夹具，不能直接将两张表的比值拼接。每种后端采集 0–10 s 的 101 帧，剔除前 10 帧后统计剩余 91 帧。三个束数组均逐时间戳、逐束比对有噪声结果，本次最大差为 0；`reset` 重放初始扫描也通过。完整数据为 `textbook/data/ch20_ros.csv`。

**验收与练习** 分别运行 CPU 和 CUDA，先核对数组及 NaN/+∞ 类别，再看计时。将诊断关闭、打开，比较性能测量自身的开销；不要把 `profiler` 的开销算成传感器物理延迟。若增加逐束采集畸变，应改变每束的采集状态和时间语义，不能仅把本章 `snapshot` 内核改成更快就宣称完成了畸变模拟。GPU 后端与无 GPU 的完整主线应分别在构建矩阵中检验。

## 附录 A 数学回查与数值检查

**用途** 忘记符号时回查，不必先读完本附录再运行课程。

**小实验** 用中央差分检查一个二维点变换的雅可比。

### A.1 坐标变换先写清方向

位姿  $T_{AB}$  表示 B 在 A 中的位置和朝向，点变换为

$$p_A = R(\psi)p_B + t, \quad R(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi \\ \sin \psi & \cos \psi \end{bmatrix}. \quad (\text{A.1})$$

平移单位为 m，yaw 单位为 rad、逆时针为正。两个变换合成时，顺序与下标一致： $T_{AC} = T_{AB}T_{BC}$ 。逆变换的平移是  $-R^T t$ ，不是简单的  $-t$ ；速度或加速度只旋转，不加平移。源码对应 `geometry/se2` 中的 `transformPoint`、`compose`、`inverse`。

若采用地图坐标的平移加法与 yaw 加法，点  $p_B = (x, y)$  的雅可比为

$$\frac{\partial p_A}{\partial (t_x, t_y, \psi)} = \begin{bmatrix} 1 & 0 & -x \sin \psi - y \cos \psi \\ 0 & 1 & x \cos \psi - y \sin \psi \end{bmatrix}. \quad (\text{A.2})$$

前两列无量纲，最后一列为 m/rad。若改成机体坐标系扰动或 SE(2) 左乘/右乘扰动，雅可比会改变；同样叫“位姿导数”并不意味着可以直接交换公式。

### A.2 最小二乘在解什么

测量模型  $z = Hx + \epsilon$ ，误差协方差为  $\Sigma$ 。加权最小二乘最小化  $(Hx - z)^T \Sigma^{-1} (Hx - z)$ ，正规方程为

$$(H^T W H)x = H^T W z, \quad W = \Sigma^{-1}. \quad (\text{A.3})$$

实现中用分解解线性方程，通常不显式构造逆矩阵；噪声协方差可通过分解做白化。矩阵病态或欠约束时，数值结果并不会因为“解出了一个向量”就可信。第 08 章的退化几何、第 10 章固定首帧和第 18 章正定二次项，分别处理不同的可观测性或唯一性问题。

## A.3 协方差与权重

$P_{ii}$  是第  $i$  个状态分量的方差,  $P_{ij}$  描述二者相关性。位置方差单位为  $\text{m}^2$ , 位置与角度的交叉协方差为  $\text{m}\cdot\text{rad}$ ; 它们不能不加说明地与位置距离相加。一阶传播写为  $P^- = FPF^T + Q$ , 测量创新协方差为  $S = HP^-H^T + R$ 。这里  $F/H$  是相应状态与观测的局部雅可比,  $Q/R$  表示建模后的过程和测量不确定性。仿真器每采样白噪声标准差, 与连续噪声密度的单位和离散缩放不同, 回查第 06、09 章。

## A.4 用差分检查, 用尺度理解误差

对输出向量  $f$  和第  $i$  个输入, 用

$$J_{:,i} \approx \frac{f(x + he_i) - f(x - he_i)}{2h} \quad (\text{A.4})$$

检查解析雅可比。 $h$  太大时有截断误差, 太小时两次接近数值相减会放大舍入误差。各维单位不同, 应分别选取合适步长, 不对所有变量机械地套用一个无单位常数。

```
1  c++ -std=c++17 -I/usr/include/eigen3 \
2    -I exercises/appA/starter exercises/appA/check_jacobian.cpp \
3    -o tmp/appA-check
4  ./tmp/appA-check
5  # Use solutions instead of starter to check the reference answer.
```

练习 appA-1 的固定点为 (2,-1) m, 位姿为 (0.3,-0.8,0.7), 依次尝试四个步长。验收在  $h = 10^{-5}$  时要求矩阵误差范数小于  $10^{-7}$ ; 这个例子的坐标与角度按各自单位扰动, 阈值只服务于该数值检查。差分通过说明局部导数与所写函数一致, 不代表函数的物理模型、时间关联或坐标约定就正确。

## A.5 凸集、二次型与课程接口

凸集中的任意两点, 其连线仍在集合内。半空间  $Ap \leq b$  的交集是凸集, 这使第 13 章凸走廊与第 16 章 Bézier 控制点证书可以连接起来。正定二次型满足  $x^T Px > 0$  ( $x \neq 0$ ); 半正定只要求非负, 允许零方向。第 18 章自写 QP 解算器要求强凸, 并没有承诺支持所有半正定 QP。

| 要回查的量    | 关键问题                      | 课程位置        |
|----------|---------------------------|-------------|
| 位姿与速度    | 表达在哪个坐标系?                 | 02、07       |
| 力、质量与加速度 | 输入是 N 还是 $\text{m/s}^2$ ? | 04、17       |
| 白噪声与偏置   | 每采样方差, 还是随时间积累?           | 06、09       |
| 雅可比      | 扰动方向、单位、差分误差?             | 08-10、15-16 |
| ESDF 与走廊 | 软代价, 还是连续区域证书?            | 12-16       |
| QP 约束    | 节点约束能否保证采样间安全?            | 18          |
| 未来交接     | 同一个时刻的 p/v/a 是否一致?        | 19          |

**理解练习** 任选一个源文件, 在纸上列出输入、输出、单位与坐标系, 再写一个可以手算的极端例子。例如把旋转角设为 0 或  $\pi/2$ , 把力设为 0, 把协方差设为对角矩阵; 若这些例子解释不通, 先修正模型含义, 再考虑增加复杂算法。

## 附录 B 差速扫地机器人：先约束运动方向

**目标** 从左右轮速得到圆心位姿，理解不能横移的含义。

**范围** 独立无滑移运动学实验；主线仍为全向圆盘，未接入轮力矩或 NMPC。

### B.1 圆盘外形相同，能做的动作不同

若希望先学建图与轨迹优化，全向圆盘能把问题分开；若研究扫地机器人，差速模型更贴近驱动约束。圆盘半径仍负责碰撞，车轮半径  $r_w$  和左右接地点间距  $b$  则负责运动换算，三者不要混用。轮速  $w_L, w_R$  单位为 rad/s，同号为前进，理想无滑移模型为

$$v = \frac{r_w(w_R + w_L)}{2}, \quad \omega = \frac{r_w(w_R - w_L)}{b}, \quad \dot{x} = v \cos \psi, \quad \dot{y} = v \sin \psi, \quad \dot{\psi} = \omega. \quad (\text{B.1})$$

横向速度满足  $-\sin \psi \dot{x} + \cos \psi \dot{y} = 0$ 。它约束瞬时速度，不能写成一个简单的位置等式消掉；因此称为非完整约束。机器人可以通过前进和转向改变横向位置，但不能直接侧移。

```
12 VelocityCommand differentialVelocity(double left,double right,const WheelGeometry & g) {
13     validate(g,left,right);
14     return {{g.radius*(right+left)/2,0},{g.radius*(right-left)/g.track,true}};
15 }
16 std::array<double,2> differentialWheels(double speed,double yaw_rate,const WheelGeometry & g) {
17     validate(g,speed,yaw_rate);
18     return {(speed-g.track*yaw_rate/2)/g.radius,(speed+g.track*yaw_rate/2)/g.radius};
19 }
20 //
```

stepDifferential 复用第 04 章恒定机体系速度的精确 SE(2) 积分，避免另写一套近似积分器。本函数没有电机、轮胎滑移或质量；阶跃轮速也不能凭空生成物理正确的瞬时 IMU 加速度。

### B.2 手算一次圆弧

取  $r_w = 0.05$  m、 $b = 0.4$  m、 $w_L = 2$ 、 $w_R = 6$  rad/s，得到  $v = 0.2$  m/s、 $\omega = 0.5$  rad/s、转弯半径  $v/\omega = 0.4$  m。从原点、yaw=0 出发， $t = \pi$  s 时应到达 (0.4,0.4) m、yaw= $\pi/2$ ； $t = 4\pi$  s 恰好一圈。速度初始样本为施加轮速前的静止状态，随后为保持命令的运动学状态。

```
1 ros2 run motion2d differential_drive_demo > textbook/data/appB_circle.csv
2 /usr/bin/python3 scripts/plot_appB.py
```

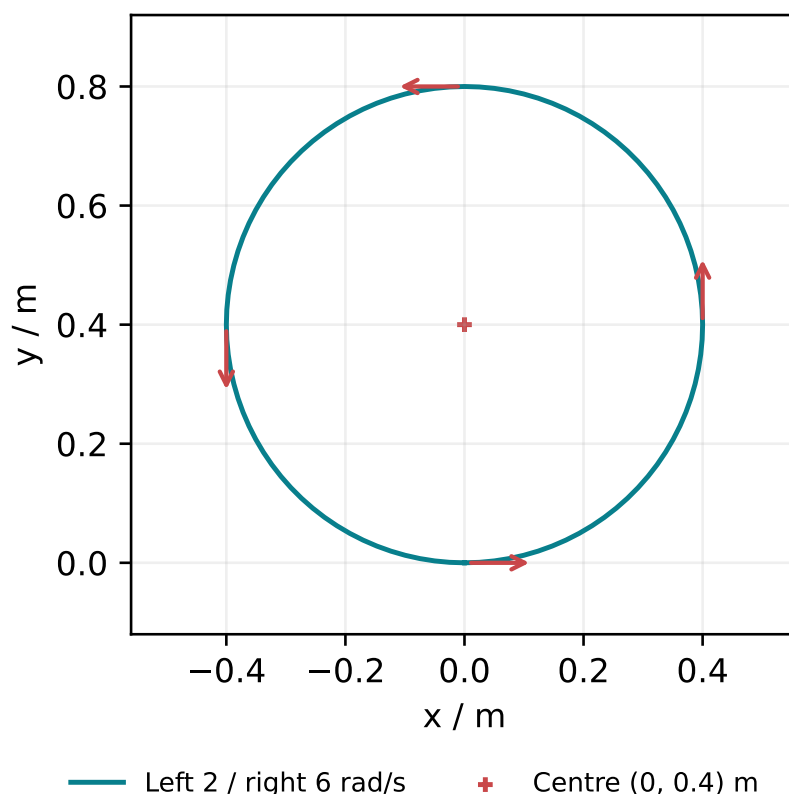


图 B.1: 实际积分得到的圆弧, 坐标等比例; 箭头表示朝向, 始终与运动方向相切。这是独立几何实验, 不包含障碍、轮动力学或 ROS 轨迹跟踪。

## B.3 怎样接到轨迹与控制

对非零、正向运动的平滑参考  $p(t)$ , 可从速度与加速度计算

$$\psi = \text{atan2}(v_y, v_x), \quad v = \|\dot{p}\|, \quad \omega = \frac{v_x a_y - v_y a_x}{v_x^2 + v_y^2}, \quad \kappa = \omega/v. \quad (\text{B.2})$$

此时 yaw 不能再独立任意指定。低速处上述分母接近零, 停点和倒车必须定义单独的运动模式; 不能靠一个很小的分母下限, 把不一致轨迹变成“可行”。原地旋转则应使用  $v = 0$ 、左右反向轮速。

第 18 章预测矩阵是全向受力模型, 直接换话题名不会变成差速控制。一种扩展是以  $(x, y, \psi)$  为状态、 $(v, \omega)$  为输入做 NMPC: 在每次滚动优化中保留三角函数运动方程, 加入轮速及变化率限制, 并在求解失败时制动。若输入轮力矩, 还应增加轮惯量、质量和轮地接触模型; 这些扩展本课程未实现。

**练习与验收** appB-1 补全逆运动学, 手算直行、倒车、原地转动与圆弧四组轮速。核心测试另外检查四分之一圆解析解、分步积分一致性和零侧向速度。改变两轮速的公共倍率, 解释为何圆半径不变、转完一圈所需时间改变。

## 附录 C 进阶感知：一次只增加一种误差

**已实现** 独立偏置随机游走实验与统计验收。

**选做设计** 延迟、逐束畸变、复杂障碍与重定位；基础模拟器仍保持第 06 章语义。

### C.1 白噪声不会自己变成偏置

第 06 章每采样白噪声  $n_k$  独立、零均值，其标准差直接配置。真实传感器还可能有随时间缓慢漂移的偏置。本附录只取一个轴，以布朗模型近似：

$$y_k = y_k^{\text{ideal}} + b_k + n_k, \quad b_{k+1} = b_k + \sigma_b \sqrt{\Delta t} \xi_k, \quad \xi_k \sim \mathcal{N}(0, 1). \quad (\text{C.1})$$

陀螺偏置  $b$  的单位为 rad/s,  $\sigma_b$  为 (rad/s)/ $\sqrt{\text{s}}$ ；它既不是每采样白噪声标准差，也不是角速度测量的连续白噪声密度。从确定的初值出发，独立增量相加给出

$$\text{Var}[b(T) - b(0)] = \sigma_b^2 T, \quad Q_b = \sigma_b^2 \Delta t. \quad (\text{C.2})$$

若误写成乘  $\Delta t$ ，固定时长的方差就变成  $\sigma_b^2 T \Delta t$ ，增加采样率会错误地让偏置更稳定。第 09 章偏置状态的过程噪声也应按模型一致离散化。

```
6 double stepBiasWalk(double bias, double density, double dt, double unit_noise) {
7     if(!std::isfinite(bias) || !std::isfinite(density) || density<0 ||
8         !std::isfinite(dt) || dt<0 || !std::isfinite(unit_noise))
9         throw std::invalid_argument("Bias walk needs finite inputs and nonnegative density/dt");
10    return bias+density*std::sqrt(dt)*unit_noise;
11 }
12 //
```

随机数由调用方传入，便于固定样本做解析检验。六轴可分别使用同一公式，每轴及白噪声使用独立随机流；有缓存的正态分布对象也应独立，不能只换引擎而共用其缓存。偏置是持久状态，每次采样保存到下一步；reset 必须同时恢复偏置初值和随机流。

### C.2 统计的是许多终点，不是一条曲线

从零偏置出发， $\sigma_b = 0.002 \text{ (rad/s)}/\sqrt{\text{s}}$ 、 $T = 10 \text{ s}$ ，100/200 Hz 各运行 5000 条独立轨迹，比较终点无偏样本方差和理论值  $4 \times 10^{-5} \text{ (rad/s)}^2$ 。源码固定种子；改变频率时不要求每条随机轨迹重合，只要求统计分布一致。实际两组方差为  $3.8782 \times 10^{-5}$  和  $3.9963 \times 10^{-5} \text{ (rad/s)}^2$ ，均接近理论值；这种有限样本的一致性不等于真实器件都服从该模型。

```
1 ros2 run motion2d bias_walk_demo tmp/appC_bias
2 # path.csv: one path; summary.csv: independent endpoint ensembles
```

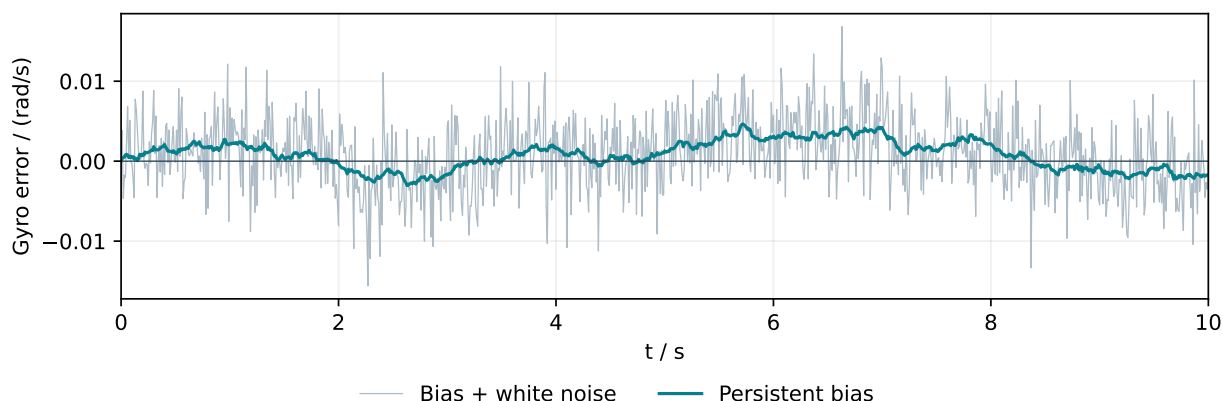


图 C.1: 静止陀螺仪单轴误差的独立实验。浅色为偏置加每采样  $0.004 \text{ rad/s}$  白噪声，深色偏置在相邻样本间持续存在；不能拿同一路径上的相关样本代替独立试验。

**练习 appC-1** 补全步长缩放；先检查同一单位噪声下  $\Delta t$  增为四倍时增量加倍，再运行统计实验。本工具未作为主线 ROS 参数接入；若将其用于 IMU，必须同步扩展参数、reset、日志和滤波过程噪声实验。

## C.3 进一步扩展时，先写验收边界

以下是选做方向，不代表基础模拟器已经具备这些功能：

### 延迟与丢包

保留采集时间，单独安排到达时间；测试乱序、缺测、历史缓存到期。滤波器对延迟校正已有处理，但模拟器尚未自动注入网络延迟。

### 逐束扫描畸变

每束有  $t_i$ ，统一到参考时刻的点为  $p_{\text{ref}} = T_{\text{ref}}^{-1} T(t_i) p_i$ 。真实估计使用估计运动，不能读模拟真值。当前雷达为 snapshot，不能只填写 time\_increment 就称为旋转扫描。

### 凹多边形

渲染、点内判断、扫掠碰撞和最近射线交点都要一致；用凹口自由点做反例。当前随机多边形是凸多边形。

### 动态障碍

世界几何要随采集时间变化；规划还需预测与时空碰撞检查。静态距离场的正净空不保证将来安全。

### 全局重定位

局部 ICP 的初值范围有限，回环候选也不等于任意位置重定位。先设计“搬走机器人”测试，再考虑全局候选搜索、几何验证和多假设管理。

## 附录 D 记录、回放与外部工具

**目标** 关闭模拟器后，仅凭记录的激光和 IMU 重新生成里程计与观测地图。

**先修** 第 06、09、10 章；终端已 source 主机 ROS 与课程工作空间。

### D.1 传感器时间与文件时间是两件事

rosbag2 保存消息及存储时间戳。传感器 header.stamp 表示采集时刻；录制器收到消息时，还会记录当时看到的时钟值。使用仿真时间录制，避免把从零开始的传感器时刻与系统日历时间混用。

```
1 # With one course simulator already running:
2 rosbag2 bag record --use-sim-time -o tmp/my_sensors \
3   --topics /scan /imu/data_raw
4 # Ctrl+C closes the bag; then stop the simulator and old estimators.
5 rosbag2 bag info tmp/my_sensors
```

只录制两类观测，使重新运行的估计器自己计算 TF 与地图。原始姿态和地图不是 SLAM 输入，记录全部话题再一起回放会混入旧结果或引入发布者冲突。消息内部的 frame、采集时间、协方差与噪声样本均应保留。

### D.2 谁来发布时钟与变换

```
1 # Terminal A, no simulator in this launch:
2 rosbag2 launch motion2d_bringup replay_slam.launch.py rviz:=true
3 # Terminal B:
4 rosbag2 bag play tmp/my_sensors --clock 200 --rate 0.5
```

播放器拥有唯一 /clock；所有估计节点使用仿真时间。半速播放改变消息到达的墙钟速度，不能把 IMU 积分步长也乘二。录制时采用的每条消息采集时间仍是滤波器的时间依据。

此启动文件复用第 10 章雷达里程计、融合、估计里程计适配和 SLAM 四个节点。估计适配器独占 odom 到 base\_link，SLAM 独占 map 到 odom；两个静态发布者只提供圆心重合、朝向一致的 laser 与 imu\_link 外参。它们适用于本课程的记录，不代表真实传感器不需要外参标定。RViz 预期显示重建地图、点云和估计轨迹；本机此次验收使用无界面模式。

## D.3 做一次真正的录制与回放验收

在仓库根目录运行下面的脚本；输出目录中的 `sensors` 不应已存在。

```
1 /usr/bin/python3 scripts/run_appD.py --output tmp/my_replay
```

脚本在独立 ROS 域内先启动暂停的 `reference` 模型，等待录制器订阅两类传感器后运行六秒，关闭录制器与模拟器，再启动仅含估计的栈并半速回放。使用第 10 章参数：720 束、10 Hz 雷达，200 Hz 白噪声 IMU，`seed=42` 的随机世界。

本机 `rosbag2 0.33.3` 实测记录了 59 帧激光、1180 帧 IMU，存储时间从 0.105 到 6.000 s。早期等待首个时钟/订阅建立的样本没有进入记录；因此不能用“六秒乘频率”替代文件计数。存储时刻与采集时刻最大相差 0.005 s，录制器使用最近收到的 `/clock`；这些不是新的物理采样时间，回放保留 `header`。

回放所有消息的字段值与记录一致，最后里程计时刻为 6.000 s；观测地图有 12556 个已观测格，累计点云 537 点。检查还确认估计器不订阅真值、模拟器已停止、估计 TF 边存在且播放器只有一个时钟。这些指标验证回放链路，不用于宣称 SLAM 精度、回环能力或不同系统性能。

**一个容易误判的检查** CDR 序列化有对齐填充字节，重新编码时这些字节未必相同。本脚本将解码后的每个字段转为固定次序的数据表示再求散列，包含 `stamp`、`frame`、量测和协方差。它检查量测语义一致，不要求无意义的填充字节一致；NaN 和无回波编码也保留。

## D.4 与现有二维 SLAM 对照的接入练习

例如接入 `slam_toolbox` 时，先按其版本文档准备扫描、标定外参及所要求的里程计 TF。若外部算法只使用雷达而课程算法融合 IMU，应分别列出传感器与先验差异，不能把两者结果当作同输入下的纯算法比较。禁止用真值里程计帮助一个系统，再隐藏这个条件。

在同一 `bag` 上串行运行两个系统，避免争抢 TF；记录版本、参数、初值、回放速率、计算预算、失败次数和第 19 章定义的 ATE/RPE。地图质量只在已观测区域评估。本附录提供接入协议，外部 SLAM 的安装和对照是选做实验，本机未宣称已运行。

**练习 appD-1** 从含时钟、TF、地图和真值的话题列表中选出两类传感器，缺少 IMU 时应明确失败。再说明为什么重放旧 `map` 到 `odom` 会使“重新建图”实验失去意义。

## D.5 从教材跳到 API 文档

教材按概念安排，API 文档按函数和类型检索。两者共同解释输入、输出、单位与前提。仓库根目录的 Doxyfile 扫描算法和 ROS 边界头文件，输出到临时目录：

```
1 bash scripts/build_api.sh
2 # Open tmp/doxygen/html/index.html in a browser.
```

脚本优先使用系统 doxygen；本主机还准备了局部 1.15.0 工具，不修改系统安装。其他主机需安装 Doxygen；公式图片还使用 LaTeX 工具链。本次生成已覆盖 52 个头文件并完成公式渲染，文档语法警告作为错误处理。这不代表每个函数说明都完整，更不能代替单元测试。

例如附录 C 的偏置函数，调用者必须知道以下内容：

| 字段            | 应解释的意义                       |
|---------------|------------------------------|
| brief         | 一次标量偏置演化，区别于每采样白噪声           |
| param         | 当前偏置、密度、实际间隔和标准正态样本；逐项给出单位   |
| return        | 更新后的偏置，保持原量纲；由调用方保存到下一步      |
| details / pre | 有限值、非负密度/时间、独立样本；该工具未改变基础模拟器 |

不要只把变量名翻译成中文。比如“dt: 时间”不如“非负实际间隔，单位 s，缺失后不能仍用名义周期”明确。公式使用 Doxygen 数学语法，项目配置加载 amsmath，避免算子命令生成失败。

## D.6 让实验报告可以被重新运行

报告模板位于仓库 docs 目录：

```
1 docs/EXPERIMENT_REPORT_TEMPLATE.md
2 docs/VALIDATION.md
```

前者是待填写模板，后者是本课程已经执行的验证记录，二者不能混淆。报告先写问题和通过标准，再列 Git 版本、构建方式、主机、参数文件、随机种子、模型、传感器与时钟。结果需包含总试验次数、全部失败、指标定义与对齐方式；给出原始数据及其生成命令。

若比较性能，分别列准备、内核、传输、消息与求解阶段，说明预热和计时区间；若比较轨迹，区分离散采样与连续证书。记录 PDF 散列和本次新增/修改页面的视觉复核结果，不需要反复检查无改动的旧章节。

**选做练习** 在生成的 API 中找到一个函数，指出其坐标系、单位、前提和失败方式。复制模板，填写附录 C 的两采样率实验；随后只改密度并重新测量。先预测方差应如何缩放，再用实际数据判断预测；模板中的空白项目不得冒充测量结果。